CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中
最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、
Elementwise、Softmax、GEMM、FlashAttention 等
面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释（WHY + HOW），
并以递进式优化路线（naive → tiling → vectorize →
tensor core → warp specialized）展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页：`https://github.com/xlite-dev/LeetCUDA`

更新日期：2026 年 7 月 23 日

```
// ================================================================================
// notes-v2.cu — CUDA Kernel 面试背题笔记
// ================================================================================
//
// 整理自 LeetCUDA 项目（https://github.com/xlite-dev/LeetCUDA），涵盖：
//   - 面试高频 CUDA kernel 的完整实现（~30 个 kernel）
//   - 每类 kernel 附带详细的面试要点注释（WHY + HOW）
//   - 优化技术的递进式讲解（naive → tiling → vectorize → tensor core → ws）
//   - BLAS 语义：N=col-major(Normal), T=row-major(Transposed)
//
// 10 个 Phase 覆盖：
//   Phase 0 — 面试框架速查（GPU 架构 / Memory Hierarchy / Roofline / 优化清单）
//   Phase 1 — 基础原语：Warp Reduce / Block Reduce / Dot Product（含 broadcast 增强版）
//   Phase 2 — Elementwise：ReLU / Elementwise Add / Histogram（基础 + float4 向量化 + atomic）
//   Phase 3 — Softmax：naive → safe → online + RMS/Layer Norm
//   Phase 4 — RoPE：旋转位置编码（Llama 风格 theta=10000）
//   Phase 5 — Mat Transpose：基础版 + BCF merge_write 最佳版（Bank Conflict专题）
//   Phase 6 — GEMV：SGEMV K32/K128/K16（warp-per-row）
//   Phase 7 — GEMM ★：SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMA m64n128k16
//   Phase 8 — FlashAttention-2split_q（FA-2，含 online softmax + P@V 寄存器复用）
//
// ================================================================================
// Phase 0：面试框架速查（纯注释，面试开场必备的基础知识）
// ================================================================================

// ---- GPU 架构速查 ----
//
// SM (Streaming Multiprocessor) 内部结构：
//   - Warp Scheduler ×4：每 SM 4 个 warp scheduler，每个每周期可发射 1 条指令
//   - Register File：每 SM 65536 × 32-bit (4 bytes) = 256KB
//   - Shared Memory / L1：可配置，最大 shared memory ~228KB (Hopper)
//   - Tensor Cores：Hopper 每 SM 4 个；Blackwell 数量随型号/定义不同，建议以官方 ISV guide 为准
//   - Warp = 32 threads：最小调度单元，SIMT 执行模型
//
// Memory Hierarchy 带宽数量级（H100 参考）：
//   HBM3:         ~3.35 TB/s（理论），实际 ~2.5-3.0 TB/s
//   L2 Cache:     ~12 TB/s（50MB，跨 SM 共享）
//   L1/SMEM:      ~19 TB/s（每 SM ~228KB）
//   Register:     ~0 延迟，~100+ TB/s 等效带宽
//
// 关键瓶颈判断：
//   Memory-bound：AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
//   Compute-bound：AI 足够大，受限于计算吞吐
//   Latency-bound：线程不够多，无法隐藏内存延迟
//
// Occupancy 公式：
//   occupancy = active_warps / max_warps_per_SM
//   受三类资源分别取下限：每线程寄存器数 → threads/SM；每 block shared memory
//   → blocks/SM；block 大小 → blocks/SM

// ---- 常见优化手段速查清单 ----
//
// 1. Coalesced Memory Access（合并访问）
//    - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
//    - 否则产生多次事务（最坏 32 次）
//
// 2. Tiling（分块）
//    - 将数据从 HBM 分块加载到 shared memory 复用，减少 HBM 访问
//    - GEMM: Block Tile (BM×BN) + K Tile (BK)
//
// 3. Vectorized Memory Access（向量化）
//    - 使用 float4/half2 等向量类型，减少 load/store 指令数
```

```
//    - float4 = 128-bit，单条指令加载 16 bytes
//
// 4. Thread Tile（寄存器分块）
//    - 每个线程计算多个输出元素（TM×TN），提高计算密度
//    - 减少线程总数，降低同步开销
//
// 5. Bank Conflict Avoidance
//    - Shared memory 有 32 banks × 4 bytes
//    - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
//    - 解决方案：PAD（在每行末尾加 1 个元素打破对齐）
//
// 6. Pipeline / Double Buffering（流水线）
//    - cp.async 异步拷贝下一批数据时同时做当前批的计算
//    - Stage 数 = 2/3/4，权衡 shared memory 占用和延迟隐藏
//
// 7. Tensor Core（MMA / WGMMA）
//    - MMA m16n8k16 (Ampere): warp 级指令，单 warp 完成 16×8×16 的矩阵乘
//    - WGMMA m64n128k16 (Hopper): warpgroup 级指令（128 threads），异步执行
//
// 8. Warp Specialization（Hopper+）
//    - Producer warpgroup 做 TMA 数据搬运，Consumer warpgroup 做计算
//    - 通过 cuda::barrier 同步，完全解耦数据搬运和计算
//
// 9. TMA (Tensor Memory Accelerator, Hopper+)
//    - 硬件 DMA 引擎，支持 1D~5D 寻址，低寄存器开销
//    - 配合 cp.async.bulk 实现异步数据搬运

// ---- Roofline 分析公式 ----
//
// AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
//
// GEMM (M=N=K=4096):
//   FLOPs = 2 × M × N × K = 2 × 4096³ ≈ 137 GFLOPS
//   Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
//   AI    ≈ 137G / 200M ≈ 685 FLOPS/Byte → compute-bound（远超 H100 ridge point:
//   FP16 TC ≈ 295:1, FP32 ≈ 20:1）
//
// GEMV (M=4096, K=4096):
//   Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
//   AI    ≈ 33M / 67M ≈ 0.5 FLOPS/Byte → severely memory-bound
//
// Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPS/Byte → memory-bound

// ============================================================================
// Phase 1: 头文件 + 宏定义 + 基础原语（Warp Reduce / Block Reduce）
// ============================================================================
// 面试要点：
//   - warp_reduce: 用 __shfl_xor_sync 做蝶形归约，O(logN) 步，无需 shared
//   memory
//   - block_reduce: 两级归约（warp → shared memory → warp0 broadcast），
//     注意最后必须 broadcast 回所有线程（__shfl_sync），否则只有 warp0 知道结果
//   - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量，更均衡

#include <algorithm>
#include <cstring>
#include <cuda_fp16.h>
#include <cuda_runtime.h>
#include <float.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cublas_v2.h>
#include <cuda.h>
```

```cpp
#include <cuda/barrier>
#include <cuda/ptx>

#if defined(NOTES_V2_ENABLE_CUDNN)
#pragma nv_diag_suppress 128
#include <cudnn_frontend.h>
#pragma nv_diag_default 128
namespace fe = cudnn_frontend;
#endif

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
#error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
#endif

#define INT4(value) (reinterpret_cast<int4 *>(&(value))[0])
#define FLOAT4(value) (reinterpret_cast<float4 *>(&(value))[0])
#define HALF2(value) (reinterpret_cast<half2 *>(&(value))[0])

static constexpr int kWarpSize = 32;

// ============================================================================
// Phase 1a: Warp Reduce (warp 内归约，纯寄存器操作，无需 shared memory)
// ============================================================================

// Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
// 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
// 复杂度 O(logN)，N=32 时仅需 5 步
// 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
// kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width，限制 shuffle 在同一 segment 内
// 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时，只有同 segment 的 lane 参与通信
//   蝶形归约示意 (以 warpSize=8 为例，实际 warpSize=32 有 5 次迭代)：
//
//   初始: 每个 lane 持有自己的值 v0..v7
//   lane:  0    1    2    3    4    5    6    7
//   val:  v0   v1   v2   v3   v4   v5   v6   v7
//
//   mask=4 (第1次迭代，lane i 与 lane i^4 交换并累加):
//          ┌─────────────────┐
//   对:   (0,4) (1,5) (2,6) (3,7)
//
//   lane:  0    1    2    3    4    5    6    7
//   val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
//
//   mask=2 (第2次迭代，lane i 与 lane i^2 交换并累加):
//          ┌───────┐  ┌───────┐  ┌───────┐  ┌───────┐
//   对:   (0,2) (1,3) (4,6) (5,7)
//       ─── 前4个一组 ───   ─── 后4个一组 ───
//
//   lane:  0    1    2    3    4    5    6    7  (每lane持有前一轮2个值的和再加本轮配对)
//   val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7}
//        = v0+v2+v4+v6 ... (逐步归约)
//
//   mask=1 (第3次迭代，lane i 与 lane i^1 交换并累加):
//          ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
//   对:   (0,1)(2,3)(4,5)(6,7)
//
//   lane:  0    1    2    3    4    5    6    7
//   val:  Σall Σall Σall Σall Σall Σall Σall Σall  ← 所有 lane 拥有全归约结果!
//
//   mask=0: 循环终止，归约完成。
//
//   关键性质:
//   - XOR 配对是对称的: lane i 的配对对象是 lane i^mask，而 (i^mask)^mask = i
```

```cpp
//   - 每轮每个 lane 只和恰好 1 个其他 lane 通信（一对一，无冲突）
//   - 每轮信息传递距离减半：16→8→4→2→1（距离减半，信息翻倍）
//   - O(log₂ N) 步完成，无需 shared memory，纯寄存器操作
//   - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width：
//     当 kWarpWidth=4 时，只有同 segment(4个一组)内的 lane 参与 shuffle
template <const int kWarpWidth = kWarpSize, typename T = float>
__device__ __forceinline__ T warp_reduce_sum(T val) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
  }
  return val;
}

// Warp Reduce Max — generic
template <const int kWarpWidth = kWarpSize, typename T = float>
__device__ __forceinline__ T warp_reduce_max(T val) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
  }
  return val;
}

// ============================================================================
// Phase 1b: Block Reduce（block 内归约，两级：warp → shared memory → warp reduce）
// ============================================================================

// Block Reduce Sum — FP32（增强版，带 broadcast）
// 两级归约流程：
//   1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
//   2. warp leader (lane=0) 写入 shared memory
//   3. syncthreads 后，lane 0~kNumWarps-1 读取 shared memory
//   4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
//   5. __shfl_sync broadcast 到所有线程（关键！否则每个warp只有lane<kNumWarps 知道结果）
template <const int kNumThreads = 256>
__device__ float block_reduce_sum(float val) {
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  int warp = threadIdx.x / kWarpSize;
  int lane = threadIdx.x % kWarpSize;
  __shared__ float shared[kNumWarps];

  float value = warp_reduce_sum<kWarpSize>(val);
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < kNumWarps) ? shared[lane] : 0.0f;
  value = warp_reduce_sum<kNumWarps>(value);
  // 关键：broadcast 结果到所有线程，后续用 result
  // 做除法等操作时所有线程都能拿到
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// Block Reduce Max — FP32（增强版，带 broadcast）
template <const int kNumThreads = 256>
__device__ float block_reduce_max(float val) {
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  int warp = threadIdx.x / kWarpSize;
  int lane = threadIdx.x % kWarpSize;
  __shared__ float shared[kNumWarps];

  float value = warp_reduce_max<kWarpSize>(val);
```

```cpp
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
  value = warp_reduce_max<kNumWarps>(value);
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
// 多 block 各自做 warp→smem→warp0 reduce，然后 atomicAdd 到全局 y
// 跨 block 求和的常见模式，适合 N 较大时使用；
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/reduce/block_all_reduce.cu
template <const int kNumThreads = 256>
__global__ void block_reduce_all(float *a, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + tid;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float val = (idx < N) ? a[idx] : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  val = warp_reduce_sum<kWarpSize>(val);
  if (lane == 0)
    shared[warp] = val;
  __syncthreads();

  val = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0)
    val = warp_reduce_sum<kNumWarps>(val);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
    atomicAdd(y, val);
}

// ---- Dot Product: y = sum(a[i] * b[i]) ----
// 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/dot-product/dot_product.cu
template <const int kNumThreads = 256>
__global__ void dot(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + tid;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  prod = warp_reduce_sum<kWarpSize>(prod);
  if (lane == 0)
    shared[warp] = prod;
  __syncthreads();

  prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
    prod = warp_reduce_sum<kNumWarps>(prod);
  if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
```

```
    atomicAdd(y, prod);
}

// Dot Product + float4
// Grid:  ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
// Block: (64, 1, 1)，256/4=64
// 注意：该版本默认输入地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/dot-product/dot_product.cu
template <const int kNumThreads = 256 / 4>
__global__ void dot_vec4(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * kNumThreads + tid) * 4;
  constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ float shared[kNumWarps];

  float4 reg_a = FLOAT4(a[idx]);
  float4 reg_b = FLOAT4(b[idx]);
  float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
                            reg_a.z * reg_b.z + reg_a.w * reg_b.w)
                         : 0.0f;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  prod = warp_reduce_sum<kWarpSize>(prod);
  if (lane == 0)
    shared[warp] = prod;
  __syncthreads();

  prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
  if (warp == 0)
    prod = warp_reduce_sum<kNumWarps>(prod);
  if (tid == 0)
    atomicAdd(y, prod);
}


// ============================================================================
// Phase 2: Elementwise Ops（逐元素操作，演示 coalesced access + vectorize）
// ============================================================================
// 面试要点：
//   - 逐元素操作是最简单的 kernel，核心考点是 memory coalescing
//   - float4 向量化可将内存事务数减为 1/4，大幅提升 bandwidth utilization
//   - grid/block 维度设计：grid(N/threads), block(threads)，一维即可

// ---- ReLU: y = max(0, x) ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu(float *x, float *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    y[idx] = fmaxf(0.0f, x[idx]);
}

// ReLU + float4 向量化：每个线程处理 4 个元素，减少 75% 的 load/store 指令
// block(64)×4(float4)=256 元素/block，与基础版吞吐相同
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1)
// 注意：该版本默认地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu_vec4(float *x, float *y, int N) {
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  if (idx < N) {
```

```
    float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
    float4 reg_y;
    reg_y.x = fmaxf(0.0f, reg_x.x);
    reg_y.y = fmaxf(0.0f, reg_x.y);
    reg_y.z = fmaxf(0.0f, reg_x.z);
    reg_y.w = fmaxf(0.0f, reg_x.w);
    FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
  }
}

// ---- Elementwise Add: c = a + b ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add(float *a, float *b, float *c, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    c[idx] = a[idx] + b[idx];
}

// Elementwise Add + float4 向量化
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
// 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
  int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
  if ((idx + 3) < N) {
    float4 reg_a = FLOAT4(a[idx]);
    float4 reg_b = FLOAT4(b[idx]);
    float4 reg_c;
    reg_c.x = reg_a.x + reg_b.x;
    reg_c.y = reg_a.y + reg_b.y;
    reg_c.z = reg_a.z + reg_b.z;
    reg_c.w = reg_a.w + reg_b.w;
    FLOAT4(c[idx]) = reg_c;
  } else if (idx < N) {
    for (int i = 0; (idx + i) < N; i++) {
      c[idx + i] = a[idx + i] + b[idx + i];
    }
  }
}

// ---- Histogram: y[a[i]]++ ----
// 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/histogram/histogram.cu
__global__ void histogram(int *a, int *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    atomicAdd(&(y[a[idx]]), 1);
}

// ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
// 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
//
// 面试要点:
//   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
//     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
//   - 数学公式 (5 步推导):
//     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
//              L_max = max(LSE_1, LSE_2)
```

```
//     Step 2 — 还原指数权重 (un-normalized) :
//              w_1 = exp(LSE_1 - L_max),  w_2 = exp(LSE_2 - L_max)
//     Step 3 — 归一化为混合比例:
//              alpha = w_1 / (w_1 + w_2),  beta = w_2 / (w_1 + w_2)
//              满足 alpha + beta = 1
//     Step 4 — 逐元素加权合并:
//              O = alpha * O_1 + beta * O_2
//   - LSE 布局 [num_heads, num_tokens] (与 FlashAttention-2惯例一致,
//     lse[head_idx][token_idx])
//   - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
//     展平为 [T0H0, T0H1, ..., T1H0, ...])
//   - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score
//     为 -inf) 的 LSE 可能为 +inf, 替换后 exp(-inf - L_max) = 0,
//     该段权重退化为 0
//   - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
//     pack load → FMA → pack store 一条流水线
//   - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
//
// 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例) :
//   kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
//   threads_per_head = head_size / 4 = 2
//   total_threads = num_tokens * num_heads * threads_per_head = 8
//
//   global_idx:       0      1      2      3      4      5      6      7
//   token_head_idx:   0      0      1      1      2      2      3      3
//        (第几个 (token,head) 对, 行优先)
//   pack_idx:         0      1      0      1      0      1      0      1
//        (该 (token,head) 对内第几个 pack)
//   token_idx:        0      0      0      0      1      1      1      1
//        (token_head_idx / num_heads, token 变化慢)
//   head_idx:         0      0      1      1      0      0      1      1
//        (token_head_idx % num_heads, head 变化快)
//   pack_offset:      0      4      0      4      0      4      0      4
//        (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
//   head_offset:      0      0      8      8     16     16     24     24
//        (token_idx * num_heads * head_size + head_idx * head_size,
//        该 (token,head) 对在 output 展平数组中的起始偏移)
//
// Grid:  ((total_threads + 127) / 128, 1, 1)
// Block: (128, 1, 1)
// source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
__global__ void merge_attn_states(
    float *output,              // [num_tokens, num_heads, head_size]
    const float *prefix_output, // [num_tokens, num_heads, head_size]
    const float *prefix_lse,    // [num_heads, num_tokens]
    const float *suffix_output, // [num_tokens, num_heads, head_size]
    const float *suffix_lse,    // [num_heads, num_tokens]
    int num_tokens, int num_heads, int head_size) {

  constexpr int kNumThreads = 128;
  constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
  using pack_t = uint4;

  // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
  const int threads_per_head = head_size / kPackSize;
  // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
  // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
  const int total_threads = num_tokens * num_heads * threads_per_head;

  const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
  if (global_idx >= total_threads)
    return;

```

```
  // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
  // token_head_idx: 第几个 (token, head) 对，行优先展平
  const int token_head_idx = global_idx / threads_per_head;
  // pack_idx: 该 (token, head) 对内第几个 pack，0 ~ threads_per_head-1
  const int pack_idx = global_idx % threads_per_head;

  // token_head_idx 分解为 (token_idx, head_idx)
  const int token_idx = token_head_idx / num_heads; // token 变化慢（外维）
  const int head_idx = token_head_idx % num_heads;  // head 变化快（内维）

  // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移（0, 4, 8, ...）
  const int pack_offset = pack_idx * kPackSize;
  // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
  const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;

  // 定位到当前 (token, head) 的输出段起始
  const float *prefix_head = prefix_output + head_offset;
  const float *suffix_head = suffix_output + head_offset;
  float *output_head = output + head_offset;

  // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
  float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
  float s_lse = suffix_lse[head_idx * num_tokens + token_idx];

  // inf → -inf: 空 attention 段 LSE 可能为 +inf，替换后权重退化为 0
  p_lse = isinf(p_lse) ? -INFINITY : p_lse;
  s_lse = isinf(s_lse) ? -INFINITY : s_lse;

  // Step 1: max 归一化（与 safe softmax 同理，防 exp 溢出）
  const float max_lse = fmaxf(p_lse, s_lse);
  p_lse -= max_lse;
  s_lse -= max_lse;

  // Step 2-3: 指数还原 → 混合比例 alpha, beta
  const float p_se = expf(p_lse);
  const float s_se = expf(s_lse);
  const float out_se = p_se + s_se;
  const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
  const float s_scale = s_se / out_se; // beta  = w_2 / (w_1 + w_2)

  // Step 4: 逐元素加权合并 O = alpha * O_1 + beta * O_2
  // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
  if (pack_offset < head_size) {
    pack_t p_pack =
        reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
    pack_t s_pack =
        reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
    pack_t o_pack;

#pragma unroll
    for (int i = 0; i < kPackSize; ++i) {
      const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
      const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
      const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
      reinterpret_cast<float *>(&o_pack)[i] = o_v;
    }

    reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
  }
}

// ================================================================================
// Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
```

```cpp
// ================================================================================
// 面试要点:
//   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
//   online (1-pass, 增量更新)
//   - Online Softmax 是 FlashAttention-2的数学基础
//   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
//   + variance)
//   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步

// ================================================================================
// Phase 3a: Softmax — 三级递进 (面试核心考点)
// ================================================================================
// 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」
// 回答线索: naive(溢出) → safe → online

// ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
// grid(S*h/h, h), block(h), 一个 block 处理一个 token
// 问题: x 值很大时 exp(x) 溢出为 inf
template <const int kNumThreads = 256>
// Grid:  (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
// source: LeetCUDA/kernels/softmax/softmax.cu
__global__ void softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
  float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
// 面试重点: 为什么 Softmax 需要 Safe?
//   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38) , exp(89)≈4.5e38 已溢出为 inf
//   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
//   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
//   - 代价: 2 次 block reduce (先 max, 再 sum) , 但仍 O(N/B) 高效
// Grid:  (S, 1, 1)
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
template <const int kNumThreads = 256>
__global__ void safe_softmax_per_token(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  // Pass 1: block reduce max — 找最大值
  float val = (idx < N) ? x[idx] : -FLT_MAX;
  float max_val = block_reduce_max<kNumThreads>(val);

  // Pass 2: exp(x - max) → block reduce sum
  float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
  float exp_sum = block_reduce_sum<kNumThreads>(exp_val);

  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 3: Online Safe Softmax (FlashAttention-2的数学基础) ----
// 面试重点:
// 两种使用场景 (公式不同, 但结果等价) :
//   1) 单元素增量更新 (online update, 处理新元素 x_i):
//      m_new = max(m_old, x_i)
```

```
//      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
//   2) 二元合并 (binary merge，合并两个部分累加器，warp/block reduce 使用)：
//      m = max(m1, m2) safe softmax用的max值
//      d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
//      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
// 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
// Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
//
// 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new))，
// warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**：
//   (m1,d1): max 和 Σexp(x_j - m1)，覆盖集合 S1
//   (m2,d2): max 和 Σexp(x_k - m2)，覆盖集合 S2
//
// 合并公式（对称，不依赖"新/旧"概念）：
//   m = max(m1, m2)
//   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
//
// 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
struct __align__(8) MD {
  float m; // running max
  float d; // running denominator (sum of exp(x - max))
};

template <const int kWarpWidth = kWarpSize>
__device__ __forceinline__ MD warp_reduce_md(MD md1) {
#pragma unroll
  for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
    MD md2;
    md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
    md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);

    MD b_m = (md1.m > md2.m) ? md1 : md2; // max
    MD s_m = (md1.m > md2.m) ? md2 : md1;

    // b_m.d 无需 rescale（其基准 max 已是全局 max），
    // s_m.d 需 rescale: exp(s_m.m - b_m.m)
    md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
    md1.m = b_m.m;
  }
  return md1;
}

// 注意：这里默认一个 block 处理一个 token；边界线程的 d=0 只参与归约，不会写回 y
// Grid:  (S, 1, 1)
// Block: (H, 1, 1)，由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
template <const int kNumThreads = 256>
__global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * kNumThreads + threadIdx.x;
  const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
  __shared__ MD shared[kNumWarps];

  float val = (idx < N) ? x[idx] : -FLT_MAX;
  int warp = tid / kWarpSize;
  int lane = tid % kWarpSize;

  // 初始化：每个线程持有一个 (max, denom) 对
  MD md;
  md.m = idx < N ? val : -FLT_MAX;
  md.d = idx < N ? 1.0f : 0.0f;

  // 第一级规约：warp_reduce_md 在归约中自动更新 m 和 d
```

```
  md = warp_reduce_md<kWarpSize>(md);

  if (lane == 0)
    shared[warp] = md;
  __syncthreads();

  // 第二级归约：每个 warp 结果再做一次 warp_reduce_md（复用 block_reduce 模式）
  md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
  md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果

  // 用全局 max 和 denom 做最终 softmax
  float d_inv = __fdividef(1.0f, md.d);
  // 边界线程即使看到 d=0 的填充值，也不会走到写回路径
  if (idx < N) {
    y[idx] = __expf(val - md.m) * d_inv;
  }
}

// ============================================================================
// Phase 3b: RMS Normalization (1-pass reduce)
// ============================================================================
// 面试要点:
//   - RMS Norm: y = (x / rms(x)) * g, 1/rms(x) = rsqrt(mean(x²))
//   - 只需 1 次 block reduce (sum of squares)，比 Layer Norm 少 1 次同步
//   - Llama 系列使用 RMS Norm
//   - grid(N, K/K), block(K): 一行一个 block
// Grid:  (N, 1, 1), N=batch*seq_len, 每行一个 block
// Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int kNumThreads = 128>
__global__ void rms_norm(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;
  float variance = value * value;
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
  __syncthreads();
  if (idx < N * K)
    y[idx] = (value * s_variance) * g;
}

// RMS Norm + float4
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意：该版本默认 K 按 4 对齐，且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
template <const int kNumThreads = 128 / 4>
__global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
  float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
                                    reg_x.z * reg_x.z + reg_x.w * reg_x.w)
                                 : 0.0f;
  variance = block_reduce_sum<kNumThreads>(variance);
```

```
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();
  float4 reg_y;
  reg_y.x = reg_x.x * s_variance * g;
  reg_y.y = reg_x.y * s_variance * g;
  reg_y.z = reg_x.z * s_variance * g;
  reg_y.w = reg_x.w * s_variance * g;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// ==========================================================================
// Phase 3c: Layer Normalization (2-pass reduce)
// ==========================================================================
// 面试要点:
//   - Layer Norm: y = ((x - mean) / std) * g + b, std = sqrt(variance), variance = mean((x - mean)²)
//   - 需要 2 次 block reduce: 先 mean (sum/K) , 再 variance (sum((x-mean)²)/K)
//   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
// Grid:  (N, 1, 1), 一行一个 block
// Block: (128, 1, 1), kNumThreads=K=128
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
template <const int kNumThreads = 128>
__global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;

  // Pass 1: compute mean
  float sum = block_reduce_sum<kNumThreads>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads(); // 必须等待 s_mean 对所有线程可见

  // Pass 2: compute variance = (x - mean)²
  float variance = (value - s_mean) * (value - s_mean);
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
  __syncthreads(); // 必须等待 s_variance 对所有线程可见

  if (idx < N * K)
    y[idx] = ((value - s_mean) * s_variance) * g + b;
}

// Layer Norm + float4
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
template <const int kNumThreads = 128 / 4>
__global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
```

```cpp
  float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;

  float sum = block_reduce_sum<kNumThreads>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads();

  float4 reg_x_hat;
  reg_x_hat.x = reg_x.x - s_mean;
  reg_x_hat.y = reg_x.y - s_mean;
  reg_x_hat.z = reg_x.z - s_mean;
  reg_x_hat.w = reg_x.w - s_mean;
  float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
                   reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
  variance = block_reduce_sum<kNumThreads>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();

  float4 reg_y;
  reg_y.x = reg_x_hat.x * s_variance * g + b;
  reg_y.y = reg_x_hat.y * s_variance * g + b;
  reg_y.z = reg_x_hat.z * s_variance * g + b;
  reg_y.w = reg_x_hat.w * s_variance * g + b;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// ============================================================================
// Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
// ============================================================================
// 面试要点:
//   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
//     [x1']   [cos(θ)  -sin(θ)] [x1]
//     [x2'] = [sin(θ)   cos(θ)] [x2]
//   - θ_i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
//   - token_pos = idx / N: token 在序列中的位置
//   - token_idx = idx % N: token 内的维度对索引
//   - 输入 [seq_len, hidden_size], 输出同形状

// Grid:  ((seq_len * N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/rope/rope.cu
__global__ void rope(float *x, float *out, int seq_len, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  float x1 = x[idx * 2];
  float x2 = x[idx * 2 + 1];
  int token_pos = idx / N; // 序列位置
  int token_idx = idx % N; // 维度对索引

  // 频率计算: θ_i = 1 / (10000^(2i/d))
  float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
  float sin_v = sinf(token_pos * exp_v);
  float cos_v = cosf(token_pos * exp_v);

  // 2D 旋转
  float out1 = x1 * cos_v - x2 * sin_v;
  float out2 = x1 * sin_v + x2 * cos_v;
  out[idx * 2] = out1;
  out[idx * 2 + 1] = out2;
}

// ============================================================================
```

```cpp
// Phase 5: Mat Transpose — 矩阵转置（Bank Conflict 专题）
// ============================================================================
// 面试要点（Bank Conflict 专题）：
//   - Shared memory 有 32 个 bank，每个 bank 4 bytes（32-bit）
//   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
//   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
//   - 解决方案：PAD（在每行末尾加 1 个元素，打破地址对齐）
//
// 转置的四步演进：
//   naive: 非合并写入（列优先写）→ 每个 warp 产生 32 次内存事务
//   shared: 写入 smem（行优先）→ 从 smem 读取（列优先）→ 合并写入 gmem
//   BCF:   smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17]，PAD=1 加在第二维消除 bank conflict
//   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
// 注：本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write)，Level 2/3 省略

// 每个线程处理 1 个元素，block(16,16)
// 读：x 按行优先访问，warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
// 写：y 按列优先写入，32 线程跨 row 行分散 → 非合并写入 ✗（32 次内存事务）
// Grid:  ((col + 15) / 16, (row + 15) / 16, 1)，每线程 1 元素
// Block: (16, 16, 1)
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose(float *x, float *y, const int row, const int col) {
  const int c = blockIdx.x * blockDim.x + threadIdx.x;  // col in x
  const int r = blockIdx.y * blockDim.y + threadIdx.y;  // row in x
  if (r < row && c < col) {
    // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
    y[c * row + r] = x[r * col + c];
  }
}

// Grid:  ((col + 15) / 16, (row + 63) / 64, 1)，每线程 4 元素(float4)
// Block: (16, 16, 1)
// 注意：该版本默认按 float4 打包写回；最适合 row 能按 4 对齐的场景
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose_padded(
    float *x, float *y, const int row, const int col) {
  const int tx = threadIdx.x;
  const int ty = threadIdx.y;

  constexpr int TILE = 16;
  constexpr int PAD = 1;
  // Bank conflict fix: 每行多 1 个元素，打破 32-bank 对齐
  __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16

  // x 空间坐标；每线程覆盖 4 行 (actual row = x_r * 4)
  const int x_c = blockIdx.x * TILE + tx;
  const int x_r = blockIdx.y * TILE + ty;

  if (x_r * 4 < row && x_c < col) {
    // Step 1: 4 rows × 1 col → smem (row-major);
#pragma unroll
    for (int i = 0; i < 4; ++i) {
      tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
    }
    __syncthreads();

    // Step 2: Transposed read from smem
    float4 reg_trans;
    reg_trans.x = tile[tx * 4 + 0][ty];
    reg_trans.y = tile[tx * 4 + 1][ty];
    reg_trans.z = tile[tx * 4 + 2][ty];
    reg_trans.w = tile[tx * 4 + 3][ty];

```

```cpp
    // y 空间坐标: y_r = x 的列, y_c = x 的行
    const int y_r = blockIdx.x * TILE + ty;          // = x col
    const int y_c = (blockIdx.y * TILE + tx) * 4;    // = x row

    // Coalesced write: y[y_r][y_c .. y_c+3]
    FLOAT4(y[y_r * row + y_c]) = reg_trans;
  }
}

// ============================================================================
// Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
// ============================================================================
// 面试要点:
//   - GEMV 是典型的 memory-bound 算子: AI ≈ O(1), 瓶颈在内存带宽
//   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
//   - 不同 K 值对应不同分块策略:
//     K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
//     K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
//     K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
//
// a: M×K, x: K×1, y: M×1, 计算: y = a * x; N = 1

// ---- SGEMV K32: 基础 warp-per-row ----
// 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
// grid(M/4), 每个 warp 负责一行
// K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
// Grid:  ((M + 3) / 4, 1, 1), 每 block 处理 4 行
// Block: (32, 4, 1), 每行 1 warp
// 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int lane = tx % kWarpSize; // 0~31
  int m = blockIdx.x * blockDim.y + ty; // 全局行号
  if (m < M) {
    float sum = 0.0f;
    // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
    // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
    const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
      int k = w * kWarpSize + lane;
      sum += a[m * K + k] * x[k];
    }
    sum = warp_reduce_sum<kWarpSize>(sum);
    // 每个 warp 处理一行, lane 0 写回结果
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K128: float4 向量化 ----
// 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
// Grid:  ((M + 3) / 4, 1, 1)
// Block: (32, 4, 1)
// 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int lane = tx % kWarpSize; // 0~31
```

```cpp
  int m = blockDim.y * blockIdx.x + ty;

  if (m < M) {
    float sum = 0.0f;
    // 沿 K 维的迭代数 = ceil(K/128)，每个 warp 每轮用 float4 覆盖 128 个 K 元素
    const int NUM_ITERS = (((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
#pragma unroll
    for (int w = 0; w < NUM_ITERS; ++w) {
      int k = (w * kWarpSize + lane) * 4;
      float4 reg_x = FLOAT4(x[k]);
      float4 reg_a = FLOAT4(a[m * K + k]);
      sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
              reg_a.w * reg_x.w);
    }
    sum = warp_reduce_sum<kWarpSize>(sum);
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
// 面试亮点：K=16 < 32，一个 warp 可以处理多行
// kRowPerWarp=2，kNumLanePerRow=16，前 16 个 lane 处理 row0，后 16 个 lane 处理 row1
// Grid:  ((M + 7) / 8, 1, 1)，NUM_ROWS=8
// Block: (32, 4, 1)
// 注意：这一版是面向 K=16 的专用写法；kRowPerWarp=2 时一个 warp 同时处理 2 行
// source: LeetCUDA/kernels/sgemv/sgemv.cu
template <const int kRowPerWarp = 2>
__global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
  constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int lane = tx % kWarpSize;
  int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
  int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
  if (m < M) {
    float sum = A[m * K + k] * x[k];
    // 按照kNumLanePerRow=16，分2组各自做 warp reduce sum，k==0的lane写回结果
    sum = warp_reduce_sum<kNumLanePerRow>(sum);
    // 注意：判断条件是 k == 0，不是 lane == 0!
    if (k == 0)
      y[m] = sum;
  }
}

// ========================================================================
// Phase 7: GEMM — 矩阵矩阵乘（GPU 最重要的算子，面试核心考点）
// ========================================================================
// 面试要点（GEMM 优化五层金字塔）：
//   Level 1 — Tiling（分块 + shared memory）：将数据从 HBM 搬到 SMEM 复用
//   Level 2 — Thread Tile（寄存器分块）：每个线程计算 TM×TN
//   个元素，提高计算密度 Level 3 — Vectorize（向量化访存）：float4/half2，减少
//   load/store 指令数 Level 4 — Tensor Core（MMA
//   m16n8k16）：硬件矩阵乘单元，warp 级指令 Level 5 — Warp Specialization +
//   TMA（WGMMA m64n128k16）：Hopper 异步执行
//
// 计算密度递进：
//   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
//   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
//   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐

// ========================================================================
// Phase 7a: SGEMM（非 Tensor Core 路径）
```

```cpp
// ================================================================================

// ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
// 最基础的 tiling 实现，演示 shared memory 的核心用法
// C = A x B, C[M, N] = A[M, K] x B[K, N]
// BM=BN=32, BK=32, block(32, 32)，一个线程计算 c 的一个元素
// Grid:  ((N + 31) / 32, (M + 31) / 32, 1)
// Block: (32, 32, 1), 1024 线程
// source: LeetCUDA/kernels/sgemm/sgemm.cu
__global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 32; // vec 版: 32x4 = 128
  constexpr int BN = 32; // vec 版: 32x4 = 128
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK], s_b[BK][BN]; //  32x32x4=4KB smem, float = 4 bytes

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int tid = threadIdx.y * blockDim.x + tx;

  // 线程到 smem 的映射: 32×32 线程，每个线程加载 a 和 b 各 1 个元素
  // 技巧: 一般来说 “/” 表示线程不是连续排布的，"%" 表示线程是连续排布的
  // 因此，在需要考虑连续访问的维度使用 “%”，比如，连续的线程访问列方向连续的元素
  // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
  // 线程不连续访问 M 维度 → 用 /;
  int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
  int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
  int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
  int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
  int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
  int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;

  float sum = 0.f; // 遍历完整的K, slice K;
  // 这里不用pragma unroll，因为K不是编译器常量，编译器无法展开循环
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
    int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
    __syncthreads(); // 确保整个 smem tile 加载完毕

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
      int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
      sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
    }
    __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
  }
  int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
  int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
  int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
  c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
}

// ---- Level 1+: SGEMM Vec4 — Block Tile 128×128 + K Tile 32 + Thread Tile 4×4 ----
// 在 Level 1 基础上引入两层优化:
//   1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
//   2) Thread Tile 4×4: 每线程计算 16 个 C 元素，提升计算/访存比 (AI 从
//   BK/2≈16 提升到 TM*TN*BK/2≈256)，减少线程总数带来的同步开销
// C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
```

```cpp
// BM=BN=128, BK=32, block(32, 32)=1024 线程，每线程负责 4×4=16 个 C 元素
//   1024 × 16 = 16384 = 128 × 128 ✓
//
// 线程到 4×4 tile 的映射（与加载映射解耦，独立计算更清晰）：
//   m_tile = tid / 32 (0~31)，每 tile 4 行 → 行 [m_tile*4, m_tile*4+3]，覆盖 0~127
//   n_tile = tid % 32 (0~31)，每 tile 4 列 → 列 [n_tile*4, n_tile*4+3]，覆盖 0~127
//
// 加载映射（每线程 4 个元素，float4）：
//   A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
//   B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
//   row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
//
// ⚠ Bank Conflict 提示（面试加分点）：
//   s_b[32][128] 上 warp 内 32 线程按 stride=4 访问（tid%32 决定列 0,4,8,...,124）
//   → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
//   s_b[BK][BN+1] PAD 打散，这里保持最简布局便于讲解。
//
// Grid:  ((N + 127) / 128, (M + 127) / 128, 1)
// Block: (32, 32, 1), 1024 线程
// 假设: M/N 为 128 的倍数，K 为 32 的倍数（与 Level 1 naive 版一致的边界约定）
// source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
__global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 128;
  constexpr int BN = 128;
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
  __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int tid = threadIdx.y * blockDim.x + tx; // 0~1023

  // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
  int load_smem_a_m = tid / 8;        // 0~127, 8 线程/行
  int load_smem_a_k = (tid % 8) * 4;  // 0,4,...,28
  // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
  int load_smem_b_k = tid / 32;       // 0~31, 32 线程/行
  int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124

  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n = bx * BN + load_smem_b_n;

  // 4×4 Thread Tile 基址（独立于加载映射），这里compute索引的计算逻辑要和
  // load索引的计算逻辑分开，load/compute是可以独立索引的，理解这点很重要。
  // 目标C Tile为[BM,BN]=[128x128], 有32x32线程，则每个线程处理4x4 tile
  // 那么，就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
  int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
  int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124

  float sum[4][4] = {0.f};
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
    int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
    __syncthreads();

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      // 每次迭代加载 4 个 A 元素 + 4 个 B 元素，再做 4×4=16 次 FMA
```

```
      float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
                         s_a[comp_smem_a_m_base + 1][k],
                         s_a[comp_smem_a_m_base + 2][k],
                         s_a[comp_smem_a_m_base + 3][k]};
      float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
                         s_b[k][comp_smem_b_n_base + 1],
                         s_b[k][comp_smem_b_n_base + 2],
                         s_b[k][comp_smem_b_n_base + 3]};
#pragma unroll
      for (int i = 0; i < 4; ++i) {
#pragma unroll
        for (int j = 0; j < 4; ++j) {
          sum[i][j] += a_vals[i] * b_vals[j];
        }
      }
    }
    __syncthreads();
  }

  // 存储 4×4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
  int store_gmem_c_m = by * BM + comp_smem_a_m_base;
  int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
#pragma unroll
  for (int i = 0; i < 4; ++i) {
    int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
    float4 reg_c;
    reg_c.x = sum[i][0];
    reg_c.y = sum[i][1];
    reg_c.z = sum[i][2];
    reg_c.w = sum[i][3];
    FLOAT4(c[store_gmem_c_addr]) = reg_c;
  }
}

// ============================================================================
// Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
// ============================================================================
// 面试要点:
//   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
//   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
//   - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit
//   寄存器)
//   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
//   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
//
// ★ TN 布局详解 (面试高频考点):
//   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
//     BLAS 源自 Fortran, 默认列优先 (column-major) 存储
//     N = Normal (列优先, BLAS 原生格式)
//     T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
//     第一个字母 → A 的 op, 第二个字母 → B 的 op
//     所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
//     BLAS=Normal)  即: C = op(A) × op(B) = A^T × B? 不对!  在 row-major
//     视角下: TN = A row-major [M×K], B^T row-major [N×K] (等价于 B col-major [K×N])  在 cuBLAS
//     调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
//       T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
//       N on B: B 已经是 BLAS 原生的 col-major, 无需转置
//
//   记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
//     T = row-major (行优先, 对 BLAS 来说是 transposed)
//     N = col-major (列优先, BLAS native = normal)
//
//   LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储) : C[M×N] = A[M×K] × B[K×N]
```

```
//     - A: row-major [M, K], B: row-major [K, N]
//     - 按 N=col-major/T=row-major 约定，二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
//     - 问题: ldmatrix 默认加载 col-major，B 是 row-major 需要 .trans
//
//   TN 布局: C[M×N] = A[M×K] × B[K×N]，B 以 B^T=[N×K] row-major 存储（A 行优先，B 列优先）
//     - A [M×K]: row-major → 全局索引 A[m*K + k]，smem s_a[BM][BK]
//     - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n)（⚠ 内维连续的是 K）
//     - 优势: B^T 已是 row-major，ldmatrix 无需 .trans，天然匹配 MMA row.col
//   MMA 指令: mma.sync.aligned.m16n8k16.row.col
//     - row.col: A 输入 row-major，B 输入 col-major → 与 TN 布局天然匹配
//
// WGMMA (Warp Group MMA, Hopper+):
//   - warpgroup 级指令（128 threads = 4 warps），异步执行
//   - m64n128k16: 一次处理 64×128×16 的矩阵乘（总 tile 量 131072，是 MMA m16n8k16 的 64 倍）
//   - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
//   threads) 做计算
//   - TMA: 硬件 DMA，~零寄存器开销，支持 1D~5D 寻址

// ============================================================================
// Phase 7b-1: MMA PTX 宏定义
// ============================================================================

// ---- gmem → smem: cp.async ----
// cp.async.commit_group / wait_group / wait_all 语义（PTX ISA §9.7.9.25.3）：
//   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group（per-thread）。
//   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成（即 pending ≤ N）。
//     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
//   - wait_all: 等价于 commit_group + wait_group 0。
#define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
#define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
#define CP_ASYNC_WAIT_GROUP(n)                                                 \
  asm volatile("cp.async.wait_group %0;\n" ::"n"(n))

// 注意: cg 只支持 16 bytes，ca 支持 4/8/16 bytes
#define CP_ASYNC_CG(dst, src, bytes)                                           \
  asm volatile(                                                                \
      "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" ::"r"(dst),       \
      "l"(src), "n"(bytes))

// ---- ldmatrix: smem → register（Tensor Core 专用）----
// ldmatrix.sync.aligned.xN.m8n8.shared.b16
// 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
// aligned: 要求 128-bit 对齐, trans:  转置加载
#define LDMATRIX_X4(R0, R1, R2, R3, addr)                                      \
  asm volatile(                                                                \
      "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n"     \
      : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)                                 \
      : "r"(addr))

#define LDMATRIX_X2(R0, R1, addr)                                              \
  asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n"    \
               : "=r"(R0), "=r"(R1)                                            \
               : "r"(addr))

// ldmatrix.x2.trans: 转置加载（用于 NN 布局中需要 col-major B 矩阵的场景）
// FA 中 V[Bc,d] 为 row-major，但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
#define LDMATRIX_X2_T(R0, R1, addr)                                            \
  asm volatile(                                                                \
      "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n"       \
      : "=r"(R0), "=r"(R1)                                                     \
      : "r"(addr))

// ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
```

```cpp
// m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
// row.col: A 是 row-major, B 是 col-major
// f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
// 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
// C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
#define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)            \
  asm volatile(                                                                \
      "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, "  \
      "%4, %5}, {%6, %7}, {%8, %9};\n"                                         \
      : "=r"(RD0), "=r"(RD1)                                                   \
      : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0),  \
        "r"(RC1))

// mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
// f32 accumulator variant: 4 output registers (RD0-RD3), A/B still f16.
// C matrix 16x8=128 elements, 32 threads × 4 f32 = 4 uint32 per thread.
#define HMMA16816F32(RD0, RD1, RD2, RD3, RA0, RA1, RA2, RA3, RB0, RB1, RC0,    \
                     RC1, RC2, RC3)                                            \
  asm volatile(                                                                \
      "mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32 {%0, %1, %2, "        \
      "%3}, {%4, %5, %6, %7}, {%8, %9}, {%10, %11, %12, %13};\n"               \
      : "=r"(RD0), "=r"(RD1), "=r"(RD2), "=r"(RD3)                             \
      : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0),  \
        "r"(RC1), "r"(RC2), "r"(RC3))

// ---- MMA 辅助函数 ----
// div_ceil: 整数除法向上取整
#define HOST_DEVICE_INLINE __device__ __host__ inline
HOST_DEVICE_INLINE int div_ceil(int a, int b) {
  return (a % b != 0) ? (a / b + 1) : (a / b);
}

// ============================================================================
// Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
// ============================================================================
// 面试重点 — Tile Hierarchy:
//   MMA Atom:         m16n8k16 (1 条 MMA 指令处理的最小 tile)
//   MMA Tile (more warps):  2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
//   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
//   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
//         → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
//
// ★ TN 布局 (T=A 行优先, N=B 列优先):
//   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
//   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
//   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
//   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
//   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
//
// ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
//   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
//   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
//   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
//   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
//
// ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
//   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
//    (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
//   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
//
//   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
//   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
//
```

```
//       N / bx →  0          1          2          3        ...
//     M  ↓      ┌──────────┬──────────┬──────────┬──────────┐
//     by = 0    │  SM0     │  SM1     │  SM2     │  SM3     │  ...
//               │ A 0,B0   │ A 0,B1   │ A 0,B2   │ A 0,B3   │
//               └──────────┴──────────┴──────────┴──────────┘
//
//   同一 by 的 CTA 复用 A[by]，但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
//   推进到下一条 by 行、再次遇到相同 bx 时，才有机会复用 B；当 tiles_n 很大时，这段时间内
//   L2 更容易被其他 B tile 挤占。
//
//   Thread Block Swizzle: 先把 N 切成 S 个连续窗口；一个 z slice 只含 grid_x 个 bx。
//   下图用 grid_x=2 展示一个窄窗口，scheduler 更早进入下一条 by 行:
//
//       N / local x →  0          1
//     M  ↓          ┌──────────┬──────────┐
//     by = 0        │  SM0     │  SM1     │  → A 0; B^T 0, B^T 1
//                   ├──────────┼──────────┤
//     by = 1        │  SM2     │  SM3     │  → A 1; B^T 0, B^T 1
//                   └──────────┴──────────┘
//
//   对于这个 2x2 窗口：同一行横向复用 A（SM0/SM1、SM2/SM3），同一列纵向复用 B
//   （SM0/SM2、SM1/SM3）。真实 grid_x 不必等于 2，但缩小它会缩短再次访问相同 bx 的距离。
//   此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会，不保证 CTA 的 z 顺序、
//   缓存驻留或 L2 hit。
//
//   Launch（kBlockSwizzle=false，当前默认路径）：
//     grid = (ceil(N / BN), ceil(M / BM), 1)，blockIdx.x 直接就是 N-tile 编号 bx。
//   Launch（kBlockSwizzle=true，需由 launch 端显式构造 3D grid）：
//     tiles_n = ceil(N / BN), S = ceil(N / 2048)
//     grid = (ceil(tiles_n / S), ceil(M / BM), S)
//     bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
//   例如 N=8192、BN=128: tiles_n=64，S=4，grid.x=16；z=0/1/2/3 分别覆盖
//   bx=0..15/16..31/32..47/48..63，即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
//   grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
//   对齐；ceil 只保证逻辑 tile 编号不遗漏，并不使部分尾 tile 变为安全。
// Block: (256, 1, 1), 8 warps
// source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
// ============================================================================
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim, also BK tile = kMmaK
          const int kMmaTileM = 2,          // warps along M, 2 → warp tile M = 32
          const int kMmaTileN = 4,          // warps along N, 4 → warp tile N = 32
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 4,          // value-repeat along N, BN = 8*4*4 = 128
          const int kStages = 3,            // cp.async pipeline depth
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(256)
    hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
  static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
  // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
  // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
  constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
  constexpr int BK = kMmaK;                         // 16

  // Dynamic shared memory: kStages 个 stage 的 A 和 B
  // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
  // row-major，即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
  // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD；尤其 B 在 TN 布局下常见会额外
```

```cpp
  // 加 B_PAD 来打散 bank 映射，避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + kStages * BM * BK;     // A 和 B 连续存放
  constexpr int s_a_stage_offset = BM * BK; // 128*16
  constexpr int s_b_stage_offset = BN * BK; // 128*16  ⚠ BN(128)×BK(16) = B^T row-major
  const int tid = threadIdx.y * blockDim.x + threadIdx.x;
  const int warp_id = tid / kWarpSize; // 0~7
  const int lane_id = tid % kWarpSize; // 0~31
  // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
  // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
  //                          N direction (warp_n)
  //                          0       1       2       3
  // M direction (warp_m)  0  MMA0    MMA2    MMA4    MMA6
  //                       1  MMA1    MMA3    MMA5    MMA7
  // MMA的排布方式不是唯一的，现在这样排是因为结合MMA/VAL Tile之后，刚好能覆盖整个
  // C Tile[128,128]。只要调整MMA/VAL Tile，这里的排布方式也可以跟着调整。要注意的
  // 是，MMA0-7逻辑上是可以认为是并行执行的，各自的计算结果累计加到对应的C Tile位置上。
  const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
  const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)

  // 线程到 global memory 的映射（用于加载 A 和 B）共 256 个线程
  // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major（内维连续的是 K）
  int load_smem_a_m = tid / 2;                 // 0~127
  int load_smem_a_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8
  int load_smem_b_n = tid / 2;                 // 0~127 → B^T 的 N 方向（row-major 的行）
  int load_smem_b_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8  → B^T 的 K 方向（row-major 的列）
  int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
  int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
  if (load_gmem_a_m >= M || load_gmem_b_n >= N)
    return;

  // 8个Warps(MMAs)的排布是M方向2个，N方向4个，则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
  // CUDA中每个block能放的线程数是有上限的（warp数量有限），一般为4或者8个warps。为了能处理更大的C tile，
  // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是Value Tile的作用。
  // MMA Tile对应到cutlass cute中的TiledMMA的概念，Value Tile对应到cutlass cute中的PermuteMNK的概念。
  uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0

  // CVTA: 一次转换 smem 基地址，避免每次 cp.async 都做转换
  uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
  uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);

  // 预加载前 (kStages-1) 个 stage
  // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
#pragma unroll
  for (int k = 0; k < (kStages - 1); ++k) {
    int load_gmem_a_k = k * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
    uint32_t load_smem_a_ptr = (smem_a_base_ptr +
      (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    int load_gmem_b_k = k * BK + load_smem_b_k;
    // B^T: [n][k] row-major (即 B[k][n] col-major) ⚠
    int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
    uint32_t load_smem_b_ptr = (smem_b_base_ptr +
      (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
    );
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);

    CP_ASYNC_COMMIT_GROUP();
  }
```

```
1512
1513    CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1514    __syncthreads();
1515
1516    // 统一循环: k 从 0 开始，每次迭代负责 tile k（加载 + 计算合并为单循环）
1517    const int NUM_K_TILES = div_ceil(K, BK);
1518    // 此处不用pragma unroll，因为 K 不是编译期常量，因此NUM_K_TILES 也不是编译期常量
1519    for (int k = 0; k < NUM_K_TILES; ++k) {
1520      int smem_sel = k % kStages;                       // 计算 tile k 的 stage
1521      int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1522
1523      // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
1524      // load了(kStages-1) 个 stage，因此这里是从 tile (k+kStages-1) 开始预加载
1525      // TN 布局: A 的 gmem 地址用 m*K+k（row-major），B^T 的 gmem 地址用 n*K+k
1526      // (row-major，内维连续的是 K)
1527      if (k + kStages - 1 < NUM_K_TILES) {
1528        int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1529        int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1530        int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1531        int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1532
1533        uint32_t load_smem_a_ptr =
1534            (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1535                                load_smem_a_m * BK + load_smem_a_k) *
1536                                   sizeof(half));
1537        CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1538
1539        uint32_t load_smem_b_ptr =
1540            (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1541                                load_smem_b_n * BK + load_smem_b_k) *
1542                                   sizeof(half));
1543        CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1544        CP_ASYNC_COMMIT_GROUP();
1545      }
1546
1547      // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1548      // TN 布局关键: A 用 x4（非转置），因为 A 是 row-major; B 用 x2（非转置），smem 中
1549      // B^T 为 row-major，逐行加载即得 B 的列，天然匹配 col-major B
1550      uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1551      uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1552
1553      // ldmatrix.x4: 加载 A 的 m16k16 片段（row-major A，非转置）
1554  #pragma unroll
1555      for (int i = 0; i < kValTileM; ++i) {
1556        // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1557        // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1558        int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1559        // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1560        // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与，每个线程提供一个有效的，不重叠的addr
1561        int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1562        int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1563        uint32_t lane_smem_a_ptr =
1564            (smem_a_base_ptr +
1565             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1566                 sizeof(half));
1567        LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1568      }
1569
1570      // ldmatrix.x2: 加载 B 的 k16n8 片段（非转置）
1571      // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1572      // ldmatrix 逐行加载 B^T 的行 = B 的列，天然给出 col-major B fragment → 直接匹配 MMA row.col
1573  #pragma unroll
1574      for (int j = 0; j < kValTileN; ++j) {
```

```
      // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
      // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
      // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
      int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
      // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
      // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与，后16个线程传的addr会被忽略
      int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
      int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
      uint32_t lane_smem_b_ptr =
          (smem_b_base_ptr +
           (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
               sizeof(half));
      LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
    }

    // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次，在N方向重复kValTileN(4)次
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
                  RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
                  RB[j][0], RB[j][1], // B fragment
                  RC[i][j][0], RC[i][j][1]);
      }
    }

    // 自适应等待：流水线满载期用 kStages-2，尾部排空用 0
    if (k + kStages - 1 < NUM_K_TILES) {
      CP_ASYNC_WAIT_GROUP(kStages - 2);
    } else { // 对于尾部的 k，等待所有剩余的 cp.async 完成
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();
  }

  // Epilogue: 寄存器 → global memory（通过 warp shuffle + 128-bit store）
  {
    for (int i = 0; i < kValTileM; ++i) {
      // RC[kValTileM][kValTileN][2] 中  RC[...][...][2] 中保存了2个 uint32
      // 寄存器，每个uint32 寄存器表示两个临近的fp16值：{c0,c1}，然后RC[...][...][0]
      // 和RC[...][...][1]代表的是按照col-major排布的2个8x8子矩阵上（不同物理行，跨8x8）
      // 同一个位置上的元素，也就是，实际代表了2个不同的行的元素，因此要分开RC0, RC1;
      // RC0表示第一行，8个half，可以用4个uint32寄存器来装；同理，RC1表示第二行。
      uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
      uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
      // ========================================================================
      // MMA m16n8k16 C fragment — a single 16×8 matrix (registers per warp):
      // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
      // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
      //
      //       cols: c0-1    c2-3    c4-5    c6-7
      //   r0:  T0{c0,c1}   T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ┐
      //   r1:  T4{c0,c1}   T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  │
      //   r2:  T8{c0,c1}   T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
      //   ...                                               |
      //   r7:  T28{c0,c1}  T29{c2,c3} T30{c4,c5} T31{c6,c7} ┘
      //   r8:  T0{c0,c1}   T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ┐
      //   r9:  T4{c0,c1}   T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  │
      //  r10:  T8{c0,c1}   T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
      //   ...                                               |
      //  r15:  T28{c0,c1}  T29{c2,c3} T30{c4,c5} T31{c6,c7} ┘
      //
```

```
      // Within a 4-lane group (e.g. T0-T3 for row 0):
      //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
      //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
      // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0，4 次 shuffle 凑齐
      // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}，再由 lane%4==0 128-bit store。
      // ================================================================
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        RC0[j][0] = RC[i][j][0];
        RC1[j][0] = RC[i][j][1];
        RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
        RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
      }
      // 每 4 个 lane 中只有 lane 0 做 128-bit store
      // lane_id / 4 → 行号映射（每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行）：
      //   ┌──────────┬────────────┬──────────────┬──────────────┐
      //   | lane_id  | lane_id/4  | RC[0] 的行   | RC[1] 的行   |
      //   ├──────────┼────────────┼──────────────┼──────────────┤
      //   | 0~3      | 0          | row 0        | row 8        |
      //   | 4~7      | 1          | row 1        | row 9        |
      //   | 8~11     | 2          | row 2        | row 10       |
      //   | 12~15    | 3          | row 3        | row 11       |
      //   | 16~19    | 4          | row 4        | row 12       |
      //   | 20~23    | 5          | row 5        | row 13       |
      //   | 24~27    | 6          | row 6        | row 14       |
      //   | 28~31    | 7          | row 7        | row 15       |
      //   └──────────┴────────────┴──────────────┴──────────────┘
      if (lane_id % 4 == 0) {
        // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
        int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
        // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}，对应 RC0/RC1 (+8) 的行号，表示2个8x8矩阵的行号
        int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
#pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
          int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
          int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
          int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
          int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
          // 128-bit store: 一次写入 8 个 half
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RC0[j][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RC1[j][0]);
        }
      }
    }
  }
}

// ================================================================================
// Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
// + XOR swizzle 消除 smem bank conflict（统一循环版）
// ================================================================================
// 面试要点（Swizzle 原理 — smem Bank Conflict 消除）：
//   - 问题：ldmatrix 以 8×8 片段（m8n8）从 smem 加载 16×16 的矩阵。同一 warp 中
//     不同 lane 访问的地址在 bank 上产生冲突 → 串行化（most common: 2/4-way）。
//   - XOR swizzle 方案：对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换，
//     将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替：
```

```cpp
//        rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
//        rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
//        rows 8~11: repeat rows 0~3 pattern
//        rows 12~15: repeat rows 4~7 pattern
//   - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
//   - 优势: 无需 smem PAD (不浪费空间) , 原理上完全消除特定 pattern 的 bank conflict。
//   - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16) , kStep ∈ {4,8}。
//     对 MMA m16n8k16 的 BK=16 而言刚好满足。
//
// 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
// 参考: https://zhuanlan.zhihu.com/p/4746910252

// ---- Swizzle 辅助函数 ----

// i: row index; j: col index.
// Returns the chunk-level (8 fp16 = 16 B per chunk) column swizzle, i.e.
// 0 / 8 / 16 / ... that the caller adds to the chunk-aligned base column.
// The within-chunk offset (``j & 7``) is preserved by the caller.
//
// kColStride matches the equivalent CuTe ``Swizzle<B, M, S>`` swizzle
// hardware mode used by TMA bulk-tensor descriptors:
//   * kColStride = 16 fp16 = 32 B/row -> Swizzle<1, 4, 3>  = SWIZZLE_32B
//   * kColStride = 32 fp16 = 64 B/row -> Swizzle<2, 4, 3>  = SWIZZLE_64B
//   * kColStride = 64 fp16 = 128 B/row -> Swizzle<3, 4, 3> = SWIZZLE_128B
// The XOR mask is derived from the byte-address bit positions used by the
// underlying CuTe swizzle pattern: bits {4..(4+B-1)} XOR bits
// {7..(7+B-1)} of the absolute byte address. Translating to (i, j) with
// row stride ``kColStride * 2`` bytes:
//   * 32B (B=1) : (j>>3) ^ (i>>2)              [chunk in {0, 1}]
//   * 64B (B=2) : chunk(2b) ^ {(i>>1)&1, (i>>2)&1}      [{0..3}]
//   * 128B(B=3) : chunk(3b) ^ {i&1, (i>>1)&1, (i>>2)&1} [{0..7}]
// source: LeetCUDA/ffpa-attn/cuffpa/swizzle.cuh
//
// v1/v2 实现 (由编译宏 NOTES_V2_ENABLE_SWIZZLE_V2 门控):
//   swizzle_v1_impl : 原手写 XOR 分支 (permuted<kColStride,8>), 按列宽展开.
//   swizzle_v2_impl : 用本地 SwizzleBMS<B,M,S> 镜像 cute::Swizzle<B,M,S>::apply
//                     位级公式统一表达 (16/32/64); kColStride=8 回退 v1.
//   swizzle         : 公开派发器, 宏门控, 接口与原 swizzle<kColStride> 一致.
// v2 不是新算法, 而是把 v1 的手写展开重新表达为 cute 的统一位置换公式,
// 便于脱离 cute 框架理解 swizzle 本质. v1/v2 bit-exact 等价.
template <const int kColStride = 16, const int kStep = 8>
static __host__ __device__ __forceinline__ int permuted(int i, int j) {
  static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64 ||
                kColStride == 8,
                "kColStride must be one of {8, 16, 32, 64} (matches "
                "SWIZZLE_32B/64B/128B + the "
                "kStep=4 narrow legacy case).");
  static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
  static_assert(kColStride % kStep == 0,
                "kColStride must be multiple of kStep.");
  if constexpr (kStep == 4) {
    static_assert(kColStride <= 16, "kStep=4 only supports (kColStride <= 16).");
    return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
  } else if constexpr (kColStride == 16) {
    return (((j >> 3) ^ (i >> 2)) & 1) << 3;  // SWIZZLE_32B
  } else if constexpr (kColStride == 32) {
    const int chunk = (j >> 3) & 3;
    const int xor_mask = ((i >> 1) & 1) | (((i >> 2) & 1) << 1);
    return (chunk ^ xor_mask) << 3;  // SWIZZLE_64B
  } else {                           // kColStride == 64
    const int chunk = (j >> 3) & 7;
    const int xor_mask =
        (i & 1) | (((i >> 1) & 1) << 1) | (((i >> 2) & 1) << 2);
```

```
    return (chunk ^ xor_mask) << 3;  // SWIZZLE_128B
  }
}

// Manually SMEM swizzling for bank conflict free.
// ---------------------------------------------------------------
// [INFO] Assert smem store layout col_stride <= 16, prefer 16.   |
// [INFO] For logical_col_stride > 16, we have to permute the     |
// [INFO] smem store layout using col major ZigZag method:        |
// [INFO] e.g, --> Q smem logical layout [Br][64].                |
// [INFO]      --> col major ZigZag permuted -->                  |
// [INFO]      --> Q smem store layout [4][Br][16].               |
// ---------------------------------------------------------------
// ---------------------------------------------------------------
// -------------------------swizzle layout------------------------
// --------------------logical col 0~64, step 8-------------------
// --------------------smem col 0~16, step 8----------------------
// ---------------------------------------------------------------
// |bank  |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
// |row 0 |  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// |bank  |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
// |row 1 |  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// |bank  |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
// |row 2 |  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// |bank  |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
// |row 3 |  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// ---------------------------------------------------------------
// |bank  |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
// |row 4 |  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// |bank  |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
// |row 5 |  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// |bank  |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
// |row 6 |  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// |bank  |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
// |row 7 |  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// ---------------------------------------------------------------
// |bank  |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
// |row 8 |  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// |bank  |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
// |row 9 |  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// |bank  |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
// |row 10|  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// |bank  |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
// |row 11|  0   |  8   |  0   |  8   |  0   |  8   |  0   |  8   |
// ---------------------------------------------------------------
// |bank  |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
// |row 12|  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// |bank  |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
// |row 13|  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// |bank  |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
// |row 14|  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// |bank  |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
// |row 15|  8   |  0   |  8   |  0   |  8   |  0   |  8   |  0   |
// ---------------------------------------------------------------
template <const int kColStride = 16>
static __host__ __device__ __forceinline__ int swizzle_v1_impl(int i, int j) {
  return permuted<kColStride, 8>(i, j);
}

// Swizzle v2: cute::Swizzle<B,M,S> 位级镜像（脱离 cute 框架独立理解）
//
// v2 双重目的:
//   (1) 探索优化 v1 的可能 -- v1 按 kColStride 手写 XOR 分支展开, 已被 nvcc
```

```
//       strength-reduce 到极简位运算; v2 用 cute 的统一位置换公式重新表达,
//       预期中性, 但统一表达式可能更利于合并到地址算术 (bonus).
//   (2) 把 cute Swizzle 的核心位逻辑从 cute 框架提取出来独立理解 -- 让 notes-v2
//       不依赖 cute 头即可展示 Swizzle<B,M,S> 的位级语义, 便于读者脱离 cute
//       模板体系直接掌握 swizzle 原理.
//
// 注释叙述参考: 上方 hgemm_mma_stages_tn_cute 处的 Swizzle<B,M,S> 原理解释
// (二维逻辑空间 + 列置换), 此处适配 v2 的 (B,M,S)=(1/2/3, 4, 3) 语境.
//
// Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 byte offset
// 重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理
// offset:
//   1. 连续的 2^M 个元素组成二维空间中的一个基本元素 (元素宽度);
//   2. 连续的 2^S 个基本元素组成一行 (列数);
//   3. 二维空间包含 2^B 行 (行数, 这些行的 chunk 互相置换);
//   4. 对二维坐标做列置换: icol' = irow ^ icol;
//   5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset.
//
// 位级实现 (与 cutlass/include/cute/swizzle.hpp 一致):
//   bit 布局:   0bxxxxxxxxxxxxxxxxYYYxxxxxxxZZZxxxx
//                                  ^--^ MBase: 保留不参与置换的低位数
//                      ^-^       ^-^     BBits: XOR 掩码位数
//                        ^---------^       SShift: YYY 相对 ZZZ 的位移
//   apply(off) = off ^ shiftr(off & yyy_msk, S)
//   yyy_msk    = ((1<<B)-1) << (M + max(0,S))    // YYY 掩码位置
//   zzz_msk    = ((1<<B)-1) << (M - min(0,S))    // ZZZ 掩码位置 (文档用)
//
// 模板参数物理/语义含义:
//   B (BBits)  : 二维逻辑空间的行数位 = 2^B 行, 即 swizzle 周期覆盖 2^B 个 chunk
//                (这些 chunk 互相置换); 同时是 TMA 硬件 swizzle 模式
//                SWIZZLE_{32,64,128}B 的位数 (B=1/2/3).
//   M (MBase)  : 基本元素宽度位 = 2^M 个连续元素组成一个基本元素. 对 fp16 +
//                8-element chunk, 基本元素 = 16B = 2^4, 故 M=4; 地址 bit[0..M-1]
//                在 apply 中保持不变 (保留).
//   S (SShift) : 二维空间每行列数位 = 2^S 个基本元素组成一行; 同时是 YYY 掩码
//                相对 ZZZ 掩码的位移. S>0 表示 YYY 在更高位, 向右移 S 位到 ZZZ
//                位置后 XOR. 本 notes-v2 三模式 S 恒为 3 (YYY 在 bit[7..7+B-1],
//                即行索引 i 的低 B 位; ZZZ 在 bit[4..4+B-1], 即列 chunk 索引).
//
// 完整 swizzle 周期覆盖 2^(M+S+B) 个元素 (fp16 下 2^(M+S+B+1) 字节):
//   (B,M,S)=(1,4,3): 256 elem / 512B   (SWIZZLE_32B)
//   (B,M,S)=(2,4,3): 512 elem / 1024B  (SWIZZLE_64B)
//   (B,M,S)=(3,4,3): 1024 elem / 2048B (SWIZZLE_128B)
template <int B, int M, int S>
struct SwizzleBMS {
  static_assert(M >= 0, "MBase must be non-negative.");
  static_assert(B > 0, "BBits must be positive.");
  static_assert((S > 0 ? S : -S) >= B,
                "abs(SShift) must be >= BBits (cute::Swizzle constraint).");

  static constexpr int bit_msk = (1 << B) - 1;
  static constexpr int yyy_msk = bit_msk << (M + (S > 0 ? S : 0));
  static constexpr int zzz_msk = bit_msk << (M - (S < 0 ? S : 0));

  // apply: 对 byte offset 做 XOR 位置换, ZZZ ^= (YYY >> S).
  static __host__ __device__ __forceinline__ int apply(int offset) {
    if constexpr (S >= 0) {
      return offset ^ ((offset & yyy_msk) >> S);
    } else {
      return offset ^ ((offset & yyy_msk) << (-S));
    }
  }
};
```

```
1890
1891 // swizzle_v2_impl: 用 SwizzleBMS<B,M,S>::apply 计算 chunk 级列 swizzle.
1892 //
1893 // 流程: (i,j) -> byte offset off=(i*kColStride+j)*sizeof(half) -> SwizzleBMS
1894 //       位级置换 -> 取 [M, M+B) 位作为物理 chunk 索引 -> 乘 kStep 还原为列偏移.
1895 //
1896 // kColStride -> (B,M,S) 映射表 (对应 TMA 硬件 swizzle 模式):
1897 //   kColStride=16 fp16=32B/row  -> (B,M,S)=(1,4,3) = SWIZZLE_32B
1898 //   kColStride=32 fp16=64B/row  -> (B,M,S)=(2,4,3) = SWIZZLE_64B
1899 //   kColStride=64 fp16=128B/row -> (B,M,S)=(3,4,3) = SWIZZLE_128B
1900 //
1901 // kColStride=8 (kStep=4 legacy): 无对应 cute 标准 TMA swizzle 模式, 回退到
1902 //   swizzle_v1_impl. 该路径当前无任何 kernel 使用, 回退保证接口契约零变更.
1903 //
1904 // 等价性: 对 (16,32,64), v2 输出与 v1 bit-exact 相同 (见 test_swizzle_equiv).
1905 //   证明: off 的 bit[4..4+B-1] 恰是 (j>>3) 低 B 位 (因 i*kColStride 在这些位
1906 //   为 0); bit[7..7+B-1] 恰是 i 低 B 位 (因 j<kColStride 不贡献高位). apply
1907 //   把 YYY(bit[7..]) XOR 到 ZZZ(bit[4..]), 取 [M,M+B)=bit[4..4+B-1] 即得
1908 //   (j>>3)^i 的低 B 位 = v1 的 chunk^xor_mask.
1909 template <const int kColStride = 16>
1910 static __host__ __device__ __forceinline__ int swizzle_v2_impl(int i, int j) {
1911   if constexpr (kColStride == 8) {
1912     return swizzle_v1_impl<kColStride>(i, j);
1913   } else {
1914     static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64,
1915                   "v2 supports kColStride in {16, 32, 64} (cute TMA swizzle); "
1916                   "8 falls back to v1.");
1917     constexpr int B = (kColStride == 16) ? 1 : (kColStride == 32) ? 2 : 3;
1918     constexpr int M = 4;
1919     constexpr int S = 3;
1920     constexpr int kStep = 8;
1921     const int off = (i * kColStride + j) * (int)sizeof(half);
1922     const int sw = SwizzleBMS<B, M, S>::apply(off);
1923     return ((sw >> M) & ((1 << B) - 1)) * kStep;
1924   }
1925 }
1926
1927 // swizzle: 公开派发器, 由编译宏 NOTES_V2_ENABLE_SWIZZLE_V2 门控 v1/v2.
1928 //   未定义宏 (默认) -> swizzle_v1_impl (原手写 XOR 分支, 历史默认路径).
1929 //   定义宏          -> swizzle_v2_impl (cute Swizzle<B,M,S> 位级镜像).
1930 // 接口与原 swizzle<kColStride>(i,j) 完全一致, 所有 kernel 调用点无需改动.
1931 // v1/v2 等价性由 host 端 test_swizzle_equiv (--swizzle-eq-check) 验证.
1932 template <const int kColStride = 16>
1933 static __device__ __forceinline__ int swizzle(int i, int j) {
1934 #if defined(NOTES_V2_ENABLE_SWIZZLE_V2)
1935   return swizzle_v2_impl<kColStride>(i, j);
1936 #else
1937   return swizzle_v1_impl<kColStride>(i, j);
1938 #endif
1939 }
1940
1941 // ============================================================================
1942 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1943 //              + Register Double Buffering (generic kValTileK, default BK=64)
1944 // ============================================================================
1945 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1946 //   - kValTileK: 每个 BK tile 包含 kValTileK 个 kMmaK slice (BK = kMmaK * kValTileK)
1947 //   - 统一 BK tile: 所有 k_step slice 连续存放在同一个 BK 宽的 smem tile 中
1948 //   - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换 (与 kValTileK 无关)
1949 //   - ldmatrix 与 MMA 计算重叠: 加载 k_step+1 的同时用另一组寄存器做 k_step 计算
1950 //   - 支持任意 kValTileK >= 2; 默认 kValTileK=4 (BK=64), kStages=2 推荐 64KB smem
1951 //
1952 // ★ smem 布局:
```

```
//   s_a: [stage0][stage1]...[stage(kStages-1)], 每个 stage = BM × BK = 128 × BK
//   s_b: 紧接 s_a 之后
//   每个 stage 内 k_step slice 列偏移 = k_step * kMmaK
//
// ★ 寄存器双缓冲时间线（每 k 迭代内）：
//   Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step（条件）
//   Step 2: MMA k_step=0（用 reg[load]，已在上轮 post-wait 中 ldmatrix）
//   Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
//   Step 4: adaptive wait + __syncthreads
//   Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store]（条件）
//
// 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
// Block: (256, 1, 1), 8 warps
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim
          const int kMmaTileM = 2,          // warps along M, 2 → warp tile M = 32
          const int kMmaTileN = 4,          // warps along N, 4 → warp tile N = 32
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 4,          // value-repeat along N, BN = 8*4*4 = 128
          const int kValTileK = 4,          // MMA_K slices per BK tile, BK = kMmaK * kValTileK
          const int kStages = 2,            // cp.async pipeline depth (2 for kValTileK>=4 to fit smem)
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(256)
    hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
  static_assert(kValTileK >= 2, "Register double buffering requires kValTileK >= 2");
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
  static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
  // Block Swizzle: 在 grid x 维度做 swizzle，改善 L2 cache 局部性
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
  constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
  constexpr int BK = kMmaK * kValTileK;             // 16 * kValTileK

  // kStages stages, each with BM×BK for A, BN×BK for B
  // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK 宽 tile 中
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + kStages * BM * BK;
  constexpr int s_a_stage_offset = BM * BK;
  constexpr int s_b_stage_offset = BN * BK;

  const int tid = threadIdx.y * blockDim.x + threadIdx.x;
  const int warp_id = tid / kWarpSize;
  const int lane_id = tid % kWarpSize;
  const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
  const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)

  // 线程到 global memory 的映射（用于加载 A 和 B）共 256 个线程
  // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
  // 注意: smem_a_k 和 smem_b_k 依然使用 0/8，在后续的 kValTileK 循环中
  // 会加上 k_step*kMmaK，最终覆盖全部 BK 列
  int load_smem_a_m = tid / 2;                // 0~127
  int load_smem_a_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8
  int load_smem_b_n = tid / 2;                // 0~127 → B^T 的 N 方向 (row-major 的行)
  int load_smem_b_k = (tid % 2 == 0) ? 0 : 8;  // 0, 8  → B^T 的 K 方向 (row-major 的列)
  int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
  int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
  if (load_gmem_a_m >= M || load_gmem_b_n >= N)
    return;

```

```
  // 8个Warps(MMAs)的排布是M方向2个，N方向4个，则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
  // CUDA中每个block能放的线程数是有上限的（warp数量有限），一般为4或者8个warps。为了能处理更大的C tile，
  // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是Value Tile的作用。
  // MMA Tile对应到cutlass cute中的TiledMMA的概念，Value Tile对应到cutlass cute中的PermuteMNK的概念。
  uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0

  uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
  uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);

  // Prefetch (kStages-1) stages: load all k_step slices for each early stage
#pragma unroll
  for (int k = 0; k < (kStages - 1); ++k) {
#pragma unroll
    for (int k_step = 0; k_step < kValTileK; ++k_step) {
      int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
      int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
      int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
      int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;

      uint32_t load_smem_a_ptr =
          (smem_a_base_ptr +
           (k * s_a_stage_offset + load_smem_a_m * BK +
            k_step * kMmaK +
            swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
              sizeof(half));
      CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

      uint32_t load_smem_b_ptr =
          (smem_b_base_ptr +
           (k * s_b_stage_offset + load_smem_b_n * BK +
            k_step * kMmaK +
            swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
              sizeof(half));
      CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
    }
    CP_ASYNC_COMMIT_GROUP();
  }

  CP_ASYNC_WAIT_GROUP(kStages - 2);
  __syncthreads();

  // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
  uint32_t RA[2][kValTileM][4];
  uint32_t RB[2][kValTileN][2];
  int reg_st_idx = 0; // write target for ldmatrix
  int reg_ld_idx = 1; // read source for MMA

// Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) → reg[0]
// 此时，reg_st_idx = 0，对0位置的寄存器buffer做初始化。
#pragma unroll
  for (int i = 0; i < kValTileM; ++i) {
    int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
    int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
    int lane_smem_a_k = (lane_id / 16) * 8;
    uint32_t lane_smem_a_ptr =
        (smem_a_base_ptr +
         (0 * s_a_stage_offset + lane_smem_a_m * BK +
          swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
            sizeof(half));
    LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                lane_smem_a_ptr);
  }
```

```
#pragma unroll
  for (int j = 0; j < kValTileN; ++j) {
    int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
    int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
    int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
    uint32_t lane_smem_b_ptr =
        (smem_b_base_ptr +
         (0 * s_b_stage_offset + lane_smem_b_n * BK +
          swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
             sizeof(half));
    LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                lane_smem_b_ptr);
  }

  // 统一循环: k 从 0 开始，条件 G→S / S→R / wait 处理所有边界情况
  // BK = kMmaK * kValTileK，每个 k 迭代覆盖 BK 个 K 元素
  const int NUM_K_TILES = div_ceil(K, BK);
  for (int k = 0; k < NUM_K_TILES; ++k) {
    int smem_sel = k % kStages;
    int smem_sel_next = (k + kStages - 1) % kStages;

    // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
    if (k + kStages - 1 < NUM_K_TILES) {
#pragma unroll
      for (int k_step = 0; k_step < kValTileK; ++k_step) {
        int load_gmem_a_k =
            (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
        int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
        int load_gmem_b_k =
            (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
        int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;

        uint32_t load_smem_a_ptr =
            (smem_a_base_ptr +
             (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
              k_step * kMmaK +
              swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
                 sizeof(half));
        CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

        uint32_t load_smem_b_ptr =
            (smem_b_base_ptr +
             (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
              k_step * kMmaK +
              swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
                 sizeof(half));
        CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
      }
      CP_ASYNC_COMMIT_GROUP();
    }

    // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
    // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
    // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld]（k_step+1 的 ldmatrix 条件化）

#pragma unroll
    for (int k_step = 0; k_step < kValTileK; ++k_step) {
      reg_st_idx ^= 1; // 0 -> 1; 1 -> 0
      reg_ld_idx ^= 1; // 1 -> 0; 0 -> 1

      if (k_step + 1 < kValTileK) {
        int smem_k_offset = (k_step + 1) * kMmaK;
#pragma unroll
```

```
        for (int i = 0; i < kValTileM; ++i) {
          int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
          int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
          int lane_smem_a_k = (lane_id / 16) * 8;
          uint32_t lane_smem_a_ptr =
              (smem_a_base_ptr +
              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
                smem_k_offset +
                swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
                  sizeof(half));
          LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                      RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                      lane_smem_a_ptr);
        }
  #pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
          int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
          int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
          uint32_t lane_smem_b_ptr =
              (smem_b_base_ptr +
              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
                smem_k_offset +
                swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
                  sizeof(half));
          LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                      lane_smem_b_ptr);
        }
      }

#pragma unroll
      for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileN; ++j) {
          HMMA16816(RC[i][j][0], RC[i][j][1],
                    RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
                    RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
                    RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
                    RC[i][j][0], RC[i][j][1]);
        }
      }
    }

    // 自适应等待：满载期用 kStages-2，尾部排空用 0
    if (k + kStages - 1 < NUM_K_TILES) {
      CP_ASYNC_WAIT_GROUP(kStages - 2);
    } else {
      CP_ASYNC_WAIT_GROUP(0);
    }
    __syncthreads();

    // 从下一阶段加载 k_step=0 数据，供下一轮 k 迭代使用，此时 reg_st_idx=0
    if (k + 1 < NUM_K_TILES) {
      smem_sel = (k + 1) % kStages; // old smem_sel + 1
#pragma unroll
      for (int i = 0; i < kValTileM; ++i) {
        int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
        int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
        int lane_smem_a_k = (lane_id / 16) * 8;
        uint32_t lane_smem_a_ptr =
            (smem_a_base_ptr +
             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
              swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
```

```
                  sizeof(half));
        LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
                    RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
                    lane_smem_a_ptr);
      }
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
        int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
        uint32_t lane_smem_b_ptr =
            (smem_b_base_ptr +
             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
              swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
                 sizeof(half));
        LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
                    lane_smem_b_ptr);
      }
    }
  }

  // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
  // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间，RA[2][4][4]正好可以复用
  for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileN; ++j) {
      RA[0][j][0] = RC[i][j][0];
      RA[1][j][0] = RC[i][j][1];
      RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
      RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
      RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
      RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
      RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
      RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
    }
    if (lane_id % 4 == 0) {
      int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
      int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
        int store_gmem_c_addr_0 =
            store_lane_gmem_c_m * N + store_lane_gmem_c_n;
        int store_gmem_c_addr_1 =
            (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
        *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
            *reinterpret_cast<float4 *>(&RA[0][j][0]);
        *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
            *reinterpret_cast<float4 *>(&RA[1][j][0]);
      }
    }
  }
}

// ============================================================================
// Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
// ============================================================================
// 面试要点 (CuTe vs 手写 MMA PTX):
//   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL，提供编译期
//     Tensor 抽象，自动推导 MMA、Copy、Swizzle 的线程-数据映射
//   - 与手写 MMA PTX (Phase 7b) 的对比:
//     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
```

```
//     - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout，编译器自动推导线程-数据映射，
//       cute::copy / cute::gemm 自动展开为高效指令序列
//   - 核心抽象（"CuTe 五要素"）：
//     1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
//     2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
//     3. TiledCopy: 描述数据搬运（G→S, S→R, R→S, S→G）的线程-元素映射
//     4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
//     5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
//
//   CuTe kernel 的参数推导链（launch wrapper 负责实例化，kernel 只接收实例化后的类型）：
//     MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
//       → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
//       → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
//       → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
//
//   调参口诀（面试速记）：
//     BM/BN/BK 越大 → smem 越大 → occupancy 越低，但 K 循环更少 → 指令开销更低
//     Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape，而是把一维 offset
//     重新解释为一个二维逻辑空间，再把二维坐标映射回 bank-conflict-free 的物理 offset:
//       1. 连续的 2^M 个元素组成二维空间中的一个基本元素；（元素宽度）
//       2. 连续的 2^S 个基本元素组成一行；（列数）
//       3. 二维空间包含 2^B 行；（行数）
//       4. 对二维坐标做列置换: icol' = irow ^ icol;
//       5. 保留基本元素内部的低 M 位，再将置换后的二维坐标编码回 offset。
//     因此 M 描述基本元素宽度，S 描述二维空间的列数，B 描述二维空间的行数。
//     CuTe 的位级实现等价于:
//       offset' = offset ^ ((offset & YYY) >> S),
//       YYY = ((1 << B) - 1) << (M + S)。
//     对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值，每行有 2^3=8 个基本元素，
//     二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5]，低 3 位保持不变。
//     一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素（FP16 下为 1024B）。
//     kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
//     EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局，决定 TiledMMA 的线程排布
//       与逻辑 tile 组织; MMA atom越多，需要的线程数就越多
//
// ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
//   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
//   - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern，cute::copy 自动应用
//   - CuTe 优势: 类型安全，编译器可在编译期推导映射; 布局变更通常只需修改类型定义
//
// ★ 本实现的 Tile 配置:
//   - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
//   - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
//   - TiledMMA 的逻辑 MMA tile 是 32×32×16，G2S/S2R copy 再把它映射到更大的 BM×BN tile;
//     BN=256 不是由“8×2×16”直接推导出的 MMA tile，而是本 wrapper 选择的 B tile 尺寸。
//   - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排，针对 ldmatrix
//     的访问模式降低 bank conflict; 512 是 swizzle 周期，不等于 A atom 的逻辑元素数。
//   - Threads: size(MMA{}) = 128（4 warps × 2×2 EU repeat）。
//   - Dynamic smem: Stage=2 时 A[128×32] + B[256×32] 共 49,152 bytes（约 48 KiB）。
//
// 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
// 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
// ================================================================================

#if defined(NOTES_V2_ENABLE_CUTE)

#include <cute/tensor.hpp>
#include <cutlass/arch/barrier.h>
#include <cutlass/device_kernel.h>
#include <cutlass/numeric_conversion.h>

// ================================================================================
// Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
```

```cpp
// ============================================================================
// TN 布局: C[M×N] = A[M×K] × B^T[N×K]
//   - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [K×N])
//   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
//
// Pipeline 流程 (stage 是 K tile 的循环缓冲槽，不是一条完整计算链)：
//   1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async)，填满 stage。
//   2. 主循环 over K tiles:
//      a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
//      b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
//      c. 在处理当前 tile 的首个 MMA slice 时，异步提交下一个 tile 的 G→S;
//      d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
//   3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store) 。
//
// 面试对比 — 手写 MMA vs CuTe 代码量:
//   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
//   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
template <typename T,                       // element type (half)
          int BM, int BN, int BK,           // block tile: BM=128, BN=256, BK=32
          int kStage,                       // cp.async pipeline depth (2)
          typename TiledMMA,                // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
          typename G2SCopyA,                // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
          typename G2SCopyB,                // G→S B: same as G2SCopyA
          typename SmemLayoutA,             // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
          typename SmemLayoutB,             // smem B: same atom → (BN,BK,kStage)
          typename SmemLayoutC,             // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
          typename S2RCopyAtomA,            // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
          typename S2RCopyAtomB,            // S→R B: same as S2RCopyAtomA
          typename R2SCopyAtomC,            // R→S C: UniversalCopy<int> (32-bit store)
          typename S2GCopyAtomC,            // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
          typename S2GCopyC,                // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
          const int BlockSwizzle = 0>       // 1 enables 3D grid swizzle for L2 locality
__global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
                                         int n, int k) {
  using namespace cute;
  static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
  static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");

  // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
  extern __shared__ T shm_data[];
  T *Ashm = shm_data;
  T *Bshm = shm_data + cute::cosize(SmemLayoutA{});

  int idx = threadIdx.x;
  // BlockSwizzle: 0/1 控制是否启用 thread block swizzle，改善 L2 cache 局部性
  int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  int iy = blockIdx.y; // M 方向 block 索引
  // 当前 kernel 只有 CTA 级边界判断，没有 tile 内的逐元素 predication; 调用方需保证
  // M % BM == 0、N % BN == 0、K % BK == 0，才能避免边界 tile 的越界访问或未写回。
  if (iy * BM >= m || ix * BN >= n) return;

  // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
  // make_stride(leading_dim, Int<1>{})  → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
                         make_stride(k, Int<1>{}));
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
                         make_stride(k, Int<1>{}));
  Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
                         make_stride(n, Int<1>{}));

  // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
  // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标，_ 保留 K 方向的 tile remainder:
  //   gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
```

```
  Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
  Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
  Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));

  // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
  auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
  auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)

  // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
  TiledMMA tiled_mma;
  auto thr_mma = tiled_mma.get_slice(idx);
  auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
  auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
  auto tCrD = thr_mma.partition_fragment_C(gD);          // (MMA, MMA_M, MMA_N)
  clear(tCrD); // 累加器清零

  // G2S TiledCopy: 描述 global → shared memory 的数据搬运（128-bit cp.async）。
  G2SCopyA g2s_tiled_copy_a;
  auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
  auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
  auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)

  G2SCopyB g2s_tiled_copy_b;
  auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
  auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
  auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);

  // S2R TiledCopy: 描述 shared → register 的数据搬运（使用 ldmatrix）
  // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
  auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
  auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
  auto tAsA = s2r_thr_copy_a.partition_S(sA);     // (CPY, CPY_M, CPY_K, kStage)
  auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐

  auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
  auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
  auto tBsB = s2r_thr_copy_b.partition_S(sB);
  auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);

  // PREFETCH: 预加载前 (kStage - 1) 个 K tile，填满循环缓冲；每次 copy 提交一个异步 G→S 操作。
  int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
#pragma unroll
  for (int istage = 0; istage < kStage - 1; ++istage) {
    cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
               tAsA_copy(_, _, _, istage));
    cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
               tBsB_copy(_, _, _, istage));
    cp_async_fence();
    ++itile_to_read;
    ++ismem_write;
  }
  cp_async_wait<kStage - 2>();
  __syncthreads();

  // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
  // 外层 k_tile 遍历这些 BK-tile，内层 k_step 遍历 tile 内的 MMA_K slice。
  // 当前实现使用整除，要求 K % BK == 0；没有为尾部 K tile 做 predication/padding。
  int num_k_tiles = k / BK;
  // 循环前预加载首轮数据：将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
  cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
  cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));

#pragma unroll 1
```

```
  for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
    // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数，取自 fragment 的第三 mode（K-mode）。
    //
    // 为什么只显式展开 K？MMA_M 和 MMA_N 去哪了？
    //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
    //   - K 方向：每个 k_step 需要从 smem 加载**不同的** A/B 数据（ldmatrix），
    //     所以 K 循环必须显式写，每次迭代做 S→R copy + cute::gemm。
    //   - M/N 方向：同一个 k_step 内，所有 M、N 位置的 MMA atom 共享
    //     同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息（EURepeat=2×2）
    //     在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列，
    //     无需手动写 M/N 循环。——这等价于手写 MMA 中的：
    //       for (i=0; i<kValTileM; ++i)
    //         for (j=0; j<kValTileN; ++j)
    //           HMMA16816(RC[i][j][0], RC[i][j][1], ...);
    //
    // CUTLASS 推导链（mma_atom.hpp）：
    //   make_tiled_mma 将 MMA_P_T::<2>（即 kMmaPK）存入 PermutationMNK
    //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
    //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
    //     做 logical_divide，即按 (kMmaPM,kMmaPK) tile 化 K 维：
    //        K 被分为 BK/kMmaPK 个 perm-K slice
    //     随后按 AtomShape_MNK<2>（MMA_K_atom）做 zipped_divide
    //     每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
    //   → partition_A 选当前线程后，K-mode size = BK / kMmaPK = 32 / 16 = 2
    // 本配置：kMmaPK=16（MMA_K_atom=16 × kMmaEURepeatK=1），BK=32 → num_k_steps=2
    int num_k_steps = size<2>(tCrA);

#pragma unroll
    for (int k_step = 0; k_step < num_k_steps; ++k_step) {
      int k_step_next = (k_step + 1) % num_k_steps;

      if (k_step == num_k_steps - 1) {
          // 等待下一个 K tile 的 smem 数据就绪，然后切换到下一个 stage。
        cp_async_wait<kStage - 2>();
        __syncthreads();
        ismem_read = (ismem_read + 1) % kStage;
      }

      // S→R: 加载下一组 A/B fragment 到寄存器。
      //
      // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice：
      //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
      // 但这里刻意逐 k_step_next 加载，目的是做**寄存器双缓冲**：
      //   当前 k_step 做 MMA 计算时，ldmatrix 预加载 k_step_next 的数据，
      //   让 S→R 延迟与 MMA 计算重叠。
      // 如果一次性加载全部 K slice，S→R 和 MMA 就会彻底串行。
      cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
                 tCrA_view(_, _, k_step_next));
      cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
                 tCrB_view(_, _, k_step_next));

      if (k_step == 0) {
        // G→S: 提交下一个 K tile（整个 BK）的异步拷贝。
        if (itile_to_read < num_k_tiles) {
          cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
                     tAsA_copy(_, _, _, ismem_write));
          cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
                     tBsB_copy(_, _, _, ismem_write));
          ++itile_to_read;
          ismem_write = (ismem_write + 1) % kStage;
        }
        cp_async_fence();
      }
```

```
2520
2521        // MMA: cute::gemm 内部根据 tiled_mma 的类型信息，自动展开为覆盖
2522        // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列，累加到 tCrD。
2523        cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2524      }
2525    }
2526
2527    // Epilogue: D 寄存器 → shared memory → global memory。
2528    // 使用当前 A stage 作为 C scratchpad，避免为 epilogue 单独分配完整的 C shared memory。
2529    // 这里是“复用 storage”，不是把 A Tensor 转换成 C Tensor：
2530    //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2)，而 sA(_,_,ismem_read)
2531    //   只选出其中一个 A-stage 的 storage 起点；sC 随后以完全独立的 SmemLayoutC
2532    //   解释这同一段地址。当前 wrapper 的固定配置下：
2533    //     一个 A stage: 128 x 32 = 4096 个 half；
2534    //     C scratchpad: 32 x 32 x 4 = 4096 个 half。
2535    //   两者容量刚好相等，但 logical shape 和 layout 不是同一个：A 的 layout atom
2536    //   是 8x32，C 的 layout atom 是 32x32，二者都含 Swizzle<3,3,3>，却不能把
2537    //   C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2538    //   检查 C scratchpad 能放进一个 A pipe；此处只别名该单个 stage，不是整块双缓冲 sA。
2539    //
2540    // mainloop 已结束，不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2541    // 从这一行开始，对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成：R2S
2542    // 用它写 C，S2G 也用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2543    auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2544
2545    // R2S TiledCopy: 寄存器 → shared memory（使用 UniversalCopy 做类型转换）
2546    // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2547    auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2548    auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2549    // tCrD 是 TiledMMA 产生的 accumulator fragment，逻辑 shape 为
2550    // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位，但该 layout 是为 MMA
2551    // accumulator 服务的，并不直接按 R2S copy atom 的 source-value 顺序表达。
2552    //
2553    // retile_S 中的 S 指“这个 TiledCopy 的 source side”，不是 shared memory。
2554    // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2555    // zero-copy layout view，将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2556    // 或写入寄存器/共享内存，不进行 dtype conversion，也不交换 lane 间数据；真正的
2557    // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2558    // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照：二者都是为了让已有
2559    // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2560    // 角色，而不是另一次 memory transfer。
2561    // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2562    auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2563    // get_slice(idx) 已固定 CTA 内当前 logical copy thread；partition_D 再按
2564    // R2S TiledCopy 的 destination mapping 切分 sC。推导链：
2565    //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2566    //     → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2567    //     → make_tiled_copy_C 的 tiler = (32, 32)
2568    //     → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2569    //     → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2570    //     → 结果 shape 的 M/N remainder = _1, _1
2571    // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch，而不是
2572    // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2573    // 是 1，并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2574    auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2575
2576    // S2G TiledCopy: shared memory → global memory（128-bit store）
2577    // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2578    S2GCopyC s2g_tiled_copy_c;
2579    auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2580    // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生，因而保留相同的 pipe=4；前者是
2581    // S2G source mapping，后者是 R2S destination mapping。tCgC_s2g 则面向完整
2582    // gD=(BM,BN)=(128,256)，相对于 S2G copy tiler 仍有非平凡的 M/N repetition，
```

```
  // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
  // 的逻辑 mode，不应直接理解为固定的 lane 数、warp 数或物理连续维度。
  auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
  auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)

  // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode：
  //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
  //       -> Tensor(CPY, (CPY_M, CPY_N))
  // 它只重组 Tensor 的 layout，不分配内存、不移动 data()，也不改变元素访问的物理地址。
  // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系，只是现在可以用一个坐标
  // i+j 访问这个组合 mode；因此这里的 group_modes 更准确地说是“合并 mode”，
  // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
  // 从 type/shape 结构看，结果确实是 Tensor(CPY, (CPY_M, CPY_N))；但 grouped
  // mode 的 cardinality 是两个子 mode 的乘积，因此 size<1>(...) 可作为一个标量
  // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
  // 的线性 logical coordinate，不是在 C++ 中对二维 tuple 做加法，更不是 raw
  // pointer offset；最终物理地址仍由各自 Tensor 的 grouped layout 计算。
  //
  // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
  //   - tCrC_r2s: 每个线程持有的寄存器 C fragment，原始形状为 (CPY, CPY_M, CPY_N);
  //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置，形状与源 Tensor 对齐;
  //   - 两者使用相同的 [1,3) 合并后，源/目的 Tensor 仍保持相同的坐标空间，
  //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
  //
  // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode，因为它们还需要显式保留
  // shared-memory C scratchpad 的 pipeline 维度；step=size<3>(tCsC_r2s) 表示
  // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
  // 按 step 前进，内层 j 选择当前 pipeline slot：寄存器先写入 sC(_,0,0,j)，
  // 再从同一个 slot 写回 global memory。
  // 当前 kSmemLayoutCBatch=4，因此 step=4。代码未对 i+j 做尾部 predication，
  // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除；不应将
  // “每轮正好处理 4 个 fragment”误认为任意 tile/copy 配置都自动成立的通用规则。
  auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
  auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);

  int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数（由 kSmemLayoutCBatch=4 指定）
  // 双层循环的语义（注意 step 与 CPY_N 无关）：
  //   group_modes<1,3> 之后，tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
  //   size<1>(tCrC_r2sx) = CPY_M * CPY_N，即该线程需处理的 C fragment 总数。
  //   外循环以 step=4 为步长，内循环 j=0..3 每次处理 1 个 fragment，将其写入
  //   第 j 号 C scratchpad pipe slot，再读出写回 global。
  //
  //   这里 step=4 是 scratchpad pipeline 深度，不是 CPY_N。CPY_N 的值取决于
  //   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上，
  //   通常 ≠ 4。循环之所以成立，是因为 grouped mode 用线性坐标 i+j 遍历整个
  //   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向，也不要求 CPY_N == step。
  //   唯一前提是 CPY_M * CPY_N 能被 step 整除（当前静态 layout 编译期保证）。
  //
  //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
  //   各自推导；因为二者都从同一个 sC（SmemLayoutC）出发，pipe mode 保持一致，
  //   写入和读取命中同一段 shared memory。
#pragma unroll
  for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
    // 每轮处理 step 个 fragment，内层 j 选 pipe slot。
#pragma unroll
    for (int j = 0; j < step; ++j) {
      // 这两行是 R2R staging：make_tensor_like<T> 分配当前线程私有、拥有 storage
      // 的 register Tensor；普通 cute::copy 将 accumulator fragment materialize
      // 成输出元素类型 T 的临时 payload。
      //
      // cute::copy（无 copy-atom 的普通版）内部按元素做
      //   dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
      // 因此当 accumulator 与 T 类型不同时，它自动完成 dtype 转换；当二者相同
```

```
      //  (如本 kernel 的 f16 accumulator + T=half) , cast 退化为 no-op。
      //
      // 即使类型一致，t 还有一个不可省略的作用: layout 归一化。
      // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
      // 叠加的 composed layout，而 make_tensor_like<T> + cute::copy 把它
      // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
      // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
      // 拆分 source 元素，简单 owning layout 比复杂的 composed layout 更容易
      // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
      // output data type difference"，实际承担了类型适配和 layout 归一化两个职责。
      //
      // 这不是 retile_S 的一部分，也不是 warp shuffle: 源/目的都在当前线程的
      // register 中，代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
      // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担，而是
      // 由随后的 R2S -> shared memory -> S2G 路径完成。若特定 accumulator/output
      // type 与 R2S copy-atom contract 已直接兼容，可以设计直接 R2S 的变体; 本例
      // 保持同源实现的通用 staging 写法。
      auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
      cute::copy(tCrC_r2sx(_, i + j), t);
      // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
      // slot; SmemLayoutC 定义写入后的跨线程地址重组。
      //
      // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
      //   r2s_tiled_copy_c 是 TiledCopy，但继承自 Copy_Atom<UniversalCopy<int>,T>
      //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
      //   → src(t) 与 dst(tCsC_r2s(_,0,0,j)) 都是 rank-1 → 直接 copy_atom.call
      //   → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count) ,
      //     走 copy_unpack; 否则递归剥 mode，最终都落到 UniversalCopy<int>::copy
      //     → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
      //
      //   这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
      //   shared memory offset，是因为 pre-partition 阶段已经把映射烧进 layout 了:
      //     - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
      //       → 元素顺序已按 R2S atom 的 source-side value ordering 排好
      //     - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
      //       → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
      //         计算了当前线程每个 val 在 smem 中的物理 offset
      //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义，
      //   因此逐元素 copy 天然把正确的数据写到了正确的地址。
      cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
    }
    // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后，S2G 才能从
    // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
    __syncthreads();

    // S→G: 从同一个 pipe slot 读回 global memory，保持源/目的坐标一一对应。
#pragma unroll
    for (int j = 0; j < step; ++j) {
      // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
      //  (32-bit) 不同，它一次搬运 128-bit (8 个 half) , 映射为一条 wide store
      //  (如 st.global.v4.f32 或 st.global.b128) 。
      //
      // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
      //   s2g_tiled_copy_c 是 TiledCopy，继承自 Copy_Atom<UniversalCopy<uint128_t>,T>
      //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
      //   → src=tCsC_s2g(_,0,0,j) 与 dst=tCgC_s2gx(_,i+j) 都是 rank-1
      //   → 直接 copy_atom.call(src, dst)
      //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
      //     → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
      //     → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
      //
      // S2G 的 pre-partition 与 R2S 对称，但 source/destination 角色互换:
      //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
```

```
      //     用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
      //     在 sC (shared memory) 中的物理 offset。
      //     S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
      //                       = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
      //     因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
      //     M/N remainder。
      //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
      //     → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
      //     因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
      //     repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
      //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
      //   因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
      cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
    }
    // 所有线程都完成当前 pipe slots 的 S2G 读取后,下一轮 R2S 才能覆盖这 4 个
    // scratchpad slots,避免一部分线程仍在读取旧数据时被其他线程提前重写。
    __syncthreads();
  }

  // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
  // 的物理分布,使用 RC0/RC1 暂存,再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
  // 8 个 half,并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
  // CuTe 版没有把这些 lane 映射写死在 kernel 源码中: retile_S 表达 accumulator
  // fragment 与 R2S copy 的对应,R2S/sC/S2G 通过 shared-memory scratchpad 完成
  // CTA 范围的数据重组,S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
  // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
  // 区别是手写版直接编排 shuffle/store,CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
}

// ============================================================================
// Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
// ============================================================================
// CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
// 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
// kernel 接收这些类型的实例 (通常为空 struct) , 在编译期完成全部映射推导。
//
// 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法,形成 1:1 对照。
//
// 面试常问: 「CuTe 为什么比手写 PTX 简洁? 」
// 答: 手写需要:
//   1) 手动计算每线程的 smem 地址偏移
//   2) 手动计算 ldmatrix 的 lane 映射
//   3) 手动插入 swizzle XOR 到每个地址计算点
//   4) 手动做 epilogue 的 warp shuffle 编排
// CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
// 在编译期自动完成上述全部推导,生成与手写等价的 PTX 指令序列。
// ============================================================================
template <typename T, const int Stages = 2, const int BlockSwizzle = 0>
void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
  using namespace cute;

  // ── Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) ──
  //
  // kernel 用法对照:
  //   BM=128:   local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
  //             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
  //   BN=256:   local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
  //             local_tile(D, ...) → gD 的列方向
  //   BK=32:    num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
  //   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
  //   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
  auto BM = Int<128>{};
  auto BN = Int<256>{};
```

```cpp
  auto BK = Int<32>{};
  auto KStage = Int<Stages>{};
  auto kSmemLayoutCBatch = Int<4>{};

  // ── SmemLayoutA / SmemLayoutB ──
  // kernel 用法:
  // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
  // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
  // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
  //
  // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
  //   - M=3: 连续 2^3=8 个 half 组成一个基本元素，即 16 bytes;
  //   - S=3: 连续 2^3=8 个基本元素组成一行，即 128 bytes;
  //   - B=3: 共有 2^3=8 行。
  // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B，覆盖全部 32 个 shared-memory bank。
  // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
  // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
  // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
  // 地址位 [6:8] XOR 到 [3:5]，地址位 [0:2] 不参与置换。
  // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
  // 逻辑坐标仍由 base_layout 给出，只有最终的 shared-memory offset 被重排。
  // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
  // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
  using SmemLayoutAtom = decltype(composition(
      Swizzle<3, 3, 3>{},
      make_layout(make_shape(Int<8>{}, Int<BK>{}),
                  make_stride(Int<BK>{}, Int<1>{}))));
  using SmemLayoutA = decltype(tile_to_shape(
      SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
  using SmemLayoutB = decltype(tile_to_shape(
      SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));

  // ── MMA (TiledMMA) ──
  // kernel 用法:
  // TiledMMA tiled_mma;
  // auto thr_mma = tiled_mma.get_slice(threadIdx.x);  // 每线程的 MMA slice
  // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
  // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
  // auto tCrD = thr_mma.partition_fragment_C(gD);        // (MMA, MMA_M, MMA_N)
  // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
  // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
  // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
  //
  // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
  //   → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
  //   → TiledMMA: 128 threads = 4 warps × (2×2 EU slices)，逻辑 MMA tile = 32×32×16
  // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
  // B^T[N,K] 的 row-major 存储（等价于 GEMM 语义中的 B[K,N] col-major）。
  using mma_op = SM80_16x8x16_F16F16F16F16_TN;
  using mma_traits = MMA_Traits<mma_op>;
  using mma_atom = MMA_Atom<mma_traits>;
  using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)

  static constexpr int kMmaEURepeatM = 2;
  static constexpr int kMmaEURepeatN = 2;
  static constexpr int kMmaEURepeatK = 1;
  static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
  static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
  static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
  // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2

  using MMA_EU_RepeatT = decltype(make_layout(make_shape(
      Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
```

```
2835    using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2836    using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2837
2838    // ── G2SCopyA / G2SCopyB ──
2839    // kernel 用法:
2840    // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2841    // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2842    // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2843    // G→S: 128-bit cp.async。ThrLayout{32,4} × ValLayout{1,8}:
2844    //   128 个线程，每线程搬运 1×8=8 个 half = 128 bits。
2845    using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2846    using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2847    using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2848    using G2SCopyA = decltype(make_tiled_copy(
2849        g2s_copy_atom{},
2850        make_layout(make_shape(Int<32>{}, Int<4>{}),
2851                    make_stride(Int<4>{}, Int<1>{})),
2852        make_layout(make_shape(Int<1>{}, Int<8>{}))));
2853    using G2SCopyB = G2SCopyA;
2854
2855    // ── S2RCopyAtomA / S2RCopyAtomB ──
2856    // kernel 用法:
2857    // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2858    // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2859    // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2860    // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2861    // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2862    // 与 MMA fragment 对齐的 shared→register 映射，无需手动指定 ThrLayout/ValLayout。
2863    using s2r_copy_op = SM75_U32x4_LDSM_N;
2864    using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2865    using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2866    using S2RCopyAtomA = s2r_copy_atom;
2867    using S2RCopyAtomB = s2r_copy_atom;
2868
2869    // ── SmemLayoutC ──
2870    // kernel 用法:
2871    // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2872    // 复用当前 A stage 的空间作为 C scratchpad
2873    // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2874    // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2875    // 与 A/B 相同的 Swizzle<3,3,3> 规则，但 base layout 是 32×32（MMA tile 大小），
2876    // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2877    // 的 kStage K-tile pipeline。
2878    using SmemLayoutAtomC = decltype(composition(
2879        Swizzle<3, 3, 3>{},
2880        make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2881                    make_stride(Int<kMmaPN>{}, Int<1>{}))));
2882    using SmemLayoutC = decltype(tile_to_shape(
2883        SmemLayoutAtomC{},
2884        make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2885
2886    // ── R2SCopyAtomC ──
2887    // kernel 用法:
2888    // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2889    // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2890    // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2891    // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler，
2892    // 因此 R2S copy 的 tiler = (32,32)，与 SmemLayoutC 的 (32,32) 恰好匹配。
2893    using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2894
2895    // ── S2GCopyC ──
2896    // kernel 用法:
2897    // S2GCopyC s2g_tiled_copy_c;
```

```
  // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
  // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared→global wide store。
  // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
  // 与 R2S tiler 相同，因此 partition_S(sC) 得到的 pipe 维度一致。
  using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
  using S2GCopyC = decltype(make_tiled_copy(
      S2GCopyAtomC{},
      make_layout(make_shape(Int<32>{}, Int<4>{}),
                  make_stride(Int<4>{}, Int<1>{})),
      make_layout(make_shape(Int<1>{}, Int<8>{}))));

  // ── Grid/Block 配置 ──
  // kernel 用法:
  // int idx = threadIdx.x;         // 0..127
  // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  // int iy = blockIdx.y;           // M 方向 CTA 坐标
  // if (iy * BM >= m || ix * BN >= n) return;  // CTA 级边界保护
  // 由于 kernel 没有 tile 内的逐元素 predication，调用方需保证
  // M % BM == 0、N % BN == 0、K % BK == 0。
  // 当不启用 BlockSwizzle 时，BZ=1 退化为纯 2D grid，行为不变。
  constexpr int kSwizzleStride = 2048;
  int BX = (N + BN - 1) / BN;
  int BY = (M + BM - 1) / BM;
  int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
  BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
  dim3 block(size(MMA{}));   // 128 threads (= size(TiledMMA))
  dim3 grid(BX, BY, BZ);

  // ── Dynamic Shared Memory 大小 ──
  // kernel 用法:
  // extern __shared__ T shm_data[];
  // T *Ashm = shm_data;
  // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
  // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
  // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
  // 合计 A+B = 24576 个 T; C epilogue 复用一个 A stage (128*32 = 4096 个 T)。
  // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
  // 能放进一个 A stage (128*32 = 4096)。
  static constexpr int shm_size_AB =
      cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
  static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
  static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
                "C shared memory must fit within one A pipe");
  static constexpr int kShmSize =
      cute::max(shm_size_AB, shm_size_C) * sizeof(T);

  cudaFuncSetAttribute(
      hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
                               SmemLayoutA, SmemLayoutB, SmemLayoutC,
                               S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
                               S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
      cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);

  hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
                           SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
                           S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
                           BlockSwizzle>
      <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
}

#endif /* NOTES_V2_ENABLE_CUTE */

#if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
```

```
// TMA/mbarrier helpers.
//
// Two implementations gated by NOTES_V2_FORCE_INLINE_ASYNC_PROXY:
//   - Defined (default): raw `asm volatile` PTX. ptxas keeps setmaxnreg
//     because raw asm carries no source annotation that ptxas would treat as
//     an "extern call" boundary (warning C7506 otherwise drops the hint).
//   - Undef'd: cuda::ptx:: / cuda::device:: C++ wrappers. Cleaner API but
//     ptxas drops setmaxnreg with C7506 even though the wrappers inline to
//     identical PTX; useful for comparing behavior / debugging.
//
// cuda::barrier (init/arrive/wait) is shared by both paths: it inlines to raw
// mbarrier PTX and does not trigger C7506, so it needs no gating.
//
// Mirrors the raw-PTX style of tmp/LeetGPU/CUDA/22_GEMM/sm90_wgmma_tma_ws_pingpong.cu.
#if defined(NOTES_V2_FORCE_INLINE_ASYNC_PROXY)

static __device__ __forceinline__ uint32_t
cast_smem_ptr_to_uint(void const *ptr) {
  return static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
}

static __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
  asm volatile("fence.proxy.async.shared::cta;\n" ::: "memory");
}

// cp.async.bulk.tensor.2d: 2D TMA copy from global to shared::cluster.
//   [dst]            = shared memory destination (smem addr, 32-bit)
//   [desc, {c0, c1}]  = CUtensorMap + (minor_coord, major_coord) coords
//   [mbar]            = mbarrier in shared memory (smem addr, 32-bit)
// mbarrier::complete_tx::bytes: barrier flips phase when TMA bytes land.
// L2::cache_hint omitted (EVICT_NORMAL); matches the cuda::ptx default.
static __device__ __forceinline__ void tma_load_2d(
    void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
    cuda::barrier<cuda::thread_scope_block> &barrier) {
  uint64_t gmem_int_desc = reinterpret_cast<uint64_t>(tensor_map);
  uint32_t smem_int_ptr = cast_smem_ptr_to_uint(dst);
  uint32_t smem_int_mbar =
      cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
  asm volatile(
      "cp.async.bulk.tensor.2d.shared::cluster.global.mbarrier::complete_tx::bytes"
      " [%0], [%1, {%3, %4}], [%2];"
      :
      : "r"(smem_int_ptr), "l"(gmem_int_desc), "r"(smem_int_mbar),
        "r"(minor_coord), "r"(major_coord)
      : "memory");
}

// mbarrier.arrive.expect_tx: register one arrival on the mbarrier and declare
// that `bytes` of async copy traffic is expected before the phase flips.
// Equivalent to cuda::ptx::mbarrier_arrive_expect_tx(release, cta, shared).
static __device__ __forceinline__ void tma_arrive_expect_tx(
    cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
  uint32_t smem_int_mbar =
      cast_smem_ptr_to_uint(reinterpret_cast<uint64_t *>(&barrier));
  asm volatile(
      "mbarrier.arrive.expect_tx.shared::cta.b64 _, [%0], %1;\n"
      :
      : "r"(smem_int_mbar), "r"(bytes)
      : "memory");
}

#else // !NOTES_V2_FORCE_INLINE_ASYNC_PROXY: use cuda::ptx / cuda::device wrappers

```

```
static __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
#if CUDART_VERSION >= 13020
  cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);
#else
  cuda::device::experimental::fence_proxy_async_shared_cta();
#endif
}

static __device__ __forceinline__ void tma_load_2d(
    void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
    cuda::barrier<cuda::thread_scope_block> &barrier) {
#if CUDART_VERSION >= 13020
  const int32_t coords[]{minor_coord, major_coord};
  auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
  cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
                                  cuda::ptx::space_global, dst, tensor_map,
                                  coords, barrier_handle);
#else
  cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
      dst, tensor_map, minor_coord, major_coord, barrier);
#endif
}

static __device__ __forceinline__ void tma_arrive_expect_tx(
    cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
#if CUDART_VERSION >= 13020
  auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
  [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
      cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
      barrier_handle, bytes);
#else
  [[maybe_unused]] auto token =
      cuda::device::barrier_arrive_tx(barrier, 1, bytes);
#endif
}

#endif // NOTES_V2_FORCE_INLINE_ASYNC_PROXY

// Warpgroup-level register rebalancing for warp specialization.
// setmaxnreg.dec releases registers (producer TMA path needs few),
// setmaxnreg.inc requests registers (consumer MMA path needs many).
// Both require all warps in a warpgroup to execute the same instruction,
// and require __launch_bounds__(N, 1) so the compiler permits up to 256
// regs/warp (otherwise the hint may have no effect). Supported on sm_90a/sm_120a.
// __forceinline__ is mandatory: without it ptxas drops setmaxnreg with
// warning C7506 "ignored to maintain compatibility into 'extern' call",
// because a non-inlined call boundary forces a fixed register convention.
//
// On sm_120a (Blackwell, CUDA 13.2) ptxas drops setmaxnreg with C7506 even
// when the PTX is fully inlined (no call.uni), because ptxas treats
// cp.async.bulk.tensor (TMA) usage as an implicit extern-call boundary.
// sm_90a (Hopper) is unaffected. Gate the *call sites* with
// NOTES_V2_ENABLE_SETMAXNREGS so sm_120a builds stay warning-free and avoid
// the register-allocation side effects of __launch_bounds__(N,1) until ptxas
// is fixed. The function templates themselves are always defined.
#if defined(NOTES_V2_ENABLE_SETMAXNREGS)
  #define NOTES_V2_REG_DEALLOC(N) warpgroup_reg_dealloc<N>()
  #define NOTES_V2_REG_ALLOC(N)   warpgroup_reg_alloc<N>()
#else
  #define NOTES_V2_REG_DEALLOC(N) ((void)0)
  #define NOTES_V2_REG_ALLOC(N)   ((void)0)
#endif

```

```cpp
template <uint32_t kNumRegs>
__device__ __forceinline__ void warpgroup_reg_dealloc()
{
  asm volatile("setmaxnreg.dec.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
}

template <uint32_t kNumRegs>
__device__ __forceinline__ void warpgroup_reg_alloc()
{
  asm volatile("setmaxnreg.inc.sync.aligned.u32 %0;\n" : : "n"(kNumRegs));
}
#endif

#if defined(NOTES_V2_ENABLE_WGMMA)
// ============================================================================
// Phase 7d: HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
// ============================================================================
// 面试要点（WGMMA vs MMA 对比）：
//   - MMA: warp 级（32 threads），同步执行
//   - WGMMA: warpgroup 级（128 threads = 4 warps），异步执行（fire-and-forget）
//   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加（MMA m16n8k16的 4x16=64倍）
//   - TMA (Tensor Memory Accelerator): 硬件 DMA，2D 寻址，零寄存器开销
//   - Warp Specialization: Producer 做 TMA，Consumer 做 WGMMA，通过 barrier 同步
//   - 128B swizzle: shared memory 的 128B swizzle 模式，避免 bank conflict

// ---- WGMMA 辅助函数 ----
// wgmma.commit_group / wait_group 语义（PTX ISA §9.7.15.7）：
//   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
//     （per-warpgroup，故需 .sync.aligned 确保所有线程同步执行）。
//   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成（pending ≤ N）。
//     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致（PTX ISA §9.7.9.25.3.3），
//     都是 "wait until only N or fewer groups are pending"。
#define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
#define WGMMA_COMMIT_GROUP()                                                    \
  asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
#define WGMMA_WAIT_GROUP(n)                                                     \
  asm volatile("wgmma.wait_group.sync.aligned %0;\n" ::"n"(n) : "memory")

// SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
// 编码规则（PTX ISA §9.7.15.5.1.2.2）：
//   encoded = (x & 0x3FFFF) >> 4
// 其中 0x3FFFF = 2^18 - 1 = 262143，只保留低 18 bit。
// >>4 是因为 shared memory 地址/偏移必须 16B 对齐（2^4），低 4 bit 恒为 0，
// 可省略以扩大可表示范围。
// 解码：decoded = encoded << 4（回到原始 byte 值）。
#define SMEM_DESC_ENCODE(x) ((((uint64_t)(x)) & 0x3FFFF) >> 0x4)

// make_smem_desc: 构造 WGMMA（warpgroup MMA）的 64-bit shared memory 矩阵描述符。
// 硬件 WGMMA（如 wgmma.mma_async.m64n128k16）需要 descA/descB 两个 64-bit 描述符，
// 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
//
// 64-bit descriptor 位域布局（参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor）：
//   ------------------------------------------------------------------------
//   | 位域                | 范围      | 大小 | 含义
//   |---------------------|-----------|------|-----------------------------
//   | start_address       | [0, 14)   | 14   | smem 基址编码
//   | (unused)            | [14, 16)  |  2   | -
//   | leading_byte_offset | [16, 30)  | 14   | 主要维度（K）的字节步长；swizzle 下硬件未使用（assumed=1）
//   | (unused)            | [30, 32)  |  2   | -
//   | stride_byte_offset  | [32, 46)  | 14   | 跨越维度（M/N）的字节步长：K-Major 下为 8-row stripe 间距
//   | (unused)            | [46, 49)  |  3   | -
//   | base_offset         | [49, 52)  |  3   | swizzle pattern 偏移
//   | (unused)            | [52, 62)  | 10   | -
```

```
//   | layout_type         | [62, 64)  |  2   | swizzle 模式
//   ------------------------------------------------------------------------
//   layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
//
// K-Major + 128B swizzle + half(16-bit) 的几何推导（PTX ISA §9.7.15.5.1.2）：
//   - 128B swizzle atom 在 128-bit 单位下为 8×8
//   - 归一化到 half(2B) 后：atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
//   - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
//   - 这是 stride_byte_offset=1024 的来源
//
//   - leading_byte_offset 在 K-Major swizzle 下 unused（hardware assumes 1），
//     此处填 16 仅为占位，编码后为 1。
//
//   - base_offset=0（bits [49,52) 未显式置位），表示 smem ptr 已 1024B 对齐。
//     若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
//
// 参考：CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
// 注：descA/descB 共用此函数；对 B 而言物理存储为 [BN, BK]，详见 WgmmaSMem 注释。
__device__ inline uint64_t make_smem_desc(half *ptr) {
  // __cvta_generic_to_shared：将通用地址空间中的指针转换为 shared memory 地址
  // （一个 32-bit 的 smem byte 偏移量，相对于当前 CTA 的 shared memory 基址）。
  uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));

  // 从全零开始：所有位域初始化为 0。
  uint64_t desc = 0x0000000000000000;

  // ── bits [0, 14): start_address ──
  // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位（丢弃低 4 个 0 bit），
  // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
  // 可寻址范围：2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
  desc |= SMEM_DESC_ENCODE(addr);

  // ── bits [16, 30): leading_byte_offset ──
  // 原始值 16（字节），编码后为 (16 & 0x3FFFF) >> 4 = 1。
  // << 16 将编码值放到 bits [16, 30)。
  // K-Major + 128B swizzle 下该字段硬件未使用（assumed to be 1，参见 PTX ISA §9.7.15.5.1.2.1.1），
  // 此处填 16 仅为占位，使编码值为 1 满足硬件预期。
  // 若为 MN-Major 或 INTERLEAVE 布局，LBO 有实际含义：
  //   对 INTERLEAVE：同一 8×2 brick 内第一列到第二列的字节偏移。
  //   对 MN-Major swizzle：偏移从首 (swizzle-byte-size/16) 行到下一组行。
  desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;

  // ── bits [32, 46): stride_byte_offset ──
  // 原始值 1024（字节），编码后为 (1024 & 0x3FFFF) >> 4 = 64。
  // << 32 将编码值放到 bits [32, 46)。
  // K-Major 下定义为"从首 8 行到次 8 行的字节偏移"（PTX ISA §9.7.15.5.1.2.1.2）。
  // 1024 的来源（K-Major + 128B swizzle + half）：
  //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
  //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
  // 若为 MN-Major：SBO 是从首列到下一列的偏移（8 列一组）。
  desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;

  // ── bits [62, 64): layout_type (swizzle mode) ──
  // 1llu << 62  = 二进制 01 在 bits [62, 64) = layout_type = 1。
  // 1 = 128B swizzle mode。
  // 其他取值：0=None, 2=64B swizzle, 3=32B swizzle。
  // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射，
  // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
  desc |= 1llu << 62;

  return desc;
}
```

```
// ---- WGMMA PTX 指令宏 ----
// wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
// m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
// f16.f16.f16: A=f16, B=f16, D=f16（累加器也是 f16 精度）。
// 每线程 32 个 uint32 输出: D=64×128=8192 half，warpgroup 128 线程分担，
// 每线程 64 half = 32 uint32。
// descA/descB: shared memory 描述符（由 make_smem_desc 生成，64-bit 编码）。
// ScaleD: 0=清零累加器再写入，1=与累加器中的旧值累加（K 维迭代必须用 1）。
// ScaleA/ScaleB: 1=正常符号，-1=翻转符号（此处始终用 1）。
// TransA/TransB: 0=K-major（K 维连续），1=MN-major（M/N 维连续）。本实现始终用 0。
// 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
#define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA,  \
                                   TransB)                                     \
  {                                                                            \
    uint64_t desc_a = make_smem_desc(&(sA)[0]);                                \
    uint64_t desc_b = make_smem_desc(&(sB)[0]);                                \
    asm volatile(                                                              \
        "{\n"                                                                  \
        "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 "                 \
        "{%0,   %1,   %2,   %3,   %4,   %5,   %6,   %7,   "                    \
        " %8,   %9,   %10,  %11,  %12,  %13,  %14,  %15,  "                    \
        " %16,  %17,  %18,  %19,  %20,  %21,  %22,  %23,  "                    \
        " %24,  %25,  %26,  %27,  %28,  %29,  %30,  %31},"                     \
        " %32,"                                                                \
        " %33,"                                                                \
        " %34, %35, %36, %37, %38;\n"                                          \
        "}\n"                                                                  \
        : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]),  \
          "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]),  \
          "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]),  \
          "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]),  \
          "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]),  \
          "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]),  \
          "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]),  \
          "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3])   \
        : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)),                      \
          "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)),    \
          "n"(int32_t(TransB)));                                               \
  }

// ---- TMA Shared Memory Layout ----
// Multi-stage pipeline: kStages 个 stage，每个 stage 存储 A[BM×BK] + B[BK×BN]
//
// 每个 stage 包含两块 smem:
//   A tile: [BM, BK] row-major → BK×BM 个 half，地址连续
//   B tile: TMA 按 [BN, BK] 物理写入（详见下方 ★），BN×BK 个 half
//
// ★ 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
//
//   上层数据: 源矩阵 B^T [N, K] row-major（TN 布局，B 的列以行优先形式存储）。
//   物理存储: TMA 将 B^T 的 tile 写入 smem，物理布局为 [BN, BK] row-major
//             （BN=128 行，BK=64 列，每行 64 个连续的 half）。
//            TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape，
//            TMA 按行写入，行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
//   逻辑视图: WGMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
//            通过 imm-trans-b=0 指定 B 为 K-major，即逻辑上按 [BK, BN] 寻址。
//            实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
//            swizzle (layout_type=1) 在硬件地址重映射层完成的，无需软件转置。
//
//   stride_byte_offset=1024 的来源（K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2）:
//     128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
//     stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
//                        = 8 rows × (BK=64) halfs × 2 bytes = 1024。
```

```
//     这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
//
//   与 MMA(TN) 的对比:
//     MMA (TN):  smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
//     WGMMA:     smem 物理存 [BN, BK] (TMA 直写), WGMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
//   简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
  alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
  // B tile 注记:
  //   - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
  //   - 逻辑视图: WGMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
  //     以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
  //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
  //     但写 BN*BK 更能体现物理存储形态
  alignas(128) half B[BN * BK * QSIZE];
};

// ---- WGMMA Kernel: Warp Specialization + TMA ----
// 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
//
// 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
// 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMA 支持将工作拆分为两个
// warpgroup, 让数据搬运和矩阵乘完全异步执行:
//
//   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
//     将 A/B tile 从 HBM 搬到 shared memory。
//   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMA 矩阵乘。
//
// Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
//   - full[stage]:  Producer 发信号表示 stage stage 的数据已就绪, 可被使用
//   - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕, 可以被覆盖
//
// 本节重点理解:
//   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
//      即可搬运整个 2D tile, 无需所有线程参与。
//   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
//   3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时,
//      Producer 可以搬运 stage (stage+1), 隐藏 HBM→SMEM 延迟。
//
// Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
//   WGMMA Atom:       m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
//   K Tile:           BK=64, 每个 K tile 包含 BK/kWgmmaK=4 个 WGMMA atom
//   M Tile:           BM=128, 每个 M tile 包含 BM/kWgmmaM=2 个 WGMMA atom
//   N Tile:           BN=128, 单个 kWgmmaN=128 即可覆盖, 无需在 N 方向分块
//   Block Tile:       C[128,128] = A[128,64] × B[64,128]
//   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
//     Producer(WG0):  128 threads (仅 thread 0 做 TMA 提交)
//     Consumer(WG1):  128 threads = 4 warps (全部参与 WGMMA 和写回)
//
// kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
// 的延迟被计算完全掩盖。
//
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
//   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
// Block: (256, 1, 1), 2 warpgroups
// source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
template <const int kWgmmaM = 64,           // WGMMA atom M dim (m64n128k16)
          const int kWgmmaN = 128,          // WGMMA atom N dim
          const int kWgmmaK = 16,           // WGMMA atom K dim
          const int BM = 128,               // block tile M, 2 × kWgmmaM
          const int BN = 128,               // block tile N, 1 × kWgmmaN
          const int BK = 64,                // block tile K, 4 × kWgmmaK
          const int kNumThreads = 256,      // 2 wargroups × 128 threads
```

```
          const int kStages = 3,            // TMA pipeline depth (full/empty barriers)
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(kNumThreads, 1)
    hgemm_wgmma_stages_tn(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
        const CUtensorMap *__restrict__ tensorMapB) {
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
  // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
  // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
  // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
  // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
  // 不展开宿主侧 create_tensor_map 细节。

  // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
  // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
  const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
  // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
  // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
  constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
  // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
  // 当只有一个 consumer 时, 它负责全部 BM=128 行
  constexpr int kWarpgroupM = BM / kNumConsumers; // 128

  // 边界检查: 确保当前 block 不超出 M/N 范围
  if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
    return;

  // ---- Shared Memory 分配 ----
  // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
  // TMA + WGMMA 仅需 __align__(128), 而非 __align__(1024)。原因:
  //   make_smem_desc() 构造 WGMMA descriptor 时, start_address 字段
  //   完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量,
  //   硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移 (等价于
  //   descriptor 内置的 base_offset 机制)。因此即使 smem 基址未落到
  //   1024B 边界, WGMMA 硬件也能正确读取 TMA 写入的数据。
  // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle<64>()
  //   , 该函数无法感知 smem 绝对地址, 硬编码了 phase=0 的假设, 因此必须
  //   __align__(1024) 来保证 phase 确实为 0。从健壮性角度看本 kernel 也
  //   应该用 __align__(1024); 这里使用 128 也可以正常运行。
  extern __shared__ __align__(1024) uint8_t smem_tma_wgmma_ws[];
  WgmmaSMem<BM, BN, BK, kStages> &s =
      *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages> *>(smem_tma_wgmma_ws);
  half *s_a = s.A;
  half *s_b = s.B;

  // ---- cuda::barrier 初始化 ----
  //
  // ★ Barrier 机制速查 (arrive / wait 核心语义):
  //
  // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替) ** 工作:
  //
  //   init(bar, N): 设置 arrive_count = N。
  //     含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
  //
  //   arrive(): 线程声明"我已到达此 barrier 点"。
  //     - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
  //     - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
  //
  //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
  //     - 即: 等待 phase 翻转。
```

```
  //     - Phase 翻转条件: (1) arrive 次数达到 arrive_count
  //                       (2) 若使用了 barrier_arrive_tx，还需 TMA 字节全部写完
  //
  //   典型用法: b.wait(b.arrive())
  //     - arrive() 注册自己到达，拿到 token（当前 phase = P）
  //     - wait(token) 阻塞直到 phase 翻转（P → P+1）
  //     - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回（不阻塞）
  //     - 如果自己不是最后一个 → wait 阻塞，直到第 N 个到达者触发翻转
  //
  // ★ 本 kernel 的 Pipeline 同步协议（kStages=3, arrive_count=129）：
  //
  //   Producer (1 thread)               Consumer (128 threads)
  //   ────────────────────              ──────────────────────
  //                                      C0: arrive(empty[*]) ×128  [init: 标记所有 stage 为空]
  //   ┌─ for each k_tile: ─┐            ┌─ for each k_tile: ─┐
  //   │ P1: arrive+wait(empty[q])       │ C1: arrive+wait(full[q])
  //   │     ↓ 等 stage q 被消费完       │     ↓ 等 TMA 数据就绪
  //   │ P2: TMA(A[q]) + TMA(B[q])       │ C2: WGMMA 计算
  //   │     ↓ 异步拷贝                  │ C3: wait WGMMA 完成
  //   │ P3: arrive_tx(full[q])          │ C4: arrive(empty[q])
  //   │     ↑ 通知: stage q 数据就绪    │     ↑ 通知: stage q 已消费完
  //   └─────────────────────┘           └─────────────────────┘
  //
  //   关键不变式（invariant）：
  //     - Producer 不会覆盖 Consumer 正在读的 stage
  //     - Consumer 不会读 Producer 还没写完的 stage
  //     - 通过 full/empty 两个 barrier 的 phase 交替来保证
  //
  //   每个 stage 有两个 barrier:
  //      full[stage]: TMA 数据就绪信号。Producer 发（arrive_tx），Consumer 等（wait）。
  //     empty[stage]: Stage 空闲信号。 Consumer 发（arrive），   Producer 等（wait）。
  //
  //   arrive_count = 128 (consumer) + 1 (producer) = 129:
  //     每 phase，128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
#pragma nv_diag_suppress static_var_with_dynamic_init
  __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
#pragma nv_diag_default static_var_with_dynamic_init

  // K 方向总 tile 数。要求 K 能被 BK 整除，否则尾 tile 被丢弃。
  const int NUM_K_TILES = div_ceil(K, BK);
  const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
  const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup

  // 初始化 barriers（仅 thread 0 执行）
  if (threadIdx.x == 0) {
    for (int i = 0; i < kStages; ++i) {
      // arrive_count = 129: 128 consumer + 1 producer
      // init 是 CUDA barrier 的 hidden friend 函数（定义在 cuda::barrier 类体内），
      // 只能通过 ADL（Argument-Dependent Lookup）调用。因为第一个参数 &full[i] 的类型是
      // cuda::barrier<cuda::thread_scope_block>*，编译器通过 ADL 在 cuda 命名空间中自动找到它。
      init(&full[i], kConsumerThreads + 1);  // 128 consumer + 1 producer
      init(&empty[i], kConsumerThreads + 1); // same
    }
    // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
    // 对 async proxy（TMA/WGMMA）可见。
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  // ======================================================================
  // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
  // ======================================================================
```

```
  //
  // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工，不是先后顺序：
  //   - WG0 (threadIdx.x 0~127):   全部走 if 分支，做 Producer。
  //   - WG1 (threadIdx.x 128~255): 全部走 else 分支，做 Consumer。
  //
  // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp，两者**同时**推进：
  //   Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 }  ← 自己的循环
  //   Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
  //
  // 两个 warpgroups 各自独立迭代 K-tiles，通过 full[] / empty[] barrier 同步：
  //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞，等 Consumer 消费完
  //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞，等 TMA 搬运完
  //
  // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
  // 交替天然保证了流水线串行化（at most kStages-1 steps ahead）。
  // ============================================================
  // Producer Warpgroup (WG0, threadIdx.x 0~127)
  // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
  // 只有 wg_tid==0 执行实际拷贝提交，其余 127 个线程空闲。
  // ============================================================
  if (wg_idx == 0) {
    NOTES_V2_REG_DEALLOC(40);
    if (wg_tid == 0) {
      // stage: 当前操作的 stage 索引（round-robin 0 -> 1 -> 2 -> 0 -> ...）
      int stage = 0;
      for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
        if (stage == kStages)
          stage = 0;

        // Step P1: 等待 stage stage 变为"空"（可被覆盖写入）
        //
        // 模式 empty[stage].wait(empty[stage].arrive()):
        //   - arrive(): Producer（1 个线程）在 empty barrier 上注册到达，
        //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
        //     128 次 arrive（来自上一轮迭代或 C0 初始化），则加上这次共 129 次
        //     → phase 立即翻转。
        //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
        //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
        //     wait 看到 phase 已变 → 立即返回（首轮不阻塞）。
        //
        // 语义：Producer 说"我准备好写 stage stage 了，Consumer 用完没有？"
        //       如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
        //       如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
        empty[stage].wait(empty[stage].arrive());

        // Step P2: 提交 TMA 2D 拷贝指令
        // cp_async_bulk_tensor_2d_global_to_shared:
        //   参数1(dst): smem 目标地址（当前 stage 的 A/B tile 起始位置）
        //   参数2(tensorMap): Host 预创建的 TMA descriptor（描述源矩阵的 shape/stride/dtype）
        //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
        //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
        //
        // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile，
        // 无需线程逐元素搬运，零寄存器开销。
        // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
        tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
                    full[stage]);

        // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
        tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
                    full[stage]);

        // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
```

```
        //
        // barrier_arrive_tx(bar, arrive_count_update, byte_count):
        //   - 在 bar 上注册 1 次到达（arrive_count_update=1）
        //   - 同时声明预期有 byte_count 字节将通过 async copy（TMA）写入 smem
        //   - Phase 翻转条件：
        //       (a) 总 arrive 次数达到 129（128 consumer + 1 producer）
        //       (b) 所有声明的 async 字节已写入 smem
        //     两个条件都满足后 phase 才翻转，Consumer 的 wait() 才返回。
        //   - 即：Consumer 不会在 TMA 数据完整到达前就开始读 smem。
        tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
      }
    }
  }
  // ================================================================
  // Consumer Warpgroup (WG1, threadIdx.x 128~255)
  // 职责：等待 TMA 数据就绪 -> 发射 WGMMA 做矩阵乘 -> 积累结果 -> 写回 C
  // 所有 128 个线程（4 warps）全部参与。
  // ================================================================
  else {
    // Step C0: Consumer 初始化 — 标记所有 stage 为"空"（可被 Producer 写入）
    //
    // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
    // 这是 Pipeline 的"预热"步骤——没有它，Producer 的 empty[stage].wait()
    // 在第一轮会永远阻塞（因为 Producer 的 1 次 arrive 不足以凑够 129）。
    //
    // 注意：此时每个 empty[i] 只有 128 次 arrive，未达 129，phase 不翻转。
    // Producer 后续的 empty[stage].arrive() 作为第 129 次，触发 phase 翻转。
    NOTES_V2_REG_ALLOC(232);

    for (int i = 0; i < kStages; ++i) {
      [[maybe_unused]] auto token = empty[i].arrive();
    }

    // 累加器寄存器声明
    // d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4]:
    //   - d[0][*][*]: M 方向第 1 个 WGMMA atom（rows 0~63）
    //   - d[1][*][*]: M 方向第 2 个 WGMMA atom（rows 64~127）
    //   - d[*][g][*]: N 方向第 g 组 16 列
    //   - d[*][*][0..3]: 4 条 uint32 寄存器，共 8 个 half（覆盖 16×16 子块）
    // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half（每次WGMMA Atom 32 uint32 输出）
    // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN（刚好覆盖整个 C tile）
    uint32_t d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4] = {};

    int stage = 0;
    // K 维外循环：沿 K tile 迭代（BK=64，每个 K tile 做 4 次 WGMMA 累加）
    for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
      if (stage == kStages)
        stage = 0;

      // Step C1: 等待 TMA 数据就绪（full 信号）
      //
      // 模式 full[stage].wait(full[stage].arrive()):
      //   - 128 个 Consumer 线程各调用 arrive()，共 128 次到达。
      //   - 当前 phase 累积：128/129，phase 尚未翻转。
      //   - wait(token) 阻塞，直到 Producer 的 barrier_arrive_tx（Step P3）
      //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
      //
      // 语义：Consumer 说"我准备好读 stage stage 了，数据到了没有？"
      full[stage].wait(full[stage].arrive());

      // Step C2: 发射 WGMMA 指令序列
      //
      // WGMMA 是异步指令（fire-and-forget），发射后立即返回，不等待计算完成。
```

```
      // 标准流程: FENCE → 发射 WGMMA → COMMIT → WAIT。
      //
      // WGMMA_FENCE (wgmma.fence.sync.aligned) :
      //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMA) 可见
      //   - 确保累加器寄存器 d[] 已准备好接收 WGMMA 输出
      //   - 本质是 proxy 间的内存序 (memory ordering) , 不是传统 __syncthreads
      //
      // 每个 WGMMA_M64N128K16_F16F16F16 指令:
      //   D[64×128] += A[64×16] × B[16×128]
      //   A/B 均为 K-major (imm-trans=0) , 由 descA/descB 描述 smem 中的布局。
      //   ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都用 ScaleD=1。
      WGMMA_FENCE();

      // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMA atom
      // 每个 atom 处理 64 行 M 方向数据。
#pragma unroll
      for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
        // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
        // s_a 布局: [kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
        half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;

        // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMA (累加)
        // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
#pragma unroll
        for (int k_step = 0; k_step < BK / kWgmmaK; ++k_step) {
          // 第 k_step 次 WGMMA:
          //   A: wgmma_sA + k_step * kWgmmaK (A 的 K 维起始位置)
          //   B: s_b + stage * BK * BN + k_step * kWgmmaK (B 的 K 维起始位置)
          //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
          //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
          //   ScaleD=1: 累加 (K 维迭代需要累积结果)
          WGMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWgmmaK,
                                     s_b + stage * BK * BN + k_step * kWgmmaK,
                                     1, // ScaleD=1: accumulate (不清零)
                                     1, 1, 0, 0);
        }
      }

      // Step C3: 提交并等待 WGMMA 完成
      //
      // WGMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
      //   将自上一次 COMMIT 以来发射的所有 WGMMA 归为一组, 提交到 async proxy 执行。
      //   类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
      //
      // WGMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
      //   阻塞直到最多 N 个 group 尚未完成 (pending ≤ N) 。N=0 即等待所有 group。
      //   ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
      //     两者都是 "wait until only N or fewer of the most recent groups are pending"。
      //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
      //   必须等它全部完成才能进入下一步 (读下一 stage 的数据) 。
      WGMMA_COMMIT_GROUP();
      WGMMA_WAIT_GROUP(0);

      // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
      //
      // 128 个 Consumer 线程在 empty[stage] 上 arrive(), 为 **下一 phase** 积累到达次数。
      // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive) 。
      //
      // 语义: Consumer 说"stage stage 我已经读完了, 你可以放心覆盖了。"
      [[maybe_unused]] auto token = empty[stage].arrive();
    }

    // ================================================================
```

```
    // Epilogue: 将寄存器中的累加结果写回 global memory C
    // ================================================================
    //
    // WGMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
    //
    // 对 warpgroup 内每个线程，输出 64 个 half = 32 个 uint32。
    // 这些 half 分布在 d[2][8][4] 数组中:
    //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
    //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
    //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
    //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
    //
    // 其中:
    //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
    //   col = g * 16 + 2 * (lane % 4)  (0~126, step 2, 8 g's * 16 cols)
    //
    // 每个线程覆盖一个 16x16 子块中的 16 个 half:
    //   +-------+-------+
    //   | reg[0] | reg[2] |  <- rows [row, row+8)
    //   |(col,+2)|(col+8)|
    //   +-------+-------+
    //   | reg[1] | reg[3] |  <- rows [row+8, row+16)
    //   |(col,+2)|(col+8)|
    //   +-------+-------+
    //   每个 reg 包含 2 个连续 half（col 和 col+2），用 uint32 存储。
    //
    // 三维遍历:
    //   m_it=0: rows 0~63,  m_it=1: rows 64~127
    //   g=0..7: 每个覆盖 16 列，8*16=128 列 ok
    // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
    // 128 线程 * 128 half = 16384 half = 128 * 128 ok

    const int lane = wg_tid % 32;
    const int warp = wg_tid / 32;
    // row: 当前线程在当前 WGMMA atom 中负责的起始行。
    // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
    // lane/4: 每 4 个连续 lane 负责同一行（因为 WGMMA fragment 中每行有 4 个 uint32，
    //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}，由 lane%4 区分）。
    //         因此 lane/4 给出该行在 16-row block 内的行号（0~7 对应 rows 0~7，
    //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3，都负责 row 0）。
    const int row = warp * 16 + lane / 4;
    // block_C: 当前 C tile 在 global memory 中的起始地址
    // by*BM 是 M 方向偏移，bx*BN 是 N 方向偏移
    half *block_C = C + by * BM * N + bx * BN;

    // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
    // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
#pragma unroll
    for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
      int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
#pragma unroll
      for (int g = 0; g < kWgmmaN / 16; ++g) {
        int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
        // 一次 uint32 store 写入 2 个 half（连续列 col 和 col+2）
        // 左上象限: (row, col)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col]) = d[m][g][0];
        // 左下象限: (row+8, col)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
        // 右上象限: (row, col+8)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
        // 右下象限: (row+8, col+8)
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
      }
```

```
    }
  }
}

#endif /* NOTES_V2_ENABLE_WGMMA */

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// SM120 不支持 WGMMA，但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
// 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
  static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
  half A[BM * BK * QSIZE];
  half B[BN * BK * QSIZE];
};

// 消费者 MMA 层级映射参考 hgemm_mma_stages_tn_swizzle。与那个
// 八 warp kernel 不同，本 warp-specialized 消费者仅拥有四个 warp。
template <const int kMmaM = 16,             // MMA atom M dim (m16n8k16)
          const int kMmaN = 8,              // MMA atom N dim
          const int kMmaK = 16,             // MMA atom K dim
          const int kMmaTileM = 2,          // consumer warps along M, warp tile M = 32
          const int kMmaTileN = 2,          // consumer warps along N, warp tile N = 16
          const int kValTileM = 4,          // value-repeat along M, BM = 16*2*4 = 128
          const int kValTileN = 8,          // value-repeat along N, BN = 8*2*8 = 128
          const int kValTileK = 4,          // MMA_K slices per BK tile, BK = 16*4 = 64
          const int kStages = 2,            // TMA full/empty pipeline depth
          const int kNumThreads = 256,      // 128 producer + 128 consumer threads
          const int kBlockSwizzle = 0>      // 1 enables 3D grid swizzle for L2 locality
__global__ void __launch_bounds__(kNumThreads)
    hgemm_tma_mma_ws_tn(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
        const CUtensorMap *__restrict__ tensorMapB) {

  constexpr int BM = kMmaM * kMmaTileM * kValTileM;
  constexpr int BN = kMmaN * kMmaTileN * kValTileN;
  constexpr int BK = kMmaK * kValTileK;
  static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
  static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
  static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
  static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
  static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");

  const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int kConsumerThreads = kNumThreads / 2;
  static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
                "Consumer threads must cover the MMA warp grid");

  if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
    return;

  // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐，
  // 以确保硬件 swizzle phase 从零开始。消费者 swizzle<64>()
  // 假设零 phase；其他对齐（如 128）会导致 phase 偏移，产生不匹配的
  // 物理地址，破坏整个输出。
  //
  // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128)，而本 kernel
  // 必须 __align__(1024)？核心区别在于消费者如何感知 swizzle phase：
  //
  // WGMMA 消费者：通过 make_smem_desc() 构造 64-bit WGMMA descriptor，
  //   其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
  //   1024B 边界的偏移量（(addr >> 7) & 0x7）。硬件在读取 smem 时会
```

```
  //   用这个 base_offset 自动补偿 swizzle phase，因此 WGMMA 路径不
  //   需要 1024 字节对齐也能得到正确数据。
  //
  // 本 kernel 消费者：使用 ldmatrix + 手写 swizzle<64>() 来计算
  //   smem 物理地址。这个函数是纯软件公式，没有任何 base_offset
  //   补偿机制。它假设 smem 基址恰好落在 1024B 边界上（phase=0）。
  //   如果基址只满足 128B 对齐，TMA 硬件会以非零 phase 写入数据，
  //   而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
  //
  // 简言之：WGMMA 用硬件 descriptor base_offset 自动解决 phase 偏移；
  // ldmatrix 路径没有等价机制，必须靠 1024B 对齐来保证 phase=0。
  extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
  TmaMmaWSSMem<BM, BN, BK, kStages> &s =
      *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
  half *s_a = s.A;
  half *s_b = s.B;

#pragma nv_diag_suppress static_var_with_dynamic_init
  __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
#pragma nv_diag_default static_var_with_dynamic_init

  const int NUM_K_TILES = div_ceil(K, BK);
  const int wg_idx = threadIdx.x / kConsumerThreads;
  const int wg_tid = threadIdx.x % kConsumerThreads;

  if (threadIdx.x == 0) {
    for (int stage = 0; stage < kStages; ++stage) {
      init(&full[stage], kConsumerThreads + 1);
      init(&empty[stage], kConsumerThreads + 1);
    }
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  if (wg_idx == 0) {
    // 生产者 Warpgroup（wg_idx==0）
    NOTES_V2_REG_DEALLOC(40);
    if (wg_tid == 0) {
      int stage = 0;
      for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
        if (stage == kStages)
          stage = 0;

        empty[stage].wait(empty[stage].arrive());

        tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
                    full[stage]);
        tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
                    full[stage]);
        tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
      }
    }
  } else {
    // 消费者 Warpgroup（wg_idx==1）
    // 初始化：标记所有 stage 为"空"（可被 Producer 写入）
    NOTES_V2_REG_ALLOC(232);

    for (int stage = 0; stage < kStages; ++stage) {
      [[maybe_unused]] auto token = empty[stage].arrive();
    }

    const int warp_id = wg_tid / kWarpSize;
```

```
    const int lane_id = wg_tid % kWarpSize;
    // WG1 内形成 2x2 warp grid，warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
    const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
    const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
    uint32_t RA[kValTileM][4];
    uint32_t RB[kValTileN][2];
    uint32_t RC[kValTileM][kValTileN][2] = {0};

    const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
    const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
    int stage = 0;
    for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
      if (stage == kStages)
        stage = 0;

      full[stage].wait(full[stage].arrive());
      tma_fence_proxy_async_shared_cta();

      // 与 cp.async pipeline（hgemm_mma_stages_tn 等）不同，这里 ldmatrix +
      // mma.sync 之前/后 不需要 __syncthreads，原因在于 TMA + async proxy 的
      // 同步机制与 warp specialization 的线程分工：
      //
      // 1. 生产者-消费者解耦：TMA 拷贝由 producer（wg_idx==0 的单个线程）
      //    通过 tma_load_2d + tma_arrive_expect_tx 提交，而非所有 256 个线程
      //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
      //    拷贝都完成后再进入计算；而 TMA 路径用 cuda::barrier 的 full/empty
      //    协议完成 CTA 级同步——full[stage].wait() 返回时，producer 已通过
      //    expect_tx 声明了完整的传输字节数，且硬件保证这些字节已全部写入 smem。
      //
      // 2. async proxy fence 替代 __syncthreads：fence_proxy_async(space_shared)
      //    在 async proxy（TMA 写入通道）和后续 smem load（ldmatrix 读取通道）
      //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作（TMA）
      //    对 fence 之后的所有 smem 读操作（ldmatrix）可见。这是一个轻量级
      //    proxy 间 ordering，不需要阻塞线程，也不会让 warp 等待其他 warp。
      //
      // 3. 消费者内部 warp 间无依赖：WG1 内的 4 个 warp（2×2 grid）各自独立
      //    读取 smem 中互不重叠的 A/B 区域（warp_m 和 warp_n 分别索引不同的
      //    行区间）。它们之间不需要读写共享数据，因此不需要 __syncthreads 来
      //    对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令，保证同一
      //    warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。

      // 注意：这里已经没有NUM_K_TILES循环了，因为K loop load已经被WG0生产者处理了
#pragma unroll
      for (int k_step = 0; k_step < kValTileK; ++k_step) {

#pragma unroll
        for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
          // kMmaM * kValTileM = 16*4 = 64，每个消费者 warp 在 M 方向覆盖 64 行。
          const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
          const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
          // 与 hgemm_mma_stages_tn_swizzle 不同：k_step * kMmaK 必须放在
          // lane_smem_a_k 中传给 swizzle<64>()，而非像 swizzle kernel
          // 那样作为 smem_k_offset 加在 swizzle 外部。原因：
          //   swizzle<64> 作用于完整 BK=64，swizzle 周期覆盖全部 64 列，
          //   k_step=0 和 k_step=1 的 chunk 会被 XOR 交叉混合 → k_step
          //   偏移必须在 swizzle 内部参与 chunk 计算。
          //   swizzle<kMmaK> 作用于 kMmaK=16，每个 kMmaK slice 独立
          //   swizzle，slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
          const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
          const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
              (stage * BM * BK + lane_smem_a_m * BK +
              swizzle<64>(lane_smem_a_m, lane_smem_a_k)) *
                  sizeof(half);
```

```
          LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
        }

#pragma unroll
        for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
          // kMmaN * kValTileN = 8*8 = 64, 每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
          const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
          const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
          const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
          const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
              (stage * BN * BK + lane_smem_b_n * BK +
              swizzle<64>(lane_smem_b_n, lane_smem_b_k)) *
                  sizeof(half);
          LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
        }

#pragma unroll
        for (int i = 0; i < kValTileM; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileN; ++j) {
            HMMA16816(RC[i][j][0], RC[i][j][1],
                      RA[i][0], RA[i][1], RA[i][2], RA[i][3],
                      RB[j][0], RB[j][1],
                      RC[i][j][0], RC[i][j][1]);
          }
        }
      }
      [[maybe_unused]] auto token = empty[stage].arrive();
    }

    // Epilogue: 复用 RA[0] 和 RA[1]（已在寄存器中）作为 shuffle 缓冲，
    // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
    // lane 各持有两个 uint32 fragment; shuffle 汇聚为每行一个 float4，
    // 由 lane 0 发出对齐的 128-bit store。
#pragma unroll
    for (int i = 0; i < kValTileM; ++i) {
      const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
      const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
#pragma unroll
      for (int j = 0; j < kValTileN; ++j) {
        const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
        RA[0][0] = RC[i][j][0];
        RA[1][0] = RC[i][j][1];
        RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
        RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
        if (lane_id % 4 == 0) {
          const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
          const int store_gmem_c_addr_0 =
              store_lane_gmem_c_m * N + store_lane_gmem_c_n;
          const int store_gmem_c_addr_1 =
              (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RA[0][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RA[1][0]);
        }
      }
    }
  }
```

```
}

#endif /* NOTES_V2_ENABLE_TMA_MMA_WS */

#if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// ---- Host-side TMA Tensor Map helpers (WGMMA test) ----
// 面试要点（TMA descriptor 创建 — cuTensorMapEncodeTiled）：
//   - TMA descriptor 描述 global memory 中矩阵的 shape/stride/dtype，
//     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
//   - 关键易错点：TMA shape 参数写的是 (W,H) 而不是 (H,W)！
//     对 row-major [M,K] 矩阵，TMA shape = (K,M)，minor=K，major=M
//   - cuTensorMapEncodeTiled 是 CUDA Driver API，需 #include <cuda.h> 并链接 -lcuda
//   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
//   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMA 的 128B swizzle
//
// ★ gmem_prob_stride + 1 的含义：
//   语法层面 — 数组名退化 + 指针算术：
//     gmem_prob_stride 类型为 uint64_t[5]，在表达式中退化为 uint64_t*
//     （指向首元素 gmem_prob_stride[0]）。+ 1 偏移 1 个 uint64_t，指向
//     gmem_prob_stride[1]，等价于 &gmem_prob_stride[1]。
//   语义层面 — 跳过隐式的最内维 stride：
//     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
//     stride 值。最内维（fastest-changing dimension）的 stride 是隐式的，
//    固定等于 sizeof(element_type)，不需要显式传入。
//     对于 tensorRank=2 的 FP16 矩阵：
//       dim 0（内维，K）：stride = sizeof(half)    ← 隐式，API 不需要
//       dim 1（外维，M）：stride = sizeof(half)*K   ← 需要传给 API
//     gmem_prob_stride 数组的布局是内维在前：
//       [0] = sizeof(half)                              ← 内维，不传
//       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维，传给 API
//   所以 + 1 的作用是跳过 [0]，让 API 从 [1] 开始读取，正好得到外维 stride。
//
// ★ smem_box_stride 为什么不需要 +1？
//   与 globalStrides 不同，smemBoxStrides 参数接收完整的 tensorRank 个 stride，
//   最内维 stride 也必须显式传入（不隐式）。
//   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
//   都是连续存放的，维度间没有 padding/stride。
//   对于 tensorRank=2：strides[0]=1（内维 stride），strides[1]=1（外维 stride），
//   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j（元素连续排列）。
//   所以这里直接传 smem_box_stride（即 &smem_box_stride[0]），API 会读到全部
//   两个 stride 值，不需要跳过任何一个。
//
// 参考：CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor

template <int BlockMajorSize, int BlockMinorSize>
__host__ static inline void create_tensor_map(CUtensorMap *tma_map,
                                              half *gmem_ptr,
                                              int blocks_height,
                                              int blocks_width) {
  void *gmem_address = (void *)gmem_ptr;
  uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
                                 (uint64_t)BlockMajorSize * blocks_height,
                                 1, 1, 1};
  uint64_t gmem_prob_stride[5] = {
      sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
  uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
                                uint32_t(BlockMajorSize), 1, 1, 1};
  uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
  CUresult result = cuTensorMapEncodeTiled(
      tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
      gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
      CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
      CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
```

```
  if (result != CUDA_SUCCESS)
    printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
}

// BlockMajorSize = major dim (outer), BlockMinorSize = minor dim (inner, contiguous).
// For row-major [Major, Minor] matrices.
// Default <128, 64> matches HGEMM A/B^T and FA Q; FA K/V use <64, 64>.
template <int BlockMajorSize = 128, int BlockMinorSize = 64>
__host__ static inline CUtensorMap *allocate_and_create_tensor_map(
    half *src, int blocks_height, int blocks_width) {
  CUtensorMap *tma_map_d;
  cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
  CUtensorMap tma_map_host;
  create_tensor_map<BlockMajorSize, BlockMinorSize>(&tma_map_host, src,
                                                    blocks_height,
                                                    blocks_width);
  cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
             cudaMemcpyHostToDevice);
  return tma_map_d;
}
#endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */

// ================================================================================
// Phase 8: FlashAttention-2  + MMA m16n8k16)
// ================================================================================
// 面试要点（FlashAttention-2算法）：
//   1. 核心问题：标准 Attention 的 O(N^2) 中间矩阵 (S=QK^T) 必须写入 HBM，
//      但 HBM 带宽是瓶颈 → FlashAttention-2用 tiling + online softmax 避免写回
//   2. FA 三板斧：
//      a) Tiling：Q 分块 [Br,d]，K/V 沿 seqlen 分块 [Bc,d]
//      b) Online Softmax：迭代更新 m(行max) 和 l(行sum)，无需全局同步
//      c) Recomputation(反向)：反向传播时重新计算 S/P，而非存储中间矩阵
//   3. Split-Q 设计：所有 warp 共享同一块 K，各 warp 处理 Q 的不同行片段
//      - warp_KV=0（所有 warp 共享 K），warp_QP=warp_id（各 warp 不同 Q 行）
//      - 优点：减少 warp 间通信和 shuffle
//   4. Online rescaling 公式（FA 核心，arXiv:2307.08691）：
//      for each K,V tile:
//        S_cur = Q @ K^T                          // 未缩放，存入 R_S
//        m_new = max(m_old, row_max(S_cur * scale))
//        P_cur = exp(S_cur * scale - m_new)       // ← 写回 R_S 寄存器!
//        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
//        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
//      O_final = O_new / l_final
//   5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
//      - R_S 经过 softmax 后，存储的是 P = exp(S - m)，数据仍然是 half 精度
//      - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局，使 softmax
//        后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
//      - 这是此实现的寄存器布局复用技巧，不要背成“所有 MMA A/C fragment
//        都天然同构”的通用结论
//
// 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
// 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
// Grid:  ((N + 63) / 64, B * H, 1), Br=64
// Block: (128, 1, 1), kNumThreads=kWarpSize×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
// source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu

// ================================================================================
// FlashAttention-2 Split-Q Kernel（完整实现）
// ================================================================================
// Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
//
// Tile 设计（以 kHeadDim=64 为例）：
//   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*{4,8}*1 = {64,128}
```

```
//   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8  = 64
//   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
//
// 执行流程:
//   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
//   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
//   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
//   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
//   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
//          → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
//   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
//          → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
//   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
//   8) 最终 rescale: O_final = (1/l_final) * O_final
//   9) Epilogue: warp shuffle + 128-bit collective store
//
// ★ Pad 与手动 XOR swizzle 的 profile 结论 (SM120, B=1,H=32,N=4096,D=64) :
//   - XOR 能消除目标 ldmatrix lane pattern 的 bank conflict, 但收益是局部的;
//     XOR 本身及 `% 2` 的计算不是主要瓶颈, nvcc 已能将该式化简为 bit operations。
//   - compact Q/K/V XOR 改变了 cp.async 的 shared destination pattern: LDGSTS
//     wavefronts 从 pad 的 30.15M 增至 68.16M (2.26x), long-scoreboard、
//     LG-throttle、MIO-throttle 分别约为 pad 的 3.25x、2.74x、1.60x。
//   - compact XOR 虽节省约 5 KiB smem, 但没有提高此 kernel 的 occupancy; 最终
//     约 120.0 TFLOPS, 显著低于 Q/K/V kPad=8 的 166.6 TFLOPS。
//   - 所以当前 kernel 默认对 Q/K/V 都用 kPad=8。保留 swizzle 路径是为了学习和消融: 评价
//     shared layout 必须同时观察 ldmatrix reads 与 cp.async/LDGSTS writes, 不能只看
//     bank-conflict counter, 也不能只优化 XOR 地址算术。
// ---- 寄存器填充辅助函数 (FlashAttention 用, 前移以供 TMA_MMA_WS kernel 使用) ----
template <typename T, int M, const int N, const int K = 2>
__device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
#pragma unroll
      for (int k = 0; k < K; ++k)
        R[i][j][k] = val;
}

template <typename T, int M, const int N = 2>
__device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
      R[i][j] = val;
}

template <
    const int kHeadDim,          // head dim: 32, 64, 128
    const int kMmaAtomM,         // 16 (MMA instruction M dimension)
    const int kMmaAtomN,         // 8  (MMA instruction N dimension)
    const int kMmaAtomK,         // 16 (MMA instruction K dimension)
    const int kMmaAccF32,        // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
    const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
    const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
    const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
    const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
    const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
    const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
    const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
    const int kStagesK,          // pipeline stages for K: >= 1; NO stages required for Q/V
```

```
    const int kPadQ,             // Q row padding; 0 selects compact XOR swizzle
    const int kPadK,             // K row padding; 0 selects compact XOR swizzle
    const int kPadV>             // V row padding; 0 selects compact XOR swizzle
__global__ void __launch_bounds__(kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK)
flash_attn_mma_stages_split_q(
  half *Q, half *K, half *V, half *O, int N, int H) {
  static_assert(kStagesK >= 1, "kStagesK must be >= 1");
  static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
  static_assert(kPadQ >= 0 && kPadK >= 0 && kPadV >= 0,
                "Q/K/V padding must be non-negative");
  constexpr int kNRegs = kMmaAccF32 ? 4 : 2;
  constexpr bool kSwizzleQ = kPadQ == 0;
  constexpr bool kSwizzleK = kPadK == 0;
  constexpr bool kSwizzleV = kPadV == 0;
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 16*8*1=128
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 8*1*8=64
  constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 32*8*1=256
  const int Tc = (N + Bc - 1) / Bc;
  // 原始实现默认 seqlen 与 Bc 对齐；最后一个不完整 tile 需要额外 pad/边界处理。
  // 这里保留 ceil 写法是为了说明 tile 划分方式，不等于当前实现已经完整处理了尾 tile。
  const float scale = 1.0f / sqrtf((float)kHeadDim);

  // Block indexing
  const int Nb_id = blockIdx.y / H; // batch id
  const int Nh_id = blockIdx.y % H; // head id
  const int Q_tile_id = blockIdx.x; // tile id along Q's M dim (Br)
  const int tid = threadIdx.x;
  const int warp_id = tid / kWarpSize;
  const int lane_id = tid % kWarpSize;
  const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
  const int warp_KV = 0; // 所有 warp 共享 K（减少跨 warp 通信）

  // Global memory base offsets for this (batch, head)
  // 这里默认 Q/K/V 共享同一 per-head 基址布局，对应 self-attention 场景
  const int Q_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
  const int K_gmem_offset = Q_gmem_offset;
  const int V_gmem_offset = Q_gmem_offset;
  const int O_gmem_offset = Q_gmem_offset;

  // Thread-to-smem mapping for cooperative load
  int load_smem_Q_Br = tid / (kNumThreads / Br);
  int load_smem_Q_d = (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
  int load_smem_K_Bc = tid / (kNumThreads / Bc);
  int load_smem_K_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
  int load_smem_V_Bc = tid / (kNumThreads / Bc);
  int load_smem_V_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));

  int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
  if (load_gmem_Q_Br >= N)
    return;

  // Shared memory layout
  // Q/K/V independently use padded row-major when kPad* > 0 and compact XOR
  // swizzle when kPad* == 0. Padding and XOR are never combined per operand.
  // The swizzled physical layout is [col / 16][row][16]; swizzle<16>() selects
  // the 0/8 phase inside the final 16-half tile.
  extern __shared__ half smem[];
  constexpr int Q_tile_size = Br * (kHeadDim + kPadQ);
  constexpr int K_tile_size = Bc * (kHeadDim + kPadK);
  constexpr int kSmemStrideQ = kHeadDim + kPadQ;
  constexpr int kSmemStrideK = kHeadDim + kPadK;
  constexpr int kSmemStrideV = kHeadDim + kPadV;
  half *Q_tile_smem = smem;
```

```
  half *K_tile_smem = Q_tile_smem + Q_tile_size;
  half *V_tile_smem = K_tile_smem + kStagesK * K_tile_size;

  uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
  uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
  uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);

  // Online Softmax persistent state
  // lane_block_row_max_old[i][r]: running max for row r of warp tile i
  // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
  float lane_block_row_max_old[kValTileSeqLenQ][2];
  float lane_block_row_sum_old[kValTileSeqLenQ][2];
  fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
  fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

  uint32_t R_Q[kValTileSeqLenQ][4];  // Q regs
  uint32_t R_K[kValTileSeqLenK][2];  // K regs
  uint32_t R_V[kValTileHeadDimV][2]; // V regs
  // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
  // f16 acc: 每 tile 2 个 uint32 (4 half)；f32 acc: 每 tile 4 个 uint32 (4 f32)。
  // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
  uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs]; // S=Q@K^T / P=softmax(S)
  uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O for current tile
  uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs]; // O accumulator (final output)

  fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
  fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);

  // ========================================================================
  // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
  // ========================================================================
  {
    int load_gmem_Q_addr =
        Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
    if constexpr (kSwizzleQ) {
#pragma unroll
      for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
        int col = load_smem_Q_d + i;
        uint32_t ptr = smem_Q_base_ptr +
            ((col / kMmaAtomK) * Br * kMmaAtomK +
             load_smem_Q_Br * kMmaAtomK +
             swizzle<kMmaAtomK>(load_smem_Q_Br, col % kMmaAtomK)) *
                sizeof(half);
        CP_ASYNC_CG(ptr, &Q[load_gmem_Q_addr + i], 16);
      }
    } else {
      uint32_t load_smem_Q_ptr =
          smem_Q_base_ptr +
          (load_smem_Q_Br * kSmemStrideQ + load_smem_Q_d) * sizeof(half);
#pragma unroll
      for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
        CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
      }
    }
    CP_ASYNC_COMMIT_GROUP();
  }

  // ========================================================================
  // Step 2: 预加载前 (kStagesK-1) 个 K tile (多 stage pipeline 预热)
  // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
  // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
  // ========================================================================
  if constexpr (kStagesK > 1) {
```

```
#pragma unroll
    for (int stage = 0; stage < (kStagesK - 1); ++stage) {
      int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
      int load_gmem_K_addr =
          K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
      if constexpr (kSwizzleK) {
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          int col = load_smem_K_d + i;
          uint32_t ptr = smem_K_base_ptr +
              (stage * K_tile_size +
               (col / kMmaAtomK) * Bc * kMmaAtomK +
               load_smem_K_Bc * kMmaAtomK +
               swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
            sizeof(half);
          CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
        }
      } else {
        uint32_t load_smem_K_ptr =
            smem_K_base_ptr +
            (stage * K_tile_size + load_smem_K_Bc * kSmemStrideK +
             load_smem_K_d) *
                sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
        }
      }
      CP_ASYNC_COMMIT_GROUP();
    }
    CP_ASYNC_WAIT_GROUP(kStagesK - 2);
    __syncthreads();
  }

  // ================================================================
  // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
  //   每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
  // ================================================================
#pragma unroll 1
  for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
    int smem_sel = tile_K_seqlen % kStagesK;
    int smem_sel_next = (tile_K_seqlen + (kStagesK - 1)) % kStagesK;

    // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
    // kStagesK>1: V 加载当前 tile, K 预取下一 tile (pipeline)
    // kStagesK=1: V 和 K 都加载当前 tile (smem_sel==smem_sel_next==0),
    //   无 pipeline 预取, 每轮 QK 前必须 wait 当前 K 就绪。
    // Load current V tile (no pipeline for V — one stage is enough)
    {
      int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
      int load_gmem_V_addr =
          V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
      if constexpr (kSwizzleV) {
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          int col = load_smem_V_d + i;
          uint32_t ptr = smem_V_base_ptr +
              ((col / kMmaAtomK) * Bc * kMmaAtomK +
               load_smem_V_Bc * kMmaAtomK +
               swizzle<kMmaAtomK>(load_smem_V_Bc, col % kMmaAtomK)) *
                  sizeof(half);
          CP_ASYNC_CG(ptr, &V[load_gmem_V_addr + i], 16);
        }
```

```
      } else {
        uint32_t load_smem_V_ptr =
            smem_V_base_ptr +
            (load_smem_V_Bc * kSmemStrideV + load_smem_V_d) * sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
        }
      }
      CP_ASYNC_COMMIT_GROUP();
    }

    if constexpr (kStagesK > 1) {
      // Prefetch next K tile (pipelined)
      if ((tile_K_seqlen + 1) < Tc) {
        int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
        int load_gmem_K_addr =
            K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
        if constexpr (kSwizzleK) {
#pragma unroll
          for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
            int col = load_smem_K_d + i;
            uint32_t ptr = smem_K_base_ptr +
               (smem_sel_next * K_tile_size +
                (col / kMmaAtomK) * Bc * kMmaAtomK +
                load_smem_K_Bc * kMmaAtomK +
                swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
                  sizeof(half);
            CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
          }
        } else {
          uint32_t load_smem_K_ptr =
              smem_K_base_ptr +
              (smem_sel_next * K_tile_size +
               load_smem_K_Bc * kSmemStrideK +
               load_smem_K_d) *
                  sizeof(half);
#pragma unroll
          for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
            CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
          }
        }
        CP_ASYNC_COMMIT_GROUP();
      }
    } else {
      // kStagesK==1: 加载当前 K tile (smem_sel_next == smem_sel == 0) 。
      // Step 2 预加载循环 (kStagesK-1=0) 不执行，所以每轮 3a 必须加载当前 K。
      {
        int load_gmem_K_Bc = tile_K_seqlen * Bc + load_smem_K_Bc;
        int load_gmem_K_addr =
            K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
        if constexpr (kSwizzleK) {
#pragma unroll
          for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
            int col = load_smem_K_d + i;
            uint32_t ptr = smem_K_base_ptr +
               (smem_sel * K_tile_size +
                (col / kMmaAtomK) * Bc * kMmaAtomK +
                load_smem_K_Bc * kMmaAtomK +
                swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
                  sizeof(half);
            CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
          }
```

```
        } else {
          uint32_t load_smem_K_ptr =
              smem_K_base_ptr +
              (smem_sel * K_tile_size +
               load_smem_K_Bc * kSmemStrideK +
               load_smem_K_d) *
                 sizeof(half);
#pragma unroll
          for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
            CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
          }
        }
        CP_ASYNC_COMMIT_GROUP();
      }
    }

    // 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环
    // kStagesK==1: 3a 刚提交当前 K 的 cp.async，必须 wait 后才能 ldmatrix。
    // kStagesK>1: K 已在上一轮预取并 wait（Step 2 或 3e 末尾的 wait），无需再 wait。
    if constexpr (kStagesK == 1) {
      CP_ASYNC_WAIT_GROUP(0);
      __syncthreads();
    }
    fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
#pragma unroll
    for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
      // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        int warp_smem_Q_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
        int lane_smem_Q_Br =
            warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
        int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
        uint32_t lane_smem_Q_ptr;
        if constexpr (kSwizzleQ) {
          lane_smem_Q_ptr = smem_Q_base_ptr +
              ((lane_smem_Q_d / kMmaAtomK) * Br * kMmaAtomK +
               lane_smem_Q_Br * kMmaAtomK +
               swizzle<kMmaAtomK>(lane_smem_Q_Br,
                                  lane_smem_Q_d % kMmaAtomK)) *
                 sizeof(half);
        } else {
          lane_smem_Q_ptr = smem_Q_base_ptr +
              (lane_smem_Q_Br * kSmemStrideQ + lane_smem_Q_d) * sizeof(half);
        }
        LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                    lane_smem_Q_ptr);
      }

      // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
      // K[Bc,d] row-major = K^T[d,Bc] col-major（NT 布局的 B 矩阵）
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        int warp_smem_K_Bc =
            warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
        int lane_smem_K_Bc =
            warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
        int lane_smem_K_d =
            tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
        uint32_t lane_smem_K_ptr;
        if constexpr (kSwizzleK) {
          lane_smem_K_ptr = smem_K_base_ptr +
```

```
              (smem_sel * K_tile_size +
               (lane_smem_K_d / kMmaAtomK) * Bc * kMmaAtomK +
               lane_smem_K_Bc * kMmaAtomK +
               swizzle<kMmaAtomK>(lane_smem_K_Bc,
                                  lane_smem_K_d % kMmaAtomK)) *
                  sizeof(half);
        } else {
          lane_smem_K_ptr = smem_K_base_ptr +
            (smem_sel * K_tile_size + lane_smem_K_Bc * kSmemStrideK +
               lane_smem_K_d) *
                  sizeof(half);
        }
        LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
      }

      // MMA: S[tile] += Q[tile] @ K^T[tile]
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          if constexpr (kMmaAccF32) {
            HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
                         R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                         R_K[j][0], R_K[j][1],
                         R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
          } else {
            HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
                      R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
                      R_S[i][j][1]);
          }
        }
      }
    } // end loop over d
    __syncthreads();

    // ================================================================
    // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
    // ================================================================
    // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
    //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
    //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
    //   - lane 0~3 持有 row 0 的片段，lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
    //   - 对于 rows 8~15 也是同样的 lane 分组，只是读取的是 reg[1]
    // 这就是为什么后面做 row max / row sum 时，warp 内真正参与同一行归约的是
    // {0,4,8,...,28} 这一类 4-lane 子组，而不是整 warp 32 个线程。
    // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
    // kValTileSeqLenK tiles.

    float lane_row_max_new[kValTileSeqLenQ][2];
    float lane_row_sum_new[kValTileSeqLenQ][2];
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);

    // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        float tmp_max_0, tmp_max_1;
        if constexpr (kMmaAccF32) {
          float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
          tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
          tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
```

```
        } else {
          float2 t_reg_S_0 =
              __half22float2(HALF2(R_S[i][j][0])); // rows 0~7:  {c0, c1}
          float2 t_reg_S_1 =
              __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
          tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
          tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
        }
        lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
        lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
      }
      // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
      // {0,4,8,...,28} hold valid data)
      lane_row_max_new[i][0] =
          warp_reduce_max<4, float>(lane_row_max_new[i][0]);
      lane_row_max_new[i][1] =
          warp_reduce_max<4, float>(lane_row_max_new[i][1]);
    }

    // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
    // 面试关键点：这里将 P 写回 R_S 寄存器!
    // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
    // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费，无需额外重组。
#pragma unroll
    for (int i = 0; i < kValTileSeqLenQ; ++i) {
      // m_new = max(m_old, m_cur)
      float block_row_max_new_0 =
          max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
      float block_row_max_new_1 =
          max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);

#pragma unroll
      for (int j = 0; j < kValTileSeqLenK; ++j) {
        if constexpr (kMmaAccF32) {
          float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
          half *t_hptr_S = reinterpret_cast<half *>(&R_S[i][j][0]);
          t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
          t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
          t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
          t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
          lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
          lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
          t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
          t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
          t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
          t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
        } else {
          float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
          float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
          t_reg_S_0.x =
              __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
          t_reg_S_0.y =
              __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
          t_reg_S_1.x =
              __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
          t_reg_S_1.y =
              __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
          lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
          lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
          HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
          HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
        }
      }
```

```
4599
4600        // Warp-level reduce sum (kWarpWidth = 4, same as max)
4601        lane_row_sum_new[i][0] =
4602            warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
4603        lane_row_sum_new[i][1] =
4604            warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
4605      }
4606      __syncthreads();
4607
4608      // ============================================================================
4609      // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
4610      // ============================================================================
4611      // Wait for V to be ready before computing P@V
4612      if constexpr (kStagesK > 1) {
4613        if ((tile_K_seqlen + 1) < Tc) {
4614          CP_ASYNC_WAIT_GROUP(1);
4615        } else {
4616          CP_ASYNC_WAIT_GROUP(0);
4617        }
4618      } else {
4619        CP_ASYNC_WAIT_GROUP(0);
4620      }
4621      __syncthreads();
4622
4623      fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_O, 0);
4624
4625      // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
4626      // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
4627  #pragma unroll
4628      for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
4629        // ldmatrix.x2.trans: load V[Bc,d] with transposition
4630        // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
4631        // transposed ldmatrix
4632  #pragma unroll
4633        for (int j = 0; j < kValTileHeadDimV; ++j) {
4634          int warp_smem_V_d =
4635              warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4636          int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
4637          int lane_smem_V_d = warp_smem_V_d;
4638          uint32_t lane_smem_V_ptr;
4639          if constexpr (kSwizzleV) {
4640            lane_smem_V_ptr = smem_V_base_ptr +
4641                ((lane_smem_V_d / kMmaAtomK) * Bc * kMmaAtomK +
4642                 lane_smem_V_Bc * kMmaAtomK +
4643                 swizzle<kMmaAtomK>(lane_smem_V_Bc,
4644                                    lane_smem_V_d % kMmaAtomK)) *
4645                    sizeof(half);
4646          } else {
4647            lane_smem_V_ptr = smem_V_base_ptr +
4648                (lane_smem_V_Bc * kSmemStrideV + lane_smem_V_d) * sizeof(half);
4649          }
4650          LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
4651        }
4652
4653        // P matrix layout in R_S[i][j][2]:
4654        //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
4655        //   |   64x64   |      warp_KV 0       |
4656        //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
4657        //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
4658        //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
4659        //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
4660        // tile_V_Bc selects which 16-column slice of P to use:
4661        //   tile_V_Bc=0 → cols  0:16 → MMA indices 0,1  → w=0
```

```
      //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3  → w=2
      //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5  → w=4
      //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7  → w=6
      // 对应的 MMA A fragment 布局可以这样记:
      //   - rows 0~7:  lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
      //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
      // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
      // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
      // fragment 都无条件成立的通用结论。layout转换逻辑:
      //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
      int w = tile_V_Bc * 2;
#pragma unroll
      for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          if constexpr (kMmaAccF32) {
            HMMA16816F32(R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3],
                         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
                         R_V[j][0], R_V[j][1],
                         R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3]);
          } else {
            HMMA16816(R_O[i][j][0], R_O[i][j][1],
                      R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
                      R_V[j][0], R_V[j][1],
                      R_O[i][j][0], R_O[i][j][1]);
          }
        }
      }
    } // end for tile_V_Bc
    __syncthreads();

    // ============================================================================
    // 3e: Online rescaling — O_new = exp(m_old - m_new) * O_old + P@V
    // ============================================================================
    // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 O 与 l 的 rescale
#pragma unroll
    for (int i = 0; i < kValTileSeqLenP; ++i) {
      float block_row_max_new_0 = lane_row_max_new[i][0];
      float block_row_max_new_1 = lane_row_max_new[i][1];
      float block_row_sum_new_0 = lane_row_sum_new[i][0];
      float block_row_sum_new_1 = lane_row_sum_new[i][1];

      float block_row_max_old_0 = lane_block_row_max_old[i][0];
      float block_row_max_old_1 = lane_block_row_max_old[i][1];

      block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
      block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);

      // Handle first iteration: m_old = -inf, need to use m_new directly
      block_row_max_old_0 =
          (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
      block_row_max_old_1 =
          (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);

      float rescale_o_factor_0 =
          __expf(block_row_max_old_0 - block_row_max_new_0);
      float rescale_o_factor_1 =
          __expf(block_row_max_old_1 - block_row_max_new_1);

      // Rescale O_old + Add P@V in one fused step
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        if constexpr (kMmaAccF32) {
```

```
          float *t_fptr_O = reinterpret_cast<float *>(&R_O[i][j][0]);
          float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
          t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_O[0]);
          t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_O[1]);
          t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_O[2]);
          t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_O[3]);
        } else {
          float2 t_reg_O_0 = __half22float2(HALF2(R_O[i][j][0]));
          float2 t_reg_O_1 = __half22float2(HALF2(R_O[i][j][1]));
          float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
          float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
          t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_O_0.x);
          t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_O_0.y);
          t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_O_1.x);
          t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_O_1.y);
          HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
          HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
        }
      }

      // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
      float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
      float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
      lane_block_row_sum_old[i][0] = __fmaf_rn(
          rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
      lane_block_row_sum_old[i][1] = __fmaf_rn(
          rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);

      // Update m
      lane_block_row_max_old[i][0] = block_row_max_new_0;
      lane_block_row_max_old[i][1] = block_row_max_new_1;
    }

    // Wait for next K tile to be ready in smem before next iteration
    if constexpr (kStagesK > 1) {
      if ((tile_K_seqlen + 1) < Tc) {
        CP_ASYNC_WAIT_GROUP(0);
      }
      __syncthreads();
    }
  } // end outer loop over K seqlen
  __syncthreads();

  // ============================================================================
  // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
  // ============================================================================
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
    float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
    float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      if constexpr (kMmaAccF32) {
        float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
        half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
        t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
        t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
        t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
        t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
      } else {
        float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
        float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
        t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
```

```
        t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
        t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
        t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
        HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
        HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
      }
    }
  }

  // ============================================================================
  // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
  // ============================================================================
  // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
  // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
#pragma unroll
  for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
    for (int j = 0; j < kValTileHeadDimV; ++j) {
      uint32_t R_Z[2][4];
      R_Z[0][0] = R_D[i][j][0];
      R_Z[1][0] = R_D[i][j][1];
      R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
      R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
      R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
      R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
      R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
      R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

      // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
      if (lane_id % 4 == 0) {
        int store_warp_regs_O_Br =
            warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
        int store_lane_gmem_O_Br =
            Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
        int store_warp_regs_O_d =
            warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
        int store_lane_gmem_O_d = store_warp_regs_O_d;
        int store_gmem_O_addr_0 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 0) * kHeadDim +
                                  store_lane_gmem_O_d;
        int store_gmem_O_addr_1 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 8) * kHeadDim +
                                  store_lane_gmem_O_d;
        // LDST128BITS = reinterpret_cast<float4*>
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_0]) =
            *reinterpret_cast<float4 *>(&R_Z[0][0]);
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_1]) =
            *reinterpret_cast<float4 *>(&R_Z[1][0]);
      }
    }
  }
}

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// ============================================================================
// Phase 8b: FlashAttention-2 TMA + MMA Warp Specialization (SM120)
// ============================================================================
// 面试要点 (FA TMA MMA WS vs cp.async FA):
//   - Producer (WG0, 128T) 用 TMA 128B SWIZZLE 搬运 Q/K/V tile 到 smem
//   - Consumer (WG1, 256T) 用 ldmatrix + mma.sync.m16n8k16 做 Q@K^T / P@V
//   - 与 hgemm_tma_mma_ws_tn 的对比:
//     * HGEMM 只做一次 GEMM, K 维迭代; FA 做 QK^T 和 PV 两次 GEMM, KV seqlen 迭代
//     * HGEMM 的 A/B 都 staged; FA 中 Q 只 load 一次 (split-Q), K staged, V 单 buffer
```

```
//
// ★ split-Q 语义（关键，避免误解）：
//   - grid = (seqlen/Br, B*H)，每个 block 处理一个 **block-level Q tile [Br, d]**
//     （不是全局 Q）。对 Q 的 seqlen 做 block-level 并行。
//   - 每个 block 遍历**完整的 KV seqlen**（Tc = seqlen/Bc 次外循环）。
//   - 因此每个 block 的 Q tile 只需 **load 一次到 smem**，在整个 KV 遍历中复用
//     → 只需 full_Q barrier（无 empty_Q）。
//
// ★ Pipeline 同步协议（arrive_count = 257 = 256 consumer + 1 producer）：
//   - Q:  full_Q — Producer 发 arrive_tx，Consumer 在主循环前 wait 一次
//   - K:  full_K[kStagesK] / empty_K[kStagesK] — 多 stage pipeline
//   - V:  full_V / empty_V — 单 buffer 串行（QK 前发 TMA → QK 重叠 → PV 前等 → PV 后释放）
//
// ★ V ldmatrix.x2.trans 正确性（已论证无风险）：
//   TMA 128B SWIZZLE 是 1-1 映射，ldmatrix 用 swizzle<64>(row,col) 计算的物理地址
//   = TMA 写入的物理地址（smem 1024B 对齐保证 phase=0）。ldmatrix.x2.trans 的转置
//   语义在寄存器层面工作，与 smem 物理布局无关 → 必然正确。
//
// ★ Shared Memory 占用分析（D=64/128, kStagesK=1/2/3/4, kStagesV=1/2）：
//   smem = (Q[Br,D] + K[kStagesK,Bc,D] + V[kStagesV,Bc,D]) * sizeof(half)
//        = D * (Br + kStagesK*Bc + kStagesV*Bc) * 2
//        = D * (128 + (kStagesK+kStagesV)*64) * 2     (Br=128, Bc=64)
//
//   | kStagesK | kStagesV | D=64  bytes | D=128 bytes |
//   |----------|----------|-------------|-------------|
//   |    1     |    1     |   32 KB     |   64 KB     |
//   |    2     |    1     |   40 KB     |   80 KB     |
//   |    2     |    2     |   48 KB     |   96 KB     |
//   |    3     |    1     |   48 KB     |   96 KB     |
//   |    3     |    2     |   56 KB     |  112 KB     |
//   |    4     |    1     |   56 KB     |  112 KB     |
//
//   SM120 (RTX PRO 5000) optin smem 上限 ~100KB:
//   - D=64: S=1/2/3/4 全部可行 (32-56 KB)
//   - D=128: S=1/2/3 可行 (64-96 KB); S=4 (112 KB) 超出 optin 上限，会被
//     check_smem_feasible 兜底判为 SMEM SKIP。D=128 推荐 S=2（甜点: 80 KB,
//     2 blocks/SM，pipeline 深度足够隐藏 TMA 延迟）。
//
// v1 限制: kHeadDim=64 or 128; seqlen % Br == 0 且 seqlen % Bc == 0（不处理尾 tile）

// ============================================================================
// FA TMA WS smem offset helper (D=64/128 通用)
// ============================================================================
// 背景: CU_TENSOR_MAP_SWIZZLE_128B 硬件要求 box innermost dim ≤ 128B = 64 half
// （见 /usr/local/cuda/include/cuda.h 的 cuTensorMapEncodeTiled 文档）。因此 D=128
// 不能用单个 TMA box=(128, Br) 覆盖整行。
//
// 方案: D=128 时 box 固定为 (64, Br)，沿 head_dim 方向连续发 kTmaChunks 次 TMA
// （minor_coord = c*64），写入 chunk-major smem 布局 [kTmaChunks, Br, 64]:
//   - chunk c 的 smem 起始偏移 = c * Br * 64
//   - chunk c 内部按 [Br, 64] row-major + SWIZZLE_128B 存放（64 half = 128B 周期）
//
// 本 helper 计算 (row, col) 在该 chunk-major 布局下的 swizzled smem 元素偏移:
//   chunk   = col / 64        ∈ [0, kTmaChunks)
//   col_in  = col % 64        ∈ [0, 64)
//   offset  = chunk * Br * 64 + row * 64 + swizzle<64>(row, col_in)
//
// D=64 退化: kTmaChunks=1, chunk=0, offset = row*64 + swizzle<64>(row, col)
//           与原公式 (row*64 + swizzle<64>(row, col)) 完全一致 → 无回归。
template <int kHeadDim, int Br>
__device__ __forceinline__ int swizzle_fa(int row, int col) {
  static_assert(kHeadDim == 64 || kHeadDim == 128, "D=64 or 128 only");
  if constexpr (kHeadDim == 64) {
```

```
    return row * 64 + swizzle<64>(row, col);
  } else {  // kHeadDim == 128
    const int chunk  = col >> 6;        // col / 64 ∈ {0, 1}
    const int col_in = col & 63;        // col % 64
    return chunk * (Br * 64) + row * 64 + swizzle<64>(row, col_in);
  }
}

template <
    const int kHeadDim,          // head dim: v1 only 64
    const int kMmaAtomM,         // 16 (MMA instruction M dim)
    const int kMmaAtomN,         // 8  (MMA instruction N dim)
    const int kMmaAtomK,         // 16 (MMA instruction K dim)
    const int kMmaAccF32,        // 0=f16 acc, 1=f32 acc (mma.sync.f32.f16.f16.f32)
    const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 8 → Br=16*8=128
    const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
    const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, == kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
    const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
    const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
    const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
    const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
    const int kStagesK,          // K pipeline depth (>=1)
    const int kStagesV,          // V pipeline depth (>=1; 1=single buffer, >=2=pipelined)
    const int kNumThreads>       // 384, 128 producer + 256 consumer
__global__ void __launch_bounds__(kNumThreads, 1)
    flash_attn_tma_mma_ws_stages_split_q(
        half *Q, half *K, half *V, half *O, int N, int H,
        const CUtensorMap *__restrict__ tensorMapQ,
        const CUtensorMap *__restrict__ tensorMapK,
        const CUtensorMap *__restrict__ tensorMapV) {
  static_assert(kHeadDim == 64 || kHeadDim == 128,
                "v1 supports kHeadDim == 64 or 128");
  static_assert(kNumThreads == 384, "128 producer + 256 consumer");
  static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
  static_assert(kStagesK >= 1, "kStagesK must be >= 1");
  static_assert(kStagesV >= 1, "kStagesV must be >= 1");
  constexpr int kNRegs = kMmaAccF32 ? 4 : 2;

  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 128
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
  // BK for QK: kHeadDim=64, split into kHeadDim/kMmaAtomK=4 MMA K slices
  // BK for PV: Bc=64, split into Bc/kMmaAtomK=4 MMA K slices (P rows)
  constexpr int kConsumerThreads = 256;  // 8 warps
  constexpr int kProducerThreads = 128;  // 4 warps, only thread 0 issues TMA
  constexpr int kQTileBytes = Br * kHeadDim * sizeof(half);       // 16 KB
  constexpr int kKTileBytes = Bc * kHeadDim * sizeof(half);       // 8 KB
  constexpr int kVTileBytes = Bc * kHeadDim * sizeof(half);       // 8 KB
  // TMA box innermost 固定 64 half (128B)，满足 CU_TENSOR_MAP_SWIZZLE_128B 的
  // box innermost ≤ 128B 硬约束。D=128 时沿 head_dim 发 kTmaChunks 次 TMA，
  // 写入 chunk-major smem 布局 [kTmaChunks, Br, 64]。
  constexpr int kTmaBoxMinor = 64;
  constexpr int kTmaChunks   = kHeadDim / kTmaBoxMinor;

  // Block indexing — split-Q: blockIdx.x indexes Q tile along seqlen
  const int Nb_id = blockIdx.y / H;
  const int Nh_id = blockIdx.y % H;
  const int Q_tile_id = blockIdx.x;
  const int Tc = (N + Bc - 1) / Bc;  // number of KV tiles along seqlen
  const float scale = 1.0f / sqrtf((float)kHeadDim);

  // Per-head gmem base offset (self-attention: Q=K=V share layout)
  const int QKV_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
```

```
  const int O_gmem_offset = QKV_gmem_offset;
  // TMA descriptor covers the entire [B*H*N, D] gmem as one 2D matrix; the
  // producer must offset major_coord by this head/batch base so each block
  // loads its own (batch, head) Q/K/V tile instead of always reading head 0.
  const int qkv_major_base = (Nb_id * H + Nh_id) * N;

  // ---- Shared memory layout ----
  // TMA CU_TENSOR_MAP_SWIZZLE_128B requires 1024B-aligned smem base so the
  // hardware swizzle phase starts at zero. Consumer swizzle<64>() assumes
  // zero phase (no base_offset compensation like WGMMA descriptor).
  // Q: [Br, kHeadDim] = [128, 64] = 16 KB (1024B aligned ✓)
  // K: [kStagesK, Bc, kHeadDim] = [kStagesK, 64, 64] (8 KB per stage)
  // V: [kStagesV, Bc, kHeadDim] = [kStagesV, 64, 64] (8 KB per stage)
  extern __shared__ __align__(1024) uint8_t smem_fa_tma_ws[];
  half *Q_smem = reinterpret_cast<half *>(smem_fa_tma_ws);
  half *K_smem = Q_smem + Br * kHeadDim;
  half *V_smem = K_smem + kStagesK * Bc * kHeadDim;

  const uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_smem);
  const uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_smem);
  const uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_smem);

  // ---- Barriers ----
  // arrive_count = kConsumerThreads + 1 = 257 (256 consumer arrives + 1 producer arrive_tx)
#pragma nv_diag_suppress static_var_with_dynamic_init
  __shared__ cuda::barrier<cuda::thread_scope_block> full_Q;
  __shared__ cuda::barrier<cuda::thread_scope_block> full_K[kStagesK];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty_K[kStagesK];
  __shared__ cuda::barrier<cuda::thread_scope_block> full_V[kStagesV];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty_V[kStagesV];
#pragma nv_diag_default static_var_with_dynamic_init

  // Thread partition: first kProducerThreads (0~127) = Producer WG,
  // remaining kConsumerThreads (128~383) = Consumer WG.
  // WARN: Must use kProducerThreads (not kConsumerThreads) as the divisor, otherwise
  // the split is wrong: 384/256=1.5 would give Producer 256 threads and
  // Consumer only 128, but barriers expect 256 consumer arrives → deadlock.
  const int wg_idx = (threadIdx.x < kProducerThreads) ? 0 : 1;
  const int wg_tid = (threadIdx.x < kProducerThreads) ? threadIdx.x :
                     (threadIdx.x - kProducerThreads);

  if (threadIdx.x == 0) {
    init(&full_Q, kConsumerThreads + 1);
    for (int s = 0; s < kStagesV; ++s) {
      init(&full_V[s], kConsumerThreads + 1);
      init(&empty_V[s], kConsumerThreads + 1);
    }
    for (int s = 0; s < kStagesK; ++s) {
      init(&full_K[s], kConsumerThreads + 1);
      init(&empty_K[s], kConsumerThreads + 1);
    }
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  // ================================================================
  // Producer Warpgroup (WG0, threadIdx.x 0~127)
  // Only wg_tid == 0 issues TMA. Split-Q: Q tile loaded once, K staged, V single buffer.
  // D=128 时每个 tile 沿 head_dim 发 kTmaChunks 次 TMA (box innermost=64 half),
  // 共享同一 barrier: N 次 cp.async.bulk.tensor 各 reduce chunk_bytes 到 tx count,
  // 1 次 mbarrier_arrive_expect_tx 声明总字节 + producer thread arrive。
  // ================================================================
  if (wg_idx == 0) {
```

```
    NOTES_V2_REG_DEALLOC(40);
    if (wg_tid == 0) {
      // Step P0: Load block-level Q tile [Br, d] once. D=128 时发 kTmaChunks 次。
      //   minor_coord = c * 64, major_coord = qkv_major_base + Q_tile_id * Br
      //   smem dst = Q_smem + c * Br * 64 (chunk-major 布局)
      for (int c = 0; c < kTmaChunks; ++c) {
        tma_load_2d(Q_smem + c * Br * kTmaBoxMinor, tensorMapQ,
                    c * kTmaBoxMinor, qkv_major_base + Q_tile_id * Br, full_Q);
      }
      tma_arrive_expect_tx(full_Q, kQTileBytes);

      // Step P1: Prefetch first (kStagesK-1) K tiles.
      // K tile coords: minor = c*64, major = qkv_major_base + s*Bc
      for (int s = 0; s < kStagesK - 1; ++s) {
        empty_K[s].wait(empty_K[s].arrive());
        for (int c = 0; c < kTmaChunks; ++c) {
          tma_load_2d(K_smem + s * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
                      tensorMapK, c * kTmaBoxMinor,
                      qkv_major_base + s * Bc, full_K[s]);
        }
        tma_arrive_expect_tx(full_K[s], kKTileBytes);
      }

      // Step P1b: Prefetch first (kStagesV-1) V tiles (only if kStagesV > 1).
      // kStagesV=1: no prefetch; V[k] loaded in P2 overlaps only QK gemm.
      // kStagesV>=2: V prefetched here; in P2 V[k+kStagesV-1] overlaps
      //   QK+softmax+PV of current iter → true V pipeline.
      if constexpr (kStagesV > 1) {
        for (int s = 0; s < kStagesV - 1; ++s) {
          empty_V[s].wait(empty_V[s].arrive());
          for (int c = 0; c < kTmaChunks; ++c) {
            tma_load_2d(V_smem + s * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
                        tensorMapV, c * kTmaBoxMinor,
                        qkv_major_base + s * Bc, full_V[s]);
          }
          tma_arrive_expect_tx(full_V[s], kVTileBytes);
        }
      }

      // Step P2: Main loop over KV seqlen tiles
      for (int k = 0; k < Tc; ++k) {
        [[maybe_unused]] const int stage = k % kStagesK;

        // Issue V[k+kStagesV-1] TMA to stage_next_v (pipelined).
        // kStagesV=1: loads V[k] to stage 0, overlaps with QK of same iter.
        // kStagesV>=2: loads V[k+kStagesV-1], overlaps with QK+softmax+PV.
        {
          const int v_tile = k + kStagesV - 1;
          if (v_tile < Tc) {
            const int stage_next_v = v_tile % kStagesV;
            empty_V[stage_next_v].wait(empty_V[stage_next_v].arrive());
            for (int c = 0; c < kTmaChunks; ++c) {
              tma_load_2d(V_smem + stage_next_v * Bc * kHeadDim +
                              c * Bc * kTmaBoxMinor,
                          tensorMapV, c * kTmaBoxMinor,
                          qkv_major_base + v_tile * Bc,
                          full_V[stage_next_v]);
            }
            tma_arrive_expect_tx(full_V[stage_next_v], kVTileBytes);
          }
        }

        // Prefetch K[k+kStagesK-1] (pipelined).
```

```
        if (k + kStagesK - 1 < Tc) {
          const int stage_next = (k + kStagesK - 1) % kStagesK;
          empty_K[stage_next].wait(empty_K[stage_next].arrive());
          for (int c = 0; c < kTmaChunks; ++c) {
            tma_load_2d(K_smem + stage_next * Bc * kHeadDim +
                            c * Bc * kTmaBoxMinor,
                        tensorMapK, c * kTmaBoxMinor,
                        qkv_major_base + (k + kStagesK - 1) * Bc,
                        full_K[stage_next]);
          }
          tma_arrive_expect_tx(full_K[stage_next], kKTileBytes);
        }
      }
    }
  }
  // ================================================================
  // Consumer Warpgroup (WG1, threadIdx.x 128~383, 256 threads = 8 warps)
  // All 256 threads participate in ldmatrix + mma.sync.
  // ================================================================
  else {
    const int warp_id = wg_tid / kWarpSize;  // 0~7
    const int lane_id = wg_tid % kWarpSize;  // 0~31
    // Split-Q warp layout: 8 warps form 8x1 grid (warp_QP = 0~7, warp_KV = 0)
    // Br = 16 * 8 * 1 = 128 → 8 warps each handle 16 rows of Q
    const int warp_QP = warp_id;
    const int warp_KV = 0;

    // Step C0: init — mark all K stages and V stages as "empty" (consumable by producer)
    // Consumer register budget per Triton flash_attn_v2 maxnreg strategy on
    // Blackwell warp_specialize: D=128 -> 168, otherwise -> 80.
    if constexpr (kHeadDim == 128) {
      NOTES_V2_REG_ALLOC(168);
    } else {
      NOTES_V2_REG_ALLOC(80);
    }

    for (int s = 0; s < kStagesK; ++s) {
      [[maybe_unused]] auto token = empty_K[s].arrive();
    }
    for (int s = 0; s < kStagesV; ++s) {
      [[maybe_unused]] auto token = empty_V[s].arrive();
    }

    // Wait for Q tile ready once (split-Q: Q smem is reused across all KV iters)
    full_Q.wait(full_Q.arrive());

    // Online softmax persistent state (per-warp, per-row-pair)
    float lane_block_row_max_old[kValTileSeqLenQ][2];
    float lane_block_row_sum_old[kValTileSeqLenQ][2];
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

    uint32_t R_Q[kValTileSeqLenQ][4];
    uint32_t R_K[kValTileSeqLenK][2];
    uint32_t R_V[kValTileHeadDimV][2];
    uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs];
    uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
    uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs];

    fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);

    // ---- Main loop over KV seqlen ----
    for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
```

```
      const int stage = tile_K_seqlen % kStagesK;
      const int stage_v = tile_K_seqlen % kStagesV;

      // Wait for K[stage] ready
      full_K[stage].wait(full_K[stage].arrive());
      tma_fence_proxy_async_shared_cta();

      // ---- 3b: Q @ K^T = S[Br, Bc] ----
      // Q smem reused (split-Q). K from stage `stage`.
      // smem addr (TMA 128B swizzle, swizzle<64>)
      fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
#pragma unroll
      for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
        // ldmatrix.x4: load Q m16k16 fragment (Q row-major, non-transpose)
#pragma unroll
        for (int i = 0; i < kValTileSeqLenQ; ++i) {
          const int warp_smem_Q_Br =
              warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
          const int lane_smem_Q_Br = warp_smem_Q_Br + lane_id % 16;
          const int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8;
          const uint32_t lane_smem_Q_ptr =
              smem_Q_base_ptr +
              swizzle_fa<kHeadDim, Br>(lane_smem_Q_Br, lane_smem_Q_d) *
                  sizeof(half);
          LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                      lane_smem_Q_ptr);
        }

        // ldmatrix.x2: load K k16n8 fragment (K row-major = K^T col-major)
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          const int warp_smem_K_Bc =
              warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
          const int lane_smem_K_Bc = warp_smem_K_Bc + lane_id % 8;
          const int lane_smem_K_d =
              tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8;
          const uint32_t lane_smem_K_ptr =
              smem_K_base_ptr +
              (stage * Bc * kHeadDim +
               swizzle_fa<kHeadDim, Bc>(lane_smem_K_Bc, lane_smem_K_d)) *
                  sizeof(half);
          LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
        }

#pragma unroll
        for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileSeqLenK; ++j) {
            if constexpr (kMmaAccF32) {
              HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
                           R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                           R_K[j][0], R_K[j][1],
                           R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
            } else {
              HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1],
                        R_Q[i][2], R_Q[i][3], R_K[j][0], R_K[j][1],
                        R_S[i][j][0], R_S[i][j][1]);
            }
          }
        }
      }

      // Release K[stage] for producer reuse as early as possible: QK^T GEMM has
```

```
      // consumed all K smem data; the subsequent softmax (3c) and PV GEMM (3d)
      // only touch registers (R_S, R_O) and V smem, so K[stage] is free to be
      // overwritten by the producer's next TMA prefetch.
      {
        [[maybe_unused]] auto token = empty_K[stage].arrive();
      }

      // ---- 3c: Online Safe Softmax — row max + exp + sum, P back to R_S ----
      // Softmax only touches R_S (registers), so V TMA latency can be overlapped
      // with this compute. Defer full_V.wait() until just before P@V (3d).
      float lane_row_max_new[kValTileSeqLenQ][2];
      float lane_row_sum_new[kValTileSeqLenQ][2];
      fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
      fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);

      // Pass 1: row-wise max across kValTileSeqLenK tiles (warp_reduce_max<4>)
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          float tmp_max_0, tmp_max_1;
          if constexpr (kMmaAccF32) {
            float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
            tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
            tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
          } else {
            float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
            float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
            tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
            tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
          }
          lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
          lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
        }
        lane_row_max_new[i][0] =
            warp_reduce_max<4, float>(lane_row_max_new[i][0]);
        lane_row_max_new[i][1] =
            warp_reduce_max<4, float>(lane_row_max_new[i][1]);
      }

      // Pass 2: P = exp(S*scale - m_new), store back to R_S, accumulate row sums
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        float block_row_max_new_0 =
            max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
        float block_row_max_new_1 =
            max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);

#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          if constexpr (kMmaAccF32) {
            float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
            half *t_hptr_S = reinterpret_cast<half *>(&R_S[i][j][0]);
            t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
            t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
            t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
            t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
            lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
            lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
            t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
            t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
            t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
            t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
```

```
          } else {
            float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
            float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
            t_reg_S_0.x = __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
            t_reg_S_0.y = __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
            t_reg_S_1.x = __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
            t_reg_S_1.y = __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
            lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
            lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
            HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
            HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
          }
        }
        lane_row_sum_new[i][0] =
            warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
        lane_row_sum_new[i][1] =
            warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
      }

      // ---- 3d: P @ V = O[Br, d] ----
      // Wait for V[stage_v] ready just before P@V.
      // kStagesV=1: V was loaded in same iter, TMA overlaps QK+softmax.
      // kStagesV>=2: V was loaded in previous iter, TMA overlaps QK+softmax+PV.
      full_V[stage_v].wait(full_V[stage_v].arrive());
      tma_fence_proxy_async_shared_cta();

      // ldmatrix.x2.trans: V row-major → col-major B fragment for NN matmul
      fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_O, 0);
#pragma unroll
      for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          const int warp_smem_V_d =
              warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
          const int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
          const int lane_smem_V_d = warp_smem_V_d;
          const uint32_t lane_smem_V_ptr =
              smem_V_base_ptr +
              (stage_v * Bc * kHeadDim +
               swizzle_fa<kHeadDim, Bc>(lane_smem_V_Bc, lane_smem_V_d)) *
                  sizeof(half);
          LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
        }

        // P fragment (R_S) directly feeds A of P@V MMA (layout reuse trick)
        const int w = tile_V_Bc * 2;
#pragma unroll
        for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileHeadDimV; ++j) {
            if constexpr (kMmaAccF32) {
              HMMA16816F32(R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3],
                           R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
                           R_V[j][0], R_V[j][1],
                           R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3]);
            } else {
              HMMA16816(R_O[i][j][0], R_O[i][j][1],
                        R_S[i][w][0], R_S[i][w][1],
                        R_S[i][w + 1][0], R_S[i][w + 1][1],
                        R_V[j][0], R_V[j][1],
                        R_O[i][j][0], R_O[i][j][1]);
            }
          }
```

```
        }
      }

      // Release V[stage_v] for producer reuse: PV GEMM has consumed all
      // V smem data; the subsequent online rescaling (3e) only touches registers
      // (R_O, R_D, R_S, lane_block_*), so V[stage_v] is free to be overwritten.
      {
        [[maybe_unused]] auto token = empty_V[stage_v].arrive();
      }

      // ---- 3e: Online rescaling — O_new = exp(m_old - m_new) * O_old + P@V ----
#pragma unroll
      for (int i = 0; i < kValTileSeqLenP; ++i) {
        float block_row_max_new_0 = lane_row_max_new[i][0];
        float block_row_max_new_1 = lane_row_max_new[i][1];
        float block_row_sum_new_0 = lane_row_sum_new[i][0];
        float block_row_sum_new_1 = lane_row_sum_new[i][1];
        float block_row_max_old_0 = lane_block_row_max_old[i][0];
        float block_row_max_old_1 = lane_block_row_max_old[i][1];

        block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
        block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);

        // First iteration: m_old = -inf, use m_new directly
        block_row_max_old_0 = (tile_K_seqlen > 0 ? block_row_max_old_0
                                                 : block_row_max_new_0);
        block_row_max_old_1 = (tile_K_seqlen > 0 ? block_row_max_old_1
                                                 : block_row_max_new_1);

        float rescale_o_factor_0 =
            __expf(block_row_max_old_0 - block_row_max_new_0);
        float rescale_o_factor_1 =
            __expf(block_row_max_old_1 - block_row_max_new_1);

#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          if constexpr (kMmaAccF32) {
            float *t_fptr_O = reinterpret_cast<float *>(&R_O[i][j][0]);
            float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
            t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_O[0]);
            t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_O[1]);
            t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_O[2]);
            t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_O[3]);
          } else {
            float2 t_reg_O_0 = __half22float2(HALF2(R_O[i][j][0]));
            float2 t_reg_O_1 = __half22float2(HALF2(R_O[i][j][1]));
            float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
            float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
            t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_O_0.x);
            t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_O_0.y);
            t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_O_1.x);
            t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_O_1.y);
            HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
            HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
          }
        }

        float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
        float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
        lane_block_row_sum_old[i][0] =
            __fmaf_rn(rescale_o_factor_0, block_row_sum_old_0,
                      block_row_sum_new_0);
        lane_block_row_sum_old[i][1] =
```

```
            __fmaf_rn(rescale_o_factor_1, block_row_sum_old_1,
                      block_row_sum_new_1);
        lane_block_row_max_old[i][0] = block_row_max_new_0;
        lane_block_row_max_old[i][1] = block_row_max_new_1;
      }
    }

    // ---- Step 4: Final rescale — O_final = (1/l_final) * O_final ----
#pragma unroll
    for (int i = 0; i < kValTileSeqLenP; ++i) {
      float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
      float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        if constexpr (kMmaAccF32) {
          float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
          half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
          t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
          t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
          t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
          t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
        } else {
          float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
          float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
          t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
          t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
          t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
          t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
          HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
          HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
        }
      }
    }

    // ---- Step 5: Epilogue — warp shuffle + 128-bit collective store ----
#pragma unroll
    for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
      for (int j = 0; j < kValTileHeadDimV; ++j) {
        uint32_t R_Z[2][4];
        R_Z[0][0] = R_D[i][j][0];
        R_Z[1][0] = R_D[i][j][1];
        R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
        R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
        R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
        R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
        R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
        R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

        if (lane_id % 4 == 0) {
          const int store_warp_regs_O_Br =
              warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
          const int store_lane_gmem_O_Br =
              Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
          const int store_warp_regs_O_d =
              warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
          const int store_lane_gmem_O_d = store_warp_regs_O_d;
          const int store_gmem_O_addr_0 =
              O_gmem_offset + store_lane_gmem_O_Br * kHeadDim +
              store_lane_gmem_O_d;
          const int store_gmem_O_addr_1 =
              O_gmem_offset + (store_lane_gmem_O_Br + 8) * kHeadDim +
              store_lane_gmem_O_d;
```

```
          *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_0]) =
              *reinterpret_cast<float4 *>(&R_Z[0][0]);
          *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_1]) =
              *reinterpret_cast<float4 *>(&R_Z[1][0]);
        }
      }
    }
  }
}

// ============================================================================
// Phase 8c: FlashAttention-3-style TMA + MMA Warp Specialization (SM120)
//   Dual 128-thread consumer warpgroups processing disjoint KV tiles,
//   with per-WG independent K pipeline (kStagesK) and single V buffer (kStagesV=1).
//
// 与 flash_attn_tma_mma_ws_stages_split_q (Phase 8b) 的核心区别:
//   - 8b: 1 producer WG (128T) + 1 consumer WG (256T, 8 warps), 全 KV 遍历
//   - 8c: 1 producer WG (128T) + 2 consumer WGs (各128T, 4 warps), KV tile 奇偶拆分
//
// 线程布局 (384 threads):
//   WG0 [0,127]:   TMA producer (仅 thread 0 发 TMA)
//   WG1 [128,255]: consumer_id=0, 处理偶数 KV tile (0,2,4,...)
//   WG2 [256,383]: consumer_id=1, 处理奇数 KV tile (1,3,5,...)
//
// per-WG 独立 stage 语义 (关键改进):
// ----------------------------------------------------------------
// 每个 consumer WG 拥有独立的 K stage slots (kStagesK 个) 和 1 个 V buffer。
// K/V smem 按 consumer_id 分 bank: K_smem[cid][stage], V_smem[cid]。
// Barrier 也按 consumer_id 隔离: full_K[cid][s], empty_K[cid][s], full_V[cid], empty_V[cid]。
// 两个 WG 绝不争抢同一 smem slot 或 barrier，producer 通过 cid = tile&1 路由 TMA。
//
// 这样 stage 语义从 "跨 WG 共享的 2 slot (每 WG 实际 depth=1)" 变为
// "每 WG 独立拥有 kStagesK 个 K slot (每 WG 真正 depth=kStagesK)"。
//
// Pipeline 时序 (Sk=1, Sv=1):
//   K[tile] TMA 被 softmax+PV+rescale 隐藏 (Sk=1 复用同 slot，用完即释放);
//   V[tile] TMA 被 rescale+QK[next]+softmax[next] 隐藏 (Sv=1 同理)。
//
// Pipeline 时序 (Sk=2, Sv=1, D=64 only):
//   K[tile+2] 在 QK[tile] 之前就 prefetch 到另一个 slot，被整个 tile compute 隐藏;
//   V[tile] TMA 同 Sk=1，被 QK+softmax 隐藏。
//
// 为什么是奇偶 tile 拆分 (而非前/后半区各自一半)?
// ----------------------------------------------------------------
// 关键约束: producer 按 global tile 顺序 0,1,2,3,... 连续 TMA。
// 奇偶拆分让 cid = tile & 1 静态路由: tile 0,2,4,... -> WG1; tile 1,3,5,... -> WG2。
// 每 WG 的下一个 tile (tile+2) 正好是 producer 当前/下一轮要加载的 tile,
// 保证 producer/consumer 流水无停顿。
//
// 若改为前/后半区拆分: 过渡区两个 WG 可能同时需要同一 cid 的 slot,
// 且 WG2 跳跃访问远端 tile 时数据尚未被 producer TMA 到 smem。
//
// Partial state merge (Split-KV / FlashDecoding-style reduction):
// ----------------------------------------------------------------
// 每个 WG 各自处理一半 KV tile 序列, 独立维护 "未归一化" 的 online-softmax
// 中间状态 (m, l, Oacc), 合并后等价于全 KV 遍历。
//
// 稳定归并公式 (避免 exp 溢出):
//   m     = max(m_0, m_1)            // 全局 row max
//   alpha = exp(m_0 - m)  (<=1)      // WG1 rescale factor
//   beta  = exp(m_1 - m)  (<=1)      // WG2 rescale factor
//   l     = alpha * l_0 + beta * l_1
//   Oacc  = alpha * Oacc_0 + beta * Oacc_1
```

```cpp
//   O     = Oacc / l                  // 最终归一化
//
// Tc==1 退化: WG2 无 tile, m_1=-inf, l_1=0, Oacc_1=0;
//   beta = exp(-inf) = 0, 退化为 WG1 结果。
//
// 限制 (首版):
//   - kStagesV == 1 (V 只需 1 buffer, TMA 被 QK+softmax 隐藏)
//   - kStagesK >= 1 (K pipeline depth; D=128 在 SM120 只支持 Sk=1)
//   - D=128 时 Sk=1 (Sk=2 需 112KB > 101KB optin 上限)
//   - Br=64, Bc=64, D=64/128, aligned seqlen (N % Br == 0 && N % Bc == 0)
//   - 仅 self-attention, 无 causal/varlen/GQA
// ============================================================================
template <
    const int kHeadDim,
    const int kMmaAtomM, const int kMmaAtomN, const int kMmaAtomK,
    const int kMmaAccF32,
    const int kMmaTileSeqLenQ, const int kMmaTileSeqLenK,
    const int kMmaTileSeqLenP, const int kMmaTileHeadDimV,
    const int kValTileSeqLenQ, const int kValTileSeqLenK,
    const int kValTileSeqLenP, const int kValTileHeadDimV,
    const int kStagesK, const int kStagesV,
    const int kNumThreads>
__global__ void __launch_bounds__(kNumThreads, 1)
    flash_attn_3_tma_ws_stages_split_q(
        half *Q, half *K, half *V, half *O, int N, int H,
        const CUtensorMap *__restrict__ tensorMapQ,
        const CUtensorMap *__restrict__ tensorMapK,
        const CUtensorMap *__restrict__ tensorMapV) {
  static_assert(kHeadDim == 64 || kHeadDim == 128,
                "v1 supports kHeadDim == 64 or 128");
  static_assert(kNumThreads == 384, "128 producer + 256 consumer (2 WGs)");
  static_assert(kMmaAccF32 == 0 || kMmaAccF32 == 1, "kMmaAccF32 must be 0 or 1");
  static_assert(kStagesV == 1, "per-WG V pipeline depth must be 1");
  static_assert(kStagesK >= 1, "kStagesK must be >= 1");
  constexpr int kNRegs = kMmaAccF32 ? 4 : 2;

  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
  constexpr int kConsumerThreadsPerWG = 128;
  constexpr int kProducerThreads = 128;
  constexpr int kNumConsumerWGs = 2;
  constexpr int kQTileBytes = Br * kHeadDim * sizeof(half);
  constexpr int kKTileBytes = Bc * kHeadDim * sizeof(half);
  constexpr int kVTileBytes = Bc * kHeadDim * sizeof(half);
  constexpr int kTmaBoxMinor = 64;
  constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;

  const int Nb_id = blockIdx.y / H;
  const int Nh_id = blockIdx.y % H;
  const int Q_tile_id = blockIdx.x;
  const int Tc = (N + Bc - 1) / Bc;
  const float scale = 1.0f / sqrtf((float)kHeadDim);

  const int QKV_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
  const int O_gmem_offset = QKV_gmem_offset;
  const int qkv_major_base = (Nb_id * H + Nh_id) * N;

  // ---- Shared memory layout (per-WG independent K/V banks) ----
  // [Q_shared: Br*D]
  // [K[cid=0][s=0..Sk-1]: Sk*Bc*D]  [K[cid=1][s=0..Sk-1]]
  // [V[cid=0]: Bc*D]                 [V[cid=1]: Bc*D]
  extern __shared__ __align__(1024) uint8_t smem_fa3_tma_ws[];
  half *Q_smem = reinterpret_cast<half *>(smem_fa3_tma_ws);
```

```
  half *K_smem_base = Q_smem + Br * kHeadDim;
  // K: [numConsumerWGs][kStagesK][Bc*kHeadDim], linearized
  //   K_smem_base + cid * (kStagesK * Bc * kHeadDim) + stg * (Bc * kHeadDim)
  half *V_smem_base = K_smem_base + kNumConsumerWGs * kStagesK * Bc * kHeadDim;
  // V: [numConsumerWGs][Bc*kHeadDim]
  //   V_smem_base + cid * (Bc * kHeadDim)

  const uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_smem);
  const uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_smem_base);
  const uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_smem_base);

#pragma nv_diag_suppress static_var_with_dynamic_init
  __shared__ cuda::barrier<cuda::thread_scope_block> full_Q;
  // per-WG K barriers: [cid][stage]
  __shared__ cuda::barrier<cuda::thread_scope_block> full_K[kNumConsumerWGs][kStagesK];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty_K[kNumConsumerWGs][kStagesK];
  // per-WG V barriers: [cid] (kStagesV=1)
  __shared__ cuda::barrier<cuda::thread_scope_block> full_V[kNumConsumerWGs];
  __shared__ cuda::barrier<cuda::thread_scope_block> empty_V[kNumConsumerWGs];
#pragma nv_diag_default static_var_with_dynamic_init

  const bool is_producer = threadIdx.x < kProducerThreads;
  const int consumer_id = is_producer ? 0
                    : (threadIdx.x - kProducerThreads) / kConsumerThreadsPerWG;
  const int wg_tid = is_producer ? threadIdx.x
                    : (threadIdx.x - kProducerThreads) % kConsumerThreadsPerWG;

  if (threadIdx.x == 0) {
    init(&full_Q, 256 + 1);  // 2 consumer WGs * 128 + 1 producer
    for (int cid = 0; cid < kNumConsumerWGs; ++cid) {
      for (int s = 0; s < kStagesK; ++s) {
        init(&full_K[cid][s], kConsumerThreadsPerWG + 1);
        init(&empty_K[cid][s], kConsumerThreadsPerWG + 1);
      }
      init(&full_V[cid], kConsumerThreadsPerWG + 1);
      init(&empty_V[cid], kConsumerThreadsPerWG + 1);
    }
    tma_fence_proxy_async_shared_cta();
  }
  __syncthreads();

  // Helper: K_smem ptr for (cid, stg)
  // K layout: K_smem_base + cid*(Sk*Bc*D) + stg*(Bc*D)
  // D=128 chunk-major: each tile's 2 chunks at stg*Bc*D + c*Bc*64

  // ================================================================
  // Producer Warpgroup (WG0, threadIdx.x 0~127)
  // Single global tile loop over all KV tiles (0..Tc-1).
  // cid = tile & 1 routes TMA to the owning consumer WG's smem bank.
  //
  // Per-tile load order: V first, then K (prefetch next).
  //   - V[tile] TMA: blocked until consumer releases V[cid] (after PV).
  //     But K[tile] signal already sent in warmup/prev iter, so consumer
  //     can do QK[tile] while producer waits for V release.
  //   - K[tile+2] prefetch (Sk>=2): blocked until consumer releases K[cid][stg_next]
  //     (after QK). QK is first compute step, always done before producer reaches here.
  // ================================================================
  if (is_producer) {
    NOTES_V2_REG_DEALLOC(40);
    if (wg_tid == 0) {
      // P0: Load Q tile once
      for (int c = 0; c < kTmaChunks; ++c) {
        tma_load_2d(Q_smem + c * Br * kTmaBoxMinor, tensorMapQ,
```

```
                    c * kTmaBoxMinor, qkv_major_base + Q_tile_id * Br, full_Q);
      }
      tma_arrive_expect_tx(full_Q, kQTileBytes);

      // P1: Warmup - prefetch first K tile for each consumer WG (Sk-1 tiles)
      // tile 0 -> cid=0, tile 1 -> cid=1
      if constexpr (kStagesK > 1) {
        for (int cid = 0; cid < kNumConsumerWGs; ++cid) {
          const int tile0 = cid;  // first tile for this WG
          if (tile0 < Tc) {
            const int stg = 0;  // first slot
            empty_K[cid][stg].wait(empty_K[cid][stg].arrive());
            for (int c = 0; c < kTmaChunks; ++c) {
              tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
                              stg * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
                          tensorMapK, c * kTmaBoxMinor,
                          qkv_major_base + tile0 * Bc, full_K[cid][stg]);
            }
            tma_arrive_expect_tx(full_K[cid][stg], kKTileBytes);
          }
        }
      }

      // P2: Main loop over all global KV tiles
      // K 优先于 V: K 的 release 依赖 QK (tile 第一步 compute, 早完成),
      // V 的 release 依赖 PV (tile 最后一步 compute, 晚完成)。
      // 先发 K 让 consumer 尽早开始 QK, V 的 TMA 在 K 之后发出,
      // 被 QK+softmax 的 compute 时间隐藏。
      for (int tile = 0; tile < Tc; ++tile) {
        const int cid = tile & 1;

        // Step 1: Load/prefetch K first (release depends on QK, which is early)
        if constexpr (kStagesK == 1) {
          // Sk=1: load K[tile] to the single slot (reuse after consumer QK release)
          empty_K[cid][0].wait(empty_K[cid][0].arrive());
          for (int c = 0; c < kTmaChunks; ++c) {
            tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
                            c * Bc * kTmaBoxMinor,
                        tensorMapK, c * kTmaBoxMinor,
                        qkv_major_base + tile * Bc, full_K[cid][0]);
          }
          tma_arrive_expect_tx(full_K[cid][0], kKTileBytes);
        } else {
          // Sk>=2: prefetch K[tile+2] to next slot (true ping-pong pipeline!)
          const int local_tile = (tile - cid) / 2;
          const int stg_next = (local_tile + 1) % kStagesK;
          const int next_tile = tile + 2;
          if (next_tile < Tc) {
            empty_K[cid][stg_next].wait(empty_K[cid][stg_next].arrive());
            for (int c = 0; c < kTmaChunks; ++c) {
              tma_load_2d(K_smem_base + cid * (kStagesK * Bc * kHeadDim) +
                              stg_next * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
                          tensorMapK, c * kTmaBoxMinor,
                          qkv_major_base + next_tile * Bc,
                          full_K[cid][stg_next]);
            }
            tma_arrive_expect_tx(full_K[cid][stg_next], kKTileBytes);
          }
        }

        // Step 2: Load V[tile] (release depends on PV, which is late)
        empty_V[cid].wait(empty_V[cid].arrive());
        for (int c = 0; c < kTmaChunks; ++c) {
```

```
          tma_load_2d(V_smem_base + cid * Bc * kHeadDim + c * Bc * kTmaBoxMinor,
                      tensorMapV, c * kTmaBoxMinor,
                      qkv_major_base + tile * Bc, full_V[cid]);
        }
        tma_arrive_expect_tx(full_V[cid], kVTileBytes);
      }
    }
  }
  // ============================================================
  // Consumer Warpgroups (WG1 consumer_id=0, WG2 consumer_id=1)
  // Each WG processes every other KV tile (consumer_id, consumer_id+2, ...).
  // K stage index: local_iter % kStagesK (Sk=1: always 0; Sk=2: 0,1,0,1,...)
  // V stage index: always consumer_id (only 1 V buffer per WG)
  // ============================================================
  else {
    const int warp_id = wg_tid / kWarpSize;  // 0~3
    const int lane_id = wg_tid % kWarpSize;  // 0~31
    const int warp_QP = warp_id;
    const int warp_KV = 0;

    if constexpr (kHeadDim == 128) {
      NOTES_V2_REG_ALLOC(168);
    } else {
      NOTES_V2_REG_ALLOC(80);
    }

    // C0: init - mark OWN K/V slots as empty (consumer arrives, producer is 129th)
    for (int s = 0; s < kStagesK; ++s) {
      [[maybe_unused]] auto tk = empty_K[consumer_id][s].arrive();
    }
    {
      [[maybe_unused]] auto tv = empty_V[consumer_id].arrive();
    }

    // Wait for Q tile (shared by both WGs)
    full_Q.wait(full_Q.arrive());

    // Partial online-softmax state (per-WG, independent)
    float lane_block_row_max_old[kValTileSeqLenQ][2];
    float lane_block_row_sum_old[kValTileSeqLenQ][2];
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
    fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

    uint32_t R_Q[kValTileSeqLenQ][4];
    uint32_t R_K[kValTileSeqLenK][2];
    uint32_t R_V[kValTileHeadDimV][2];
    uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][kNRegs];
    uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
    uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV][kNRegs];
    fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_D, 0);

    // K smem offset for this WG: base + cid*(Sk*Bc*D)
    // stg within: local_iter % kStagesK
    // V smem offset for this WG: V_smem_base + cid*(Bc*D)
    const uint32_t smem_K_wg_base = smem_K_base_ptr +
        consumer_id * (kStagesK * Bc * kHeadDim) * sizeof(half);
    const uint32_t smem_V_wg_base = smem_V_base_ptr +
        consumer_id * (Bc * kHeadDim) * sizeof(half);

    int local_iter = 0;
    for (int tile = consumer_id; tile < Tc; tile += 2) {
      const int k_stg = local_iter % kStagesK;

```

```
      // K smem offset for this stage: wg_base + stg*(Bc*D)
      const uint32_t smem_K_stage_ptr = smem_K_wg_base +
          k_stg * Bc * kHeadDim * sizeof(half);

      // Wait for K[consumer_id][k_stg] ready
      full_K[consumer_id][k_stg].wait(full_K[consumer_id][k_stg].arrive());
      tma_fence_proxy_async_shared_cta();

      // ---- Q @ K^T = S[Br, Bc] ----
      // K early release: 在最后一个 tile_K_d 的 ldmatrix_K 完成后立即 release K,
      // 不等整个 QK MMA 循环结束。K 数据已全部读入 R_K 寄存器, smem 可被
      // producer 复用。这对 Sk=1 尤其重要: producer 可更早开始下一个 K TMA。
      fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, kNRegs>(R_S, 0);
#pragma unroll
      for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
#pragma unroll
        for (int i = 0; i < kValTileSeqLenQ; ++i) {
          const int warp_smem_Q_Br =
              warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
          const int lane_smem_Q_Br = warp_smem_Q_Br + lane_id % 16;
          const int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8;
          const uint32_t lane_smem_Q_ptr =
              smem_Q_base_ptr +
              swizzle_fa<kHeadDim, Br>(lane_smem_Q_Br, lane_smem_Q_d) *
                  sizeof(half);
          LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                      lane_smem_Q_ptr);
        }

#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          const int warp_smem_K_Bc =
              warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
          const int lane_smem_K_Bc = warp_smem_K_Bc + lane_id % 8;
          const int lane_smem_K_d =
              tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8;
          // K smem: stage_ptr + swizzle_fa offset (no stage*Bc*D since stage_ptr already includes it)
          const uint32_t lane_smem_K_ptr =
              smem_K_stage_ptr +
              swizzle_fa<kHeadDim, Bc>(lane_smem_K_Bc, lane_smem_K_d) *
                  sizeof(half);
          LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
        }

        // Early release K after last ldmatrix_K: K data now fully in registers
        if (tile_K_d == (kHeadDim / kMmaAtomK) - 1) {
          [[maybe_unused]] auto token = empty_K[consumer_id][k_stg].arrive();
        }

#pragma unroll
        for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileSeqLenK; ++j) {
            if constexpr (kMmaAccF32) {
              HMMA16816F32(R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3],
                           R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                           R_K[j][0], R_K[j][1],
                           R_S[i][j][0], R_S[i][j][1], R_S[i][j][2], R_S[i][j][3]);
            } else {
              HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1],
                        R_Q[i][2], R_Q[i][3], R_K[j][0], R_K[j][1],
                        R_S[i][j][0], R_S[i][j][1]);
            }
```

```
          }
        }
      }

      // K already released above (after last ldmatrix_K)

      // ---- Online Safe Softmax ----
      // This compute overlaps with producer's V[tile] TMA (which waits for
      // empty_V from previous PV, or is already in-flight)
      float lane_row_max_new[kValTileSeqLenQ][2];
      float lane_row_sum_new[kValTileSeqLenQ][2];
      fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
      fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);

#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          float tmp_max_0, tmp_max_1;
          if constexpr (kMmaAccF32) {
            float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
            tmp_max_0 = max(t_fptr_S[0], t_fptr_S[1]) * scale;
            tmp_max_1 = max(t_fptr_S[2], t_fptr_S[3]) * scale;
          } else {
            float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
            float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
            tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
            tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
          }
          lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
          lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
        }
        lane_row_max_new[i][0] =
            warp_reduce_max<4, float>(lane_row_max_new[i][0]);
        lane_row_max_new[i][1] =
            warp_reduce_max<4, float>(lane_row_max_new[i][1]);
      }

#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        float block_row_max_new_0 =
            max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
        float block_row_max_new_1 =
            max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);

#pragma unroll
        for (int j = 0; j < kValTileSeqLenK; ++j) {
          if constexpr (kMmaAccF32) {
            float *t_fptr_S = reinterpret_cast<float *>(&R_S[i][j][0]);
            half *t_hptr_S = reinterpret_cast<half *>(&R_S[i][j][0]);
            t_fptr_S[0] = __expf(__fmaf_rn(t_fptr_S[0], scale, -block_row_max_new_0));
            t_fptr_S[1] = __expf(__fmaf_rn(t_fptr_S[1], scale, -block_row_max_new_0));
            t_fptr_S[2] = __expf(__fmaf_rn(t_fptr_S[2], scale, -block_row_max_new_1));
            t_fptr_S[3] = __expf(__fmaf_rn(t_fptr_S[3], scale, -block_row_max_new_1));
            lane_row_sum_new[i][0] += (t_fptr_S[0] + t_fptr_S[1]);
            lane_row_sum_new[i][1] += (t_fptr_S[2] + t_fptr_S[3]);
            t_hptr_S[0] = __float2half_rn(t_fptr_S[0]);
            t_hptr_S[1] = __float2half_rn(t_fptr_S[1]);
            t_hptr_S[2] = __float2half_rn(t_fptr_S[2]);
            t_hptr_S[3] = __float2half_rn(t_fptr_S[3]);
          } else {
            float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
            float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
```

```
            t_reg_S_0.x = __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
            t_reg_S_0.y = __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
            t_reg_S_1.x = __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
            t_reg_S_1.y = __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
            lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
            lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
            HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
            HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
          }
        }
        lane_row_sum_new[i][0] =
            warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
        lane_row_sum_new[i][1] =
            warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
      }

      // ---- P @ V = O[Br, d] ----
      // Wait for V[consumer_id] ready
      full_V[consumer_id].wait(full_V[consumer_id].arrive());
      tma_fence_proxy_async_shared_cta();

      fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, kNRegs>(R_O, 0);
#pragma unroll
      for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          const int warp_smem_V_d =
              warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
          const int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
          const int lane_smem_V_d = warp_smem_V_d;
          // V smem: V_wg_base + swizzle_fa offset
          const uint32_t lane_smem_V_ptr =
              smem_V_wg_base +
              swizzle_fa<kHeadDim, Bc>(lane_smem_V_Bc, lane_smem_V_d) *
                  sizeof(half);
          LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
        }

        // Early release V after last ldmatrix_V: V data now fully in registers.
        // This lets producer start next V TMA during the remaining PV MMA +
        // online rescaling, giving more overlap window than releasing after PV.
        if (tile_V_Bc == (Bc / kMmaAtomK) - 1) {
          [[maybe_unused]] auto token = empty_V[consumer_id].arrive();
        }

        const int w = tile_V_Bc * 2;
#pragma unroll
        for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
          for (int j = 0; j < kValTileHeadDimV; ++j) {
            if constexpr (kMmaAccF32) {
              HMMA16816F32(R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3],
                           R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1],
                           R_V[j][0], R_V[j][1],
                           R_O[i][j][0], R_O[i][j][1], R_O[i][j][2], R_O[i][j][3]);
            } else {
              HMMA16816(R_O[i][j][0], R_O[i][j][1],
                        R_S[i][w][0], R_S[i][w][1],
                        R_S[i][w + 1][0], R_S[i][w + 1][1],
                        R_V[j][0], R_V[j][1],
                        R_O[i][j][0], R_O[i][j][1]);
            }
          }
```

```
        }
      }

      // V already released above (after last ldmatrix_V)

      // ---- Online rescaling - O_new = exp(m_old - m_new) * O_old + P@V ----
#pragma unroll
      for (int i = 0; i < kValTileSeqLenP; ++i) {
        float block_row_max_new_0 = lane_row_max_new[i][0];
        float block_row_max_new_1 = lane_row_max_new[i][1];
        float block_row_sum_new_0 = lane_row_sum_new[i][0];
        float block_row_sum_new_1 = lane_row_sum_new[i][1];
        float block_row_max_old_0 = lane_block_row_max_old[i][0];
        float block_row_max_old_1 = lane_block_row_max_old[i][1];

        block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
        block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);

        // First iteration per WG: m_old = -inf, use m_new directly
        block_row_max_old_0 = (local_iter > 0 ? block_row_max_old_0
                                              : block_row_max_new_0);
        block_row_max_old_1 = (local_iter > 0 ? block_row_max_old_1
                                              : block_row_max_new_1);

        float rescale_o_factor_0 =
            __expf(block_row_max_old_0 - block_row_max_new_0);
        float rescale_o_factor_1 =
            __expf(block_row_max_old_1 - block_row_max_new_1);

#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          if constexpr (kMmaAccF32) {
            float *t_fptr_O = reinterpret_cast<float *>(&R_O[i][j][0]);
            float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
            t_fptr_D[0] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[0], t_fptr_O[0]);
            t_fptr_D[1] = __fmaf_rn(rescale_o_factor_0, t_fptr_D[1], t_fptr_O[1]);
            t_fptr_D[2] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[2], t_fptr_O[2]);
            t_fptr_D[3] = __fmaf_rn(rescale_o_factor_1, t_fptr_D[3], t_fptr_O[3]);
          } else {
            float2 t_reg_O_0 = __half22float2(HALF2(R_O[i][j][0]));
            float2 t_reg_O_1 = __half22float2(HALF2(R_O[i][j][1]));
            float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
            float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
            t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_O_0.x);
            t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_O_0.y);
            t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_O_1.x);
            t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_O_1.y);
            HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
            HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
          }
        }

        float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
        float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
        lane_block_row_sum_old[i][0] =
            __fmaf_rn(rescale_o_factor_0, block_row_sum_old_0,
                      block_row_sum_new_0);
        lane_block_row_sum_old[i][1] =
            __fmaf_rn(rescale_o_factor_1, block_row_sum_old_1,
                      block_row_sum_new_1);
        lane_block_row_max_old[i][0] = block_row_max_new_0;
        lane_block_row_max_old[i][1] = block_row_max_new_1;
      }
```

```
      ++local_iter;
    }

    // ================================================================
    // Final merge of two WG partial states (split-KV reduction)
    //
    // 安全 alias dynamic smem 的前提:
    //   1) producer 已退出 mainloop (所有 TMA 已 arrive_expect_tx);
    //   2) 两个 consumer 都已 release 自己的 K/V stage (QK 后 release K,
    //      PV 后 release V), 且 mainloop 结束后不再访问任何 K/V smem;
    //   3) Q_smem 在 mainloop 内只读, 此后也不再访问。
    // 故此时整个 dynamic smem (Q/K/V 区) 生命周期结束, 可安全重命名为
    // merge scratch, 不增加 launch smem_bytes。第一次 __syncthreads
    // 保证所有 TMA/ldmatrix 完成后才开始写 scratch。
    // 完整的 split-KV merge 数学推导见 kernel 头部注释。
    // ================================================================
    __syncthreads();

    // Merge scratch 布局 (按 wg_tid 索引, 每 WG 128 thread):
    //   [merge_rd]: 128 * kRdPerThread 个 uint32  <- WG2 未归一化 R_D fragment
    //   [merge_ml]: 128 * kValTileSeqLenQ 个 float4 <- WG2 的 [m0,m1,l0,l1] interleaved
    // WG1 用自己的 R_D/m/l 作为 (Oacc_0, m_0, l_0), 读取 WG2 写出的作为
    // (Oacc_1, m_1, l_1), 按 stable merge 公式归并后由 WG1 写最终 O。
    // m/l interleaved 为 float4 布局, 写入/读取各一次 128-bit store/load。
    constexpr int kRdPerThread = kValTileSeqLenP * kValTileHeadDimV * kNRegs;
    constexpr int kRdPackPerThread = kRdPerThread / 4;  // float4 个数 (kNRegs 始终偶数)
    constexpr int kMlPackPerThread = kValTileSeqLenQ;   // 每 thread 的 float4 个数
    uint32_t *merge_rd = reinterpret_cast<uint32_t *>(smem_fa3_tma_ws);
    float4 *merge_rd_pack = reinterpret_cast<float4 *>(merge_rd);
    float4 *merge_ml_pack = merge_rd_pack + 128 * kRdPackPerThread;

    // WG2 写出未归一化 partial (R_D, m, l) 到 scratch, 按 wg_tid 索引。
    // R_D: 以 float4 (128-bit) 连续写入, kRdPackPerThread 次。
    // m/l: interleaved 为 float4 [m0, m1, l0, l1], 每次 128-bit store。
    // 按 wg_tid 索引保证 WG1 的同一 lane 读到对应行的 WG2 partial:
    //   R_D[i][j][k] fragment 对应的 Q rows 与 lane_block_row_max/sum[i][0/1]
    //   的两组 rows (rows 0-7 / rows 8-15) 严格对齐, 故 WG1/WG2 同一
    //   wg_tid 的 partial 覆盖相同 Q 行, 可直接逐元素归并。
    // Tc==1 时 WG2 未进入循环, m=-inf/l=0/R_D=0, merge 自然退化。
    // 写完后第二次 __syncthreads 让 WG1 看到 WG2 的全部写入。
    if (consumer_id == 1) {
      // R_D: 128-bit stores
      float4 *rd_dst = merge_rd_pack + wg_tid * kRdPackPerThread;
#pragma unroll
      for (int idx = 0; idx < kRdPackPerThread; ++idx)
        rd_dst[idx] = reinterpret_cast<float4 *>(&R_D[0][0][0])[idx];
      // m/l interleaved: [m0, m1, l0, l1] per i, 128-bit store
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        float4 ml;
        ml.x = lane_block_row_max_old[i][0];
        ml.y = lane_block_row_max_old[i][1];
        ml.z = lane_block_row_sum_old[i][0];
        ml.w = lane_block_row_sum_old[i][1];
        merge_ml_pack[wg_tid * kMlPackPerThread + i] = ml;
      }
    }
    __syncthreads();

    // WG1 merges WG2's partial into its own, normalizes, and writes O
    if (consumer_id == 0) {
      // 把 WG2 的未归一化 R_D (Oacc_1) 载入 R_O 寄存器复用为临时 buffer。
      // R_D: 以 float4 (128-bit) 连续读取。
```

```
      // 后续 merge 用 R_D[i][j] 作为 Oacc_0, R_O[i][j] 作为 Oacc_1, 逐元素加权后
      // 结果写回 R_D, 再做最终 O=Oacc/l 归一化。
      float4 *rd_src = merge_rd_pack + wg_tid * kRdPackPerThread;
#pragma unroll
      for (int idx = 0; idx < kRdPackPerThread; ++idx)
        reinterpret_cast<float4 *>(&R_O[0][0][0])[idx] = rd_src[idx];

      // 稳定归并 (实现层面, 完整数学见头部注释):
      //   - m0/l0 = WG1 自己的 partial (寄存器), m1/l1 = WG2 的 partial (scratch)
      //   - [i][0] 对应 Q rows [i*16 .. i*16+7], [i][1] 对应 rows [i*16+8 .. +15]
      //   - alpha=exp(m0-m)<=1, beta=exp(m1-m)<=1: 用全局 max 做 pivot 避免 exp 溢出
      //   - F32Acc: 直接对 float fragment 逐元素加权求和
      //   - F16Acc: 把 packed half2 提升到 float2 计算, 再 pack 回 half2
      //     (避免 half 精度下 alpha/beta 加权累加的精度损失)
      //   - Tc==1: m1=-inf -> beta=exp(-inf)=0, Oacc/Oacc_1 项归零, 退化为 WG1 结果
      //   m=max(m0,m1), a=exp(m0-m), b=exp(m1-m),
      //   l=a*l0+b*l1, Oacc=a*Oacc0+b*Oacc1
#pragma unroll
      for (int i = 0; i < kValTileSeqLenQ; ++i) {
        // 128-bit load: [m1_0, m1_1, l1_0, l1_1] interleaved
        float4 ml1 = merge_ml_pack[wg_tid * kMlPackPerThread + i];
        float m1_0 = ml1.x;
        float m1_1 = ml1.y;
        float l1_0 = ml1.z;
        float l1_1 = ml1.w;

        float m0_0 = lane_block_row_max_old[i][0];
        float m0_1 = lane_block_row_max_old[i][1];
        float l0_0 = lane_block_row_sum_old[i][0];
        float l0_1 = lane_block_row_sum_old[i][1];

        float m_0 = fmaxf(m0_0, m1_0);
        float m_1 = fmaxf(m0_1, m1_1);
        float alpha_0 = __expf(m0_0 - m_0);
        float alpha_1 = __expf(m0_1 - m_1);
        float beta_0 = __expf(m1_0 - m_0);
        float beta_1 = __expf(m1_1 - m_1);

        lane_block_row_max_old[i][0] = m_0;
        lane_block_row_max_old[i][1] = m_1;
        lane_block_row_sum_old[i][0] = alpha_0 * l0_0 + beta_0 * l1_0;
        lane_block_row_sum_old[i][1] = alpha_1 * l0_1 + beta_1 * l1_1;

#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          if constexpr (kMmaAccF32) {
            float *d0 = reinterpret_cast<float *>(&R_D[i][j][0]);
            float *d1 = reinterpret_cast<float *>(&R_O[i][j][0]);
            d0[0] = alpha_0 * d0[0] + beta_0 * d1[0];
            d0[1] = alpha_0 * d0[1] + beta_0 * d1[1];
            d0[2] = alpha_1 * d0[2] + beta_1 * d1[2];
            d0[3] = alpha_1 * d0[3] + beta_1 * d1[3];
          } else {
            float2 d0_0 = __half22float2(HALF2(R_D[i][j][0]));
            float2 d0_1 = __half22float2(HALF2(R_D[i][j][1]));
            float2 d1_0 = __half22float2(HALF2(R_O[i][j][0]));
            float2 d1_1 = __half22float2(HALF2(R_O[i][j][1]));
            d0_0.x = alpha_0 * d0_0.x + beta_0 * d1_0.x;
            d0_0.y = alpha_0 * d0_0.y + beta_0 * d1_0.y;
            d0_1.x = alpha_1 * d0_1.x + beta_1 * d1_1.x;
            d0_1.y = alpha_1 * d0_1.y + beta_1 * d1_1.y;
            HALF2(R_D[i][j][0]) = __float22half2_rn(d0_0);
            HALF2(R_D[i][j][1]) = __float22half2_rn(d0_1);
```

```
          }
        }
      }

      // ---- Final rescale: O = Oacc / l ----
#pragma unroll
      for (int i = 0; i < kValTileSeqLenP; ++i) {
        float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
        float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          if constexpr (kMmaAccF32) {
            float *t_fptr_D = reinterpret_cast<float *>(&R_D[i][j][0]);
            half *t_hptr_D = reinterpret_cast<half *>(&R_D[i][j][0]);
            t_hptr_D[0] = __float2half_rn(rescale_factor_0 * t_fptr_D[0]);
            t_hptr_D[1] = __float2half_rn(rescale_factor_0 * t_fptr_D[1]);
            t_hptr_D[2] = __float2half_rn(rescale_factor_1 * t_fptr_D[2]);
            t_hptr_D[3] = __float2half_rn(rescale_factor_1 * t_fptr_D[3]);
          } else {
            float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
            float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
            t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
            t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
            t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
            t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
            HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
            HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
          }
        }
      }

      // ---- Epilogue: warp shuffle + 128-bit collective store ----
#pragma unroll
      for (int i = 0; i < kValTileSeqLenP; ++i) {
#pragma unroll
        for (int j = 0; j < kValTileHeadDimV; ++j) {
          uint32_t R_Z[2][4];
          R_Z[0][0] = R_D[i][j][0];
          R_Z[1][0] = R_D[i][j][1];
          R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
          R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
          R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
          R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
          R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
          R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

          if (lane_id % 4 == 0) {
            const int store_warp_regs_O_Br =
                warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
            const int store_lane_gmem_O_Br =
                Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
            const int store_warp_regs_O_d =
                warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
            const int store_lane_gmem_O_d = store_warp_regs_O_d;
            const int store_gmem_O_addr_0 =
                O_gmem_offset + store_lane_gmem_O_Br * kHeadDim +
                store_lane_gmem_O_d;
            const int store_gmem_O_addr_1 =
                O_gmem_offset + (store_lane_gmem_O_Br + 8) * kHeadDim +
                store_lane_gmem_O_d;
            *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_0]) =
                *reinterpret_cast<float4 *>(&R_Z[0][0]);
            *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_1]) =
```

```
                *reinterpret_cast<float4 *>(&R_Z[1][0]);
          }
        }
      }
    }
  }
}

#if defined(NOTES_V2_ENABLE_CUTE)
namespace fa_cute {
using namespace cute;

// FlashAttn2CuTeTraits: FA2-style CuTe kernel 配置 (Br=128, Bc=64, 单 consumer WG)。
// 与 FlashAttn3CuTeTraits 的核心区别:
//   - Br=128 -> 8 warps along M (vs FA3 的 4), Q tile [128, D] (vs [64, D])
//   - 单 consumer WG 全 KV 遍历, 无 split-KV merge
//   - K/V tile 仍为 [64, D], SmemLayoutAtom/MmaAtom/SmemCopyAtom 与 FA3 一致
//     (设计原因见 FlashAttn3CuTeTraits 头注释)
//
// 设计差异详述:
// ── SmemLayout (Q 更大, K/V 不变) ──
//   SmemLayoutQ = tile_to_shape(atom, (128, D)): Q tile [128, D] (Br=128),
//     是 FA3 Q tile [64, D] 的 2 倍行数。8×8 atom 按 128 行重复, swizzle pattern 不变。
//   SmemLayoutKV = tile_to_shape(atom, (64, D)): K/V tile [64, D] (Bc=64),
//     与 FA3 的 SmemLayoutQKV 完全一致。
//   SmemLayoutVt: 同 FA3, composition 叠加 col-major (D, 64) 得 V^T 转置视图。
//
// ── MMA (8 warps, 单 WG 覆盖 Br=128) ──
//   TiledMma = TiledMMA(atom, EURepeat<8,1,1>, Tile<128,16,16>):
//     - EURepeat<8,1,1>: 8 warp 沿 M 重复 MMA atom
//       -> 8 warps × 32 threads = 256 threads = 1 consumer WG (FA3 为 128 threads)
//     - Tile<128,16,16>: 逻辑 MMA tile
//       M=128 = 8 warps × 16 rows = Br (单 WG 覆盖 Br=128, FA3 为 64)
//       N=16, K=16: 与 FA3 一致
//   MmaAtom / SmemCopyAtom / SmemCopyAtomTransposed 与 FA3 完全相同,
//   不再重复说明 (见 FlashAttn3CuTeTraits 头注释)。
// 复用 fa_cute 的 gemm_ss / gemm_rs / convert_layout_acc_* / convert_type。
template <int kHeadDim>
struct FlashAttn2CuTeTraits {
  static_assert(kHeadDim == 64 || kHeadDim == 128);

  using Element = cutlass::half_t;
  using SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>;
  // Q tile: [Br=128, D], K/V tile: [Bc=64, D]
  using SmemLayoutQ = decltype(tile_to_shape(
      SmemLayoutAtom{}, Shape<_128, Int<kHeadDim>>{}));
  using SmemLayoutKV = decltype(tile_to_shape(
      SmemLayoutAtom{}, Shape<_64, Int<kHeadDim>>{}));
  using SmemLayoutVt = decltype(composition(
      SmemLayoutKV{},
      make_layout(Shape<Int<kHeadDim>, _64>{}, GenRowMajor{})));

  // 8 warps along M (Br=128 = 8*16), ValTile N=2 (16 cols = 2*8)
  using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;
  using TiledMma = TiledMMA<
      MmaAtom, Layout<Shape<_8, _1, _1>>,
      Tile<_128, _16, _16>>;
  using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
  using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
};

// FlashAttn3CuTeTraits: FA3-style CuTe kernel 编译期配置 (Br=Bc=64, 双 consumer WG)。
// 所有 layout/atom/tiler 都是编译期类型, kernel body 只接收实例化后的类型。
```

```
//
// ── SmemLayout 设计 (与 TMA 128B swizzle 匹配, 保证 ldmatrix 无 bank conflict) ──
//   SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>:
//     - GMMA 128B swizzle atom: (8, 8) layout + Swizzle<3,4,3>
//     - 一个 atom 覆盖 8 行 × 8 half = 128B, 恰好一个 swizzle 周期
//     - 与 TMA CU_TENSOR_MAP_SWIZZLE_128B 写入 pattern 完全一致:
//       producer 用 TMA 按 128B 粒度写入 swizzle 地址, consumer 用 cute::copy
//       读取时由同款 swizzle layout 推导物理地址, 读写两侧 swizzle 一致 -> 无 bank conflict
//   SmemLayoutQKV = tile_to_shape(atom, (64, D)):
//     - Q/K/V tile 都是 [Br=Bc=64, D] (FA3 中 Br=Bc=64)
//     - tile_to_shape 把 8×8 atom 按 (64, D) 重复, 保持 swizzle pattern
//   SmemLayoutVt = composition(QKV, col-major (D, 64)):
//     - V 转置视图: P@V 时 V 作为 MMA 的 B 矩阵, mma.sync.row.col 要求
//       B 为 col-major。V 物理存储是 row-major [64, D], 通过 composition
//       叠加 col-major (D, 64) 逻辑视图, 得到 V^T[D, 64] 访问接口。
//     - composition 不搬数据, 只改 layout 解读, 物理 swizzle 不变。
//
// ── MMA 设计 (Ampere m16n8k16, f32 累加保证 softmax 精度) ──
//   MmaAtom = SM80_16x8x16_F32F16F16F32_TN:
//     - f16 输入 + f32 累加。FA 的 softmax 对精度敏感 (exp/sum),
//       f16 累加会导致 row sum 溢出/下溢, 必须用 f32。
//     - TN 布局: A(row-major) × B(col-major), 与 smem 物理布局天然匹配。
//   TiledMma = TiledMMA(atom, EURepeat<4,1,1>, Tile<64,16,16>):
//     - EURepeat<4,1,1>: 4 warp 沿 M 重复 MMA atom
//       -> 4 warps × 32 threads = 128 threads = 1 consumer WG
//     - Tile<64,16,16>: 逻辑 MMA tile
//       M=64 = 4 warps × 16 rows = Br (单 WG 覆盖 Br=64)
//       N=16 = 2 × kMmaN(8) -> 一次覆盖 16 列
//       K=16 = kMmaK (单次 MMA K slice)
//
// ── SmemCopy 设计 (S->R ldmatrix, 与 smem swizzle layout 配合) ──
//   SmemCopyAtom = SM75_U32x4_LDSM_N:
//     - 对应 ldmatrix.sync.aligned.x4.m8n8.shared.b16 (非转置)
//     - 一次加载 4 个 8×8 half 片段到 4 条 uint32 寄存器
//     - 用于 Q 和 K: Q/K 在 smem 是 row-major, ldmatrix.x4 非转置加载
//       得到 col-major fragment, 匹配 mma.row.col 的 A/B 输入约定
//   SmemCopyAtomTransposed = SM75_U16x8_LDSM_T:
//     - 对应 ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 (转置)
//     - 用于 V: V 在 smem 是 row-major [Bc, D], P@V 需把 V 当 col-major
//       B 矩阵。ldmatrix.x2.trans 转置加载为 col-major fragment, 直接匹配
//       mma.row.col 的 B 输入, 无需软件转置 V smem。
//     - 用 x2 (非 x4): V 的 K 维 (Bc=64) 按 kMmaK=16 切片, 每片 16×8
//       = 1 个 ldmatrix.x2 (2 个 8×8 fragment)
template <int kHeadDim>
struct FlashAttn3CuTeTraits {
  static_assert(kHeadDim == 64 || kHeadDim == 128);

  using Element = cutlass::half_t;
  using SmemLayoutAtom = GMMA::Layout_K_SW128_Atom<Element>;
  using SmemLayoutQKV = decltype(tile_to_shape(
      SmemLayoutAtom{}, Shape<_64, Int<kHeadDim>>{}));
  using SmemLayoutVt = decltype(composition(
      SmemLayoutQKV{},
      make_layout(Shape<Int<kHeadDim>, _64>{}, GenRowMajor{})));

  using MmaAtom = MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>;
  using TiledMma = TiledMMA<
      MmaAtom, Layout<Shape<_4, _1, _1>>,
      Tile<_64, _16, _16>>;
  using SmemCopyAtom = Copy_Atom<SM75_U32x4_LDSM_N, Element>;
  using SmemCopyAtomTransposed = Copy_Atom<SM75_U16x8_LDSM_T, Element>;
};

```

```
// convert_layout_acc_rowcol: 把 MMA accumulator 的 (MMA=4, MMA_M, MMA_N)
// layout 转换为 ((2, MMA_M), (2, MMA_N)) 的 (nrow, ncol) 二维布局。
//
// 为什么需要转换? FA 的 online softmax 需要按"行"操作 score 矩阵 S:
//   - 求 row max、exp(S - max)、row sum 都要求同一行的元素聚集在同一线程。
//   - 但 m16n8k16 的 MMA C fragment 的 4 个寄存器按 (MMA=4, MMA_M, MMA_N)
//     排布, lane 0/4/8/.. 持有同一行片段但散布在 MMA mode 0/1。
//   - logical_divide(_, Shape<_2>{}) 把 MMA=4 拆成 (2, 2):
//       (2, 2, MMA_M, MMA_N) -> 重组为 ((2, MMA_M), (2, MMA_N))
//       外层 (2, MMA_M) = nrow: 前 2 个 MMA mode × MMA_M 组成"行块"
//       内层 (2, MMA_N) = ncol: 后 2 个 MMA mode × MMA_N 组成"列块"
//   - 转换后 scores(r, c) 的 r 索引行、c 索引列, online softmax 可直接遍历
//     size<1>(scores) 做行内 reduce max/sum, 再用 __shfl_xor_sync 在 4-lane
//     子组内归约 (因为 m16n8k16 下每 4 个 lane 共享同一行的不同列片段)。
//
// 出处: flash-attention/csrc/flash_attn/src/utils.h:188
//       flash-attention/csrc/flash_attn/src/softmax.h:139 (调用点)
//       用于 tCrS (QK score) 和 tCrO (PV accumulator) 的 rowcol 视图。
template <typename Layout>
CUTE_DEVICE auto convert_layout_acc_rowcol(Layout acc_layout) {
  auto divided = logical_divide(acc_layout, Shape<_2>{});
  return make_layout(
      make_layout(get<0, 1>(divided), get<1>(divided)),
      make_layout(get<0, 0>(divided), get<2>(divided)));
}

// convert_layout_acc_Aregs: 把 MMA accumulator (QK 的 score S) 转换为
// P@V 所需的 A-register (左矩阵 P) layout, 使 softmax 后的 P 能直接喂给
// 下一个 MMA 做 P@V, 无需额外数据搬运。
//
// 为什么需要转换? QK 和 PV 用同一个 TiledMma, 但 C fragment 和 A fragment
// 的寄存器排布不同:
//   - QK: S = Q @ K^T, S 是 C fragment, layout = (MMA=4, MMA_M, MMA_N)
//   - PV: O = P @ V, P 是 A fragment, layout = (MMA, MMA_M, MMA_K)
//   - m16n8k16 的 MMA=4 对应 4 个寄存器, C 的 4 个按 (2行, 2列) 排布,
//     A 的 4 个按 (2行, 2K-slice) 排布, 直接用 C layout 做 A 会导致
//     寄存器对齐错误。
//   - logical_divide(_, Shape<X, X, _2>{}) 把 MMA_N 拆成 (2, MMA_N/2):
//       (MMA=4, MMA_M, (2, MMA_N/2)) -> 重组为 ((4, 2), MMA_M, MMA_N/2)
//       外层 (4, 2) = 新的 MMA mode, 把 C 的列方向 2 个 half 配对到
//       A 的 K 方向连续 2 个元素, 匹配 m16n8k16 的 K=16 内部 2-element 步长。
//   - 转换后 tCrPv 可直接作为 gemm_rs 的 tCrA 参数, P 的数据仍在原寄存器中,
//     只是逻辑坐标重解释, 零拷贝。
//
// ★ 这就是 FA "寄存器复用" 的核心技巧: softmax 后的 P 不写回 smem 再
//   ldmatrix, 而是原地重解释为 A fragment, 省一次 smem 往返。
//   但注意这依赖 m16n8k16 的特定 fragment 约定, 不是通用性质。
//
// 出处: flash-attention/csrc/flash_attn/src/utils.h:200
//       flash-attention/csrc/flash_attn/src/flash_fwd_kernel.h:365 (调用点)
//       注释 "Reshape rP from (MMA=4, MMA_M, MMA_N) to ((4, 2), MMA_M, MMA_N / 2)"
template <typename TiledMma, typename Layout>
CUTE_DEVICE auto convert_layout_acc_Aregs(Layout acc_layout) {
  using X = Underscore;
  auto divided = logical_divide(acc_layout, Shape<X, X, _2>{});
  return make_layout(
      make_layout(get<0>(divided), get<2, 0>(divided)),
      get<1>(divided), get<2, 1>(divided));
}

// convert_type: 在寄存器层面做 dtype 转换, 不搬数据只改元素类型解释。
//
// 为什么需要? MMA accumulator 是 f32 (F32F16F16F32_TN), 但下游消费需要 f16:
```

```
//   - tCrP = convert_type<Element>(tCrS): softmax 后的 score S (f32) 转 P (f16),
//     供 P@V 的 A fragment (mma 要求 f16 输入)
//   - tCrO_half = convert_type<Element>(tCrO): 最终 O (f32 acc) 转 f16, 供 store
//
// 实现: NumericArrayConverter 一次性批量转换整个 fragment (kElements 个元素),
// 返回的 tensor 共享原 layout, 只是 value_type 从 From 变为 To。
// 零拷贝, 纯寄存器内转换 (make_rmem_ptr 标记为 register memory)。
//
// 出处: flash-attention/csrc/flash_attn/src/epilogue/epilogue.hpp (同类实现)
template <typename To, typename Engine, typename Layout>
CUTE_DEVICE auto convert_type(Tensor<Engine, Layout> const &tensor) {
  using From = typename Engine::value_type;
  constexpr int kElements = decltype(size(tensor))::value;
  cutlass::NumericArrayConverter<To, From, kElements> convert;
  auto fragment = convert(
      *reinterpret_cast<cutlass::Array<From, kElements> const *>(tensor.data()));
  return make_tensor(make_rmem_ptr<To>(&fragment), tensor.layout());
}

// gemm_ss: Shared-Shared GEMM, A 和 B 都从 smem 加载到寄存器再做 MMA。
// 用于 FA 的 Q@K^T 步骤: Q 和 K 都在 smem 中 (Q 由 TMA/cp.async 预加载,
// K 由 TMA 加载到当前 stage), 需要 ldmatrix 同时搬运 A(Q) 和 B(K)。
//
// 流程 (software-pipelined ldmatrix + mma):
//   1. retile_D: 把 A/B 的 register fragment 按 TiledCopy 的 source 视图重排,
//      使 copy() 能正确写入后续 MMA 消费的寄存器位置。
//   2. 预加载 tile_k=0: copy A[0] 和 B[0] 到寄存器。
//   3. for tile_k = 0..MMA_K-1:
//        a. 若 tile_k+1 存在, 预加载 A[tile_k+1] 和 B[tile_k+1] (与当前 MMA 重叠)
//        b. gemm(mma, A[tile_k], B[tile_k], acc) - 发射 mma.sync 指令
//   每个 K slice 的 ldmatrix 与前一个 slice 的 MMA 重叠执行, 隐藏 smem->reg 延迟。
//
// 参数:
//   acc: (MMA, MMA_M, MMA_N) 累加器, 输入输出
//   fragment_a/fragment_b: A/B 寄存器 fragment, 会被 retile 后复用
//   shared_a/shared_b: smem 中的 A/B tile (带 swizzle layout)
//   tiled_mma: TiledMMA 类型实例
//   tiled_copy_a/b: A/B 的 S->R TiledCopy (make_tiled_copy_A/B 生成)
//   thread_copy_a/b: A/B 的线程级 copy slice
//
// 出处: flash-attention/csrc/flash_attn/src/utils.h:166 (gemm_rs 的对照)
//       FlashMLA/csrc/sm90/helpers.h:97 (gemm_ss 的同类实现)
//       本 notes 的 QK GEMM 调用: fa_cute::gemm_ss(tCrS, tCrQ, tCrK, ...)
template <typename TensorC, typename TensorA, typename TensorB,
          typename TensorSA, typename TensorSB, typename TiledMma,
          typename TiledCopyA, typename TiledCopyB,
          typename ThreadCopyA, typename ThreadCopyB>
CUTE_DEVICE void gemm_ss(
    TensorC &acc, TensorA &fragment_a, TensorB &fragment_b,
    TensorSA const &shared_a, TensorSB const &shared_b,
    TiledMma tiled_mma, TiledCopyA tiled_copy_a, TiledCopyB tiled_copy_b,
    ThreadCopyA thread_copy_a, ThreadCopyB thread_copy_b) {
  auto copy_view_a = thread_copy_a.retile_D(fragment_a);
  auto copy_view_b = thread_copy_b.retile_D(fragment_b);
  copy(tiled_copy_a, shared_a(_, _, _0{}), copy_view_a(_, _, _0{}));
  copy(tiled_copy_b, shared_b(_, _, _0{}), copy_view_b(_, _, _0{}));
#pragma unroll
  for (int tile_k = 0; tile_k < size<2>(fragment_a); ++tile_k) {
    if (tile_k + 1 < size<2>(fragment_a)) {
      copy(tiled_copy_a, shared_a(_, _, tile_k + 1),
           copy_view_a(_, _, tile_k + 1));
      copy(tiled_copy_b, shared_b(_, _, tile_k + 1),
           copy_view_b(_, _, tile_k + 1));
```

```
    }
    gemm(tiled_mma, fragment_a(_, _, tile_k),
         fragment_b(_, _, tile_k), acc);
  }
}

// gemm_rs: Register-Shared GEMM, A 在寄存器中（已是 fragment），B 从 smem 加载。
// 用于 FA 的 P@V 步骤：P 已经在寄存器中（softmax 后的 tCrS，经 convert_layout_acc_Aregs
// 重解释为 A fragment），V 在 smem 中（TMA 加载），只需 ldmatrix 搬运 B(V)。
//
// 与 gemm_ss 的区别：A 不从 smem 加载（P 已在寄存器），省 A 的 ldmatrix，
// 这就是 FA "寄存器复用" 的性能优势：softmax 后的 P 不回写 smem，直接做 A。
//
// 流程（与 gemm_ss 对称，但只 pipelined B）：
//   1. retile_D: B 的 register fragment 按 TiledCopy source 视图重排。
//   2. 预加载 tile_k=0: copy B[0] 到寄存器。
//   3. for tile_k = 0..MMA_K-1:
//        a. 若 tile_k+1 存在，预加载 B[tile_k+1]
//        b. gemm(mma, A[tile_k], B[tile_k], acc) - A 是 P（寄存器），B 是 V（smem）
//
// 参数：
//   acc: (MMA, MMA_M, MMA_N) 累加器，输出 O
//   fragment_a: P 寄存器 fragment（softmax 后的 score，已 Aregs 转换）
//   fragment_b: V 寄存器 fragment（会被 retile 后复用）
//   shared_b: smem 中的 V tile（带 transposed swizzle layout）
//
// 出处：flash-attention/csrc/flash_attn/src/utils.h:166
//       flash-attention/csrc/flash_attn/src/flash_fwd_kernel.h:367（调用点）
//       本 notes 的 PV GEMM 调用：fa_cute::gemm_rs(tCrO, tCrPv, tCrV, ...)
template <typename TensorC, typename TensorA, typename TensorB,
          typename TensorSB, typename TiledMma, typename TiledCopyB,
          typename ThreadCopyB>
CUTE_DEVICE void gemm_rs(
    TensorC &acc, TensorA &fragment_a, TensorB &fragment_b,
    TensorSB const &shared_b, TiledMma tiled_mma,
    TiledCopyB tiled_copy_b, ThreadCopyB thread_copy_b) {
  auto copy_view_b = thread_copy_b.retile_D(fragment_b);
  copy(tiled_copy_b, shared_b(_, _, _0{}), copy_view_b(_, _, _0{}));
#pragma unroll
  for (int tile_k = 0; tile_k < size<2>(fragment_a); ++tile_k) {
    if (tile_k + 1 < size<2>(fragment_a)) {
      copy(tiled_copy_b, shared_b(_, _, tile_k + 1),
           copy_view_b(_, _, tile_k + 1));
    }
    gemm(tiled_mma, fragment_a(_, _, tile_k),
         fragment_b(_, _, tile_k), acc);
  }
}

}  // namespace fa_cute

// ========================================================================
// FA2 CuTe cp.async MMA Stages Split-Q (no TMA, no Warp Specialization)
// ========================================================================
// 是 flash_attn_tma_mma_ws_split_q_cute 的简化版：去掉 TMA producer/consumer WS，
// 改用 cp.async + 统一 pipeline（参考 hgemm_mma_stages_tn_cute 的 cute::copy 用法
// + flash_attn_mma_stages_split_q 手写版本的 V/K group 管理策略）。
//
// ★ 核心设计：
//   - 只支持 kStagesK（V 固定单 buffer，无 kStagesV）
//   - V 在 QK 之前发起 cp.async，通过 QK+softmax 隐藏 V 延迟
//   - V/K 分开 commit_group，利用 cp.async group 的 LIFO 栈特性选择性 wait
//
```

```cpp
// ★ cp.async group 管理策略（对齐手写版 L4323-4765）：
//   提交顺序（每轮 tile）：
//     1. copy V[tile] -> sV; cp_async_fence()  (group V_tile，栈底)
//     2. copy K[tile+kStagesK-1] -> sK[write]; cp_async_fence()  (group K_next，栈顶)
//   等待策略：
//     QK 前：kStagesK==1 -> wait<0>+sync（V+K 都是当前 tile，全等）
//            kStagesK>1  -> 无 wait（K[tile] 已在上一轮循环末尾 wait<0> 完成）
//     PV 前：kStagesK>1 且非最后 tile -> wait<1>+sync（栈顶 K_next 未完成，V_tile 完成）
//            否则 -> wait<0>+sync
//     循环末尾：kStagesK>1 且非最后 tile -> wait<0>+sync（确保 K_next 完成，
//              下一轮 QK 安全；同时保护 V/K smem 不被提前覆盖）
//
// ★ 同步链（参考手写版 6 个同步点）：
//   1. Q load: copy + fence + wait<0> + sync
//   2. PREFETCH K[0..Sk-2]: copy + fence，然后 wait<Sk-2> + sync
//   3. 每轮 tile:
//      3a: V[tile] fence + K_next fence (kStagesK>1) 或 K[tile] fence (kStagesK==1)
//      3b: QK 前 wait (kStagesK==1: wait<0>+sync; kStagesK>1: 无)
//          QK gemm_ss
//      3c: softmax (纯寄存器，无 sync)
//      3d: PV 前 wait (见上) + sync
//          PV gemm_rs
//      3e: rescale (纯寄存器，无 sync)
//      循环末尾：kStagesK>1 且非最后 -> wait<0> + sync
//
// 限制：仅 self-attention，无 causal/varlen/GQA（与 TMA FA2 v1 一致）
template <int kHeadDim, int kStagesK = 2>
__global__ void __launch_bounds__(256)
flash_attn_mma_stages_split_q_cute(
    cutlass::half_t *Q, cutlass::half_t *K, cutlass::half_t *V,
    cutlass::half_t *output, int rows, int seqlen) {
  // rows: B * H * seqlen, seqlen: N, kHeadDim: D
  using namespace cute;
  using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
  using Element = typename Traits::Element;
  using SmemLayoutQ = typename Traits::SmemLayoutQ;
  using SmemLayoutKV = typename Traits::SmemLayoutKV;
  static_assert(kHeadDim == 64 || kHeadDim == 128, "D=64 or 128 only");
  static_assert(kStagesK >= 1, "kStagesK must be >= 1");

  constexpr int kBr = 128;
  constexpr int kBc = 64;
  constexpr int kQTileElements = size(SmemLayoutQ{});
  constexpr int kKVTileElements = size(SmemLayoutKV{});

  // Shared memory: Q[128,D] + K[kStagesK,64,D] + V[1,64,D]
  // __align__(1024): swizzle phase=0, cp.async 写 GMMA swizzle layout 同样要求
  extern __shared__ __align__(1024) Element shm[];
  auto sQ = make_tensor(make_smem_ptr(shm), SmemLayoutQ{});
  // K smem 带 stage mode: (64, D, kStagesK)
  using SmemLayoutKStage = decltype(tile_to_shape(
      typename Traits::SmemLayoutAtom{},
      make_shape(_64{}, Int<kHeadDim>{}, Int<kStagesK>{})));
  auto sK = make_tensor(make_smem_ptr(shm + kQTileElements), SmemLayoutKStage{});
  // V 单 buffer: (64, D)，无 stage mode
  Element *v_base = shm + kQTileElements + kStagesK * kKVTileElements;
  auto sV = make_tensor(make_smem_ptr(v_base), SmemLayoutKV{});

  int tid = threadIdx.x;
  int q_tile = blockIdx.y * (seqlen / kBr) + blockIdx.x;
  int kv_tiles = seqlen / kBc;
  // ★ 多 head K/V offset: local_tile coord 需加 blockIdx.y * kv_tiles 偏移，
  // 否则所有 head 都读 head 0 的 K/V（参考 TMA 版 L6624 注释）
```

```
  int kv_base = blockIdx.y * kv_tiles;

  // Global memory tensors: [rows=B*H*N, D] row-major
  auto mQ = make_tensor(make_gmem_ptr(Q), make_shape(rows, Int<kHeadDim>{}),
                        make_stride(Int<kHeadDim>{}, _1{}));
  auto mK = make_tensor(make_gmem_ptr(K), make_shape(rows, Int<kHeadDim>{}),
                        make_stride(Int<kHeadDim>{}, _1{}));
  auto mV = make_tensor(make_gmem_ptr(V), make_shape(rows, Int<kHeadDim>{}),
                        make_stride(Int<kHeadDim>{}, _1{}));

  auto gQ = local_tile(mQ, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
  // K/V 不在此处做 local_tile (无 remainder mode)，main loop 内按 tile 做 local_tile + partition_S
  // (参考 TMA 版 producer L6624: 每次 local_tile 传入 blockIdx.y * kv_tiles + tile)

  // G2S TiledCopy: 128-bit cp.async, 256 threads, 每线程 8 half = 128b
  using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
  using g2s_copy_atom = Copy_Atom<Copy_Traits<g2s_copy_op>, Element>;
  using G2SCopy = decltype(make_tiled_copy(
      g2s_copy_atom{},
      make_layout(make_shape(Int<32>{}, Int<8>{}),    // ThrLayout: 32*8=256 threads
                  make_stride(Int<8>{}, Int<1>{})),
      make_layout(make_shape(Int<1>{}, Int<8>{}))));  // ValLayout: 1*8=8 half=128b
  G2SCopy g2s_copy;
  auto g2s_thr = g2s_copy.get_slice(tid);

  // G2S partition: Q source(gmem) 和 destination(smem)
  auto tQgQ = g2s_thr.partition_S(gQ);
  auto tQsQ = g2s_thr.partition_D(sQ);
  // K/V 的 smem destination (source 在 main loop 内每次 partition_S)
  auto tKsK = g2s_thr.partition_D(sK);  // (CPY, CPY_M, CPY_K, kStagesK)
  auto tVsV = g2s_thr.partition_D(sV);  // (CPY, CPY_M, CPY_K)  无 stage

  // TiledMMA + S2R partition (完全复用 TMA 版 consumer 逻辑 L6696-6737)
  typename Traits::TiledMma tiled_mma;
  auto thr_mma = tiled_mma.get_thread_slice(tid);
  auto tCrQ = thr_mma.partition_fragment_A(sQ);
  // V non-swizzle 视图用于推导寄存器 layout (不能与 swizzle composition 冲突)
  auto sVt0 = make_tensor(sV.data(), typename Traits::SmemLayoutVt{});
  auto sVt0_ns = make_tensor(
      sV.data(), get_nonswizzle_portion(typename Traits::SmemLayoutVt{}));
  auto tCrV_layout = thr_mma.partition_fragment_B(sVt0_ns).layout();
  auto tCrO = partition_fragment_C(tiled_mma, Shape<_128, Int<kHeadDim>>{});
  clear(tCrO);

  auto s2r_copy_q = make_tiled_copy_A(typename Traits::SmemCopyAtom{}, tiled_mma);
  auto s2r_thr_q = s2r_copy_q.get_thread_slice(tid);
  auto tQsQ_s2r = s2r_thr_q.partition_S(sQ);
  auto s2r_copy_k = make_tiled_copy_B(typename Traits::SmemCopyAtom{}, tiled_mma);
  auto s2r_thr_k = s2r_copy_k.get_thread_slice(tid);
  auto s2r_copy_v = make_tiled_copy_B(
      typename Traits::SmemCopyAtomTransposed{}, tiled_mma);
  auto s2r_thr_v = s2r_copy_v.get_thread_slice(tid);

  // Online softmax state
  auto tCrO_rc = make_tensor(
      tCrO.data(), fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
  constexpr int kRows = decltype(size<0>(tCrO_rc))::value;
  float row_max[kRows], row_sum[kRows];
#pragma unroll
  for (int r = 0; r < kRows; ++r) {
    row_max[r] = -INFINITY;
    row_sum[r] = 0.0f;
  }
```

```
6678
6679    float scale = rsqrtf(static_cast<float>(kHeadDim));
6680
6681    // ===== Step 1: Q 一次性 cp.async 加载 (split-Q 核心) =====
6682    cute::copy(g2s_copy, tQgQ, tQsQ);
6683    cp_async_fence();
6684    cp_async_wait<0>();
6685    __syncthreads();
6686
6687    // ===== Step 2: PREFETCH K 前 kStagesK-1 个 tile (仅 K, 不预取 V) =====
6688    int ismem_read = 0;
6689    int ismem_write = 0;
6690  #pragma unroll
6691    for (int s = 0; s < kStagesK - 1; ++s) {
6692      if (s < kv_tiles) {
6693        int kv_coord = kv_base + s;
6694        auto gK_tile = local_tile(mK, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_coord, _0{}));
6695        auto tKgK_tile = g2s_thr.partition_S(gK_tile);
6696        cute::copy(g2s_copy, tKgK_tile, tKsK(_, _, _, s));
6697        cp_async_fence();
6698        ++ismem_write;
6699      }
6700    }
6701    if constexpr (kStagesK > 1) {
6702      cp_async_wait<kStagesK - 2>();
6703      __syncthreads();
6704    }
6705
6706    // ===== Step 3: Main loop over KV tiles =====
6707    for (int tile = 0; tile < kv_tiles; ++tile) {
6708      // 3a: 提交 V[tile] (group V) + K_next (group K prefetch)
6709      // V 单 buffer, 每轮重写 sV; V 在 QK 之前发起, 通过 QK+softmax 隐藏延迟
6710      {
6711        int kv_coord = kv_base + tile;
6712        auto gV_tile = local_tile(mV, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_coord, _0{}));
6713        auto tVgV_tile = g2s_thr.partition_S(gV_tile);
6714        cute::copy(g2s_copy, tVgV_tile, tVsV);
6715      }
6716      cp_async_fence();  // group V_tile (栈底)
6717
6718      if constexpr (kStagesK > 1) {
6719        // Prefetch K[tile+kStagesK-1] -> sK[ismem_write] (group K_next, 栈顶)
6720        int k_next = tile + kStagesK - 1;
6721        if (k_next < kv_tiles) {
6722          int kv_coord = kv_base + k_next;
6723          auto gK_tile = local_tile(mK, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_coord, _0{}));
6724          auto tKgK_tile = g2s_thr.partition_S(gK_tile);
6725          cute::copy(g2s_copy, tKgK_tile, tKsK(_, _, _, ismem_write));
6726          cp_async_fence();  // group K_next (栈顶)
6727        }
6728      } else {
6729        // kStagesK==1: 加载当前 K[tile] (smem_sel_next == smem_sel == 0)
6730        int kv_coord = kv_base + tile;
6731        auto gK_tile = local_tile(mK, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_coord, _0{}));
6732        auto tKgK_tile = g2s_thr.partition_S(gK_tile);
6733        cute::copy(g2s_copy, tKgK_tile, tKsK(_, _, _, 0));
6734        cp_async_fence();  // group K_current (栈顶)
6735      }
6736
6737      // 3b: QK 前 wait K[tile] 就绪
6738      if constexpr (kStagesK == 1) {
6739        // kStagesK==1: V+K 都是当前 tile, wait<0> 等全部完成
6740        // (kStagesK==1 是退化路径, V 延迟无法隐藏)
```

```
      cp_async_wait<0>();
      __syncthreads();
    }
    // kStagesK>1: K[tile] 已在上一轮循环末尾 wait<0> 完成，无需 wait

    // Per-stage K smem (剥离 stage mode)
    auto sK_stg = make_tensor(
        make_smem_ptr(shm + kQTileElements + ismem_read * kKVTileElements),
        SmemLayoutKV{});
    auto tCrK = thr_mma.partition_fragment_B(sK_stg);
    CUTE_STATIC_ASSERT_V(size(tCrK) == size(tCrV_layout));
    auto tCrV = make_tensor(tCrK.data(), tCrV_layout);  // K 和 V 共享寄存器存储
    auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);

    // QK GEMM: S = Q @ K^T
    auto tCrS = partition_fragment_C(tiled_mma, Shape<_128, _64>{});
    clear(tCrS);
    fa_cute::gemm_ss(tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
                     tiled_mma, s2r_copy_q, s2r_copy_k, s2r_thr_q, s2r_thr_k);

    // 3c: Online softmax (纯寄存器操作，V[tile] 在后台加载)
    auto scores = make_tensor(
        tCrS.data(), fa_cute::convert_layout_acc_rowcol(tCrS.layout()));
#pragma unroll
    for (int r = 0; r < kRows; ++r) {
      float tile_max = -INFINITY;
#pragma unroll
      for (int c = 0; c < size<1>(scores); ++c)
        tile_max = fmaxf(tile_max, scores(r, c) * scale);
      tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
      tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
      float nxt = fmaxf(row_max[r], tile_max);
      float rs = __expf(row_max[r] - nxt);
#pragma unroll
      for (int c = 0; c < size<1>(tCrO_rc); ++c)
        tCrO_rc(r, c) *= rs;
      float ts = 0.0f;
#pragma unroll
      for (int c = 0; c < size<1>(scores); ++c) {
        float p = __expf(scores(r, c) * scale - nxt);
        scores(r, c) = p;
        ts += p;
      }
      ts += __shfl_xor_sync(0xffffffff, ts, 1);
      ts += __shfl_xor_sync(0xffffffff, ts, 2);
      row_sum[r] = row_sum[r] * rs + ts;
      row_max[r] = nxt;
    }

    // 3d: PV 前 wait V[tile] 就绪
    // kStagesK>1: K_next 提交条件是 tile + kStagesK - 1 < kv_tiles。
    //   已提交时栈顶有 K_next，wait<1> 只等 V_tile;
    //   未提交（尾部 Sk-1 个 tile）时栈只有 V_tile，必须 wait<0>。
    //   tile=0 必然已提交（PREFETCH 保证 kv_tiles >= kStagesK-1，且循环能跑说明 kv_tiles >= 1）。
    if constexpr (kStagesK > 1) {
      if (tile + kStagesK - 1 < kv_tiles) {
        cp_async_wait<1>();
      } else {
        cp_async_wait<0>();
      }
    }
    // kStagesK==1: 3b 已 wait<0>，V 已就绪
    __syncthreads();
```

```
6804
6805      // PV GEMM: O = P @ V
6806      // V smem 带 swizzle: partition_S 推导 smem 源地址映射，copy() 正确应用 swizzle
6807      auto sVt_stg = make_tensor(sV.data(), typename Traits::SmemLayoutVt{});
6808      auto tVsVt_s2r = s2r_thr_v.partition_S(sVt_stg);
6809      auto tCrP = fa_cute::convert_type<Element>(tCrS);
6810      auto tCrPv = make_tensor(
6811          tCrP.data(),
6812          fa_cute::convert_layout_acc_Aregs<typename Traits::TiledMma>(
6813              tCrP.layout()));
6814      fa_cute::gemm_rs(tCrO, tCrPv, tCrV, tVsVt_s2r,
6815                       tiled_mma, s2r_copy_v, s2r_thr_v);
6816      __syncthreads();  // 确保 ldmatrix V 完成后才能覆盖 V smem
6817
6818      // 3e: rescale 已合并到 3c softmax pass
6819      // 循环末尾: kStagesK>1 且非最后 tile -> wait<0> 确保 K_next 完成
6820      // (下一轮 QK 安全; 同时保护 V/K smem 不被提前覆盖)
6821      if constexpr (kStagesK > 1) {
6822        if (tile + 1 < kv_tiles) {
6823          cp_async_wait<0>();
6824          __syncthreads();
6825        }
6826      }
6827
6828      // 更新 pipeline 指针
6829      ismem_read = (ismem_read + 1) % kStagesK;
6830      ismem_write = (ismem_write + 1) % kStagesK;
6831    }
6832
6833    // ===== Step 4: Final normalize O = Oacc / l =====
6834  #pragma unroll
6835    for (int r = 0; r < kRows; ++r) {
6836      float inv_sum = 1.0f / row_sum[r];
6837  #pragma unroll
6838      for (int c = 0; c < size<1>(tCrO_rc); ++c)
6839        tCrO_rc(r, c) *= inv_sum;
6840    }
6841
6842    // ===== Step 5: Epilogue - direct R->G store =====
6843    auto tCrO_half = fa_cute::convert_type<Element>(tCrO);
6844    auto mO = make_tensor(make_gmem_ptr(output),
6845                          make_shape(rows, Int<kHeadDim>{}),
6846                          make_stride(Int<kHeadDim>{}, _1{}));
6847    auto gO = local_tile(mO, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
6848    auto tCgO = thr_mma.partition_C(gO);
6849    copy(tCrO_half, tCgO);
6850  }
6851
6852  template <int kHeadDim, typename TmaQ, typename TmaK, typename TmaV,
6853            int kStagesK = 1, int kStagesV = 1>
6854  __global__ void __launch_bounds__(384, 1)
6855  flash_attn_tma_mma_ws_split_q_cute(
6856      CUTLASS_GRID_CONSTANT TmaQ const tma_q,
6857      CUTLASS_GRID_CONSTANT TmaK const tma_k,
6858      CUTLASS_GRID_CONSTANT TmaV const tma_v,
6859      cutlass::half_t *output, int rows, int seqlen) {
6860    // rows: B * H * seqlen, seqlen: N, kHeadDim: D
6861    using namespace cute;
6862    using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
6863    using Element = typename Traits::Element;
6864    using SmemLayoutQ = typename Traits::SmemLayoutQ;
6865    using SmemLayoutKV = typename Traits::SmemLayoutKV;
6866    using TmaBarrier = cutlass::arch::ClusterTransactionBarrier;
```

```
6867  using CtaBarrier = cutlass::arch::ClusterBarrier;
6868  constexpr int kBr = 128;
6869  constexpr int kBc = 64;
6870  constexpr int kConsumerThreads = 256;
6871  constexpr int kProducerThreads = 128;
6872  constexpr int kQTileElements = cosize(SmemLayoutQ{});
6873  constexpr int kKVTileElements = cosize(SmemLayoutKV{});
6874
6875  extern __shared__ __align__(1024) Element shm[];
6876  auto sQ = make_tensor(make_smem_ptr(shm), SmemLayoutQ{});
6877  Element *k_base = shm + kQTileElements;
6878  Element *v_base = k_base + kStagesK * kKVTileElements;
6879
6880  __shared__ uint64_t q_full;
6881  __shared__ uint64_t k_full[kStagesK];
6882  __shared__ uint64_t k_empty[kStagesK];
6883  __shared__ uint64_t v_full[kStagesV];
6884  __shared__ uint64_t v_empty[kStagesV];
6885
6886  const bool is_producer = threadIdx.x < kProducerThreads;
6887  const int wg_tid = is_producer ? threadIdx.x
6888      : threadIdx.x - kProducerThreads;
6889
6890  if (threadIdx.x == 0) {
6891    TmaBarrier::init(&q_full, 1);
6892    for (int s = 0; s < kStagesK; ++s) {
6893      TmaBarrier::init(&k_full[s], 1);
6894      CtaBarrier::init(&k_empty[s], kConsumerThreads);
6895    }
6896    for (int s = 0; s < kStagesV; ++s) {
6897      TmaBarrier::init(&v_full[s], 1);
6898      CtaBarrier::init(&v_empty[s], kConsumerThreads);
6899    }
6900  }
6901  __syncthreads();
6902
6903  int q_tile = blockIdx.y * (seqlen / kBr) + blockIdx.x;
6904  int kv_tiles = seqlen / kBc;
6905
6906  // ====================================================================
6907  // Producer Warpgroup (WG0, threadIdx.x 0~127)
6908  // Only wg_tid == 0 issues TMA. Load Q once, then V-first K-after KV loop.
6909  // P1: prefetch K[0..Sk-2] + V[0..Sv-2] (if Sk/Sv > 1).
6910  // P2: V[tile+Sv-1] then K[tile+Sk-1] (prefetch ahead Sk-1/Sv-1, 对齐 8b).
6911  // ====================================================================
6912  if (is_producer) {
6913    if (wg_tid == 0) {
6914      auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6915      auto gQ = local_tile(
6916          mQ, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
6917      auto q_slice = tma_q.get_slice(_0{});
6918      auto tQgQ = q_slice.partition_S(gQ);
6919      auto tQsQ = q_slice.partition_D(sQ);
6920      TmaBarrier::arrive_and_expect_tx(
6921          &q_full, sizeof(Element) * size(sQ));
6922      copy(tma_q.with(q_full), tQgQ, tQsQ);
6923
6924      auto mK = tma_k.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6925      auto mV = tma_v.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
6926      auto k_slice = tma_k.get_slice(_0{});
6927      auto v_slice = tma_v.get_slice(_0{});
6928
6929      // ★ 易错点 (CuTe TMA multi-head): K/V local_tile 的 coord 是相对于 TMA
```

```
      // tensor row 维的全局 tile 索引 (TMA tensor shape = [B*H*N, D]), 不是
      // per-head tile 索引。多 head 场景必须加 `blockIdx.y * kv_tiles` 偏移,
      // 否则所有 head 都读 head 0 的 K/V -> correctness test (H=1) 正常但
      // bench (H>1) 误差飙升 2 个量纲。Q 的 q_tile 已含 blockIdx.y 无需修复。
      // 参考: /memories/repo/leetcuda-tma-fa-multihead.md
      // P1: Prefetch first (kStagesK-1) K tiles
      for (int s = 0; s < kStagesK - 1; ++s) {
        if (s < kv_tiles) {
          auto sK = make_tensor(
              make_smem_ptr(k_base + s * kKVTileElements), SmemLayoutKV{});
          CtaBarrier::wait(&k_empty[s], 0);
          auto gK = local_tile(
              mK, Shape<_64, Int<kHeadDim>>{},
              make_coord(blockIdx.y * kv_tiles + s, _0{}));
          auto tKgK = k_slice.partition_S(gK);
          auto tKsK = k_slice.partition_D(sK);
          TmaBarrier::arrive_and_expect_tx(
              &k_full[s], sizeof(Element) * size(sK));
          copy(tma_k.with(k_full[s]), tKgK, tKsK);
        }
      }

      // P1b: Prefetch first (kStagesV-1) V tiles
      for (int s = 0; s < kStagesV - 1; ++s) {
        if (s < kv_tiles) {
          auto sV = make_tensor(
              make_smem_ptr(v_base + s * kKVTileElements), SmemLayoutKV{});
          CtaBarrier::wait(&v_empty[s], 0);
          auto gV = local_tile(
              mV, Shape<_64, Int<kHeadDim>>{},
              make_coord(blockIdx.y * kv_tiles + s, _0{}));
          auto tVgV = v_slice.partition_S(gV);
          auto tVsV = v_slice.partition_D(sV);
          TmaBarrier::arrive_and_expect_tx(
              &v_full[s], sizeof(Element) * size(sV));
          copy(tma_v.with(v_full[s]), tVgV, tVsV);
        }
      }

      // P2: Main loop - V-first then K-after (对齐 8b producer 顺序)
      for (int tile = 0; tile < kv_tiles; ++tile) {
        // V: load V[tile+Sv-1] (prefetch ahead Sv-1)
        {
          int v_tile = tile + kStagesV - 1;
          if (v_tile < kv_tiles) {
            int stage_v = v_tile % kStagesV;
            int phase_v = (v_tile / kStagesV) & 1;
            auto sV = make_tensor(
                make_smem_ptr(v_base + stage_v * kKVTileElements),
                SmemLayoutKV{});
            CtaBarrier::wait(&v_empty[stage_v], phase_v);
            auto gV = local_tile(
                mV, Shape<_64, Int<kHeadDim>>{},
                make_coord(blockIdx.y * kv_tiles + v_tile, _0{}));
            auto tVgV = v_slice.partition_S(gV);
            auto tVsV = v_slice.partition_D(sV);
            TmaBarrier::arrive_and_expect_tx(
                &v_full[stage_v], sizeof(Element) * size(sV));
            copy(tma_v.with(v_full[stage_v]), tVgV, tVsV);
          }
        }
        // K: prefetch K[tile+Sk-1] (prefetch ahead Sk-1)
        {
```

```
6993          int k_tile = tile + kStagesK - 1;
6994          if (k_tile < kv_tiles) {
6995            int stage_k = k_tile % kStagesK;
6996            int phase_k = (k_tile / kStagesK) & 1;
6997            auto sK = make_tensor(
6998                make_smem_ptr(k_base + stage_k * kKVTileElements),
6999                SmemLayoutKV{});
7000            CtaBarrier::wait(&k_empty[stage_k], phase_k);
7001            auto gK = local_tile(
7002                mK, Shape<_64, Int<kHeadDim>>{},
7003                make_coord(blockIdx.y * kv_tiles + k_tile, _0{}));
7004            auto tKgK = k_slice.partition_S(gK);
7005            auto tKsK = k_slice.partition_D(sK);
7006            TmaBarrier::arrive_and_expect_tx(
7007                &k_full[stage_k], sizeof(Element) * size(sK));
7008            copy(tma_k.with(k_full[stage_k]), tKgK, tKsK);
7009          }
7010        }
7011      }
7012    }
7013  }
7014  // ================================================================
7015  // Consumer Warpgroup (WG1, threadIdx.x 128~383, 256 threads = 8 warps)
7016  // Single consumer: full KV traversal, no split-KV merge.
7017  // Flow: wait K -> QK -> release K -> softmax -> wait V -> PV -> release V.
7018  // ================================================================
7019  else {
7020    TmaBarrier::wait(&q_full, 0);
7021
7022    // V layout 从 stage 0 推导 (所有 stage layout 相同)
7023    auto sV0 = make_tensor(make_smem_ptr(v_base), SmemLayoutKV{});
7024    auto sVt0 = make_tensor(sV0.data(), typename Traits::SmemLayoutVt{});
7025    // ★ get_nonswizzle_portion 只用于 partition_fragment_B 推导寄存器侧 B
7026    // fragment layout (每线程持有哪些 V 元素)，寄存器布局与 smem swizzle 无关。
7027    // V 的 smem bank conflict 由 TMA 写入 (SmemLayoutKV 带 swizzle) +
7028    // ldmatrix 读取 (主循环内 partition_S(sVt_stg) 带 swizzle) 共同解决。
7029    // 这里不能用带 swizzle 的 sVt0 做 partition_fragment_B，否则 CuTe 推导
7030    // ldmatrix.trans 的线程-数据映射时会与 swizzle composition 冲突。
7031    // 参考: flash-attention/csrc/flash_attn/src/kernel_traits.h
7032    //       SmemLayoutVtransposedNoSwizzle 的同样用法。
7033    auto sVt0_ns = make_tensor(
7034        sV0.data(), get_nonswizzle_portion(typename Traits::SmemLayoutVt{}));
7035
7036    typename Traits::TiledMma tiled_mma;
7037    auto thr_mma = tiled_mma.get_thread_slice(wg_tid);
7038    auto tCrQ = thr_mma.partition_fragment_A(sQ);
7039    auto tCrV_layout = thr_mma.partition_fragment_B(sVt0_ns).layout();
7040    auto tCrO = partition_fragment_C(
7041        tiled_mma, Shape<_128, Int<kHeadDim>>{});
7042    clear(tCrO);
7043
7044    auto s2r_copy_q = make_tiled_copy_A(
7045        typename Traits::SmemCopyAtom{}, tiled_mma);
7046    auto s2r_thr_q = s2r_copy_q.get_thread_slice(wg_tid);
7047    auto tQsQ_s2r = s2r_thr_q.partition_S(sQ);
7048    auto s2r_copy_k = make_tiled_copy_B(
7049        typename Traits::SmemCopyAtom{}, tiled_mma);
7050    auto s2r_thr_k = s2r_copy_k.get_thread_slice(wg_tid);
7051    auto s2r_copy_v = make_tiled_copy_B(
7052        typename Traits::SmemCopyAtomTransposed{}, tiled_mma);
7053    auto s2r_thr_v = s2r_copy_v.get_thread_slice(wg_tid);
7054
7055    auto tCrO_rc = make_tensor(
```

```
        tCrO.data(),
        fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
    constexpr int kRows = decltype(size<0>(tCrO_rc))::value;
    float row_max[kRows], row_sum[kRows];
#pragma unroll
    for (int r = 0; r < kRows; ++r) {
      row_max[r] = -INFINITY;
      row_sum[r] = 0.0f;
    }

    // Init: mark all K/V slots as empty (256 consumer arrivals each)
    for (int s = 0; s < kStagesK; ++s)
      CtaBarrier::arrive(&k_empty[s]);
    for (int s = 0; s < kStagesV; ++s)
      CtaBarrier::arrive(&v_empty[s]);

    float scale = rsqrtf(static_cast<float>(kHeadDim));
    for (int tile = 0; tile < kv_tiles; ++tile) {
      int k_stg = tile % kStagesK;
      int k_phase = (tile / kStagesK) & 1;
      int v_stg = tile % kStagesV;
      int v_phase = (tile / kStagesV) & 1;

      // Per-stage K smem and S2R partition
      auto sK_stg = make_tensor(
          make_smem_ptr(k_base + k_stg * kKVTileElements), SmemLayoutKV{});
      auto tCrK = thr_mma.partition_fragment_B(sK_stg);
      CUTE_STATIC_ASSERT_V(size(tCrK) == size(tCrV_layout));
      auto tCrV = make_tensor(tCrK.data(), tCrV_layout);
      auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);

      // 1) Wait K, QK gemm, release K early
      TmaBarrier::wait(&k_full[k_stg], k_phase);
      auto tCrS = partition_fragment_C(tiled_mma, Shape<_128, _64>{});
      clear(tCrS);
      fa_cute::gemm_ss(
          tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
          tiled_mma, s2r_copy_q, s2r_copy_k, s2r_thr_q, s2r_thr_k);
      { CtaBarrier::arrive(&k_empty[k_stg]); }

      // 2) Online softmax (overlaps with producer's V TMA)
      auto scores = make_tensor(
          tCrS.data(),
          fa_cute::convert_layout_acc_rowcol(tCrS.layout()));
#pragma unroll
      for (int r = 0; r < kRows; ++r) {
        float tile_max = -INFINITY;
#pragma unroll
        for (int c = 0; c < size<1>(scores); ++c)
          tile_max = fmaxf(tile_max, scores(r, c) * scale);
        tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
        tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
        float nxt = fmaxf(row_max[r], tile_max);
        float rs = __expf(row_max[r] - nxt);
#pragma unroll
        for (int c = 0; c < size<1>(tCrO_rc); ++c)
          tCrO_rc(r, c) *= rs;
        float ts = 0.0f;
#pragma unroll
        for (int c = 0; c < size<1>(scores); ++c) {
          float p = __expf(scores(r, c) * scale - nxt);
          scores(r, c) = p;
          ts += p;
```

```
        }
        ts += __shfl_xor_sync(0xffffffff, ts, 1);
        ts += __shfl_xor_sync(0xffffffff, ts, 2);
        row_sum[r] = row_sum[r] * rs + ts;
        row_max[r] = nxt;
      }

      // 3) Wait V, PV gemm, release V early
      // ★ sVt_stg 带 swizzle: partition_S 推导 smem 源地址映射，copy() 会
      // 正确应用 swizzle 物理地址，与 TMA 写入 (SmemLayoutKV 带 swizzle) 匹配，
      // 解决 V 的 smem bank conflict。这是 V bank-conflict-free 的关键路径。
      auto sV_stg = make_tensor(
          make_smem_ptr(v_base + v_stg * kKVTileElements), SmemLayoutKV{});
      auto sVt_stg = make_tensor(sV_stg.data(), typename Traits::SmemLayoutVt{});
      auto tVsVt_s2r = s2r_thr_v.partition_S(sVt_stg);

      TmaBarrier::wait(&v_full[v_stg], v_phase);
      auto tCrP = fa_cute::convert_type<Element>(tCrS);
      auto tCrPv = make_tensor(
          tCrP.data(),
          fa_cute::convert_layout_acc_Aregs<typename Traits::TiledMma>(
              tCrP.layout()));
      fa_cute::gemm_rs(
          tCrO, tCrPv, tCrV, tVsVt_s2r,
          tiled_mma, s2r_copy_v, s2r_thr_v);
      { CtaBarrier::arrive(&v_empty[v_stg]); }
    }

    // ---- Final normalize: O = Oacc / l (single consumer, no merge) ----
#pragma unroll
    for (int r = 0; r < kRows; ++r) {
      float inv_sum = 1.0f / row_sum[r];
#pragma unroll
      for (int c = 0; c < size<1>(tCrO_rc); ++c)
        tCrO_rc(r, c) *= inv_sum;
    }

    auto tCrO_half = fa_cute::convert_type<Element>(tCrO);
    auto mO = make_tensor(
        make_gmem_ptr(output),
        make_shape(rows, Int<kHeadDim>{}),
        make_stride(Int<kHeadDim>{}, _1{}));
    auto gO = local_tile(
        mO, Shape<_128, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
    auto tCgO = thr_mma.partition_C(gO);
    copy(tCrO_half, tCgO);
  }
}

template <int kHeadDim, typename TmaQ, typename TmaK, typename TmaV,
          int kStagesK = 1>
__global__ void __launch_bounds__(384, 1)
flash_attn_3_tma_mma_ws_split_q_cute(
    CUTLASS_GRID_CONSTANT TmaQ const tma_q,
    CUTLASS_GRID_CONSTANT TmaK const tma_k,
    CUTLASS_GRID_CONSTANT TmaV const tma_v,
    cutlass::half_t *output, int rows, int seqlen) {
  // rows: B * H * seqlen, seqlen: N, kHeadDim: D
  using namespace cute;
  using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
  using Element = typename Traits::Element;
  using SmemLayout = typename Traits::SmemLayoutQKV;
  using TmaBarrier = cutlass::arch::ClusterTransactionBarrier;
```

```
  using CtaBarrier = cutlass::arch::ClusterBarrier;
  constexpr int kTile = 64;
  constexpr int kNumConsumers = 2;
  constexpr int kConsumerThreads = 128;
  constexpr int kProducerThreads = 128;
  constexpr int kTileElements = cosize(SmemLayout{});

  extern __shared__ __align__(1024) Element shm[];
  auto sQ = make_tensor(make_smem_ptr(shm), SmemLayout{});
  Element *k_base = shm + kTileElements;
  Element *v_base = k_base + kNumConsumers * kStagesK * kTileElements;

  __shared__ uint64_t q_full;
  __shared__ uint64_t k_full[kNumConsumers][kStagesK];
  __shared__ uint64_t k_empty[kNumConsumers][kStagesK];
  __shared__ uint64_t v_full[kNumConsumers];
  __shared__ uint64_t v_empty[kNumConsumers];

  const bool is_producer = threadIdx.x < kProducerThreads;
  const int consumer_id = is_producer ? 0
      : (threadIdx.x - kProducerThreads) / kConsumerThreads;
  const int wg_tid = is_producer ? threadIdx.x
      : (threadIdx.x - kProducerThreads) % kConsumerThreads;

  if (threadIdx.x == 0) {
    TmaBarrier::init(&q_full, 1);
    for (int cid = 0; cid < kNumConsumers; ++cid) {
      for (int s = 0; s < kStagesK; ++s) {
        TmaBarrier::init(&k_full[cid][s], 1);
        CtaBarrier::init(&k_empty[cid][s], kConsumerThreads);
      }
      TmaBarrier::init(&v_full[cid], 1);
      CtaBarrier::init(&v_empty[cid], kConsumerThreads);
    }
  }
  __syncthreads();

  int q_tile = blockIdx.y * (seqlen / kTile) + blockIdx.x;
  int kv_tiles = seqlen / kTile;

  // ====================================================================
  // Producer Warpgroup (WG0, threadIdx.x 0~127)
  // Only wg_tid == 0 issues TMA. Load Q once, then K-first KV loop.
  // Sk=1: load K[tile] + V[tile] per iteration.
  // Sk>=2: warmup prefetch K[cid][0], then V[tile] + prefetch K[tile+2].
  // ====================================================================
  if (is_producer) {
    if (wg_tid == 0) {
      auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto gQ = local_tile(
          mQ, Shape<_64, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
      auto q_slice = tma_q.get_slice(_0{});
      auto tQgQ = q_slice.partition_S(gQ);
      auto tQsQ = q_slice.partition_D(sQ);
      TmaBarrier::arrive_and_expect_tx(
          &q_full, sizeof(Element) * size(sQ));
      copy(tma_q.with(q_full), tQgQ, tQsQ);

      auto mK = tma_k.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto mV = tma_v.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
      auto k_slice = tma_k.get_slice(_0{});
      auto v_slice = tma_v.get_slice(_0{});

```

```
      // Warmup: prefetch first K tile per consumer (Sk>=2 only)
      if constexpr (kStagesK > 1) {
        for (int cid = 0; cid < kNumConsumers; ++cid) {
          if (cid < kv_tiles) {
            auto sK = make_tensor(
                make_smem_ptr(k_base + cid * kStagesK * kTileElements),
                SmemLayout{});
            CtaBarrier::wait(&k_empty[cid][0], 0);
            auto gK = local_tile(
                mK, Shape<_64, Int<kHeadDim>>{},
                make_coord(blockIdx.y * kv_tiles + cid, _0{}));
            auto tKgK = k_slice.partition_S(gK);
            auto tKsK = k_slice.partition_D(sK);
            TmaBarrier::arrive_and_expect_tx(
                &k_full[cid][0], sizeof(Element) * size(sK));
            copy(tma_k.with(k_full[cid][0]), tKgK, tKsK);
          }
        }
      }

      for (int tile = 0; tile < kv_tiles; ++tile) {
        int cid = tile & 1;
        int local = (tile - cid) / kNumConsumers;
        int kv_tile = blockIdx.y * kv_tiles + tile;

        // Step 1: K — load current (Sk=1) or prefetch next (Sk>=2)
        if constexpr (kStagesK == 1) {
          auto sK = make_tensor(
              make_smem_ptr(k_base + cid * kStagesK * kTileElements),
              SmemLayout{});
          CtaBarrier::wait(&k_empty[cid][0], local & 1);
          auto gK = local_tile(
              mK, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_tile, _0{}));
          auto tKgK = k_slice.partition_S(gK);
          auto tKsK = k_slice.partition_D(sK);
          TmaBarrier::arrive_and_expect_tx(
              &k_full[cid][0], sizeof(Element) * size(sK));
          copy(tma_k.with(k_full[cid][0]), tKgK, tKsK);
        } else {
          int stg_next = (local + 1) % kStagesK;
          int next_tile = tile + kNumConsumers;
          if (next_tile < kv_tiles) {
            int next_local = local + 1;
            int next_phase = (next_local / kStagesK) & 1;
            auto sK = make_tensor(
                make_smem_ptr(k_base +
                    (cid * kStagesK + stg_next) * kTileElements),
                SmemLayout{});
            CtaBarrier::wait(&k_empty[cid][stg_next], next_phase);
            auto gK = local_tile(
                mK, Shape<_64, Int<kHeadDim>>{},
                make_coord(blockIdx.y * kv_tiles + next_tile, _0{}));
            auto tKgK = k_slice.partition_S(gK);
            auto tKsK = k_slice.partition_D(sK);
            TmaBarrier::arrive_and_expect_tx(
                &k_full[cid][stg_next], sizeof(Element) * size(sK));
            copy(tma_k.with(k_full[cid][stg_next]), tKgK, tKsK);
          }
        }

        // Step 2: V — single buffer, release depends on PV (late step).
        // V TMA latency is hidden by consumer QK+softmax compute.
        auto sV = make_tensor(
```

```
            make_smem_ptr(v_base + cid * kTileElements), SmemLayout{});
        CtaBarrier::wait(&v_empty[cid], local & 1);
        auto gV = local_tile(
            mV, Shape<_64, Int<kHeadDim>>{}, make_coord(kv_tile, _0{}));
        auto tVgV = v_slice.partition_S(gV);
        auto tVsV = v_slice.partition_D(sV);
        TmaBarrier::arrive_and_expect_tx(
            &v_full[cid], sizeof(Element) * size(sV));
        copy(tma_v.with(v_full[cid]), tVgV, tVsV);
      }
    }
  }
  // ====================================================================
  // Consumer Warpgroups (WG1 consumer_id=0, WG2 consumer_id=1)
  // Each WG processes every other KV tile independently.
  // K staged (kStagesK depth), V single buffer (always 1).
  // Flow: wait K → QK → release K → softmax → wait V → PV → release V → rescale.
  // ====================================================================
  else {
    TmaBarrier::wait(&q_full, 0);

    auto sV = make_tensor(
        make_smem_ptr(v_base + consumer_id * kTileElements), SmemLayout{});
    auto sVt = make_tensor(sV.data(), typename Traits::SmemLayoutVt{});
    auto sVt_ns = make_tensor(
        sV.data(), get_nonswizzle_portion(typename Traits::SmemLayoutVt{}));

    typename Traits::TiledMma tiled_mma;
    auto thr_mma = tiled_mma.get_thread_slice(wg_tid);
    auto tCrQ = thr_mma.partition_fragment_A(sQ);
    auto tCrV_layout = thr_mma.partition_fragment_B(sVt_ns).layout();
    auto tCrO = partition_fragment_C(
        tiled_mma, Shape<_64, Int<kHeadDim>>{});
    clear(tCrO);

    auto s2r_copy_q = make_tiled_copy_A(
        typename Traits::SmemCopyAtom{}, tiled_mma);
    auto s2r_thr_q = s2r_copy_q.get_thread_slice(wg_tid);
    auto tQsQ_s2r = s2r_thr_q.partition_S(sQ);
    auto s2r_copy_k = make_tiled_copy_B(
        typename Traits::SmemCopyAtom{}, tiled_mma);
    auto s2r_thr_k = s2r_copy_k.get_thread_slice(wg_tid);
    auto s2r_copy_v = make_tiled_copy_B(
        typename Traits::SmemCopyAtomTransposed{}, tiled_mma);
    auto s2r_thr_v = s2r_copy_v.get_thread_slice(wg_tid);
    auto tVsVt_s2r = s2r_thr_v.partition_S(sVt);

    auto tCrO_rc = make_tensor(
        tCrO.data(),
        fa_cute::convert_layout_acc_rowcol(tCrO.layout()));
    constexpr int kRows = decltype(size<0>(tCrO_rc))::value;
    float row_max[kRows], row_sum[kRows];
#pragma unroll
    for (int r = 0; r < kRows; ++r) {
      row_max[r] = -INFINITY;
      row_sum[r] = 0.0f;
    }

    // Init: mark own K/V slots as empty
    for (int s = 0; s < kStagesK; ++s)
      CtaBarrier::arrive(&k_empty[consumer_id][s]);
    CtaBarrier::arrive(&v_empty[consumer_id]);

```

```
    float scale = rsqrtf(static_cast<float>(kHeadDim));
    int local_iter = 0;
    for (int tile = consumer_id; tile < kv_tiles;
         tile += kNumConsumers, ++local_iter) {
      int k_stg = local_iter % kStagesK;
      int k_phase = (local_iter / kStagesK) & 1;

      // Per-stage K smem and S2R partition
      auto sK_stg = make_tensor(
          make_smem_ptr(k_base +
              (consumer_id * kStagesK + k_stg) * kTileElements),
          SmemLayout{});
      auto tCrK = thr_mma.partition_fragment_B(sK_stg);
      CUTE_STATIC_ASSERT_V(size(tCrK) == size(tCrV_layout));
      auto tCrV = make_tensor(tCrK.data(), tCrV_layout);
      auto tKsK_s2r = s2r_thr_k.partition_S(sK_stg);

      // 1) Wait K, QK gemm, release K early
      TmaBarrier::wait(&k_full[consumer_id][k_stg], k_phase);
      auto tCrS = partition_fragment_C(tiled_mma, Shape<_64, _64>{});
      clear(tCrS);
      fa_cute::gemm_ss(
          tCrS, tCrQ, tCrK, tQsQ_s2r, tKsK_s2r,
          tiled_mma, s2r_copy_q, s2r_copy_k, s2r_thr_q, s2r_thr_k);
      { CtaBarrier::arrive(&k_empty[consumer_id][k_stg]); }

      // 2) Online softmax (overlaps with producer's V TMA)
      auto scores = make_tensor(
          tCrS.data(),
          fa_cute::convert_layout_acc_rowcol(tCrS.layout()));
#pragma unroll
      for (int r = 0; r < kRows; ++r) {
        float tile_max = -INFINITY;
#pragma unroll
        for (int c = 0; c < size<1>(scores); ++c)
          tile_max = fmaxf(tile_max, scores(r, c) * scale);
        tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 1));
        tile_max = fmaxf(tile_max, __shfl_xor_sync(0xffffffff, tile_max, 2));
        float nxt = fmaxf(row_max[r], tile_max);
        float rs = __expf(row_max[r] - nxt);
#pragma unroll
        for (int c = 0; c < size<1>(tCrO_rc); ++c)
          tCrO_rc(r, c) *= rs;
        float ts = 0.0f;
#pragma unroll
        for (int c = 0; c < size<1>(scores); ++c) {
          float p = __expf(scores(r, c) * scale - nxt);
          scores(r, c) = p;
          ts += p;
        }
        ts += __shfl_xor_sync(0xffffffff, ts, 1);
        ts += __shfl_xor_sync(0xffffffff, ts, 2);
        row_sum[r] = row_sum[r] * rs + ts;
        row_max[r] = nxt;
      }

      // 3) Wait V, PV gemm, release V early
      TmaBarrier::wait(&v_full[consumer_id], local_iter & 1);
      auto tCrP = fa_cute::convert_type<Element>(tCrS);
      auto tCrPv = make_tensor(
          tCrP.data(),
          fa_cute::convert_layout_acc_Aregs<typename Traits::TiledMma>(
              tCrP.layout()));
```

```
      fa_cute::gemm_rs(
          tCrO, tCrPv, tCrV, tVsVt_s2r,
          tiled_mma, s2r_copy_v, s2r_thr_v);
      { CtaBarrier::arrive(&v_empty[consumer_id]); }
    }

    // Split-KV merge
    __syncthreads();
    float *merge_o = reinterpret_cast<float *>(shm);
    float4 *merge_stats = reinterpret_cast<float4 *>(
        merge_o + 128 * size(tCrO));

    if (consumer_id == 1) {
#pragma unroll
      for (int idx = 0; idx < size(tCrO); ++idx)
        merge_o[idx * 128 + wg_tid] = tCrO(idx);
      merge_stats[wg_tid] = make_float4(
          row_max[0], row_max[1], row_sum[0], row_sum[1]);
    }
    __syncthreads();

    if (consumer_id == 0) {
      float4 st = merge_stats[wg_tid];
      float o_max[kRows] = {st.x, st.y};
      float o_sum[kRows] = {st.z, st.w};
#pragma unroll
      for (int r = 0; r < kRows; ++r) {
        float mg = fmaxf(row_max[r], o_max[r]);
        float lhs = __expf(row_max[r] - mg);
        float rhs = __expf(o_max[r] - mg);
        float ms = lhs * row_sum[r] + rhs * o_sum[r];
#pragma unroll
        for (int c = 0; c < size<1>(tCrO_rc); ++c) {
          int idx = tCrO_rc.layout()(make_coord(r, c));
          float ov = merge_o[idx * 128 + wg_tid];
          tCrO_rc(r, c) = (lhs * tCrO_rc(r, c) + rhs * ov) / ms;
        }
      }

      auto tCrO_half = fa_cute::convert_type<Element>(tCrO);
      auto mO = make_tensor(
          make_gmem_ptr(output),
          make_shape(rows, Int<kHeadDim>{}),
          make_stride(Int<kHeadDim>{}, _1{}));
      auto gO = local_tile(
          mO, Shape<_64, Int<kHeadDim>>{}, make_coord(q_tile, _0{}));
      auto tCgO = thr_mma.partition_C(gO);
      copy(tCrO_half, tCgO);
    }
  }
}


template <int kHeadDim, typename TmaQ>
__global__ void flash_attn_3_cute_tma_copy_smoke(
  CUTLASS_GRID_CONSTANT TmaQ const tma_q,
  cutlass::half_t *output, int rows) {
  using namespace cute;
  using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
  using Element = typename Traits::Element;
  using SmemLayout = typename Traits::SmemLayoutQKV;
  using TransactionBarrier = cutlass::arch::ClusterTransactionBarrier;

```

```
  extern __shared__ __align__(1024) Element shared_storage[];
  auto sQ = make_tensor(make_smem_ptr(shared_storage), SmemLayout{});
  __shared__ uint64_t tma_barrier;

  if (threadIdx.x == 0) {
    TransactionBarrier::init(&tma_barrier, 1);
  }
  __syncthreads();

  auto mQ = tma_q.get_tma_tensor(make_shape(rows, Int<kHeadDim>{}));
  auto gQ = local_tile(
      mQ, Shape<_64, Int<kHeadDim>>{},
      make_coord(blockIdx.x, _0{}));
  auto cta_tma_q = tma_q.get_slice(_0{});
  auto tQgQ = cta_tma_q.partition_S(gQ);
  auto tQsQ = cta_tma_q.partition_D(sQ);

  if (threadIdx.x == 0) {
    TransactionBarrier::arrive_and_expect_tx(
        &tma_barrier, sizeof(half) * size(sQ));
    copy(tma_q.with(tma_barrier), tQgQ, tQsQ);
  }
  TransactionBarrier::wait(&tma_barrier, 0);

  for (int linear = threadIdx.x; linear < size(sQ); linear += blockDim.x) {
    int row = linear / kHeadDim;
    int col = linear % kHeadDim;
    output[(blockIdx.x * 64 + row) * kHeadDim + col] = sQ(row, col);
  }
}
#endif
#endif // END NOTES_V2_ENABLE_TMA_MMA_WS

// ================================================================
// 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
// ================================================================

// Bench / test mode globals (placed early so all functions can reference them)
static bool g_debug = false;
static bool g_bench_hgemm = false;
static bool g_bench_hgemm_all = false;
static bool g_bench_fa = false;
static bool g_bench_fa3_cute_only = false;
static bool g_bench_all = false;
static bool g_fa_skip_check = false;
static bool g_swizzle_eq_check = false;
enum class FALayout {
  All, Pad, SwizzleQ, SwizzleK, SwizzleV, SwizzleQK, SwizzleQV, SwizzleKV, Swizzle
};
static FALayout g_fa_layout = FALayout::Pad;
static int g_bench_M = 4096, g_bench_N = 4096, g_bench_K = 4096;
static int g_bench_B = 1, g_bench_H = 48, g_bench_Nfa = 8192, g_bench_D = 128;
static int g_warmup = 2, g_repeat = 3;
static float g_fa_f16_max_tflops = 0.0f;
static float g_fa_f32_max_tflops = 0.0f;
static bool g_verbose = false;

// Decide whether to print a FA TFLOPS line. When --verbose/--debug is off,
// only print when the current TFLOPS exceeds the running max for its
// accumulator category (f16/f32). Correctness failures always print so they
// are never silently dropped (callers gate FAIL paths themselves).
static bool should_print_fa_tflops(int acc_f32, float tflops) {
  if (g_verbose || g_debug) return true;
```

```
  float &max_tflops = acc_f32 ? g_fa_f32_max_tflops : g_fa_f16_max_tflops;
  if (tflops > max_tflops) {
    max_tflops = tflops;
    return true;
  }
  return false;
}

static bool check_smem_feasible(const void *kernel_func, size_t dyn_smem_bytes) {
  int device = 0;
  int max_smem = 0;
  cudaFuncAttributes attrs{};
  cudaGetDevice(&device);
  cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin, device);
  cudaFuncGetAttributes(&attrs, kernel_func);
  return dyn_smem_bytes + attrs.sharedSizeBytes <= (size_t)max_smem;
}

static inline void check(cudaError_t err, const char *msg) {
  if (err != cudaSuccess) {
    fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
    exit(EXIT_FAILURE);
  }
}

#if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
template <int kHeadDim>
static void test_flash_attn_3_cute_tma_copy_smoke() {
  using namespace cute;
  using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
  using SmemLayout = typename Traits::SmemLayoutQKV;
  constexpr int kRows = 128;
  constexpr int kCount = kRows * kHeadDim;

  half *h_input = (half *)malloc(kCount * sizeof(half));
  half *h_output = (half *)malloc(kCount * sizeof(half));
  for (int idx = 0; idx < kCount; ++idx) {
    h_input[idx] = __float2half((idx % 251) * 0.00390625f);
  }

  half *d_input;
  half *d_output;
  check(cudaMalloc(&d_input, kCount * sizeof(half)), "cute tma smoke alloc input");
  check(cudaMalloc(&d_output, kCount * sizeof(half)), "cute tma smoke alloc output");
  check(cudaMemcpy(d_input, h_input, kCount * sizeof(half), cudaMemcpyHostToDevice),
        "cute tma smoke H2D");

  auto mQ = make_tensor(
      make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(d_input)),
      make_shape(kRows, Int<kHeadDim>{}),
      make_stride(Int<kHeadDim>{}, _1{}));
  auto tma_q = make_tma_copy(
      SM90_TMA_LOAD{}, mQ, SmemLayout{},
      Shape<_64, Int<kHeadDim>>{}, _1{});
  auto kernel = flash_attn_3_cute_tma_copy_smoke<kHeadDim, decltype(tma_q)>;
  constexpr int kSmemBytes = cosize(SmemLayout{}) * sizeof(cutlass::half_t);
  check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             kSmemBytes),
        "cute tma smoke set smem");
  kernel<<<2, 128, kSmemBytes>>>(
      tma_q, reinterpret_cast<cutlass::half_t *>(d_output), kRows);
  check(cudaGetLastError(), "cute tma smoke launch");
  check(cudaDeviceSynchronize(), "cute tma smoke sync");
```

```
  check(cudaMemcpy(h_output, d_output, kCount * sizeof(half), cudaMemcpyDeviceToHost),
        "cute tma smoke D2H");

  float max_err = 0.0f;
  for (int idx = 0; idx < kCount; ++idx) {
    max_err = max(max_err, fabsf(__half2float(h_input[idx]) -
                                 __half2float(h_output[idx])));
  }
  char label[64];
  snprintf(label, sizeof(label), "CuTe TMA copy smoke (D=%d)", kHeadDim);
  printf("| %-56s | %.3e |\n", label, max_err);

  free(h_input);
  free(h_output);
  cudaFree(d_input);
  cudaFree(d_output);
}

template <int kHeadDim>
static void test_flash_attn_3_tma_mma_ws_split_q_cute() {
  using namespace cute;
  using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
  using SmemLayout = typename Traits::SmemLayoutQKV;
  constexpr int kSeqlen = 128;
  constexpr int kCount = kSeqlen * kHeadDim;

  half *h_q = (half *)malloc(kCount * sizeof(half));
  half *h_k = (half *)malloc(kCount * sizeof(half));
  half *h_v = (half *)malloc(kCount * sizeof(half));
  half *h_o = (half *)malloc(kCount * sizeof(half));
  float *ref_o = (float *)malloc(kCount * sizeof(float));
  srand(42 + kHeadDim);
  for (int idx = 0; idx < kCount; ++idx) {
    h_q[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  float scale = 1.0f / sqrtf((float)kHeadDim);
  for (int row = 0; row < kSeqlen; ++row) {
    float scores[kSeqlen];
    float row_max = -INFINITY;
    for (int col = 0; col < kSeqlen; ++col) {
      float score = 0.0f;
      for (int dim = 0; dim < kHeadDim; ++dim) {
        score += __half2float(h_q[row * kHeadDim + dim]) *
                 __half2float(h_k[col * kHeadDim + dim]);
      }
      scores[col] = score * scale;
      row_max = max(row_max, scores[col]);
    }
    float row_sum = 0.0f;
    for (int col = 0; col < kSeqlen; ++col) {
      scores[col] = expf(scores[col] - row_max);
      row_sum += scores[col];
    }
    for (int dim = 0; dim < kHeadDim; ++dim) {
      float output = 0.0f;
      for (int col = 0; col < kSeqlen; ++col) {
        output += scores[col] * __half2float(h_v[col * kHeadDim + dim]);
      }
      ref_o[row * kHeadDim + dim] = output / row_sum;
    }
```

```
  }

  half *d_q;
  half *d_k;
  half *d_v;
  half *d_o;
  check(cudaMalloc(&d_q, kCount * sizeof(half)), "cute fa3 alloc Q");
  check(cudaMalloc(&d_k, kCount * sizeof(half)), "cute fa3 alloc K");
  check(cudaMalloc(&d_v, kCount * sizeof(half)), "cute fa3 alloc V");
  check(cudaMalloc(&d_o, kCount * sizeof(half)), "cute fa3 alloc O");
  check(cudaMemcpy(d_q, h_q, kCount * sizeof(half), cudaMemcpyHostToDevice),
        "cute fa3 H2D Q");
  check(cudaMemcpy(d_k, h_k, kCount * sizeof(half), cudaMemcpyHostToDevice),
        "cute fa3 H2D K");
  check(cudaMemcpy(d_v, h_v, kCount * sizeof(half), cudaMemcpyHostToDevice),
        "cute fa3 H2D V");

  auto make_tma = [=](half *pointer) {
    auto tensor = make_tensor(
        make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
        make_shape(kSeqlen, Int<kHeadDim>{}),
        make_stride(Int<kHeadDim>{}, _1{}));
    return make_tma_copy(
        SM90_TMA_LOAD{}, tensor, SmemLayout{},
        Shape<_64, Int<kHeadDim>>{}, _1{});
  };
  auto tma_q = make_tma(d_q);
  auto tma_k = make_tma(d_k);
  auto tma_v = make_tma(d_v);
  auto kernel = flash_attn_3_tma_mma_ws_split_q_cute<
      kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v)>;
  auto acc_o = partition_fragment_C(
      typename Traits::TiledMma{}, Shape<_64, Int<kHeadDim>>{});
  constexpr int kNumConsumers = 2;
  constexpr int kStagesK = 1;
  constexpr int kTiles = 1 + kNumConsumers * kStagesK + kNumConsumers;
  constexpr int kTilesBytes =
      kTiles * cosize(SmemLayout{}) * sizeof(cutlass::half_t);
  int merge_bytes = 128 * size(acc_o) * sizeof(float) + 128 * sizeof(float4);
  int smem_bytes = max(kTilesBytes, merge_bytes);
  check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             smem_bytes),
        "cute fa3 set smem");
  kernel<<<dim3(2, 1), 384, smem_bytes>>>(
      tma_q, tma_k, tma_v,
      reinterpret_cast<cutlass::half_t *>(d_o), kSeqlen, kSeqlen);
  check(cudaGetLastError(), "cute fa3 launch");
  check(cudaDeviceSynchronize(), "cute fa3 sync");
  check(cudaMemcpy(h_o, d_o, kCount * sizeof(half), cudaMemcpyDeviceToHost),
        "cute fa3 D2H");

  float max_err = 0.0f;
  for (int idx = 0; idx < kCount; ++idx) {
    max_err = max(max_err, fabsf(__half2float(h_o[idx]) - ref_o[idx]));
  }
  char label[80];
  snprintf(label, sizeof(label), "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=1, Sv=1, F32Acc, D=%d)", kHeadDim)
    ;
  printf("| %-56s | %.3e |\n", label, max_err);

  free(h_q);
  free(h_k);
  free(h_v);
```

```
  free(h_o);
  free(ref_o);
  cudaFree(d_q);
  cudaFree(d_k);
  cudaFree(d_v);
  cudaFree(d_o);
}
#endif

#if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// FA2-style CuTe cp.async test: single consumer, Br=128, no TMA/WS.
// kSeqlen=256 -> 2 Q tiles, covers multi-Q-tile path.
// kStagesK=2 验证 cp.async pipeline + V 单 buffer + V/K group 策略正确性。
template <int kHeadDim>
static void test_flash_attn_mma_stages_split_q_cute() {
  using namespace cute;
  using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
  using SmemLayoutQ = typename Traits::SmemLayoutQ;
  using SmemLayoutKV = typename Traits::SmemLayoutKV;
  constexpr int kBr = 128;
  constexpr int kStagesK = 2;
  constexpr int kSeqlen = 256;
  constexpr int kCount = kSeqlen * kHeadDim;

  half *h_q = (half *)malloc(kCount * sizeof(half));
  half *h_k = (half *)malloc(kCount * sizeof(half));
  half *h_v = (half *)malloc(kCount * sizeof(half));
  half *h_o = (half *)malloc(kCount * sizeof(half));
  float *ref_o = (float *)malloc(kCount * sizeof(float));
  srand(42 + kHeadDim);
  for (int idx = 0; idx < kCount; ++idx) {
    h_q[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  // CPU FP32 naive attention reference
  float scale = 1.0f / sqrtf((float)kHeadDim);
  for (int row = 0; row < kSeqlen; ++row) {
    float scores[kSeqlen];
    float row_max = -INFINITY;
    for (int col = 0; col < kSeqlen; ++col) {
      float score = 0.0f;
      for (int dim = 0; dim < kHeadDim; ++dim) {
        score += __half2float(h_q[row * kHeadDim + dim]) *
                 __half2float(h_k[col * kHeadDim + dim]);
      }
      scores[col] = score * scale;
      row_max = max(row_max, scores[col]);
    }
    float row_sum = 0.0f;
    for (int col = 0; col < kSeqlen; ++col) {
      scores[col] = expf(scores[col] - row_max);
      row_sum += scores[col];
    }
    for (int dim = 0; dim < kHeadDim; ++dim) {
      float output = 0.0f;
      for (int col = 0; col < kSeqlen; ++col) {
        output += scores[col] * __half2float(h_v[col * kHeadDim + dim]);
      }
      ref_o[row * kHeadDim + dim] = output / row_sum;
    }
  }
```

```
7811
7812    half *d_q;
7813    half *d_k;
7814    half *d_v;
7815    half *d_o;
7816    check(cudaMalloc(&d_q, kCount * sizeof(half)), "cute fa2 cpasync alloc Q");
7817    check(cudaMalloc(&d_k, kCount * sizeof(half)), "cute fa2 cpasync alloc K");
7818    check(cudaMalloc(&d_v, kCount * sizeof(half)), "cute fa2 cpasync alloc V");
7819    check(cudaMalloc(&d_o, kCount * sizeof(half)), "cute fa2 cpasync alloc O");
7820    check(cudaMemcpy(d_q, h_q, kCount * sizeof(half), cudaMemcpyHostToDevice),
7821          "cute fa2 cpasync H2D Q");
7822    check(cudaMemcpy(d_k, h_k, kCount * sizeof(half), cudaMemcpyHostToDevice),
7823          "cute fa2 cpasync H2D K");
7824    check(cudaMemcpy(d_v, h_v, kCount * sizeof(half), cudaMemcpyHostToDevice),
7825          "cute fa2 cpasync H2D V");
7826
7827    // 无需 TMA descriptor，直接传指针
7828    auto kernel = flash_attn_mma_stages_split_q_cute<kHeadDim, kStagesK>;
7829    // smem = Q[128*D] + K[Sk*64*D] + V[1*64*D]
7830    constexpr int kSmemBytes =
7831        (size(SmemLayoutQ{}) +
7832         kStagesK * size(SmemLayoutKV{}) +
7833         1 * size(SmemLayoutKV{})) * sizeof(cutlass::half_t);
7834    check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
7835                               kSmemBytes),
7836          "cute fa2 cpasync set smem");
7837    kernel<<<dim3(kSeqlen / kBr, 1), 256, kSmemBytes>>>(
7838        reinterpret_cast<cutlass::half_t *>(d_q),
7839        reinterpret_cast<cutlass::half_t *>(d_k),
7840        reinterpret_cast<cutlass::half_t *>(d_v),
7841        reinterpret_cast<cutlass::half_t *>(d_o), kSeqlen, kSeqlen);
7842    check(cudaGetLastError(), "cute fa2 cpasync launch");
7843    check(cudaDeviceSynchronize(), "cute fa2 cpasync sync");
7844    check(cudaMemcpy(h_o, d_o, kCount * sizeof(half), cudaMemcpyDeviceToHost),
7845          "cute fa2 cpasync D2H");
7846
7847    float max_err = 0.0f;
7848    for (int idx = 0; idx < kCount; ++idx) {
7849      max_err = max(max_err, fabsf(__half2float(h_o[idx]) - ref_o[idx]));
7850    }
7851    char label[96];
7852    snprintf(label, sizeof(label),
7853             "FA2 CuTe MMA Stages (Sk=%d, F32Acc, D=%d)", kStagesK, kHeadDim);
7854    printf("| %-56s | %.3e |\n", label, max_err);
7855
7856    free(h_q);
7857    free(h_k);
7858    free(h_v);
7859    free(h_o);
7860    free(ref_o);
7861    cudaFree(d_q);
7862    cudaFree(d_k);
7863    cudaFree(d_v);
7864    cudaFree(d_o);
7865  }
7866  #endif
7867
7868  #if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7869  // FA2-style CuTe test: single consumer, Br=128, no merge.
7870  // kSeqlen=256 -> 2 Q tiles, covers multi-Q-tile path.
7871  template <int kHeadDim>
7872  static void test_flash_attn_tma_mma_ws_split_q_cute() {
7873    using namespace cute;
```

```
  using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
  using SmemLayoutQ = typename Traits::SmemLayoutQ;
  using SmemLayoutKV = typename Traits::SmemLayoutKV;
  constexpr int kBr = 128;
  constexpr int kSeqlen = 256;
  constexpr int kCount = kSeqlen * kHeadDim;

  half *h_q = (half *)malloc(kCount * sizeof(half));
  half *h_k = (half *)malloc(kCount * sizeof(half));
  half *h_v = (half *)malloc(kCount * sizeof(half));
  half *h_o = (half *)malloc(kCount * sizeof(half));
  float *ref_o = (float *)malloc(kCount * sizeof(float));
  srand(42 + kHeadDim);
  for (int idx = 0; idx < kCount; ++idx) {
    h_q[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[idx] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  float scale = 1.0f / sqrtf((float)kHeadDim);
  for (int row = 0; row < kSeqlen; ++row) {
    float scores[kSeqlen];
    float row_max = -INFINITY;
    for (int col = 0; col < kSeqlen; ++col) {
      float score = 0.0f;
      for (int dim = 0; dim < kHeadDim; ++dim) {
        score += __half2float(h_q[row * kHeadDim + dim]) *
                 __half2float(h_k[col * kHeadDim + dim]);
      }
      scores[col] = score * scale;
      row_max = max(row_max, scores[col]);
    }
    float row_sum = 0.0f;
    for (int col = 0; col < kSeqlen; ++col) {
      scores[col] = expf(scores[col] - row_max);
      row_sum += scores[col];
    }
    for (int dim = 0; dim < kHeadDim; ++dim) {
      float output = 0.0f;
      for (int col = 0; col < kSeqlen; ++col) {
        output += scores[col] * __half2float(h_v[col * kHeadDim + dim]);
      }
      ref_o[row * kHeadDim + dim] = output / row_sum;
    }
  }

  half *d_q;
  half *d_k;
  half *d_v;
  half *d_o;
  check(cudaMalloc(&d_q, kCount * sizeof(half)), "cute fa2 alloc Q");
  check(cudaMalloc(&d_k, kCount * sizeof(half)), "cute fa2 alloc K");
  check(cudaMalloc(&d_v, kCount * sizeof(half)), "cute fa2 alloc V");
  check(cudaMalloc(&d_o, kCount * sizeof(half)), "cute fa2 alloc O");
  check(cudaMemcpy(d_q, h_q, kCount * sizeof(half), cudaMemcpyHostToDevice),
        "cute fa2 H2D Q");
  check(cudaMemcpy(d_k, h_k, kCount * sizeof(half), cudaMemcpyHostToDevice),
        "cute fa2 H2D K");
  check(cudaMemcpy(d_v, h_v, kCount * sizeof(half), cudaMemcpyHostToDevice),
        "cute fa2 H2D V");

  // Q tile: [128, D], K/V tile: [64, D] (both use GMMA::Layout_K_SW128_Atom)
  auto make_tma_q = [=]() {
```

```
    auto tensor = make_tensor(
        make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(d_q)),
        make_shape(kSeqlen, Int<kHeadDim>{}),
        make_stride(Int<kHeadDim>{}, _1{}));
    return make_tma_copy(
        SM90_TMA_LOAD{}, tensor, SmemLayoutQ{},
        Shape<_128, Int<kHeadDim>>{}, _1{});
  };
  auto make_tma_kv = [=](half *pointer) {
    auto tensor = make_tensor(
        make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
        make_shape(kSeqlen, Int<kHeadDim>{}),
        make_stride(Int<kHeadDim>{}, _1{}));
    return make_tma_copy(
        SM90_TMA_LOAD{}, tensor, SmemLayoutKV{},
        Shape<_64, Int<kHeadDim>>{}, _1{});
  };
  auto tma_q = make_tma_q();
  auto tma_k = make_tma_kv(d_k);
  auto tma_v = make_tma_kv(d_v);
  auto kernel = flash_attn_tma_mma_ws_split_q_cute<
      kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v)>;
  // smem = Q[128*D] + K[Sk*64*D] + V[Sv*64*D] (Sk=1, Sv=1 default)
  constexpr int kSmemBytes =
      (cosize(SmemLayoutQ{}) +
       1 * cosize(SmemLayoutKV{}) +
       1 * cosize(SmemLayoutKV{})) * sizeof(cutlass::half_t);
  check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             kSmemBytes),
        "cute fa2 set smem");
  kernel<<<dim3(kSeqlen / kBr, 1), 384, kSmemBytes>>>(
      tma_q, tma_k, tma_v,
      reinterpret_cast<cutlass::half_t *>(d_o), kSeqlen, kSeqlen);
  check(cudaGetLastError(), "cute fa2 launch");
  check(cudaDeviceSynchronize(), "cute fa2 sync");
  check(cudaMemcpy(h_o, d_o, kCount * sizeof(half), cudaMemcpyDeviceToHost),
        "cute fa2 D2H");

  float max_err = 0.0f;
  for (int idx = 0; idx < kCount; ++idx) {
    max_err = max(max_err, fabsf(__half2float(h_o[idx]) - ref_o[idx]));
  }
  char label[80];
  snprintf(label, sizeof(label), "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=1, Sv=1, F32Acc, D=%d)", kHeadDim)
    ;
  printf("| %-56s | %.3e |\n", label, max_err);

  free(h_q);
  free(h_k);
  free(h_v);
  free(h_o);
  free(ref_o);
  cudaFree(d_q);
  cudaFree(d_k);
  cudaFree(d_v);
  cudaFree(d_o);
}
#endif

static void test_block_reduce(int N) {

  srand(42);
  float *h_a = (float *)malloc((size_t)N * sizeof(float));
```

```
  for (int i = 0; i < N; i++)
    h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: sum of all elements
  double ref = 0.0;
  for (int i = 0; i < N; i++) ref += (double)h_a[i];

  float *d_a, *d_y;
  check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
  check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
  check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");

  dim3 block(128);
  dim3 grid((N + 127) / 128);
  block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
  check(cudaGetLastError(), "blockreduce launch");
  check(cudaDeviceSynchronize(), "blockreduce sync");

  float result;
  check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");

  float err = fabsf(result - (float)ref);
  printf("| %-56s | %.3e |\n", "BlockReduce", err);

  free(h_a);
  cudaFree(d_a); cudaFree(d_y);
}


static void test_dot(int N) {

  srand(42);
  float *h_a = (float *)malloc((size_t)N * sizeof(float));
  float *h_b = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++) {
    h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
    h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  }

  // CPU reference
  double ref = 0.0;
  for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];

  float *d_a, *d_b, *d_y;
  check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
  check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
  check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
  check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
  check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");

  dim3 block(128);
  dim3 grid((N + 127) / 128);
  dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
  check(cudaGetLastError(), "dot launch");
  check(cudaDeviceSynchronize(), "dot sync");

  float result;
  check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");

```

```
  float err = fabsf(result - (float)ref);
  printf("| %-56s | %.3e |\n", "Dot", err);

  // ---- Dot Vec4 ----
  check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
  dim3 block_v4(32);
  dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
  check(cudaGetLastError(), "dot_vec4 launch");
  check(cudaDeviceSynchronize(), "dot_vec4 sync");

  check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
  float err_v4 = fabsf(result - (float)ref);
  printf("| %-56s | %.3e |\n", "Dot-Vec4", err_v4);

  free(h_a); free(h_b);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
}


static void test_relu(int N) {
  srand(42);
  float *h_x = (float *)malloc((size_t)N * sizeof(float));
  float *h_y = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
  check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");

  for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
  dim3 block256(256);
  dim3 grid256((N + 255) / 256);
  relu<<<grid256, block256>>>(d_x, d_y, N);
  check(cudaGetLastError(), "relu launch");
  check(cudaDeviceSynchronize(), "relu sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float expected = fmaxf(0.0f, h_x[i]);
    float err = fabsf(h_y[i] - expected);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "ReLU", max_err);

  dim3 block64(64);
  relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
  check(cudaGetLastError(), "relu_vec4 launch");
  check(cudaDeviceSynchronize(), "relu_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float expected = fmaxf(0.0f, h_x[i]);
    float err = fabsf(h_y[i] - expected);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "ReLU-Vec4", max_err);

  free(h_x); free(h_y);
  cudaFree(d_x); cudaFree(d_y);
}

```

```
8125
8126 static void test_elementwise(int N) {
8127   srand(42);
8128   float *h_a = (float *)malloc((size_t)N * sizeof(float));
8129   float *h_b = (float *)malloc((size_t)N * sizeof(float));
8130   for (int i = 0; i < N; i++) {
8131     h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8132     h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8133   }
8134
8135   float *d_a, *d_b, *d_c;
8136   check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
8137   check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
8138   check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
8139   check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
8140   check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
8141
8142   dim3 block256(256);
8143   dim3 grid256((N + 255) / 256);
8144   elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
8145   check(cudaGetLastError(), "eadd launch");
8146   check(cudaDeviceSynchronize(), "eadd sync");
8147   float *h_c = (float *)malloc((size_t)N * sizeof(float));
8148   check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
8149   float max_err = 0.0f;
8150   for (int i = 0; i < N; i++) {
8151     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
8152     if (err > max_err) max_err = err;
8153   }
8154   printf("| %-56s | %.3e |\n", "ElemwiseAdd", max_err);
8155
8156   dim3 block64(64);
8157   check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
8158   elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
8159   check(cudaGetLastError(), "eadd_vec4 launch");
8160   check(cudaDeviceSynchronize(), "eadd_vec4 sync");
8161   check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
8162   max_err = 0.0f;
8163   for (int i = 0; i < N; i++) {
8164     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
8165     if (err > max_err) max_err = err;
8166   }
8167   printf("| %-56s | %.3e |\n", "ElemwiseAdd-Vec4", max_err);
8168
8169   free(h_a); free(h_b); free(h_c);
8170   cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
8171 }
8172
8173
8174 static void test_histogram(int N) {
8175   srand(42);
8176   int BINS = 16;
8177   int *h_hist = (int *)calloc(BINS, sizeof(int));
8178   int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
8179   int *h_idx = (int *)malloc((size_t)N * sizeof(int));
8180   for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
8181   for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
8182
8183   int *d_idx, *d_hist;
8184   check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
8185   check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
8186   check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
8187   check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
```

```
8188
8189    dim3 block256(256);
8190    dim3 grid256((N + 255) / 256);
8191    histogram<<<grid256, block256>>>(d_idx, d_hist, N);
8192    check(cudaGetLastError(), "histogram launch");
8193    check(cudaDeviceSynchronize(), "histogram sync");
8194    check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
8195
8196    float max_err = 0.0f;
8197    for (int i = 0; i < BINS; i++) {
8198      float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
8199      if (err > max_err) max_err = err;
8200    }
8201    printf("| %-56s | %.3e |\n", "Histogram", max_err);
8202
8203    free(h_hist); free(h_hist_ref); free(h_idx);
8204    cudaFree(d_idx); cudaFree(d_hist);
8205  }
8206
8207
8208  static void test_merge_attn_states(int num_tokens, int num_heads,
8209                                     int head_size) {
8210
8211    size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
8212    size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
8213
8214    float *h_prefix_out = (float *)malloc(out_size);
8215    float *h_suffix_out = (float *)malloc(out_size);
8216    float *h_prefix_lse = (float *)malloc(lse_size);
8217    float *h_suffix_lse = (float *)malloc(lse_size);
8218    float *h_output_ref = (float *)malloc(out_size);
8219
8220    srand(42);
8221    for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
8222      h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8223      h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8224    }
8225    for (int i = 0; i < num_heads * num_tokens; i++) {
8226      h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
8227      h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
8228    }
8229
8230    // CPU reference: 与 kernel 完全相同的浮点计算
8231    for (int t = 0; t < num_tokens; t++) {
8232      for (int h = 0; h < num_heads; h++) {
8233        float p_lse = h_prefix_lse[h * num_tokens + t];
8234        float s_lse = h_suffix_lse[h * num_tokens + t];
8235        p_lse = isinf(p_lse) ? -INFINITY : p_lse;
8236        s_lse = isinf(s_lse) ? -INFINITY : s_lse;
8237
8238        float max_lse = fmaxf(p_lse, s_lse);
8239        p_lse -= max_lse;
8240        s_lse -= max_lse;
8241        float p_se = expf(p_lse);
8242        float s_se = expf(s_lse);
8243        float p_scale = p_se / (p_se + s_se);
8244        float s_scale = s_se / (p_se + s_se);
8245
8246        int head_off = t * num_heads * head_size + h * head_size;
8247        for (int d = 0; d < head_size; d++) {
8248          h_output_ref[head_off + d] =
8249              h_prefix_out[head_off + d] * p_scale +
8250              h_suffix_out[head_off + d] * s_scale;
```

```
    }
  }
}

float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");

check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
                 cudaMemcpyHostToDevice),
      "merge_attn H2D p_out");
check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
                 cudaMemcpyHostToDevice),
      "merge_attn H2D s_out");
check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
                 cudaMemcpyHostToDevice),
      "merge_attn H2D p_lse");
check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
                 cudaMemcpyHostToDevice),
      "merge_attn H2D s_lse");

int threads_per_head = head_size / 4;
int total_threads = num_tokens * num_heads * threads_per_head;
dim3 block(128);
dim3 grid((total_threads + 127) / 128);
merge_attn_states<<<grid, block>>>(
    d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
    num_tokens, num_heads, head_size);
check(cudaGetLastError(), "merge_attn launch");
check(cudaDeviceSynchronize(), "merge_attn sync");

float *h_output = (float *)malloc(out_size);
check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
      "merge_attn D2H");

float max_err = 0.0f;
for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
  float err = fabsf(h_output[i] - h_output_ref[i]);
  if (err > max_err) max_err = err;
}
printf("| %-56s | %.3e |\n", "MergeAttnStates", max_err);

// 边界测试: +inf LSE → 权重退化为 0（空 attention 段）
h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
h_suffix_lse[0] = 0.0f;
check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
                 cudaMemcpyHostToDevice),
      "merge_attn H2D inf lse");
merge_attn_states<<<grid, block>>>(
    d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
    num_tokens, num_heads, head_size);
check(cudaGetLastError(), "merge_attn inf launch");
check(cudaDeviceSynchronize(), "merge_attn inf sync");
check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
      "merge_attn inf D2H");

// token 0, head 0 的所有元素应等于 suffix_output（prefix 权重 α=0）
float inf_err = 0.0f;
for (int d = 0; d < head_size; d++) {
  float err = fabsf(h_output[d] - h_suffix_out[d]);
```

```
    if (err > inf_err) inf_err = err;
  }
  printf("| %-56s | %.3e |\n", "MergeAttnStates-inf", inf_err);

  free(h_prefix_out);
  free(h_suffix_out);
  free(h_prefix_lse);
  free(h_suffix_lse);
  free(h_output_ref);
  free(h_output);
  cudaFree(d_prefix_out);
  cudaFree(d_suffix_out);
  cudaFree(d_prefix_lse);
  cudaFree(d_suffix_lse);
  cudaFree(d_output);
}


static void test_softmax(int N) {
  // Use N = blockDim.x so one block processes all elements as one token.
  constexpr int kNumThreads = 256;
  if (N != kNumThreads) {
    printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
    return;
  }

  srand(42);
  float *h_x = (float *)malloc((size_t)N * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
  for (int i = 0; i < N; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f;  // [-5, 5]

  // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
  float max_val = -FLT_MAX;
  for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
  double sum_exp = 0.0;
  for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
  for (int i = 0; i < N; i++)
    h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
  check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");

  check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");

  dim3 block(256);
  dim3 grid(1);  // one token covering all N elements
  online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "softmax launch");
  check(cudaDeviceSynchronize(), "softmax sync");

  float *h_y = (float *)malloc((size_t)N * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "OnlineSafeSoftmax", max_err);

  // ---- Safe Softmax ----
```

```
  safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "safe_softmax launch");
  check(cudaDeviceSynchronize(), "safe_softmax sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "SafeSoftmax", max_err);

  // ---- Naive Softmax ----
  softmax_per_token<<<grid, block>>>(d_x, d_y, N);
  check(cudaGetLastError(), "naive_softmax launch");
  check(cudaDeviceSynchronize(), "naive_softmax sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
  max_err = 0.0f;
  for (int i = 0; i < N; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "NaiveSoftmax", max_err);

  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_rms_norm(int N, int K) {

  srand(42);
  float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
  for (int i = 0; i < N * K; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  float g = 1.5f;  // gain

  // CPU reference: y = (x / rms(x)) * g
  float epsilon = 1e-5f;
  for (int n = 0; n < N; n++) {
    double sum_sq = 0.0;
    for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
    float rms = sqrtf((float)sum_sq / (float)K + epsilon);
    for (int k = 0; k < K; k++)
      h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
  check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");

  dim3 block(128);
  dim3 grid(N);
  rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
  check(cudaGetLastError(), "rmsnorm launch");
  check(cudaDeviceSynchronize(), "rmsnorm sync");

  float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");

  float max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
```

```
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "RMSNorm", max_err);

  // ---- RMS Norm Vec4 ----
  dim3 block_rv4(32);
  rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
  check(cudaGetLastError(), "rmsnorm_vec4 launch");
  check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "RMSNorm-Vec4", max_err);

  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_layer_norm(int N, int K) {

  srand(42);
  float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
  float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
  for (int i = 0; i < N * K; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  float g = 1.5f, b = 0.3f;  // gain and bias

  // CPU reference: y = ((x - mean) / std) * g + b
  float epsilon = 1e-5f;
  for (int n = 0; n < N; n++) {
    double sum = 0.0;
    for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
    float mean = (float)sum / (float)K;
    double sum_sq = 0.0;
    for (int k = 0; k < K; k++) {
      float diff = h_x[n * K + k] - mean;
      sum_sq += (double)diff * (double)diff;
    }
    float std = sqrtf((float)sum_sq / (float)K + epsilon);
    for (int k = 0; k < K; k++)
      h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
  check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
  check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");

  dim3 block(128);
  dim3 grid(N);
  layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
  check(cudaGetLastError(), "layernorm launch");
  check(cudaDeviceSynchronize(), "layernorm sync");

  float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");

  float max_err = 0.0f;
```

```
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "LayerNorm", max_err);

  // ---- Layer Norm Vec4 ----
  dim3 block_lv4(32);
  layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
  check(cudaGetLastError(), "layernorm_vec4 launch");
  check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
  check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
  max_err = 0.0f;
  for (int i = 0; i < N * K; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "LayerNorm-Vec4", max_err);

  free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_x); cudaFree(d_y);
}


static void test_rope(int seq_len, int N) {

  int total_pairs = seq_len * N;
  int total_elems = total_pairs * 2;
  size_t size = (size_t)total_elems * sizeof(float);

  float *h_x = (float *)malloc(size);
  float *h_y_ref = (float *)malloc(size);

  srand(42);
  for (int i = 0; i < total_elems; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: 2D rotation for each pair
  for (int idx = 0; idx < total_pairs; idx++) {
    int token_pos = idx / N;
    int token_idx = idx % N;
    float x1 = h_x[idx * 2];
    float x2 = h_x[idx * 2 + 1];
    float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
    float angle = (float)token_pos * theta;
    float cos_v = cosf(angle);
    float sin_v = sinf(angle);
    h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
    h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
  }

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, size), "rope alloc X");
  check(cudaMalloc(&d_y, size), "rope alloc Y");
  check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");

  dim3 block(256);
  dim3 grid((total_pairs + 255) / 256);
  rope<<<grid, block>>>(d_x, d_y, seq_len, N);
  check(cudaGetLastError(), "rope launch");
  check(cudaDeviceSynchronize(), "rope sync");

  float *h_y = (float *)malloc(size);
```

```
  check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");

  float max_err = 0.0f;
  for (int i = 0; i < total_elems; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "RoPE", max_err);

  free(h_x);
  free(h_y);
  free(h_y_ref);
  cudaFree(d_x);
  cudaFree(d_y);
}


static void test_mat_transpose(int row, int col) {

  size_t size_in = (size_t)row * col * sizeof(float);
  size_t size_out = (size_t)col * row * sizeof(float);

  float *h_x = (float *)malloc(size_in);
  float *h_y_ref = (float *)malloc(size_out);

  srand(42);
  for (int i = 0; i < row * col; i++)
    h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  // CPU reference: y[j][i] = x[i][j] (row-major)
  for (int i = 0; i < row; i++)
    for (int j = 0; j < col; j++)
      h_y_ref[j * row + i] = h_x[i * col + j];

  float *d_x, *d_y;
  check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
  check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
  check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");

  dim3 block(16, 16);
  dim3 grid((col + 15) / 16, (row + 15) / 16);
  mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
  check(cudaGetLastError(), "mattrans launch");
  check(cudaDeviceSynchronize(), "mattrans sync");

  float *h_y = (float *)malloc(size_out);
  check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");

  float max_err = 0.0f;
  for (int i = 0; i < col * row; i++) {
    float err = fabsf(h_y[i] - h_y_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "MatTranspose", max_err);

  free(h_x);
  free(h_y);
  free(h_y_ref);
  cudaFree(d_x);
  cudaFree(d_y);
}


```

```
8629 static void test_mat_transpose_padded(int row, int col) {
8630
8631   size_t size_in = (size_t)row * col * sizeof(float);
8632   size_t size_out = (size_t)col * row * sizeof(float);
8633
8634   float *h_x = (float *)malloc(size_in);
8635   float *h_y_ref = (float *)malloc(size_out);
8636
8637   srand(42);
8638   for (int i = 0; i < row * col; i++)
8639     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8640
8641   // CPU reference: y[j][i] = x[i][j] (row-major)
8642   for (int i = 0; i < row; i++)
8643     for (int j = 0; j < col; j++)
8644       h_y_ref[j * row + i] = h_x[i * col + j];
8645
8646   float *d_x, *d_y;
8647   check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
8648   check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
8649   check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
8650
8651   dim3 block(16, 16);
8652   dim3 grid((col + 15) / 16, (row + 63) / 64);
8653   mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
8654   check(cudaGetLastError(), "mattrans_padded launch");
8655   check(cudaDeviceSynchronize(), "mattrans_padded sync");
8656
8657   float *h_y = (float *)malloc(size_out);
8658   check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
8659
8660   float max_err = 0.0f;
8661   for (int i = 0; i < col * row; i++) {
8662     float err = fabsf(h_y[i] - h_y_ref[i]);
8663     if (err > max_err) max_err = err;
8664   }
8665   printf("| %-56s | %.3e |\n", "MatTransposePadded", max_err);
8666
8667   free(h_x);
8668   free(h_y);
8669   free(h_y_ref);
8670   cudaFree(d_x);
8671   cudaFree(d_y);
8672 }
8673
8674
8675 static void test_sgemv(int M, int K) {
8676
8677   srand(42);
8678   float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
8679   float *h_x = (float *)malloc((size_t)K * sizeof(float));
8680   float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
8681   for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8682   for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
8683
8684   // CPU reference: y[m] = sum_k A[m,k] * x[k]
8685   for (int m = 0; m < M; m++) {
8686     double sum = 0.0;
8687     for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
8688     h_y_ref[m] = (float)sum;
8689   }
8690
8691   float *d_a, *d_x, *d_y;
```

```
  check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
  check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
  check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
  check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");

  dim3 block(32, 4);
  dim3 grid((M + 3) / 4);
  sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
  check(cudaGetLastError(), "sgemv launch");
  check(cudaDeviceSynchronize(), "sgemv sync");

  float *h_y = (float *)malloc((size_t)M * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");

  float max_err = 0.0f;
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "SGEMV-K128", max_err);

  // ---- SGEMV K32 ----
  check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
  sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
  check(cudaGetLastError(), "sgemv_k32 launch");
  check(cudaDeviceSynchronize(), "sgemv_k32 sync");

  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");

  max_err = 0.0f;
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "SGEMV-K32", max_err);

  // ---- SGEMV K16 ----
  free(h_a); free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);

  int K16 = 16;
  h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
  h_x = (float *)malloc((size_t)K16 * sizeof(float));
  h_y_ref = (float *)malloc((size_t)M * sizeof(float));
  for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  for (int m = 0; m < M; m++) {
    double sum = 0.0;
    for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
    h_y_ref[m] = (float)sum;
  }

  check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
  check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
  check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");

  check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
  check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");

  dim3 grid_k16((M + 7) / 8);
```

```
  sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
  check(cudaGetLastError(), "sgemv_k16 launch");
  check(cudaDeviceSynchronize(), "sgemv_k16 sync");

  h_y = (float *)malloc((size_t)M * sizeof(float));
  check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");

  max_err = 0.0f;
  for (int m = 0; m < M; m++) {
    float err = fabsf(h_y[m] - h_y_ref[m]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "SGEMV-K16", max_err);

  free(h_a); free(h_x); free(h_y); free(h_y_ref);
  cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
}


static void test_sgemm(int M, int N, int K) {

  size_t size_a = (size_t)M * K * sizeof(float);
  size_t size_b = (size_t)K * N * sizeof(float);
  size_t size_c = (size_t)M * N * sizeof(float);

  float *h_a = (float *)malloc(size_a);
  float *h_b = (float *)malloc(size_b);
  float *h_c_ref = (float *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
  for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

  float *d_a, *d_b, *d_c;
  check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
  check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
  check(cudaMalloc(&d_c, size_c), "sgemm alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");

  // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
  cublasHandle_t handle;
  cublasCreate(&handle);
  float alpha = 1.0f, beta = 0.0f;
  cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
              &alpha, d_b, N, d_a, K, &beta, d_c, N);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");

  // Kernel
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  dim3 block(32, 32);
  dim3 grid((N + 31) / 32, (M + 31) / 32);
  sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
  check(cudaGetLastError(), "sgemm launch");
  check(cudaDeviceSynchronize(), "sgemm sync");

  float *h_c = (float *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");

```

```
  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(h_c[i] - h_c_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "SGEMM", max_err);

  // ---- SGEMM Vec4 (128×128 tile, 4×4 thread tile) ----

  dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
  check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
  sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
  check(cudaGetLastError(), "sgemm_vec4 launch");
  check(cudaDeviceSynchronize(), "sgemm_vec4 sync");

  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");

  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(h_c[i] - h_c_ref[i]);
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "SGEMM-Vec4", max_err);

  free(h_a); free(h_b); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
  cublasDestroy(handle);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
}


static void test_hgemm_mma(int M, int N, int K) {

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // Kernel expects B^T [N×K] row-major layout (TN layout convention).
  // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
  check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "hgemm alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
```

```
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");

  // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
  // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
               &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
               &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");

  // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
  size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
  dim3 block(256);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
  check(cudaGetLastError(), "hgemm launch");
  check(cudaDeviceSynchronize(), "hgemm sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "HGEMM MMA", max_err);

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cublasDestroy(handle);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
}


static void test_hgemm_swizzle(int M, int N, int K) {
  // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
  //   + Register Double Buffering (kValTileK=4, BK=64)
  // TN layout: C[M×N] = A[M×K] × B^T[N×K]
  // Kernel: hgemm_mma_stages_tn_swizzle with default template params
  //   (kValTileK=4, kStages=2, BK=64)
  // smem: kStages × (BM×BK + BN×BK) halfs

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
```

```
  for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // Kernel expects B^T [N×K] row-major layout (TN layout convention).
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
  check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");

  // cuBLAS FP16 reference
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
               &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
               &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");

  // MMA swizzle kernel (default params: kStages=2, kValTileK=4, BK=64)
  constexpr int BM = 128, BN = 128, BK = 64, K_STAGE_S = 2;
  size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
  cudaFuncSetAttribute(
      (const void *)hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, K_STAGE_S, 0>,
      cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
  dim3 block(256);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
  check(cudaGetLastError(), "hgemm_swizzle launch");
  check(cudaDeviceSynchronize(), "hgemm_swizzle sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "HGEMM Swizzle + Reg2x", max_err);

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cublasDestroy(handle);
}


#if defined(NOTES_V2_ENABLE_CUTE)
static void test_hgemm_cute(int M, int N, int K) {
  // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
  // TN layout: C[M×N] = A[M×K] × B^T[N×K]
  // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
```

```
  // Tile: BM=128, BN=256, BK=32, 128 threads/block

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // CuTe kernel expects B^T [N×K] row-major (TN layout).
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
  check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");

  // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
               CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");

  // CuTe kernel (Stages=2, TN layout)
  launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
  check(cudaGetLastError(), "hgemm_cute launch");
  check(cudaDeviceSynchronize(), "hgemm_cute sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  printf("| %-56s | %.3e |\n", "HGEMM CuTe Swizzle + Reg2x",
         max_err);

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cublasDestroy(handle);
}
```

```
#endif /* NOTES_V2_ENABLE_CUTE */


#if defined(NOTES_V2_ENABLE_WGMMA)
static void test_hgemm_wgmma(int M, int N, int K) {
  // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
  // TN layout: C[M×N] = A[M×K] × B^T[N×K]
  // Kernel: hgemm_wgmma_stages_tn with default template params

  constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;

  // M, K must be divisible by tile dims
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-56s | %-9s |\n", "HGEMM WGMMA", "SKIP");
    return;
  }

  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);

  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);

  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

  // B^T [N×K] row-major for TN layout (same as hgemm_mma kernel)
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
  check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
  check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
  check(cudaMalloc(&d_c, size_c), "wgmma alloc C");

  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
        "wgmma H2D B (cuBLAS)");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
        "wgmma H2D B_t (kernel)");

  // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
               CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
        "wgmma D2H ref");

  // Create TMA tensor maps for A and B^T
  // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
  // B^T[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
```

```
  CUtensorMap *tma_a =
      allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b =
      allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);

  // Launch WGMMA kernel
  // kBlockSwizzle=false → 2D grid, no swizzle
  size_t smem_bytes =
      kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
  cudaFuncSetAttribute(
      hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
                            false>,
      cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);

  dim3 block(kNumThreads);
  dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
  hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
      <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
  check(cudaGetLastError(), "wgmma launch");
  check(cudaDeviceSynchronize(), "wgmma sync");

  half *h_c = (half *)malloc(size_c);
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");

  // Verify
  float max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err)
      max_err = err;
  }
  printf("| %-56s | %.3e |\n", "HGEMM TMA WGMMA WS (3-stage)", max_err);

  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cudaFree(tma_a);
  cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif /* NOTES_V2_ENABLE_WGMMA */

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
template <int kStages, int kBlockSwizzle = 0>
static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,
                                    half *d_b_t, half *d_c,
                                    CUtensorMap *tma_a, CUtensorMap *tma_b) {
  constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
  constexpr int kMmaTileM = 2, kMmaTileN = 2;
  constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
  constexpr int BM = kMmaM * kMmaTileM * kValTileM;
  constexpr int BN = kMmaN * kMmaTileN * kValTileN;
  constexpr int BK = kMmaK * kValTileK;
  constexpr int kNumThreads = 256;
  constexpr size_t payload_bytes =
      kStages * (BM * BK + BN * BK) * sizeof(half);
  constexpr size_t smem_bytes = payload_bytes;
```

```
  using Kernel = void (*)(int, int, int, half *, const CUtensorMap *,
                          const CUtensorMap *);
  Kernel kernel = hgemm_tma_mma_ws_tn<
      kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
      kValTileK, kStages, kNumThreads, kBlockSwizzle>;

  int device = 0;
  int max_smem = 0;
  cudaFuncAttributes attributes{};
  check(cudaGetDevice(&device), "tma_mma_ws get device");
  check(cudaDeviceGetAttribute(&max_smem,
                               cudaDevAttrMaxSharedMemoryPerBlockOptin,
                               device),
        "tma_mma_ws max shared memory");
  check(cudaFuncGetAttributes(&attributes, kernel),
        "tma_mma_ws function attributes");
  if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
    if (g_debug)
      printf("| %-56s | %-9s |\n",
             kStages == 2 ? "HGEMM TMA MMA WS (S=2, BLK_SW=0)"
                          : "HGEMM TMA MMA WS (S=3, BLK_SW=0)",
             "SMEM SKIP");
    return false;
  }

  check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             smem_bytes),
        "tma_mma_ws set dynamic shared memory");
  dim3 block(kNumThreads);
  constexpr int kSwizzleN = 16;
  const int n_tiles = N / BN;
  const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
  const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(grid_x, M / BM, grid_z);
  kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
  check(cudaGetLastError(), "tma_mma_ws launch");
  check(cudaDeviceSynchronize(), "tma_mma_ws sync");
  return true;
}

static void test_hgemm_tma_mma_ws(int M, int N, int K) {
  constexpr int BM = 128, BN = 128, BK = 64;
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-56s | %-9s |\n", "HGEMM TMA MMA WS", "SKIP");
    return;
  }

  const size_t size_a = size_t(M) * K * sizeof(half);
  const size_t size_b = size_t(K) * N * sizeof(half);
  const size_t size_b_t = size_t(N) * K * sizeof(half);
  const size_t size_c = size_t(M) * N * sizeof(half);
  half *h_a = static_cast<half *>(malloc(size_a));
  half *h_b = static_cast<half *>(malloc(size_b));
  half *h_b_t = static_cast<half *>(malloc(size_b_t));
  half *h_c = static_cast<half *>(malloc(size_c));
  half *h_c_ref = static_cast<half *>(malloc(size_c));
  srand(42);
  for (int i = 0; i < M * K; ++i)
    h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; ++i)
    h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
  for (int n = 0; n < N; ++n)
    for (int k = 0; k < K; ++k)
```

```
      h_b_t[n * K + k] = h_b[k * N + n];

  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
  check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
  check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
        "tma_mma_ws H2D B_t");

  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha = __float2half(1.0f), beta = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
               CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
               CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
        "tma_mma_ws D2H reference");

  CUtensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
  for (int block_swizzle : {0, 1}) {
    for (int stages : {2, 3}) {
      const bool launched = stages == 2
          ? (block_swizzle
                 ? launch_hgemm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
                                                 tma_a, tma_b)
                 : launch_hgemm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
                                              tma_a, tma_b))
          : (block_swizzle
                 ? launch_hgemm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
                                                 tma_a, tma_b)
                 : launch_hgemm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,
                                              tma_a, tma_b));
      if (!launched)
        continue;
      check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
            "tma_mma_ws D2H");
      float max_err = 0.0f;
      for (int i = 0; i < M * N; ++i)
        max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
                                       __half2float(h_c_ref[i])));
      printf("| %-56s | %.3e |\n",
             block_swizzle
                 ? (stages == 2 ? "HGEMM TMA MMA WS (S=2, BLK_SW=1)"
                                : "HGEMM TMA MMA WS (S=3, BLK_SW=1)")
                 : (stages == 2 ? "HGEMM TMA MMA WS (S=2, BLK_SW=0)"
                                : "HGEMM TMA MMA WS (S=3, BLK_SW=0)"),
             max_err);
    }
  }

  free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
  cudaFree(tma_a); cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif /* NOTES_V2_ENABLE_TMA_MMA_WS */


static void test_flash_attn(int seqlen, int head_dim) {
```

```
  // FlashAttention-2 with split-Q, MMA m16n8k16
  int B = 1, H = 8;

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);

  srand(42);
  half *h_q = (half *)malloc(sz);
  half *h_k = (half *)malloc(sz);
  half *h_v = (half *)malloc(sz);
  for (int i = 0; i < B * H * seqlen * head_dim; i++) {
    h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  // CPU reference in FP32: O = softmax(Q @ K^T / sqrt(d)) @ V
  float *ref_q = (float *)malloc(sz * 4 / sizeof(half));  // 4x for float
  float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
  float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
  float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
  int count = B * H * seqlen * head_dim;
  for (int i = 0; i < count; i++) {
    ref_q[i] = __half2float(h_q[i]);
    ref_k[i] = __half2float(h_k[i]);
    ref_v[i] = __half2float(h_v[i]);
  }

  float scale = 1.0f / sqrtf((float)head_dim);
  for (int bi = 0; bi < B * H; bi++) {
    for (int qi = 0; qi < seqlen; qi++) {
      // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
      float smax = -INFINITY;
      float *S = (float *)malloc((size_t)seqlen * sizeof(float));
      for (int kj = 0; kj < seqlen; kj++) {
        float s = 0.0f;
        for (int d = 0; d < head_dim; d++)
          s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
               ref_k[bi * seqlen * head_dim + kj * head_dim + d];
        S[kj] = s * scale;
        if (S[kj] > smax) smax = S[kj];
      }
      // softmax
      double sum_exp = 0.0;
      for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
      float inv_sum = 1.0f / (float)sum_exp;
      // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
      for (int d = 0; d < head_dim; d++) {
        double o_acc = 0.0;
        for (int kj = 0; kj < seqlen; kj++)
          o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
                   ref_v[bi * seqlen * head_dim + kj * head_dim + d];
        ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
      }
      free(S);
    }
  }

  half *d_q, *d_k, *d_v, *d_o;
  check(cudaMalloc(&d_q, sz), "fa alloc Q");
  check(cudaMalloc(&d_k, sz), "fa alloc K");
  check(cudaMalloc(&d_v, sz), "fa alloc V");
  check(cudaMalloc(&d_o, sz), "fa alloc O");
  check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
```

```
  check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
  check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");

  // Template params for kHeadDim=64, kStagesK=2
  constexpr int kHeadDim = 64;
  constexpr int kStagesK = 2;
  constexpr int kPadQ = 8;
  constexpr int kPadK = 8;
  constexpr int kPadV = 8;
  constexpr int kMmaAtomM = 16;
  constexpr int kMmaAtomN = 8;
  constexpr int kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 8;
  constexpr int kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 8;
  constexpr int kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1;
  constexpr int kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);

  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
  if (seqlen < Br) return; // kernel requires seqlen >= tile size
  size_t smem_bytes =
      (Br * (kHeadDim + kPadQ) +
       kStagesK * Bc * (kHeadDim + kPadK) +
       Bc * (kHeadDim + kPadV)) * sizeof(half);

  dim3 block(256);
  dim3 grid((seqlen + Br - 1) / Br, B * H);

  // Test both accumulator variants: f16 (kMmaAccF32=0) and f32 (kMmaAccF32=1)
  for (int acc = 0; acc <= 1; ++acc) {
    const int kMmaAcc = acc;
    half *h_o = (half *)malloc(sz);

    if (kMmaAcc == 0) {
      using FAKernel = void (*)(half *, half *, half *, half *, int, int);
      FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
          kMmaAtomK, 0, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
          kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
          kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
      cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
      fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
    } else {
      using FAKernel = void (*)(half *, half *, half *, half *, int, int);
      FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
          kMmaAtomK, 1, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
          kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
          kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
      cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
      fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
    }
    check(cudaGetLastError(), "fa launch");
    check(cudaDeviceSynchronize(), "fa sync");

    check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");

    float max_err = 0.0f;
    for (int i = 0; i < count; i++) {
      float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
      if (err > max_err) max_err = err;
```

```
    }
    const char *acc_label = kMmaAcc ? "F32Acc" : "F16Acc";
    char label[64];
    snprintf(label, sizeof(label), "FA2 (kStagesK=2, Pad, %s)", acc_label);
    printf("| %-56s | %.3e |\n", label,
           max_err);
    free(h_o);
  }

  free(h_q); free(h_k); free(h_v);
  free(ref_q); free(ref_k); free(ref_v); free(ref_o);
  cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
}


#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// Test for flash_attn_tma_mma_ws_stages_split_q (SM120, D=64/128)
// Reference: cuDNN SDPA (half) if available, else CPU FP32 (float) fallback.
template <int kHeadDim>
static void test_flash_attn_tma_mma_ws_impl(int seqlen, int head_dim) {
  int B = 1, H = 8;
  constexpr int kStagesK = 2;
  constexpr int kStagesV = 1;
  constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 8, kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;  // 128
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;  // 64
  constexpr int kNumThreads = 384;

  if (seqlen % Br != 0 || seqlen % Bc != 0 || seqlen < Br) {
    printf("| %-56s | %-9s |\n",
           "FA2 TMA MMA WS (1 Consumer WG) (unaligned)", "SKIP");
    return;
  }

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
  srand(42);
  half *h_q = (half *)malloc(sz);
  half *h_k = (half *)malloc(sz);
  half *h_v = (half *)malloc(sz);
  for (int i = 0; i < B * H * seqlen * head_dim; ++i) {
    h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

  half *d_q, *d_k, *d_v, *d_o;
  check(cudaMalloc(&d_q, sz), "fa_tma_ws alloc Q");
  check(cudaMalloc(&d_k, sz), "fa_tma_ws alloc K");
  check(cudaMalloc(&d_v, sz), "fa_tma_ws alloc V");
  check(cudaMalloc(&d_o, sz), "fa_tma_ws alloc O");
  check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D Q");
  check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D K");
  check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa_tma_ws H2D V");

  // Reference output: either cuDNN (half) or CPU (float)
  half *h_o_ref = nullptr;
  float *ref_o = nullptr;
  int count = B * H * seqlen * head_dim;
```

```
9511
9512 #if defined(NOTES_V2_ENABLE_CUDNN)
9513   {
9514     // Try cuDNN SDPA first; fall back to CPU if unsupported on this SM
9515     bool cudnn_ok = false;
9516     half *d_o_ref;
9517     check(cudaMalloc(&d_o_ref, sz), "fa_tma_ws alloc O_ref");
9518     {
9519       cudnnHandle_t cudnn_handle;
9520       cudnnCreate(&cudnn_handle);
9521       auto graph = std::make_shared<fe::graph::Graph>();
9522       graph->set_io_data_type(fe::DataType_t::HALF)
9523         .set_intermediate_data_type(fe::DataType_t::FLOAT)
9524         .set_compute_data_type(fe::DataType_t::FLOAT);
9525
9526       auto Q = graph->tensor(fe::graph::Tensor_attributes()
9527         .set_uid(1).set_dim({B, H, seqlen, head_dim})
9528         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
9529       auto K = graph->tensor(fe::graph::Tensor_attributes()
9530         .set_uid(2).set_dim({B, H, seqlen, head_dim})
9531         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
9532       auto V = graph->tensor(fe::graph::Tensor_attributes()
9533         .set_uid(3).set_dim({B, H, seqlen, head_dim})
9534         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
9535
9536       auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
9537         fe::graph::SDPA_attributes()
9538           .set_name("sdpa_ref")
9539           .set_attn_scale(1.0f / sqrtf((float)head_dim)));
9540
9541       O_sdpa->set_output(true).set_uid(4)
9542         .set_dim({B, H, seqlen, head_dim})
9543         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
9544
9545       auto build_status = graph->build(cudnn_handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
9546       if (build_status.is_good()) {
9547         std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
9548           {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
9549         int64_t ws_size = 0;
9550         if (graph->get_workspace_size(ws_size).is_good()) {
9551           int8_t *d_ws = nullptr;
9552           if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "fa_tma_ws workspace");
9553           if (graph->execute(cudnn_handle, vp, d_ws).is_good()) {
9554             check(cudaDeviceSynchronize(), "fa_tma_ws cudnn sync");
9555             cudnn_ok = true;
9556           }
9557           if (d_ws) cudaFree(d_ws);
9558         }
9559       }
9560       cudnnDestroy(cudnn_handle);
9561     }
9562
9563     if (cudnn_ok) {
9564       h_o_ref = (half *)malloc(sz);
9565       check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H ref");
9566     } else {
9567       fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
9568     }
9569     cudaFree(d_o_ref);
9570   }
9571 #endif
9572
9573   // CPU reference fallback
```

```
  if (!h_o_ref) {
    float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
    float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
    float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
    ref_o = (float *)malloc(sz * 4 / sizeof(half));
    for (int i = 0; i < count; ++i) {
      ref_q[i] = __half2float(h_q[i]);
      ref_k[i] = __half2float(h_k[i]);
      ref_v[i] = __half2float(h_v[i]);
    }
    float scale = 1.0f / sqrtf((float)head_dim);
    for (int bi = 0; bi < B * H; ++bi) {
      for (int qi = 0; qi < seqlen; ++qi) {
        float smax = -INFINITY;
        float *S = (float *)malloc((size_t)seqlen * sizeof(float));
        for (int kj = 0; kj < seqlen; ++kj) {
          float s = 0.0f;
          for (int d = 0; d < head_dim; ++d)
            s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
                 ref_k[bi * seqlen * head_dim + kj * head_dim + d];
          S[kj] = s * scale;
          if (S[kj] > smax) smax = S[kj];
        }
        double sum_exp = 0.0;
        for (int kj = 0; kj < seqlen; ++kj)
          sum_exp += (double)expf(S[kj] - smax);
        float inv_sum = 1.0f / (float)sum_exp;
        for (int d = 0; d < head_dim; ++d) {
          double o_acc = 0.0;
          for (int kj = 0; kj < seqlen; ++kj)
            o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
          ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
        }
        free(S);
      }
    }
    free(ref_q);
    free(ref_k);
    free(ref_v);
  }

  // TMA descriptors: box innermost 固定 64 half (128B)，满足
  // CU_TENSOR_MAP_SWIZZLE_128B 对 box innermost ≤ 128B 的硬约束。
  // D=64: blocks_width=1, 单次 TMA 覆盖整行。
  // D=128: blocks_width=2, kernel producer 沿 head_dim 连续发 2 次 TMA，
  //        minor_coord = 0/64，写入 chunk-major smem 布局 [2, Br, 64]。
  // Q/K/V gmem 是 [B, H, seqlen, head_dim] row-major，作为 2D
  // [B*H*seqlen, head_dim] 矩阵描述。blocks_height = B*H*seqlen/tile_major。
  // kernel 用 (Nb_id*H+Nh_id)*N 偏移 major_coord。
  constexpr int kTmaBoxMinor = 64;  // box innermost = 64 half = 128B
  constexpr int kTmaChunksQ  = kHeadDim / kTmaBoxMinor;
  constexpr int kTmaChunksKV = kHeadDim / kTmaBoxMinor;
  CUtensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
      d_q, B * H * seqlen / Br, kTmaChunksQ);
  CUtensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
      d_k, B * H * seqlen / Bc, kTmaChunksKV);
  CUtensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
      d_v, B * H * seqlen / Bc, kTmaChunksKV);

  using FAKernel = void (*)(half *, half *, half *, half *, int, int,
                            const CUtensorMap *, const CUtensorMap *,
                            const CUtensorMap *);
```

```
  FAKernel fa_k = flash_attn_tma_mma_ws_stages_split_q<
      kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
      kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
      kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
      kNumThreads>;

  // smem = Q[Br*d] + K[kStagesK*Bc*d] + V[kStagesV*Bc*d]
  size_t smem_bytes = (Br * kHeadDim + kStagesK * Bc * kHeadDim +
                       kStagesV * Bc * kHeadDim) * sizeof(half);
  int device = 0, max_smem = 0;
  cudaFuncAttributes attributes{};
  cudaGetDevice(&device);
  cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin,
                         device);
  cudaFuncGetAttributes(&attributes, fa_k);
  bool smem_ok =
      (smem_bytes + attributes.sharedSizeBytes <= (size_t)max_smem);
  if (!smem_ok) {
    if (g_debug)
      printf("| %-56s | %-9s |\n",
             "FA2 TMA MMA WS (1 Consumer WG) (SMEM SKIP)", "SMEM SKIP");
  } else {
    dim3 block(kNumThreads);
    dim3 grid((seqlen + Br - 1) / Br, B * H);
    // Test both accumulator variants
    for (int acc = 0; acc <= 1; ++acc) {
      const int kMmaAcc = acc;
      half *h_o = (half *)malloc(sz);

      if (kMmaAcc == 0) {
        using FAK = void (*)(half *, half *, half *, half *, int, int,
                             const CUtensorMap *, const CUtensorMap *,
                             const CUtensorMap *);
        FAK fk = flash_attn_tma_mma_ws_stages_split_q<
            kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
            kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
            kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK,
            kStagesV, kNumThreads>;
        cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             smem_bytes);
        fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
                                        tma_k, tma_v);
      } else {
        using FAK = void (*)(half *, half *, half *, half *, int, int,
                             const CUtensorMap *, const CUtensorMap *,
                             const CUtensorMap *);
        FAK fk = flash_attn_tma_mma_ws_stages_split_q<
            kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 1, kMmaTileSeqLenQ,
            kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
            kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK,
            kStagesV, kNumThreads>;
        cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             smem_bytes);
        fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
                                        tma_k, tma_v);
      }
      check(cudaGetLastError(), "fa_tma_ws launch");
      check(cudaDeviceSynchronize(), "fa_tma_ws sync");

      check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H");
      float max_err = 0.0f;
      bool checked = h_o_ref || ref_o;
      if (checked) {
```

```
        for (int i = 0; i < count; ++i) {
          float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
          float err = fabsf(__half2float(h_o[i]) - ref_val);
          if (err > max_err) max_err = err;
        }
      }
      const char *acc_label = kMmaAcc ? "F32Acc" : "F16Acc";
      char label[64];
      snprintf(label, sizeof(label),
               "FA2 TMA MMA WS (1 Consumer WG) (%s)",
               acc_label);
      printf("| %-56s | %.3e |\n",
             label, max_err);
      free(h_o);
    }
  }

  free(h_q); free(h_k); free(h_v);
  free(h_o_ref); free(ref_o);
  cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
  cudaFree(tma_q); cudaFree(tma_k); cudaFree(tma_v);
}

// D=64/128 dispatch wrapper
static void test_flash_attn_tma_mma_ws(int seqlen, int head_dim) {
  if (head_dim == 64) {
    test_flash_attn_tma_mma_ws_impl<64>(seqlen, head_dim);
  } else if (head_dim == 128) {
    test_flash_attn_tma_mma_ws_impl<128>(seqlen, head_dim);
  } else {
    printf("| %-56s | %-9s |\n",
           "FA2 TMA MMA WS (1 Consumer WG) (D!=64/128)", "SKIP");
  }
}

// FA3-style dual-consumer correctness test (Br=64, Sk=Sv=2)
// Tests both F16Acc and F32Acc against cuDNN SDPA reference.
#if defined(NOTES_V2_ENABLE_CUDNN)
namespace fe = cudnn_frontend;
static float bench_cudnn_sdpa_tflops(half *d_q, half *d_k, half *d_v,
                                     half *d_o_ref, int B, int H, int seqlen,
                                     int head_dim, fe::DataType_t compute_type);
#endif
template <int kHeadDim, int kStagesK>
static void test_flash_attn_3_tma_ws_impl(int seqlen, int head_dim) {
  int B = 1, H = 8;
  constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 4, kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 4, kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
  constexpr int kStagesV = 1;  // per-WG V: always 1 buffer
  constexpr int kNumConsumerWGs = 2;
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;  // 64
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
  constexpr int kNumThreads = 384;

  if (seqlen % Br != 0 || seqlen % Bc != 0 || seqlen < Br) {
    printf("| %-56s | %-9s |\n",
           "FA3 TMA MMA WS (2 Consumer WG) (unaligned)", "SKIP");
    return;
  }
```

```
9763
9764    size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
9765    srand(42);
9766    half *h_q = (half *)malloc(sz);
9767    half *h_k = (half *)malloc(sz);
9768    half *h_v = (half *)malloc(sz);
9769    for (int i = 0; i < B * H * seqlen * head_dim; ++i) {
9770      h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9771      h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9772      h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
9773    }
9774
9775    half *d_q, *d_k, *d_v, *d_o;
9776    check(cudaMalloc(&d_q, sz), "fa3 alloc Q");
9777    check(cudaMalloc(&d_k, sz), "fa3 alloc K");
9778    check(cudaMalloc(&d_v, sz), "fa3 alloc V");
9779    check(cudaMalloc(&d_o, sz), "fa3 alloc O");
9780    check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa3 H2D Q");
9781    check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa3 H2D K");
9782    check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa3 H2D V");
9783
9784    // Reference: cuDNN SDPA (half output)
9785    half *h_o_ref = nullptr;
9786    int count = B * H * seqlen * head_dim;
9787  #if defined(NOTES_V2_ENABLE_CUDNN)
9788    {
9789      half *d_o_ref;
9790      check(cudaMalloc(&d_o_ref, sz), "fa3 alloc O_ref");
9791      bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref, B, H, seqlen, head_dim,
9792                              fe::DataType_t::FLOAT);
9793      h_o_ref = (half *)malloc(sz);
9794      check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "fa3 ref D2H");
9795      cudaFree(d_o_ref);
9796    }
9797  #endif
9798
9799    // TMA descriptors (Br=64 for Q, Bc=64 for K/V)
9800    constexpr int kTmaBoxMinor = 64;
9801    constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
9802    CUtensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
9803        d_q, B * H * seqlen / Br, kTmaChunks);
9804    CUtensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
9805        d_k, B * H * seqlen / Bc, kTmaChunks);
9806    CUtensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
9807        d_v, B * H * seqlen / Bc, kTmaChunks);
9808
9809    using FAKernel = void (*)(half *, half *, half *, half *, int, int,
9810                              const CUtensorMap *, const CUtensorMap *,
9811                              const CUtensorMap *);
9812
9813    // smem = Q + K[2WGs * Sk * Bc*D] + V[2WGs * 1 * Bc*D]
9814    size_t smem_bytes = (Br * kHeadDim +
9815                         kNumConsumerWGs * kStagesK * Bc * kHeadDim +
9816                         kNumConsumerWGs * kStagesV * Bc * kHeadDim) * sizeof(half);
9817    dim3 block(kNumThreads);
9818    dim3 grid((seqlen + Br - 1) / Br, B * H);
9819
9820    for (int acc = 0; acc <= 1; ++acc) {
9821      half *h_o = (half *)malloc(sz);
9822      FAKernel fk;
9823      if (acc == 0) {
9824        fk = flash_attn_3_tma_ws_stages_split_q<
9825            kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 0, kMmaTileSeqLenQ,
```

```
          kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
          kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
          kNumThreads>;
    } else {
      fk = flash_attn_3_tma_ws_stages_split_q<
          kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, 1, kMmaTileSeqLenQ,
          kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
          kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
          kNumThreads>;
    }
    bool smem_ok = check_smem_feasible((const void *)fk, smem_bytes);
    if (!smem_ok) {
      const char *acc_label = acc ? "F32Acc" : "F16Acc";
      char label[64];
      snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (%s)",
               acc_label);
      if (g_debug)
        printf("| %-56s | %-9s |\n", label, "SMEM too large");
      free(h_o);
      continue;
    }
    cudaFuncSetAttribute(fk, cudaFuncAttributeMaxDynamicSharedMemorySize,
                         smem_bytes);
    fk<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H, tma_q,
                                    tma_k, tma_v);
    check(cudaGetLastError(), "fa3 launch");
    check(cudaDeviceSynchronize(), "fa3 sync");

    check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa3 D2H");
    float max_err = 0.0f;
    bool checked = h_o_ref != nullptr;
    if (checked) {
      for (int i = 0; i < count; ++i) {
        float err = fabsf(__half2float(h_o[i]) - __half2float(h_o_ref[i]));
        if (err > max_err) max_err = err;
      }
    }
    const char *acc_label = acc ? "F32Acc" : "F16Acc";
    char label[64];
    snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (%s)",
             acc_label);
    printf("| %-56s | %.3e |\n", label, max_err);
    free(h_o);
  }

  free(h_q); free(h_k); free(h_v); free(h_o_ref);
  cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
  cudaFree(tma_q); cudaFree(tma_k); cudaFree(tma_v);
}

static void test_flash_attn_3_tma_ws(int seqlen, int head_dim) {
  // Try stages up to 4; test_flash_attn_3_tma_ws_impl internally checks
  // cudaDevAttrMaxSharedMemoryPerBlockOptin and skips oversized configs.
  if (head_dim == 64) {
    test_flash_attn_3_tma_ws_impl<64, 1>(seqlen, head_dim);
    test_flash_attn_3_tma_ws_impl<64, 2>(seqlen, head_dim);
    test_flash_attn_3_tma_ws_impl<64, 3>(seqlen, head_dim);
    test_flash_attn_3_tma_ws_impl<64, 4>(seqlen, head_dim);
  } else if (head_dim == 128) {
    test_flash_attn_3_tma_ws_impl<128, 1>(seqlen, head_dim);
    test_flash_attn_3_tma_ws_impl<128, 2>(seqlen, head_dim);
    test_flash_attn_3_tma_ws_impl<128, 3>(seqlen, head_dim);
    test_flash_attn_3_tma_ws_impl<128, 4>(seqlen, head_dim);
```

```
  } else {
    printf("| %-56s | %-9s |\n",
           "FA3 TMA MMA WS (2 Consumer WG) (D!=64/128)", "SKIP");
  }
}
#endif /* NOTES_V2_ENABLE_TMA_MMA_WS */

static float bench_hgemm_tflops(int M, int N, int K, float time_ms) {
  double flops = 2.0 * M * N * K;
  return (float)(flops / (double)time_ms / 1e9);
}

static float bench_fa_tflops(int B, int H, int N, int D, float time_ms) {
  double flops = 4.0 * B * H * N * N * D;
  return (float)(flops / (double)time_ms / 1e9);
}

static float bench_cublas_hgemm_tflops(cublasHandle_t handle, int M, int N, int K,
                                       half *d_a, half *d_b, half *d_c) {
  half alpha = __float2half(1.0f), beta = __float2half(0.0f);
  cudaStream_t stream;
  cudaStreamCreate(&stream);
  cublasSetStream(handle, stream);
  // warmup
  for (int w = 0; w < g_warmup; ++w)
    cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
                 d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta,
                 d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  cudaStreamSynchronize(stream);
  // timed repeat
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  cudaEventRecord(start, stream);
  for (int r = 0; r < g_repeat; ++r)
    cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
                 d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta,
                 d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
  cudaEventRecord(stop, stream);
  cudaEventSynchronize(stop);
  float time_ms = 0;
  cudaEventElapsedTime(&time_ms, start, stop);
  time_ms /= g_repeat;
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cublasSetStream(handle, nullptr);
  cudaStreamDestroy(stream);
  return bench_hgemm_tflops(M, N, K, time_ms);
}

// ============================================================================
// Bench: HGEMM MMA (basic m16n8k16 + multistage pipeline)
// ============================================================================
template <int kStages, int kBlockSwizzle>
static bool launch_timed_hgemm_mma(
  half *d_a, half *d_b_t, half *d_c, half *h_c,
  half *h_c_ref, int M, int N, int K, size_t size_c,
  cudaEvent_t start, cudaEvent_t stop, float &max_err,
  float &time_ms
) {
  constexpr int BM = 128, BN = 128, BK = 16;
  using Kernel = void (*)(half *, half *, half *, int, int, int);
  Kernel k = hgemm_mma_stages_tn<16, 8, 16, 2, 4, 4, 4, kStages, kBlockSwizzle>;
```

```
  size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
  if (!check_smem_feasible((const void *)k, smem)) return false;
  dim3 block(256);
  const int tiles_n = (N + BN - 1) / BN;
  constexpr int kSwizzleN = 16;
  int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
  int gz = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(gx, (M + BM - 1) / BM, gz);
  for (int w = 0; w < g_warmup; w++)
    k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
  cudaDeviceSynchronize();
  cudaEventRecord(start);
  for (int r = 0; r < g_repeat; r++)
    k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  time_ms /= g_repeat;
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  return true;
}

static void bench_hgemm_mma(int M, int N, int K) {
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench mma alloc A");
  check(cudaMalloc(&d_b, size_b), "bench mma alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench mma alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench mma alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench mma H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench mma H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench mma H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H ref");
  half *h_c = (half *)malloc(size_c);
  float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
```

```
  cudaEventCreate(&stop);
  for (int stages : {2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
      bool ok = false;
      if (stages == 2)
        ok = swizzle ? launch_timed_hgemm_mma<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_mma<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms);
      else
        ok = swizzle ? launch_timed_hgemm_mma<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_mma<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
            K, size_c, start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM MMA (S=%d, BLK_SW=%d)", stages, swizzle);
      if (!ok) {
        if (g_debug)
          printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        char tflops_str[32];
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)",
                 tflops, cublas_tflops, tflops / cublas_tflops);
        printf("| %-56s | %.3e | %-19s |\n", label, max_err,
        tflops_str);
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cublasDestroy(handle);
}

// ============================================================================
// Bench: HGEMM MMA Swizzle + Register Double Buffering (kValTileK=4, BK=64)
// ============================================================================
template <int kStages, int kBlockSwizzle>
static bool launch_timed_hgemm_swizzle(
  half *d_a, half *d_b_t, half *d_c, half *h_c,
  half *h_c_ref, int M, int N, int K, size_t size_c,
  cudaEvent_t start, cudaEvent_t stop, float &max_err,
  float &time_ms
) {
  constexpr int BM = 128, BN = 128, BK = 64;
  using Kernel = void (*)(half *, half *, half *, int, int, int);
  Kernel k = hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, kStages, kBlockSwizzle>;
  size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
  if (!check_smem_feasible((const void *)k, smem)) return false;
  cudaFuncSetAttribute((const void *)k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
  dim3 block(256);
  const int tiles_n = (N + BN - 1) / BN;
  constexpr int kSwizzleN = 16;
```

```
  int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
  int gz = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(gx, (M + BM - 1) / BM, gz);
  for (int w = 0; w < g_warmup; w++)
    k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
  cudaDeviceSynchronize();
  cudaEventRecord(start);
  for (int r = 0; r < g_repeat; r++)
    k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  time_ms /= g_repeat;
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  return true;
}

static void bench_hgemm_swizzle(int M, int N, int K) {
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench swizzle alloc A");
  check(cudaMalloc(&d_b, size_b), "bench swizzle alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench swizzle alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench swizzle alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench swizzle H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench swizzle H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench swizzle H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H ref");
  half *h_c = (half *)malloc(size_c);
  float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
      bool ok = false;
```

```
      if (stages == 2)
        ok = swizzle ? launch_timed_hgemm_swizzle<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_swizzle<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms);
      else
        ok = swizzle ? launch_timed_hgemm_swizzle<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_swizzle<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
            N, K, size_c, start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM Swizzle+Reg2x (S=%d, BLK_SW=%d)", stages, swizzle);
      if (!ok) {
        if (g_debug)
          printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        char tflops_str[32];
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)",
                 tflops, cublas_tflops, tflops / cublas_tflops);
        printf("| %-56s | %.3e | %-19s |\n", label, max_err,
        tflops_str);
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cublasDestroy(handle);
}

#if defined(NOTES_V2_ENABLE_CUTE)
// ============================================================================
// Bench: HGEMM CuTe
// ============================================================================
static void bench_hgemm_cute(int M, int N, int K) {
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench cute alloc A");
  check(cudaMalloc(&d_b, size_b), "bench cute alloc B");
```

```
  check(cudaMalloc(&d_b_t, size_b_t), "bench cute alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench cute alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench cute H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench cute H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench cute H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H ref");
  half *h_c = (half *)malloc(size_c);
  float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {2, 3}) {
    for (int swizzle : {0, 1}) {
      char label[64];
      snprintf(label, sizeof(label), "HGEMM CuTe Swizzle (S=%d, BLK_SW=%d)", stages, swizzle);
      bool ok = false;
      float time_ms = 0, max_err = 0;
      if (stages == 2) {
        auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 2, 1>
                         : launch_hgemm_mma_stages_tn_cute<half, 2, 0>;
        for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
        cudaDeviceSynchronize();
        cudaEventRecord(start);
        for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&time_ms, start, stop);
        time_ms /= g_repeat;
        ok = true;
      } else {
        auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 3, 1>
                         : launch_hgemm_mma_stages_tn_cute<half, 3, 0>;
        for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
        cudaDeviceSynchronize();
        cudaEventRecord(start);
        for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&time_ms, start, stop);
        time_ms /= g_repeat;
        ok = true;
      }
      if (ok) {
        check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H");
        max_err = 0.0f;
        for (int i = 0; i < M * N; i++) {
          float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
          if (err > max_err) max_err = err;
        }
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        char tflops_str[32];
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)",
                 tflops, cublas_tflops, tflops / cublas_tflops);
        printf("| %-56s | %.3e | %-19s |\n", label, max_err,
        tflops_str);
      } else {
        printf("| %-56s | %-9s | %-19s |\n", label, "LAUNCH ERR", "None");
```

```
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cublasDestroy(handle);
}
#endif

#if defined(NOTES_V2_ENABLE_WGMMA)
// ================================================================================
// Bench: HGEMM WGMMA (m64n128k16 + TMA + Warp Specialization, SM90+)
// ================================================================================
template <int kStages, int kBlockSwizzle>
static bool launch_timed_hgemm_wgmma(half *d_c, half *h_c, half *h_c_ref,
            CUtensorMap *tma_a, CUtensorMap *tma_b,
            int M, int N, int K, size_t size_c,
            cudaEvent_t start, cudaEvent_t stop,
            float &max_err, float &time_ms) {
  constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
  using Kernel = void (*)(int, int, int, half *, const CUtensorMap *,
          const CUtensorMap *);
  Kernel k = hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
            kBlockSwizzle>;
  size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
  if (!check_smem_feasible((const void *)k, smem)) return false;
  cudaFuncSetAttribute(k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
  dim3 block(kNumThreads);
  const int tiles_n = (N + BN - 1) / BN;
  constexpr int kSwizzleN = 16;
  int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
  int gz = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(gx, (M + BM - 1) / BM, gz);
  for (int w = 0; w < g_warmup; w++)
    k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
  cudaDeviceSynchronize();
  cudaEventRecord(start);
  for (int r = 0; r < g_repeat; r++)
    k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  time_ms /= g_repeat;
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; i++) {
    float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
    if (err > max_err) max_err = err;
  }
  return true;
}

static void bench_hgemm_wgmma(int M, int N, int K) {
  constexpr int BM = 128, BN = 128, BK = 64;
```

```
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-56s | %-9s | %-19s |\n", "HGEMM WGMMA (unaligned)", "SKIP", "None");
    return;
  }
  size_t size_a = (size_t)M * K * sizeof(half);
  size_t size_b = (size_t)K * N * sizeof(half);
  size_t size_c = (size_t)M * N * sizeof(half);
  half *h_a = (half *)malloc(size_a);
  half *h_b = (half *)malloc(size_b);
  half *h_c_ref = (half *)malloc(size_c);
  srand(42);
  for (int i = 0; i < M * K; i++)
    h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; i++)
    h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  size_t size_b_t = (size_t)N * K * sizeof(half);
  half *h_b_t = (half *)malloc(size_b_t);
  for (int n = 0; n < N; n++)
    for (int k = 0; k < K; k++)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench wgmma alloc A");
  check(cudaMalloc(&d_b, size_b), "bench wgmma alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench wgmma alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench wgmma alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench wgmma H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench wgmma H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench wgmma H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H ref");
  CUtensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
  half *h_c = (half *)malloc(size_c);
  float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {1, 2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
      bool ok = false;
      if (stages == 2)
        ok = swizzle
          ? launch_timed_hgemm_wgmma<2, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_wgmma<2, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
            size_c, start, stop, max_err, time_ms);
      else
        ok = swizzle
          ? launch_timed_hgemm_wgmma<3, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
            K, size_c, start, stop, max_err, time_ms)
          : launch_timed_hgemm_wgmma<3, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
            size_c, start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM TMA WGMMA WS (S=%d, BLK_SW=%d)", stages, swizzle);
      if (!ok) {
        if (g_debug)
          printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
```

```
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        char tflops_str[32];
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops,
                 cublas_tflops, tflops / cublas_tflops);
        printf("| %-56s | %.3e | %-19s |\n", label, max_err,
        tflops_str);
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cudaFree(tma_a);
  cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// ============================================================================
// Bench: HGEMM TMA MMA WS (mma.sync + TMA + Warp Specialization, SM120)
// ============================================================================
template <int kStages, int kBlockSwizzle>
static bool bench_launch_tma_mma_ws(
  int M, int N, int K, half *d_a, half *d_b_t,
  half *d_c, half *h_c, half *h_c_ref, size_t size_c,
  CUtensorMap *tma_a, CUtensorMap *tma_b,
  cudaEvent_t start, cudaEvent_t stop,
  float &max_err, float &time_ms) {
  constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
  constexpr size_t payload_bytes = kStages * (BM * BK + BN * BK) * sizeof(half);
  using Kernel = void (*)(int, int, int, half *, const CUtensorMap *,
          const CUtensorMap *);
  Kernel kernel = hgemm_tma_mma_ws_tn<16, 8, 16, 2, 2, 4, 8, 4, kStages,
            kNumThreads, kBlockSwizzle>;
  if (!check_smem_feasible((const void *)kernel, payload_bytes)) return false;
  cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
          payload_bytes);
  dim3 block(kNumThreads);
  constexpr int kSwizzleN = 16;
  const int n_tiles = N / BN;
  const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
  const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
  dim3 grid(grid_x, M / BM, grid_z);
  for (int w = 0; w < g_warmup; w++)
    kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
  cudaDeviceSynchronize();
  cudaEventRecord(start);
  for (int r = 0; r < g_repeat; r++)
    kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
  cudaEventRecord(stop);
  cudaEventSynchronize(stop);
  cudaEventElapsedTime(&time_ms, start, stop);
  time_ms /= g_repeat;
```

```
  check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench tma mma ws D2H");
  max_err = 0.0f;
  for (int i = 0; i < M * N; ++i)
    max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i])));
  return true;
}

static void bench_hgemm_tma_mma_ws(int M, int N, int K) {
  constexpr int BM = 128, BN = 128, BK = 64;
  if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
    printf("| %-56s | %-9s | %-19s |\n", "HGEMM TMA MMA WS (unaligned)", "SKIP", "None");
    return;
  }
  const size_t size_a = size_t(M) * K * sizeof(half);
  const size_t size_b = size_t(K) * N * sizeof(half);
  const size_t size_b_t = size_t(N) * K * sizeof(half);
  const size_t size_c = size_t(M) * N * sizeof(half);
  half *h_a = static_cast<half *>(malloc(size_a));
  half *h_b = static_cast<half *>(malloc(size_b));
  half *h_b_t = static_cast<half *>(malloc(size_b_t));
  half *h_c = static_cast<half *>(malloc(size_c));
  half *h_c_ref = static_cast<half *>(malloc(size_c));
  srand(42);
  for (int i = 0; i < M * K; ++i)
    h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
  for (int i = 0; i < K * N; ++i)
    h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
  for (int n = 0; n < N; ++n)
    for (int k = 0; k < K; ++k)
      h_b_t[n * K + k] = h_b[k * N + n];
  half *d_a, *d_b, *d_b_t, *d_c;
  check(cudaMalloc(&d_a, size_a), "bench tma mma ws alloc A");
  check(cudaMalloc(&d_b, size_b), "bench tma mma ws alloc B");
  check(cudaMalloc(&d_b_t, size_b_t), "bench tma mma ws alloc B_t");
  check(cudaMalloc(&d_c, size_c), "bench tma mma ws alloc C");
  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench tma mma ws H2D A");
  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench tma mma ws H2D B");
  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
     "bench tma mma ws H2D B_t");
  cublasHandle_t handle;
  cublasCreate(&handle);
  half alpha = __float2half(1.0f), beta = __float2half(0.0f);
  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b, CUDA_R_16F, N,
        d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
        CUBLAS_GEMM_DEFAULT);
  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
     "bench tma mma ws D2H reference");
  CUtensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
  CUtensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
  float cublas_tflops = bench_cublas_hgemm_tflops(handle, M, N, K, d_a, d_b, d_c);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);
  for (int stages : {1, 2, 3}) {
    for (int swizzle : {0, 1}) {
      float max_err = 0, time_ms = 0;
      bool ok = false;
      if (stages == 2)
        ok = swizzle ? bench_launch_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
            start, stop, max_err, time_ms)
          : bench_launch_tma_mma_ws<2, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
```

```
            start, stop, max_err, time_ms);
      else
        ok = swizzle ? bench_launch_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
            start, stop, max_err, time_ms)
          : bench_launch_tma_mma_ws<3, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
            h_c_ref, size_c, tma_a, tma_b,
            start, stop, max_err, time_ms);
      char label[64];
      snprintf(label, sizeof(label), "HGEMM TMA MMA WS (S=%d, BLK_SW=%d)", stages, swizzle);
      if (!ok) {
        if (g_debug)
          printf("| %-56s | %-9s | %-19s |\n", label, "SMEM SKIP", "None");
      } else {
        float tflops = bench_hgemm_tflops(M, N, K, time_ms);
        char tflops_str[32];
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops,
                 cublas_tflops, tflops / cublas_tflops);
        printf("| %-56s | %.3e | %-19s |\n", label, max_err,
        tflops_str);
      }
    }
  }
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  free(h_a);
  free(h_b);
  free(h_b_t);
  free(h_c);
  free(h_c_ref);
  cudaFree(d_a);
  cudaFree(d_b);
  cudaFree(d_b_t);
  cudaFree(d_c);
  cudaFree(tma_a);
  cudaFree(tma_b);
  cublasDestroy(handle);
}
#endif

// ============================================================================
// Bench: FlashAttention-2 Split-Q (template on kHeadDim for dispatch)
// ============================================================================
template <int kHeadDim, int kStagesK = 2, int kPadQ = 8, int kPadK = 8,
          int kPadV = 8, int kMmaAccF32 = 0>
static void bench_fa_launch(int B, int H, int seqlen, int head_dim,
    half *h_o_ref, float *ref_o, half *d_q, half *d_k, half *d_v, half *d_o,
    float cudnn_tflops_f16) {
  static_assert(kHeadDim == 64 || kHeadDim == 128, "Only D=64 and D=128 are supported");
  constexpr bool kSwizzleQ = kPadQ == 0;
  constexpr bool kSwizzleK = kPadK == 0;
  constexpr bool kSwizzleV = kPadV == 0;
  constexpr int kMmaAtomM = 16;
  constexpr int kMmaAtomN = 8;
  constexpr int kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 8;
  constexpr int kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 8;
  constexpr int kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1;
  constexpr int kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
```

```
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
  constexpr const char *layout_name =
      kSwizzleQ && kSwizzleK && kSwizzleV ? "Swizzle" :
      kSwizzleQ && kSwizzleK ? "SwizzleQK" :
      kSwizzleQ && kSwizzleV ? "SwizzleQV" :
      kSwizzleK && kSwizzleV ? "SwizzleKV" :
      kSwizzleQ ? "SwizzleQ" :
      kSwizzleK ? "SwizzleK" :
      kSwizzleV ? "SwizzleV" : "Pad";

  // Kernel requires seqlen >= Br (tile size); skip gracefully for short seqlen
  if (seqlen < Br) {
    char label[64];
    snprintf(label, sizeof(label), "FA2 (%s)",
             layout_name);
    printf("| %-56s | %-9s | %-19s |\n", label,
           "seqlen<Br", "None");
    return;
  }
  size_t smem_bytes =
      (Br * (kHeadDim + kPadQ) +
       kStagesK * Bc * (kHeadDim + kPadK) +
       Bc * (kHeadDim + kPadV)) * sizeof(half);

  dim3 block(256);
  dim3 grid((seqlen + Br - 1) / Br, B * H);

  cudaStream_t timing_stream;
  cudaStreamCreate(&timing_stream);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  using FAKernel = void (*)(half *, half *, half *, half *, int, int);
  FAKernel fa_k = flash_attn_mma_stages_split_q<
    kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32,
    kMmaTileSeqLenQ, kMmaTileSeqLenK,
    kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
    kValTileSeqLenP, kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
  bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
  if (smem_ok) {
    cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
  }

  // Warmup
  if (smem_ok) {
    for (int w = 0; w < g_warmup; w++)
      fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
    check(cudaStreamSynchronize(timing_stream), "fa warmup sync");
  }

  // Timed repeat
  cudaEventRecord(start, timing_stream);
  if (smem_ok) {
    for (int r = 0; r < g_repeat; r++)
      fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
  }
  cudaEventRecord(stop, timing_stream);
  cudaEventSynchronize(stop);

  float time_ms = 0;
  cudaEventElapsedTime(&time_ms, start, stop);
```

```
  time_ms /= g_repeat;

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
  half *h_o = (half *)malloc(sz);
  check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "bench fa D2H");

  int count = B * H * seqlen * head_dim;
  char label[64];
  snprintf(label, sizeof(label), "FA2 MMA Stages (Sk=%d, %s, %s)", kStagesK,
           layout_name, kMmaAccF32 ? "F32Acc" : "F16Acc");
  float max_err = 0.0f;
  bool checked = h_o_ref || ref_o;
  if (smem_ok && checked) {
    for (int i = 0; i < count; i++) {
      float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
      float err = fabsf(__half2float(h_o[i]) - ref_val);
      if (err > max_err) max_err = err;
    }
  }
  if (smem_ok && checked) {
    float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
    bool is_fail = max_err >= 5e-1f;
    if (is_fail || should_print_fa_tflops(kMmaAccF32, tflops)) {
      char tflops_str[32];
      if (cudnn_tflops_f16 > 0)
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)",
                 tflops, cudnn_tflops_f16, tflops / cudnn_tflops_f16);
      else
        snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
      printf("| %-56s | %.3e | %-19s |\n", label, max_err,
             tflops_str);
    }
  } else if (smem_ok) {
    float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
    if (should_print_fa_tflops(kMmaAccF32, tflops)) {
      char tflops_str[32];
      snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
      printf("| %-56s | %-9s | %-19s |\n", label, "unchecked", tflops_str);
    }
  } else {
    if (g_debug)
      printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large", "None");
  }

  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cudaStreamDestroy(timing_stream);
  free(h_o);
}

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// Bench for flash_attn_tma_mma_ws_stages_split_q (SM120, D=64/128)
template <int kHeadDim, int kStagesK, int kStagesV = 1, int kMmaAccF32 = 0>
static void bench_fa_tma_mma_ws_launch(int B, int H, int seqlen, int head_dim,
                                       half *h_o_ref, float *ref_o,
                                       half *d_q, half *d_k, half *d_v,
                                       half *d_o, float cudnn_tflops_f16) {
  constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 8, kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
```

```
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
  constexpr int kNumThreads = 384;

  if (seqlen < Br || seqlen % Br != 0 || seqlen % Bc != 0) {
    char label[64];
    snprintf(label, sizeof(label),
             "FA2 TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, unaligned)", kStagesK, kStagesV);
    printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
    return;
  }

  // TMA descriptors: box innermost 固定 64 half (128B), 满足
  // CU_TENSOR_MAP_SWIZZLE_128B 对 box innermost ≤ 128B 的硬约束。
  // D=64: blocks_width=1, 单次 TMA 覆盖整行。
  // D=128: blocks_width=2, kernel producer 沿 head_dim 连续发 2 次 TMA,
  //        minor_coord = 0/64, 写入 chunk-major smem 布局 [2, Br, 64]。
  // Q/K/V gmem 是 [B, H, seqlen, head_dim] row-major, 作为 2D
  // [B*H*seqlen, head_dim] 矩阵描述。blocks_height = B*H*seqlen/tile_major。
  // kernel 用 (Nb_id*H+Nh_id)*N 偏移 major_coord。
  constexpr int kTmaBoxMinor = 64;  // box innermost = 64 half = 128B
  constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
  CUtensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
      d_q, B * H * seqlen / Br, kTmaChunks);
  CUtensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
      d_k, B * H * seqlen / Bc, kTmaChunks);
  CUtensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
      d_v, B * H * seqlen / Bc, kTmaChunks);

  using FAKernel = void (*)(half *, half *, half *, half *, int, int,
                            const CUtensorMap *, const CUtensorMap *,
                            const CUtensorMap *);
  FAKernel fa_k = flash_attn_tma_mma_ws_stages_split_q<
      kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32, kMmaTileSeqLenQ,
      kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
      kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
      kNumThreads>;

  size_t smem_bytes = (Br * kHeadDim + kStagesK * Bc * kHeadDim +
                       kStagesV * Bc * kHeadDim) * sizeof(half);
  bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
  if (smem_ok) {
    cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize,
                         smem_bytes);
  }

  dim3 block(kNumThreads);
  dim3 grid((seqlen + Br - 1) / Br, B * H);

  cudaStream_t timing_stream;
  cudaStreamCreate(&timing_stream);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  if (smem_ok) {
    for (int w = 0; w < g_warmup; ++w)
      fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
                                                       seqlen, H, tma_q, tma_k,
                                                       tma_v);
    check(cudaStreamSynchronize(timing_stream), "fa_tma_ws warmup sync");
  }
  cudaEventRecord(start, timing_stream);
```

```
  if (smem_ok) {
    for (int r = 0; r < g_repeat; ++r)
      fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
                                                       seqlen, H, tma_q, tma_k,
                                                       tma_v);
  }
  cudaEventRecord(stop, timing_stream);
  cudaEventSynchronize(stop);

  float time_ms = 0;
  cudaEventElapsedTime(&time_ms, start, stop);
  time_ms /= g_repeat;

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
  half *h_o = (half *)malloc(sz);
  check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa_tma_ws D2H");
  int count = B * H * seqlen * head_dim;

  char label[64];
  snprintf(label, sizeof(label), "FA2 TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, %s)",
           kStagesK, kStagesV, kMmaAccF32 ? "F32Acc" : "F16Acc");
  float max_err = 0.0f;
  bool checked = h_o_ref || ref_o;
  if (smem_ok && checked) {
    for (int i = 0; i < count; ++i) {
      float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
      float err = fabsf(__half2float(h_o[i]) - ref_val);
      if (err > max_err) max_err = err;
    }
  }
  if (smem_ok && checked) {
    float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
    bool is_fail = max_err >= 5e-1f;
    if (is_fail || should_print_fa_tflops(kMmaAccF32, tflops)) {
      char tflops_str[32];
      if (cudnn_tflops_f16 > 0)
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)",
                 tflops, cudnn_tflops_f16, tflops / cudnn_tflops_f16);
      else
        snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
      printf("| %-56s | %.3e | %-19s |\n", label, max_err,
             tflops_str);
    }
  } else if (smem_ok) {
    float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
    if (should_print_fa_tflops(kMmaAccF32, tflops)) {
      char tflops_str[32];
      snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
      printf("| %-56s | %-9s | %-19s |\n", label, "unchecked",
             tflops_str);
    }
  } else {
    if (g_debug)
      printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large",
             "None");
  }

  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cudaStreamDestroy(timing_stream);
  free(h_o);
  cudaFree(tma_q);
  cudaFree(tma_k);
```

```
  cudaFree(tma_v);
}

// D=64/128 dispatch wrapper for bench
template <int kStagesK, int kStagesV = 1, int kMmaAccF32 = 0>
static void bench_fa_tma_mma_ws_dispatch(int B, int H, int seqlen, int head_dim,
                                         half *h_o_ref, float *ref_o,
                                         half *d_q, half *d_k, half *d_v,
                                         half *d_o, float cudnn_tflops_f16) {
  if (head_dim == 64) {
    bench_fa_tma_mma_ws_launch<64, kStagesK, kStagesV, kMmaAccF32>(
      B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
  } else if (head_dim == 128) {
    bench_fa_tma_mma_ws_launch<128, kStagesK, kStagesV, kMmaAccF32>(
      B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
  } else {
    char label[64];
    snprintf(label, sizeof(label),
             "FA2 TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, D!=64/128)", kStagesK, kStagesV);
    printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
  }
}

// FA3-style dual-consumer bench (Br=64, per-WG independent K pipeline, Sv=1)
template <int kHeadDim, int kStagesK, int kMmaAccF32 = 0>
static void bench_fa_3_tma_ws_launch(int B, int H, int seqlen, int head_dim,
                                     half *h_o_ref, float *ref_o,
                                     half *d_q, half *d_k, half *d_v,
                                     half *d_o, float cudnn_tflops_f16) {
  constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
  constexpr int kMmaTileSeqLenQ = 4, kMmaTileSeqLenK = 1;
  constexpr int kMmaTileSeqLenP = 4, kMmaTileHeadDimV = 1;
  constexpr int kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
  constexpr int kValTileSeqLenP = 1;
  constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
  constexpr int kStagesV = 1;  // per-WG V: always 1 buffer
  constexpr int kNumConsumerWGs = 2;
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;  // 64
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;  // 64
  constexpr int kNumThreads = 384;

  if (seqlen < Br || seqlen % Br != 0 || seqlen % Bc != 0) {
    char label[80];
    snprintf(label, sizeof(label),
             "FA3 TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, unaligned)", kStagesK);
    printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
    return;
  }

  constexpr int kTmaBoxMinor = 64;
  constexpr int kTmaChunks = kHeadDim / kTmaBoxMinor;
  CUtensorMap *tma_q = allocate_and_create_tensor_map<Br, kTmaBoxMinor>(
      d_q, B * H * seqlen / Br, kTmaChunks);
  CUtensorMap *tma_k = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
      d_k, B * H * seqlen / Bc, kTmaChunks);
  CUtensorMap *tma_v = allocate_and_create_tensor_map<Bc, kTmaBoxMinor>(
      d_v, B * H * seqlen / Bc, kTmaChunks);

  using FAKernel = void (*)(half *, half *, half *, half *, int, int,
                            const CUtensorMap *, const CUtensorMap *,
                            const CUtensorMap *);
  FAKernel fa_k = flash_attn_3_tma_ws_stages_split_q<
      kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaAccF32, kMmaTileSeqLenQ,
```

```
      kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ,
      kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV, kStagesK, kStagesV,
      kNumThreads>;

  // smem = Q + K[2WGs * Sk * Bc*D] + V[2WGs * 1 * Bc*D]
  size_t smem_bytes = (Br * kHeadDim +
                       kNumConsumerWGs * kStagesK * Bc * kHeadDim +
                       kNumConsumerWGs * kStagesV * Bc * kHeadDim) * sizeof(half);
  bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
  if (smem_ok) {
    cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize,
                         smem_bytes);
  }

  dim3 block(kNumThreads);
  dim3 grid((seqlen + Br - 1) / Br, B * H);

  cudaStream_t timing_stream;
  cudaStreamCreate(&timing_stream);
  cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

  if (smem_ok) {
    for (int w = 0; w < g_warmup; ++w)
      fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
                                                       seqlen, H, tma_q, tma_k,
                                                       tma_v);
    check(cudaStreamSynchronize(timing_stream), "fa3_tma_ws warmup sync");
  }
  cudaEventRecord(start, timing_stream);
  if (smem_ok) {
    for (int r = 0; r < g_repeat; ++r)
      fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o,
                                                       seqlen, H, tma_q, tma_k,
                                                       tma_v);
  }
  cudaEventRecord(stop, timing_stream);
  cudaEventSynchronize(stop);

  float time_ms = 0;
  cudaEventElapsedTime(&time_ms, start, stop);
  time_ms /= g_repeat;

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
  half *h_o = (half *)malloc(sz);
  check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa3_tma_ws D2H");
  int count = B * H * seqlen * head_dim;

  char label[80];
  snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, %s)",
           kStagesK, kMmaAccF32 ? "F32Acc" : "F16Acc");
  float max_err = 0.0f;
  bool checked = h_o_ref || ref_o;
  if (smem_ok && checked) {
    for (int i = 0; i < count; ++i) {
      float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
      float err = fabsf(__half2float(h_o[i]) - ref_val);
      if (err > max_err) max_err = err;
    }
  }
  if (smem_ok && checked) {
    float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
```

```
    bool is_fail = max_err >= 5e-1f;
    if (is_fail || should_print_fa_tflops(kMmaAccF32, tflops)) {
      char tflops_str[32];
      if (cudnn_tflops_f16 > 0)
        snprintf(tflops_str, sizeof(tflops_str), "%.1f/%.1f (%.2fx)", tflops,
                 cudnn_tflops_f16, tflops / cudnn_tflops_f16);
      else
        snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
      printf("| %-56s | %.3e | %-19s |\n", label, max_err,
             tflops_str);
    }
  } else if (smem_ok) {
    float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
    if (should_print_fa_tflops(kMmaAccF32, tflops)) {
      char tflops_str[32];
      snprintf(tflops_str, sizeof(tflops_str), "%.1f", tflops);
      printf("| %-56s | %-9s | %-19s |\n", label, "unchecked",
             tflops_str);
    }
  } else {
    if (g_debug)
      printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large",
             "None");
  }

  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cudaStreamDestroy(timing_stream);
  free(h_o);
  cudaFree(tma_q);
  cudaFree(tma_k);
  cudaFree(tma_v);
}

// D=64/128 dispatch wrapper for FA3-style bench
template <int kMmaAccF32 = 0>
static void bench_fa_3_tma_ws_dispatch(int B, int H, int seqlen, int head_dim,
                                       half *h_o_ref, float *ref_o,
                                       half *d_q, half *d_k, half *d_v,
                                       half *d_o, float cudnn_tflops_f16) {
  // Try stages up to 4; bench_fa_3_tma_ws_launch internally checks
  // cudaDevAttrMaxSharedMemoryPerBlockOptin and skips configs that
  // exceed the actual device limit (e.g. Hopper 224KB vs Blackwell 101KB).
  if (head_dim == 64) {
    bench_fa_3_tma_ws_launch<64, 1, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                 ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
    bench_fa_3_tma_ws_launch<64, 2, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                 ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
    bench_fa_3_tma_ws_launch<64, 3, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                 ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
    bench_fa_3_tma_ws_launch<64, 4, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                 ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
  } else if (head_dim == 128) {
    bench_fa_3_tma_ws_launch<128, 1, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                  ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
    bench_fa_3_tma_ws_launch<128, 2, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                  ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
    bench_fa_3_tma_ws_launch<128, 3, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                  ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
    bench_fa_3_tma_ws_launch<128, 4, kMmaAccF32>(B, H, seqlen, head_dim, h_o_ref,
                                                  ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f16);
  } else {
    char label[64];
```

```
    snprintf(label, sizeof(label), "FA3 TMA MMA WS (2 Consumer WG) (D!=64/128)");
    printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
  }
}

#if defined(NOTES_V2_ENABLE_CUTE)
template <int kHeadDim, int kStagesK = 1>
static void bench_fa_3_tma_mma_ws_cute_launch(
    int B, int H, int seqlen, half *h_o_ref, float *ref_o,
    half *d_q, half *d_k, half *d_v, half *d_o,
    float cudnn_tflops_f32) {
  using namespace cute;
  using Traits = fa_cute::FlashAttn3CuTeTraits<kHeadDim>;
  using SmemLayout = typename Traits::SmemLayoutQKV;
  constexpr int kNumConsumers = 2;
  if (seqlen < 64 || seqlen % 64 != 0) {
    char label[80];
    snprintf(label, sizeof(label),
             "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, unaligned)", kStagesK);
    printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
    return;
  }

  int rows = B * H * seqlen;
  auto make_tma = [=](half *pointer) {
    auto tensor = make_tensor(
        make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
        make_shape(rows, Int<kHeadDim>{}),
        make_stride(Int<kHeadDim>{}, _1{}));
    return make_tma_copy(
        SM90_TMA_LOAD{}, tensor, SmemLayout{},
        Shape<_64, Int<kHeadDim>>{}, _1{});
  };
  auto tma_q = make_tma(d_q);
  auto tma_k = make_tma(d_k);
  auto tma_v = make_tma(d_v);
  auto kernel = flash_attn_3_tma_mma_ws_split_q_cute<
      kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v),
      kStagesK>;
  auto acc_o = partition_fragment_C(
      typename Traits::TiledMma{}, Shape<_64, Int<kHeadDim>>{});
  constexpr int kTiles = 1 + kNumConsumers * kStagesK + kNumConsumers;
  constexpr int kTilesBytes = kTiles * cosize(SmemLayout{}) * sizeof(cutlass::half_t);
  int merge_bytes = 128 * size(acc_o) * sizeof(float) + 128 * sizeof(float4);
  int smem_bytes = max(kTilesBytes, merge_bytes);
  bool smem_ok = check_smem_feasible((const void *)kernel, smem_bytes);
  if (!smem_ok) {
    char label[80];
    snprintf(label, sizeof(label),
             "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, SMEM)", kStagesK);
    if (g_debug)
      printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large", "None");
    return;
  }
  check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             smem_bytes),
        "bench cute fa3 set smem");

  dim3 grid(seqlen / 64, B * H);
  cudaStream_t stream;
  cudaEvent_t start;
  cudaEvent_t stop;
  check(cudaStreamCreate(&stream), "bench cute fa3 stream");
```

```
  check(cudaEventCreate(&start), "bench cute fa3 event start");
  check(cudaEventCreate(&stop), "bench cute fa3 event stop");
  for (int warmup = 0; warmup < g_warmup; ++warmup) {
    kernel<<<grid, 384, smem_bytes, stream>>>(
        tma_q, tma_k, tma_v,
        reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
  }
  check(cudaStreamSynchronize(stream), "bench cute fa3 warmup sync");
  check(cudaEventRecord(start, stream), "bench cute fa3 record start");
  for (int repeat = 0; repeat < g_repeat; ++repeat) {
    kernel<<<grid, 384, smem_bytes, stream>>>(
        tma_q, tma_k, tma_v,
        reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
  }
  check(cudaEventRecord(stop, stream), "bench cute fa3 record stop");
  check(cudaEventSynchronize(stop), "bench cute fa3 timing sync");
  float time_ms = 0.0f;
  check(cudaEventElapsedTime(&time_ms, start, stop), "bench cute fa3 elapsed");
  time_ms /= g_repeat;

  size_t count = (size_t)rows * kHeadDim;
  half *h_o = (half *)malloc(count * sizeof(half));
  check(cudaMemcpy(h_o, d_o, count * sizeof(half), cudaMemcpyDeviceToHost),
        "bench cute fa3 D2H");
  float max_err = 0.0f;
  bool checked = h_o_ref || ref_o;
  if (checked) {
    for (size_t idx = 0; idx < count; ++idx) {
      float reference = h_o_ref ? __half2float(h_o_ref[idx]) : ref_o[idx];
      max_err = max(max_err, fabsf(__half2float(h_o[idx]) - reference));
    }
  }
  float tflops = bench_fa_tflops(B, H, seqlen, kHeadDim, time_ms);
  char label[80];
  snprintf(label, sizeof(label),
           "FA3 CuTe TMA MMA WS (2 Consumer WG) (Sk=%d, Sv=1, F32Acc)", kStagesK);
  bool is_fail = checked && max_err >= 5e-1f;
  if (is_fail || should_print_fa_tflops(1, tflops)) {
    char performance[32];
    if (cudnn_tflops_f32 > 0.0f) {
      snprintf(performance, sizeof(performance), "%.1f/%.1f (%.2fx)",
               tflops, cudnn_tflops_f32, tflops / cudnn_tflops_f32);
    } else {
      snprintf(performance, sizeof(performance), "%.1f", tflops);
    }
    printf("| %-56s | %.3e | %-19s |\n", label, max_err, performance);
  }

  free(h_o);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cudaStreamDestroy(stream);
}

static void bench_fa_3_tma_mma_ws_cute_dispatch(
    int B, int H, int seqlen, int head_dim,
    half *h_o_ref, float *ref_o,
    half *d_q, half *d_k, half *d_v, half *d_o,
    float cudnn_tflops_f32) {
  if (head_dim == 64) {
    bench_fa_3_tma_mma_ws_cute_launch<64, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_3_tma_mma_ws_cute_launch<64, 2>(B, H, seqlen, h_o_ref, ref_o,
```

```
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_3_tma_mma_ws_cute_launch<64, 3>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_3_tma_mma_ws_cute_launch<64, 4>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
  } else if (head_dim == 128) {
    bench_fa_3_tma_mma_ws_cute_launch<128, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_3_tma_mma_ws_cute_launch<128, 2>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
  }
}

#if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
// FA2-style CuTe cp.async bench: single consumer, Br=128, no TMA/WS.
// V 单 buffer, V 在 QK 之前发起 cp.async, 通过 QK+softmax 隐藏 V 延迟.
template <int kHeadDim, int kStagesK>
static void bench_fa_2_mma_stages_cute_launch(
    int B, int H, int seqlen, half *h_o_ref, float *ref_o,
    half *d_q, half *d_k, half *d_v, half *d_o,
    float cudnn_tflops_f32) {
  using namespace cute;
  using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
  using SmemLayoutQ = typename Traits::SmemLayoutQ;
  using SmemLayoutKV = typename Traits::SmemLayoutKV;
  constexpr int kBr = 128;
  if (seqlen < kBr || seqlen % kBr != 0 || seqlen % 64 != 0) {
    char label[96];
    snprintf(label, sizeof(label),
             "FA2 CuTe MMA Stages (Sk=%d, unaligned)", kStagesK);
    printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
    return;
  }

  int rows = B * H * seqlen;
  auto kernel = flash_attn_mma_stages_split_q_cute<kHeadDim, kStagesK>;
  int smem_bytes = (size(SmemLayoutQ{}) +
                    kStagesK * size(SmemLayoutKV{}) +
                    1 * size(SmemLayoutKV{})) *
                   sizeof(cutlass::half_t);
  bool smem_ok = check_smem_feasible((const void *)kernel, smem_bytes);
  if (!smem_ok) {
    char label[96];
    snprintf(label, sizeof(label),
             "FA2 CuTe MMA Stages (Sk=%d, SMEM)", kStagesK);
    if (g_debug)
      printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large", "None");
    return;
  }
  check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             smem_bytes),
        "bench cute fa2 cpasync set smem");

  dim3 grid(seqlen / kBr, B * H);
  cudaStream_t stream;
  cudaEvent_t start;
  cudaEvent_t stop;
  check(cudaStreamCreate(&stream), "bench cute fa2 cpasync stream");
  check(cudaEventCreate(&start), "bench cute fa2 cpasync event start");
  check(cudaEventCreate(&stop), "bench cute fa2 cpasync event stop");
  for (int warmup = 0; warmup < g_warmup; ++warmup) {
    kernel<<<grid, 256, smem_bytes, stream>>>(
        reinterpret_cast<cutlass::half_t *>(d_q),
```

```
        reinterpret_cast<cutlass::half_t *>(d_k),
        reinterpret_cast<cutlass::half_t *>(d_v),
        reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
  }
  check(cudaStreamSynchronize(stream), "bench cute fa2 cpasync warmup sync");
  check(cudaEventRecord(start, stream), "bench cute fa2 cpasync record start");
  for (int repeat = 0; repeat < g_repeat; ++repeat) {
    kernel<<<grid, 256, smem_bytes, stream>>>(
        reinterpret_cast<cutlass::half_t *>(d_q),
        reinterpret_cast<cutlass::half_t *>(d_k),
        reinterpret_cast<cutlass::half_t *>(d_v),
        reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
  }
  check(cudaEventRecord(stop, stream), "bench cute fa2 cpasync record stop");
  check(cudaEventSynchronize(stop), "bench cute fa2 cpasync timing sync");
  float time_ms = 0.0f;
  check(cudaEventElapsedTime(&time_ms, start, stop),
        "bench cute fa2 cpasync elapsed");
  time_ms /= g_repeat;

  size_t count = (size_t)rows * kHeadDim;
  half *h_o = (half *)malloc(count * sizeof(half));
  check(cudaMemcpy(h_o, d_o, count * sizeof(half), cudaMemcpyDeviceToHost),
        "bench cute fa2 cpasync D2H");
  float max_err = 0.0f;
  bool checked = h_o_ref || ref_o;
  if (checked) {
    for (size_t idx = 0; idx < count; ++idx) {
      float reference = h_o_ref ? __half2float(h_o_ref[idx]) : ref_o[idx];
      max_err = max(max_err, fabsf(__half2float(h_o[idx]) - reference));
    }
  }
  float tflops = bench_fa_tflops(B, H, seqlen, kHeadDim, time_ms);
  char label[96];
  snprintf(label, sizeof(label),
           "FA2 CuTe MMA Stages (Sk=%d, F32Acc)", kStagesK);
  bool is_fail = checked && max_err >= 5e-1f;
  if (is_fail || should_print_fa_tflops(1, tflops)) {
    char performance[32];
    if (cudnn_tflops_f32 > 0.0f) {
      snprintf(performance, sizeof(performance), "%.1f/%.1f (%.2fx)",
               tflops, cudnn_tflops_f32, tflops / cudnn_tflops_f32);
    } else {
      snprintf(performance, sizeof(performance), "%.1f", tflops);
    }
    printf("| %-56s | %.3e | %-19s |\n", label, max_err, performance);
  }

  free(h_o);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cudaStreamDestroy(stream);
}

static void bench_fa_2_mma_stages_cute_dispatch(
    int B, int H, int seqlen, int head_dim,
    half *h_o_ref, float *ref_o,
    half *d_q, half *d_k, half *d_v, half *d_o,
    float cudnn_tflops_f32) {
  if (head_dim == 64) {
    bench_fa_2_mma_stages_cute_launch<64, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_mma_stages_cute_launch<64, 2>(B, H, seqlen, h_o_ref, ref_o,
```

```
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_mma_stages_cute_launch<64, 3>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_mma_stages_cute_launch<64, 4>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
  } else if (head_dim == 128) {
    bench_fa_2_mma_stages_cute_launch<128, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_mma_stages_cute_launch<128, 2>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
  }
}
#endif

// FA2-style CuTe bench: single consumer, Br=128, no merge.
template <int kHeadDim, int kStagesK, int kStagesV = 1>
static void bench_fa_2_tma_mma_ws_cute_launch(
    int B, int H, int seqlen, half *h_o_ref, float *ref_o,
    half *d_q, half *d_k, half *d_v, half *d_o,
    float cudnn_tflops_f32) {
  using namespace cute;
  using Traits = fa_cute::FlashAttn2CuTeTraits<kHeadDim>;
  using SmemLayoutQ = typename Traits::SmemLayoutQ;
  using SmemLayoutKV = typename Traits::SmemLayoutKV;
  constexpr int kBr = 128;
  if (seqlen < kBr || seqlen % kBr != 0 || seqlen % 64 != 0) {
    char label[80];
    snprintf(label, sizeof(label),
             "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, unaligned)",
             kStagesK, kStagesV);
    printf("| %-56s | %-9s | %-19s |\n", label, "SKIP", "None");
    return;
  }

  int rows = B * H * seqlen;
  // Q tile: [128, D], K/V tile: [64, D]
  auto make_tma_q = [=]() {
    auto tensor = make_tensor(
        make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(d_q)),
        make_shape(rows, Int<kHeadDim>{}),
        make_stride(Int<kHeadDim>{}, _1{}));
    return make_tma_copy(
        SM90_TMA_LOAD{}, tensor, SmemLayoutQ{},
        Shape<_128, Int<kHeadDim>>{}, _1{});
  };
  auto make_tma_kv = [=](half *pointer) {
    auto tensor = make_tensor(
        make_gmem_ptr(reinterpret_cast<cutlass::half_t *>(pointer)),
        make_shape(rows, Int<kHeadDim>{}),
        make_stride(Int<kHeadDim>{}, _1{}));
    return make_tma_copy(
        SM90_TMA_LOAD{}, tensor, SmemLayoutKV{},
        Shape<_64, Int<kHeadDim>>{}, _1{});
  };
  auto tma_q = make_tma_q();
  auto tma_k = make_tma_kv(d_k);
  auto tma_v = make_tma_kv(d_v);
  auto kernel = flash_attn_tma_mma_ws_split_q_cute<
      kHeadDim, decltype(tma_q), decltype(tma_k), decltype(tma_v),
      kStagesK, kStagesV>;
  // smem = Q[128*D] + K[Sk*64*D] + V[Sv*64*D] (no merge scratch)
  int smem_bytes = (cosize(SmemLayoutQ{}) +
                    kStagesK * cosize(SmemLayoutKV{}) +
```

```
                    kStagesV * cosize(SmemLayoutKV{})) *
                  sizeof(cutlass::half_t);
  bool smem_ok = check_smem_feasible((const void *)kernel, smem_bytes);
  if (!smem_ok) {
    char label[80];
    snprintf(label, sizeof(label),
             "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, SMEM)",
             kStagesK, kStagesV);
    if (g_debug)
      printf("| %-56s | %-9s | %-19s |\n", label, "SMEM too large", "None");
    return;
  }
  check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
                             smem_bytes),
        "bench cute fa2 set smem");

  dim3 grid(seqlen / kBr, B * H);
  cudaStream_t stream;
  cudaEvent_t start;
  cudaEvent_t stop;
  check(cudaStreamCreate(&stream), "bench cute fa2 stream");
  check(cudaEventCreate(&start), "bench cute fa2 event start");
  check(cudaEventCreate(&stop), "bench cute fa2 event stop");
  for (int warmup = 0; warmup < g_warmup; ++warmup) {
    kernel<<<grid, 384, smem_bytes, stream>>>(
        tma_q, tma_k, tma_v,
        reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
  }
  check(cudaStreamSynchronize(stream), "bench cute fa2 warmup sync");
  check(cudaEventRecord(start, stream), "bench cute fa2 record start");
  for (int repeat = 0; repeat < g_repeat; ++repeat) {
    kernel<<<grid, 384, smem_bytes, stream>>>(
        tma_q, tma_k, tma_v,
        reinterpret_cast<cutlass::half_t *>(d_o), rows, seqlen);
  }
  check(cudaEventRecord(stop, stream), "bench cute fa2 record stop");
  check(cudaEventSynchronize(stop), "bench cute fa2 timing sync");
  float time_ms = 0.0f;
  check(cudaEventElapsedTime(&time_ms, start, stop), "bench cute fa2 elapsed");
  time_ms /= g_repeat;

  size_t count = (size_t)rows * kHeadDim;
  half *h_o = (half *)malloc(count * sizeof(half));
  check(cudaMemcpy(h_o, d_o, count * sizeof(half), cudaMemcpyDeviceToHost),
        "bench cute fa2 D2H");
  float max_err = 0.0f;
  bool checked = h_o_ref || ref_o;
  if (checked) {
    for (size_t idx = 0; idx < count; ++idx) {
      float reference = h_o_ref ? __half2float(h_o_ref[idx]) : ref_o[idx];
      max_err = max(max_err, fabsf(__half2float(h_o[idx]) - reference));
    }
  }
  float tflops = bench_fa_tflops(B, H, seqlen, kHeadDim, time_ms);
  char label[80];
  snprintf(label, sizeof(label),
           "FA2 CuTe TMA MMA WS (1 Consumer WG) (Sk=%d, Sv=%d, F32Acc)",
           kStagesK, kStagesV);
  bool is_fail = checked && max_err >= 5e-1f;
  if (is_fail || should_print_fa_tflops(1, tflops)) {
    char performance[32];
    if (cudnn_tflops_f32 > 0.0f) {
      snprintf(performance, sizeof(performance), "%.1f/%.1f (%.2fx)",
```

```
                tflops, cudnn_tflops_f32, tflops / cudnn_tflops_f32);
    } else {
      snprintf(performance, sizeof(performance), "%.1f", tflops);
    }
    printf("| %-56s | %.3e | %-19s |\n", label, max_err, performance);
  }

  free(h_o);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  cudaStreamDestroy(stream);
}

static void bench_fa_2_tma_mma_ws_cute_dispatch(
    int B, int H, int seqlen, int head_dim,
    half *h_o_ref, float *ref_o,
    half *d_q, half *d_k, half *d_v, half *d_o,
    float cudnn_tflops_f32) {
  if (head_dim == 64) {
    bench_fa_2_tma_mma_ws_cute_launch<64, 1, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_tma_mma_ws_cute_launch<64, 2, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_tma_mma_ws_cute_launch<64, 3, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_tma_mma_ws_cute_launch<64, 4, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_tma_mma_ws_cute_launch<64, 2, 2>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
  } else if (head_dim == 128) {
    bench_fa_2_tma_mma_ws_cute_launch<128, 1, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_tma_mma_ws_cute_launch<128, 2, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    bench_fa_2_tma_mma_ws_cute_launch<128, 3, 1>(B, H, seqlen, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
  }
}
#endif
#endif /* NOTES_V2_ENABLE_TMA_MMA_WS */

#if defined(NOTES_V2_ENABLE_CUDNN)
static float bench_cudnn_sdpa_tflops(half *d_q, half *d_k, half *d_v,
                                     half *d_o_ref, int B, int H, int seqlen,
                                     int head_dim, fe::DataType_t compute_type) {
  cudnnHandle_t handle;
  cudnnCreate(&handle);
  auto graph = std::make_shared<fe::graph::Graph>();
  graph->set_io_data_type(fe::DataType_t::HALF)
    .set_intermediate_data_type(fe::DataType_t::FLOAT)
    .set_compute_data_type(compute_type);

  auto Q = graph->tensor(fe::graph::Tensor_attributes()
    .set_uid(1).set_dim({B, H, seqlen, head_dim})
    .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
  auto K = graph->tensor(fe::graph::Tensor_attributes()
    .set_uid(2).set_dim({B, H, seqlen, head_dim})
    .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
  auto V = graph->tensor(fe::graph::Tensor_attributes()
    .set_uid(3).set_dim({B, H, seqlen, head_dim})
    .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));

  auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
```

```
    fe::graph::SDPA_attributes()
      .set_name("sdpa_ref")
      .set_attn_scale(1.0f / sqrtf((float)head_dim)));

  O_sdpa->set_output(true).set_uid(4)
    .set_dim({B, H, seqlen, head_dim})
    .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});

  auto build_status = graph->build(handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
  float tflops = -1.0f;
  if (build_status.is_good()) {
    std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
      {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
    int64_t ws_size = 0;
    if (graph->get_workspace_size(ws_size).is_good()) {
      int8_t *d_ws = nullptr;
      if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "bench cudnn ws");
      if (graph->execute(handle, vp, d_ws).is_good()) {
        check(cudaDeviceSynchronize(), "bench cudnn sync");
        for (int w = 1; w < g_warmup; ++w)
          (void)graph->execute(handle, vp, d_ws);
        cudaDeviceSynchronize();
        cudaEvent_t ev_s, ev_e;
        cudaEventCreate(&ev_s); cudaEventCreate(&ev_e);
        cudaEventRecord(ev_s);
        for (int r = 0; r < g_repeat; ++r)
          (void)graph->execute(handle, vp, d_ws);
        cudaEventRecord(ev_e);
        cudaEventSynchronize(ev_e);
        float time_ms = 0;
        cudaEventElapsedTime(&time_ms, ev_s, ev_e);
        time_ms /= g_repeat;
        tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
        cudaEventDestroy(ev_s); cudaEventDestroy(ev_e);
      }
      if (d_ws) cudaFree(d_ws);
    }
  }
  cudnnDestroy(handle);
  return tflops;
}
#endif

static void bench_flash_attn(int B, int H, int N, int D) {
  int seqlen = N, head_dim = D;

  if (head_dim != 64 && head_dim != 128) {
    printf("| %-56s | %-9s | %-19s |\n", "FlashAttention-2", "unsupported D", "None");
    return;
  }

  size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);

  srand(42);
  half *h_q = (half *)malloc(sz);
  half *h_k = (half *)malloc(sz);
  half *h_v = (half *)malloc(sz);
  for (int i = 0; i < B * H * seqlen * head_dim; i++) {
    h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
    h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
  }

```

```
  // Device allocations — shared by kernel and reference
  half *d_q, *d_k, *d_v, *d_o;
  check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
  check(cudaMalloc(&d_k, sz), "bench fa alloc K");
  check(cudaMalloc(&d_v, sz), "bench fa alloc V");
  check(cudaMalloc(&d_o, sz), "bench fa alloc O");
  check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
  check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
  check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");

  // Reference output: either cuDNN (half) or CPU (float)
  half *h_o_ref = nullptr;
  float *ref_o = nullptr;
  float cudnn_tflops_f16 = -1.0f, cudnn_tflops_f32 = -1.0f;
  int ref_count = B * H * seqlen * head_dim;

#if defined(NOTES_V2_ENABLE_CUDNN)
  if (!g_fa_skip_check) {
    half *d_o_ref;
    check(cudaMalloc(&d_o_ref, sz), "bench fa alloc O_ref");
    cudnn_tflops_f16 = bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref,
                                               B, H, seqlen, head_dim,
                                               fe::DataType_t::HALF);
    bool cudnn_ok = (cudnn_tflops_f16 > 0);
    if (cudnn_ok) {
      cudnn_tflops_f32 = bench_cudnn_sdpa_tflops(d_q, d_k, d_v, d_o_ref,
                                                 B, H, seqlen, head_dim,
                                                 fe::DataType_t::FLOAT);
      h_o_ref = (half *)malloc(sz);
      check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "bench fa D2H ref");
    } else {
      fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
    }
    cudaFree(d_o_ref);
  }
#endif

  // CPU reference fallback
  if (!g_fa_skip_check && !h_o_ref) {
    float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
    float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
    float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
    ref_o = (float *)malloc(sz * 4 / sizeof(half));
    for (int i = 0; i < ref_count; i++) {
      ref_q[i] = __half2float(h_q[i]);
      ref_k[i] = __half2float(h_k[i]);
      ref_v[i] = __half2float(h_v[i]);
    }

    float scale = 1.0f / sqrtf((float)head_dim);
    for (int bi = 0; bi < B * H; bi++) {
      for (int qi = 0; qi < seqlen; qi++) {
        float smax = -INFINITY;
        float *S = (float *)malloc((size_t)seqlen * sizeof(float));
        for (int kj = 0; kj < seqlen; kj++) {
          float s = 0.0f;
          for (int d = 0; d < head_dim; d++)
            s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
                 ref_k[bi * seqlen * head_dim + kj * head_dim + d];
          S[kj] = s * scale;
          if (S[kj] > smax) smax = S[kj];
        }
        double sum_exp = 0.0;
```

```
        for (int kj = 0; kj < seqlen; kj++)
          sum_exp += (double)expf(S[kj] - smax);
        float inv_sum = 1.0f / (float)sum_exp;
        for (int d = 0; d < head_dim; d++) {
          double o_acc = 0.0;
          for (int kj = 0; kj < seqlen; kj++)
            o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
          ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
        }
        free(S);
      }
    }
    free(ref_q);
    free(ref_k);
    free(ref_v);
  }

  if (g_bench_fa3_cute_only) {
#if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
    bench_fa_3_tma_mma_ws_cute_dispatch(
        B, H, seqlen, head_dim, h_o_ref, ref_o,
        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
#endif
    free(h_q);
    free(h_k);
    free(h_v);
    free(h_o_ref);
    free(ref_o);
    cudaFree(d_q);
    cudaFree(d_k);
    cudaFree(d_v);
    cudaFree(d_o);
    return;
  }

  if (head_dim == 64) {
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 1, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 1, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
```

```
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                        d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 8, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                        d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<64, 2, 0, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                         d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
#if defined(NOTES_V2_ENABLE_CUTE)
    {
      cudaDeviceSynchronize();
      bench_fa_2_mma_stages_cute_dispatch(
        B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
#endif
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
    {
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<1, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                            d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                            d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<3, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                            d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
```

```
      bench_fa_tma_mma_ws_dispatch<4, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                            d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 2, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                            d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      // F32Acc variants
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<1, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
                                            ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
                                            ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<3, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
                                            ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<4, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
                                            ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 2, 1>(B, H, seqlen, head_dim, h_o_ref,
                                            ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      // FA3-style dual-consumer
      cudaDeviceSynchronize();
      bench_fa_3_tma_ws_dispatch<0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                    d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_3_tma_ws_dispatch<1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                    d_q, d_k, d_v, d_o, cudnn_tflops_f32);
#if defined(NOTES_V2_ENABLE_CUTE)
      cudaDeviceSynchronize();
      bench_fa_2_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_3_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                d_q, d_k, d_v, d_o, cudnn_tflops_f32);
#endif
    }
#endif
  } else {
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
      cudaDeviceSynchronize();
      bench_fa_launch<128, 1, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                           d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 8, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                           d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 1, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                           d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 8, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                           d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                           d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 8, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                           d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
      cudaDeviceSynchronize();
```

```
      bench_fa_launch<128, 2, 8, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 8, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 8, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 8, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 0, 8, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 8, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 8, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 8, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
    if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_launch<128, 2, 0, 0, 0, 1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
#if defined(NOTES_V2_ENABLE_CUTE)
    {
      cudaDeviceSynchronize();
      bench_fa_2_mma_stages_cute_dispatch(
        B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    }
#endif
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
    {
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<1, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                            d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                            d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<3, 1, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
```

```
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 2, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                          d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      // F32Acc variants
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<1, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
                                          ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
                                          ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<3, 1, 1>(B, H, seqlen, head_dim, h_o_ref,
                                          ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_tma_mma_ws_dispatch<2, 2, 1>(B, H, seqlen, head_dim, h_o_ref,
                                          ref_o, d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      // FA3-style dual-consumer
      cudaDeviceSynchronize();
      bench_fa_3_tma_ws_dispatch<0>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                    d_q, d_k, d_v, d_o, cudnn_tflops_f16);
      cudaDeviceSynchronize();
      bench_fa_3_tma_ws_dispatch<1>(B, H, seqlen, head_dim, h_o_ref, ref_o,
                                    d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    #if defined(NOTES_V2_ENABLE_CUTE)
      cudaDeviceSynchronize();
      bench_fa_2_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
               d_q, d_k, d_v, d_o, cudnn_tflops_f32);
      cudaDeviceSynchronize();
      bench_fa_3_tma_mma_ws_cute_dispatch(B, H, seqlen, head_dim, h_o_ref, ref_o,
               d_q, d_k, d_v, d_o, cudnn_tflops_f32);
    #endif
    }
#endif
  }
  cudaDeviceSynchronize();

  free(h_q);
  free(h_k);
  free(h_v);
  free(h_o_ref);
  free(ref_o);
  cudaFree(d_q);
  cudaFree(d_k);
  cudaFree(d_v);
  cudaFree(d_o);
}

// Host 端 v1/v2 等价性测试: 遍历 kColStride/i/j, 断言 swizzle_v1_impl == swizzle_v2_impl.
// 用于在启用 NOTES_V2_ENABLE_SWIZZLE_V2 前后验证 v2 与 v1 bit-exact 等价.
// kColStride=8 在 v2 内回退 v1, 故 8 也应恒等.
template <int kColStride>
static void swizzle_equiv_check_one(long &total, long &fail) {
  for (int i = 0; i < 256; ++i) {
    for (int j = 0; j < kColStride; ++j) {
      int v1 = swizzle_v1_impl<kColStride>(i, j);
      int v2 = swizzle_v2_impl<kColStride>(i, j);
      ++total;
      if (v1 != v2) {
        ++fail;
        if (fail <= 10)
          printf("  MISMATCH cs=%d i=%d j=%d: v1=%d v2=%d\n", kColStride, i, j, v1, v2);
      }
```

```
    }
  }
}

static void test_swizzle_equiv() {
  long total = 0, fail = 0;
  swizzle_equiv_check_one<8>(total, fail);
  swizzle_equiv_check_one<16>(total, fail);
  swizzle_equiv_check_one<32>(total, fail);
  swizzle_equiv_check_one<64>(total, fail);
  printf("| %-56s | %-9s |\n",
         "Swizzle v1/v2 equiv (host)", fail == 0 ? "ALL PASS" : "FAIL");
  printf("  total=%ld fail=%ld\n", total, fail);
}

int main(int argc, char *argv[]) {
#if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  cuInit(0);
#endif

  for (int i = 1; i < argc; i++) {
    if (strcmp(argv[i], "--bench-hgemm") == 0) {
      g_bench_hgemm = true;
    } else if (strcmp(argv[i], "--bench") == 0) {
      g_bench_hgemm = true;
      g_bench_fa = true;
    } else if (strcmp(argv[i], "--bench-fa") == 0) {
      g_bench_fa = true;
    } else if (strcmp(argv[i], "--bench-fa3-cute") == 0) {
      g_bench_fa = true;
      g_bench_fa3_cute_only = true;
    } else if (strcmp(argv[i], "--bench-hgemm-all") == 0) {
      g_bench_hgemm_all = true;
    } else if (strcmp(argv[i], "--bench-fa-all") == 0) {
      g_bench_fa = true;
      g_fa_layout = FALayout::All;
    } else if (strcmp(argv[i], "--bench-all") == 0) {
      g_bench_all = true;
      g_fa_layout = FALayout::All;
    } else if (strcmp(argv[i], "--mnk") == 0 && i + 1 < argc) {
      sscanf(argv[++i], "%d,%d,%d", &g_bench_M, &g_bench_N, &g_bench_K);
    } else if (strcmp(argv[i], "--bhnd") == 0 && i + 1 < argc) {
      sscanf(argv[++i], "%d,%d,%d,%d", &g_bench_B, &g_bench_H, &g_bench_Nfa, &g_bench_D);
    } else if (strcmp(argv[i], "--fa-layout") == 0 && i + 1 < argc) {
      const char *layout = argv[++i];
      if (strcmp(layout, "all") == 0)
        g_fa_layout = FALayout::All;
      else if (strcmp(layout, "pad") == 0)
        g_fa_layout = FALayout::Pad;
      else if (strcmp(layout, "swizzle-q") == 0)
        g_fa_layout = FALayout::SwizzleQ;
      else if (strcmp(layout, "swizzle-k") == 0)
        g_fa_layout = FALayout::SwizzleK;
      else if (strcmp(layout, "swizzle-v") == 0)
        g_fa_layout = FALayout::SwizzleV;
      else if (strcmp(layout, "swizzle-qk") == 0)
        g_fa_layout = FALayout::SwizzleQK;
      else if (strcmp(layout, "swizzle-qv") == 0)
        g_fa_layout = FALayout::SwizzleQV;
      else if (strcmp(layout, "swizzle-kv") == 0)
        g_fa_layout = FALayout::SwizzleKV;
      else if (strcmp(layout, "swizzle") == 0)
        g_fa_layout = FALayout::Swizzle;
```

```
      else {
        fprintf(stderr, "Unsupported FA layout: %s\n", layout);
        return EXIT_FAILURE;
      }
    } else if (strcmp(argv[i], "--fa-skip-check") == 0) {
      g_fa_skip_check = true;
    } else if (strcmp(argv[i], "--swizzle-eq-check") == 0) {
      g_swizzle_eq_check = true;
    } else if (strcmp(argv[i], "--debug") == 0) {
      g_debug = true;
    } else if (strcmp(argv[i], "--verbose") == 0) {
      g_verbose = true;
    } else if (strcmp(argv[i], "--warmup") == 0 && i + 1 < argc) {
      g_warmup = atoi(argv[++i]);
    } else if (strcmp(argv[i], "--repeat") == 0 && i + 1 < argc) {
      g_repeat = atoi(argv[++i]);
    }
  }

  if (g_swizzle_eq_check) {
    printf("=== notes-v2.cu swizzle v1/v2 equivalence check ===\n");
    printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
    printf("|----------------------------------------------------------|----------|\n");
    test_swizzle_equiv();
    printf("=== Done ===\n");
    return 0;
  }

#if defined(NOTES_V2_ENABLE_CUTE) && defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  if (argc >= 2 && strcmp(argv[1], "--fa3-cute") == 0) {
    printf("=== CuTe FA3 TMA MMA WS correctness ===\n");
    printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
    printf("|----------------------------------------------------------|----------|\n");
    test_flash_attn_3_tma_mma_ws_split_q_cute<64>();
    test_flash_attn_3_tma_mma_ws_split_q_cute<128>();
    return 0;
  }
  if (argc >= 2 && strcmp(argv[1], "--fa3-cute-tma-smoke") == 0) {
    printf("=== CuTe TMA copy smoke ===\n");
    printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
    printf("|----------------------------------------------------------|----------|\n");
    test_flash_attn_3_cute_tma_copy_smoke<64>();
    test_flash_attn_3_cute_tma_copy_smoke<128>();
    return 0;
  }
  if (argc >= 2 && strcmp(argv[1], "--fa2-cute-cpasync") == 0) {
    printf("=== CuTe FA2 MMA Stages (cp.async) correctness ===\n");
    printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
    printf("|----------------------------------------------------------|----------|\n");
    test_flash_attn_mma_stages_split_q_cute<64>();
    test_flash_attn_mma_stages_split_q_cute<128>();
    return 0;
  }
  if (argc >= 2 && strcmp(argv[1], "--fa2-cute") == 0) {
    printf("=== CuTe FA2 TMA MMA WS correctness ===\n");
    printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
    printf("|----------------------------------------------------------|----------|\n");
    test_flash_attn_tma_mma_ws_split_q_cute<64>();
    test_flash_attn_tma_mma_ws_split_q_cute<128>();
    return 0;
  }
#endif

```

```
  if (g_bench_hgemm || g_bench_fa || g_bench_all) {
    printf("=== notes-v2.cu bench mode ===\n");
    printf("HGEMM: M=%d N=%d K=%d   FA: B=%d H=%d N=%d D=%d\n",
           g_bench_M, g_bench_N, g_bench_K,
           g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
    printf("| %-56s | %-9s | %-19s |\n", "Kernel", "Max Err", "TFLOPS/cu{BLAS,DNN}");
    printf(
      "|----------------------------------------------------------|-----------|---------------------|\n"
    );

    if (g_bench_hgemm || g_bench_hgemm_all || g_bench_all) {
#if defined(NOTES_V2_ENABLE_CUTE)
      bench_hgemm_cute(g_bench_M, g_bench_N, g_bench_K);
#endif
      if (g_bench_hgemm_all || g_bench_all) {
        bench_hgemm_mma(g_bench_M, g_bench_N, g_bench_K);
        bench_hgemm_swizzle(g_bench_M, g_bench_N, g_bench_K);
#if defined(NOTES_V2_ENABLE_WGMMA)
        bench_hgemm_wgmma(g_bench_M, g_bench_N, g_bench_K);
#endif
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
        bench_hgemm_tma_mma_ws(g_bench_M, g_bench_N, g_bench_K);
#endif
      }
    }
    if (g_bench_fa || g_bench_all)
      bench_flash_attn(g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);

    printf("=== Bench done ===\n");
    return 0;
  }

#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
    int M = 128, N = 128, K = 64;
    if (argc > 4) {
      M = atoi(argv[2]);
      N = atoi(argv[3]);
      K = atoi(argv[4]);
    }
    printf("=== SM120 TMA MMA WS validation ===\n");
    printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
    printf("|----------------------------------------------------------|----------|\n");
    test_hgemm_tma_mma_ws(M, N, K);
    return 0;
  }
#endif
  int M = 1024, N = 1024, K = 1024;
  if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }

  printf("=== notes-v2.cu verification harness ===\n");
  printf("| %-56s | %-9s |\n", "Kernel", "Max Err");
  printf("|----------------------------------------------------------|-----------|\n");

  test_block_reduce(N);
  test_dot(N);
  test_relu(1024);
  test_elementwise(1024);
  test_histogram(1024);
  test_merge_attn_states(512, 16, 128);
  test_softmax(256);
  test_rms_norm(8, 128);
  test_layer_norm(8, 128);
```

```
  test_rope(8, 128);
  test_mat_transpose(256, 256);
  test_mat_transpose_padded(256, 256);
  test_sgemv(256, 128);
  test_sgemm(M, N, K);
  test_hgemm_mma(M, N, K);
  test_hgemm_swizzle(M, N, K);
#if (defined(NOTES_V2_ENABLE_CUTE))
  test_hgemm_cute(M, N, K);
#endif
#if (defined(NOTES_V2_ENABLE_WGMMA))
  test_hgemm_wgmma(M, N, K);
#endif
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  test_hgemm_tma_mma_ws(M, N, K);
#endif
  test_flash_attn(1024, 64);
#if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
  test_flash_attn_tma_mma_ws(1024, 64);
  test_flash_attn_tma_mma_ws(1024, 128);
  test_flash_attn_3_tma_ws(1024, 64);
  test_flash_attn_3_tma_ws(1024, 128);
#endif

  printf("=== All tests done ===\n");
  return 0;
}

// ============================================================================
// Quick build & run reference
// ============================================================================
// # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMA) :
// nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
//   -I ../../third-party/cutlass/include \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
//
// # sm_89 单独编译 + 运行:
// nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
//
// # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
//    项目内置 third-party/cutlass) :
// nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
//   -I ../../third-party/cutlass/include \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
//
// # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
// nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
//   -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
//
// # sm_90a + CuTe + WGMMA:
// nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
//   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
//   -I ../../third-party/cutlass/include \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
//
// # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
//   RTX PRO 5000 / RTX 5090):
// nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
//   -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
// ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024
//
// # sm_120a + CUTE + TMA_MMA_WS + cuDNN SDPA (CUDA Toolkit >= 13.2;
//   requires cudnn-frontend submodule: git submodule update --init):
```

```
// nvcc -std=c++20 -O2 -arch=sm_120a \
//   -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_TMA_MMA_WS -DNOTES_V2_ENABLE_CUDNN \
//   -I ../../third-party/cutlass/include -I ../../third-party/cudnn-frontend/include \
//   -L/usr/local/cuda-13.2/targets/x86_64-linux/lib/stubs \
//   -lcublas -lcudnn -lnvrtc -lcuda notes-v2.cu -o notes_v2_cute_ws_sm120a.bin
// # Default FA bench (kPadQ=kPadK=kPadV=8 only):
// ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa --bhnd 1,48,4096,64
// # All FA layout variants:
// ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa-all --bhnd 1,48,4096,64
// # FA TMA MMA WS bench (SM120, D=64 only, kStagesK=2/3):
// ./notes_v2_cute_ws_sm120a.bin --bench-fa-tma-ws --bhnd 1,32,4096,64
```